

GPU-AWARE HIGH-PERFORMANCE COMMUNICATION AND COMPUTATION IN HPC CLUSTERS

by

AMIRHOSSEIN SOJODI

A thesis submitted to the
Graduate Program in Electrical and Computer Engineering
in conformity with the requirements for
the Degree of Doctor of Philosophy

Queen's University
Kingston, Ontario, Canada

January 2026

COPYRIGHT © AMIRHOSSEIN SOJODI, 2026

Abstract

Optimizing communication and computation is critical for advancing high-performance computing (HPC) systems and applications. This dissertation presents a comprehensive study on various techniques to enhance the performance of HPC systems, focusing on GPU-aware multi-path communication, heterogeneous computing, collective communication, and parallelization strategies for complex computational problems.

We investigate methods to optimize GPU-to-GPU communication within a single node. We introduce a novel multi-path communication framework within the MPI+UCX library, designed to leverage multiple communication paths, such as NVLink and PCIe, concurrently. To further improve point-to-point communication, we integrate CUDA Graphs into this framework, achieving up to a 2.9x improvement in GPU-to-GPU bandwidth tests. We also characterize the performance of CUDA Graphs in the context of multi-path communication to understand its benefits and limitations.

We develop the first analytical performance model for multi-path intra-node GPU communication. This model predicts data transfer times under pipelined, staged intra-node GPU transfers, enabling the dynamic configuration of data transfers across multiple paths. We prove that the optimal multi-path schedule (minimizing overall transfer time) is achieved when the per-path transfer times are equal. We then integrate this model into the MPI+UCX communication stack for automatic runtime tuning, validating its accuracy with an average error of less than 6% for messages larger than 4MB. The model-enabled multi-path communication strategy achieves up to 2.9x and 1.4x speedup in point-to-point and collective operations, respectively.

Beyond point-to-point communication, we propose a collaborative Allreduce collective communication framework that utilizes heterogeneous computing and multi-path communication for multi-GPU systems. This approach leverages GPU parallelism and high memory bandwidth, optimizing both data transfer and host system computing power. Our implementation demonstrates significant performance gains, with improvements of up

to 1.45x for large message sizes compared to the state-of-the-art NCCL library. We also introduce the first performance model for heterogeneous Allreduce operations, which accurately predicts execution times with an average error of less than 8% for messages between 4MB and 512MB.

Finally, we address the computational challenges of constructing Legendre Pairs (LPs) for large lengths. We propose a highly effective multi-GPU solution and detail the parallelized implementation of the LP search using OpenMP, MPI, and CUDA. Through systematic optimization strategies for both CPU and GPU, we achieve a substantial performance improvement, demonstrating a speedup of over 6,500 $\times$ and 1,900 $\times$ on NVIDIA H100 and A30 GPUs, respectively, compared to serial execution on an Intel Xeon Gold 6338 CPU. We also studied the resource utilization and scalability of our implementations on modern GPUs, using NVIDIA Multi-Instance GPU (MIG) and Multi-Process Service (MPS).

Acknowledgements

Dedicated to my love,

Parisa

for her tremendous support,
wonderful kindness, loyal encouragement,
and endless patience. Thank you for being my rock,
inspiration, motivation, and hope in every step of the way.
I couldn't have done this without you.

First and foremost, I extend my sincere appreciation to my supervisor, Dr. Ahmad Afsahi, for his support, insightful guidance, and profound expertise. His diligence, patience, and dedication to students' academic and professional development have been truly inspiring. I am deeply grateful for the countless hours we spent discussing ideas, refining arguments, and providing constructive feedback that shaped this research.

My heartfelt thanks also go to my dissertation committee members: Dr. Antonio J. Pena, Dr. Ahmed Hassan, Dr. Xiaodan Zhu, and Dr. Muhammad Alam. Their critical insights, challenging questions, and diverse perspectives greatly enriched this study.

To my colleagues and friends at the Parallel Processing Research Laboratory (PPRL), a sincere thanks for the stimulating discussions, collaborative spirit, and lasting friendships that made my time here so meaningful: Pedram Alizadeh, Dr. Yiltan Temucin, Benjamin Kitor, Hamed Sharifian, Amirreza Barati, Ali Farazdaghi, Keshav Aggarwal, and Leila Habibpour. Special thanks to Dr. Ryan E. Grant for his invaluable mentorship and support

throughout my research journey. I am also deeply grateful to Dr. Mohammad Akbari for his collaborations and his support.

I would like to acknowledge the support of the Natural Sciences and Engineering Research Council of Canada (NSERC), Digital Research Alliance of Canada, and the SciNet HPC Consortium. The computations that made this dissertation possible were performed partly on Beluga, Narval, and Mist clusters, from Compute Canada. I am grateful for the technical support provided by Amirreza Barati and Dr. Yiltan Temucin for their assistance with our local cluster maintenance and troubleshooting.

My immense gratitude goes to the incredible staff at Queen’s University, especially the ECE department assistants, Cheryl Wright, Debra Fraser, and Dijana Bosnjak-Krstic. I am also deeply grateful to Alyssa Foerstner, Lydia Skulstad, and the entire team at Student Academic Services (SASS) for their great support and priceless assistance. They helped me shape and polish my academic and authorial voice through days of discussion and feedback. I am forever grateful for all of their help and support, especially with the IV@Q podcast. My special thanks go to Kate Zucconi for her helpful advice and support regarding my scholarship/job applications. I would also like to thank Amanda Gray, Colette Steer, and the amazing teams at the School of Graduate Studies (SGS), Queen’s University International Center (QUIC), Career Services, and Dissertation Boot Camps (DBC) for their invaluable help throughout my academic journey, especially during the challenging times of the COVID pandemic. The last Dissertation Boot Camp (Lake Shift 2025) and the incredible team, students, and the friends I met there was particularly beneficial in writing my dissertation.

I would also like to extend my heartfelt gratitude to the amazing team at Distributive Corp., led by Dr. Dan Desjardins, where I had the privilege to do my part-time job as a GPU Software Engineer for two years during my PhD studies. Their support, encouragement, and flexibility were instrumental in allowing me to balance work and research effectively. Also, my appreciation goes to Dr. Ilias S. Kotsireas from Wilfrid Laurier University for his guidance and insights on the Legendre Pairs search algorithm.

Moreover, I am very thankful for the great team at Microsoft Research (MSR), especially Dr. Philipp Witte and Dr. Russle Hewett, for their support and collaboration during my internship. Their insights and expertise were so helpful to my research.

During my academic journey before my PhD studies, I have been fortunate to receive support and guidance from many individuals who have significantly influenced my personal and professional growth. I would like to express my deepest gratitude to my previous advisors and supervisors: Dr. Farshad Khunjush for his great supervisorship during my master's studies, Dr. Reza Azimi who introduced me to the world of GPUs, and Dr. Gholamhossein Dastghaibifard who introduced me to MPI and parallel algorithms, Morteza Tajbakhsh for his mentorship and his unique courses during my undergraduate studies, Arash Kamali for teaching me about growth mindset and self development, and Amin Abbasi for his support and friendship during my early academic years.

This work would not have been possible without the unwavering support of my amazing family. To my dear lovely parents, Aliasghar and Bahiyeh, and my very kind brother, Amirreza, thank you so much for being a constant source of encouragement and motivation. My special gratitude goes to beloved Mozhdah for her inspiring motivation and continuous support throughout my academic journey. My deep appreciation goes to Jamileh, Behzad, Azadeh, Mohsen, Behrooz, Zeinab, Parizad, and all other my family members for their love and care. This work is dedicated to them.

Finally, to my dear friends, thank you for your loyal companionship, understanding, encouragement, and love throughout this journey: Armin, Soheila, Abdi, Samira, Sareh, Amir, Mohammad, Marzieh, Hossein, Najmeh, Camelia, Mohsen, Elham, Todd, Holly, Gordon, Sandra, Juergen, Maryam, Behzad, Maryam, Ashkan, Azadeh, Hooman, Nastaran, Golsa, Javad; including my dear friends from all over the world who have always been there when I needed them: Siavash, Zohreh, Bahareh, Majid, Mahsa, Farzin, Sanaz, Iman, Hossein, Saeed, Ensieh, Tina, Fatemeh, Amin, Azam, Arash, Negar, Parastoo, Ehsan, Khooneye Mehr Family, Masih, Babak, Ghazaleh, Sarah, Sina, Roozbeh, Asma, Zohreh, Marzieh, Pedram, and Mahyar. I would also like to thank those who have supported us throughout our immigration and settlement process in Canada: Mehdi, Nastaran, John, Krista, Amy, Alireza, Arshia, Reza, Mahshid, Amirhossein, and last but not least, Dr. Alireza Bakhshai.

I am forever grateful to Queen's University for providing me with the opportunity, the resources, the community, and the funding that made this research possible. The experiences and knowledge I gained here will stay with me for the rest of my life.

In loving memory of
Ardavan Afshar, Maysam Haghdan, Mohammad-Sadegh Kiani,
Dr. Gh. Dastghaibifard, and Amir Moradi

Statement of Originality

I hereby certify that the intellectual content of this dissertation is the product of my own work. All references and contributions of other individuals have been cited and sourced appropriately.

The work in Chapter 3 was inspired by my previous collaboration on multi-path communication with Dr. Yiltan Temucin. In this work, Yiltan provided support to understand the MPI+UCX software stack and the modules involved in the implementation, and Amirreza Barati helped with submitting some of the jobs on the Narval cluster.

The work in Chapter 4 is a collaborative effort with Dr. Mohammad Akbari, Hamed Sharifian, Ali Farazdaghi, and Ryan E. Grant. Hamed Sharifian brainstormed on the design and initial ideas of the performance model. Dr. Mohammad Akbari developed and provided the proof of the theorem presented in this chapter. Ali Farazdaghi contributed to measurement of the synchronization cost on the staging devices. Ryan Grant provided reviews and guidance along the way. Also, the team worked together to discuss the overall architecture and key design decisions for the performance model.

The work in Chapter 5 is a collaboration with Ali Farazdaghi, Hamed Sharifian, and Ryan E. Grant. Ali Farazdaghi contributed to the implementation of the CPU side reduction by utilizing vector instructions. He also helped me with maintaining the build environment of the library stack, ensuring integration and functionality across different clusters. Hamed Sharifian contributed to debugging and testing the implementation, providing valuable feedback and insights throughout the process. Also, Ryan E. Grant provided feedback and reviews during the development of this work.

The work in Chapter 6 is a collaborative effort with Dr. Ilias S. Kotsireas, and its early stages were supported by Distributive Corp., where I worked as a part-time GPU Software Engineer. Dr. Kotsireas provided the initial idea and the theoretical framework for the Legendre Pairs search algorithm on a single-core CPU, presented in Section 6.2.1. The

early phases of this research contributed to the WebGPU-based solution of the Legendre Pairs search algorithm, while I was employed at Distributive Corp.

Generative AI tools were used to assist with grammar and spelling checks, as well as formatting consistency and improvements of the text. Other than the mentioned collaborations, this dissertation is the result of my own original research and work, including the design, implementation, experimentation, analysis, figures, listings, plots, and writing the entire content of this dissertation.

Table of Contents

Abstract	ii
Acknowledgements	iv
Statement of Originality	viii
Table of Contents	x
List of Figures	xvi
List of Tables	xxii
List of Algorithms	xxiv
List of Code Listings	xxv
Glossary	xxvi
Chapter 1: Introduction	1
1.1 Motivation	2
1.2 Problem Statement	4
1.3 Contributions	5
1.4 Dissertation Organization	7

Chapter 2: Background	8
2.1 High-Performance Computing	9
2.1.1 High-Performance Interconnects	9
2.1.2 GPU-enabled Clusters	11
2.1.3 Modern GPU Features and Technologies	11
2.1.4 Parallel Programming Paradigms	14
2.2 Shared-Memory Programming Models	15
2.3 GPU Programming Models	17
2.3.1 CUDA Execution Model	17
2.3.2 CUDA Memory Model	19
2.3.3 CUDA Graphs	20
2.3.4 Other CUDA Advanced Features	22
2.4 Message Passing Interface	23
2.4.1 Point-to-Point Communication	24
2.4.2 One-Sided Communication	25
2.4.3 Collective Communication	25
2.4.4 Hybrid MPI Programming Models	29
2.4.5 Message Passing Interface Implementations	31
2.5 Unified Communication X (UCX) Framework	32
2.5.1 Unified Communication Transport (UCT)	33
2.6 NVIDIA Collective Communications Library	34
2.7 Performance Measurement and Benchmarks	34
2.7.1 Performance Metrics	35
Chapter 3: Multipath Point to Point Communication	37
3.1 Introduction	37
3.2 Background and Motivation	39
3.2.1 GPU-enabled Systems, MPI, and UCX	39

3.2.2	CUDA, UCX-CUDA Modules, and Direct Communication	40
3.2.3	CUDA Graphs and Advanced Runtime Integration	42
3.2.4	Motivation	43
3.3	Related Work	44
3.3.1	Multi-Path Communication	44
3.3.2	CUDA Graphs	45
3.4	Design and Implementation	46
3.4.1	Design Objectives	47
3.4.2	Framework Architecture	48
3.4.3	2-D Pipelining Engine and CUDA Graph Execution	49
3.4.4	Runtime Tuning and Configuration	53
3.4.5	Ensuring Data Integrity and Correctness	54
3.5	Evaluation	55
3.5.1	Experimental Setup	56
3.5.2	UCX Put Bandwidth Evaluation	57
3.5.3	MPI Micro-benchmark Results	59
3.5.4	Jacobi Iterative Solver	63
3.5.5	CUDA Graph Overhead Analysis	67
3.6	Summary	72

Chapter 4: Accelerating Intra-Node GPU Communication: A Performance

Model for Multi-Path Transfers	74
4.1 Introduction	74
4.2 Background and Related Work	77
4.2.1 GPU-aware MPI Communication	77
4.2.2 Communication Performance Modeling	78
4.2.3 GPU Communication Modeling and Optimization	79

4.3	Performance Model for Multi-path Intra-node GPU Communication	80
4.3.1	Extending the Hockney’s Model for Multi-Path Communication . .	82
4.3.2	Finding The Optimal Path Configuration	83
4.3.3	Incorporating Paths with Staged Transfers	88
4.3.4	Incorporating Pipelining for Staged Transfers	90
4.4	Design and Implementation of the Proposed Communication Model	94
4.5	Performance Evaluation and Analysis	97
4.5.1	Experimental Setup	98
4.5.2	Unidirectional and Bidirectional Bandwidth Evaluation	99
4.5.3	Collective Communication Evaluation	107
4.6	Summary	110

Chapter 5: Collaborative Bandwidth-Efficient Intra-Node Allreduce 112

5.1	Introduction	112
5.2	Background and Related Work	115
5.2.1	Collective Communication Algorithms and GPU-awareness	115
5.2.2	Heterogeneous Computing in HPC	116
5.2.3	Performance Modeling of Collective Communications	118
5.3	Design and Implementation	119
5.3.1	Framework Architecture	120
5.3.2	Multi-path Heterogeneous MPI_Allreduce	124
5.4	Performance Model for Heterogeneous Multi-Path Allreduce	128
5.4.1	Modeling Communication and Computation	129
5.4.2	Overall Time and Optimization	131
5.4.3	Analytical solution for the optimal split	132
5.4.4	Incorporating Pipelining	132
5.4.5	Developing the Model in Our Framework	138

5.5	Evaluation	140
5.5.1	Experimental Setup	141
5.5.2	Performance Analysis	142
5.5.3	Performance Model Evaluation	143
5.6	Summary	144
Chapter 6: Legendre Pairs Algorithm Optimizations		146
6.1	Introduction	146
6.2	Background and Related Work	150
6.2.1	Legendre Pairs: Definition and Properties	150
6.2.2	Parallel Processing in Legendre Pairs Search Problem	153
6.2.3	Parallel Programming Models, OpenMP, MPI, and CUDA	154
6.3	CPU-Based Approaches	156
6.3.1	Experimental Framework	156
6.3.2	Baseline: Single-Core Implementation	156
6.3.3	Utilizing OpenMP	162
6.3.4	Utilizing MPI	165
6.4	GPU Acceleration - Design and Evaluation	170
6.4.1	Baseline GPU Implementation	171
6.5	GPU-based Optimizations	172
6.5.1	Removing Branch Divergence	173
6.5.2	Utilizing <i>Constant Memory</i>	174
6.5.3	Utilizing <i>Shared Memory</i>	176
6.5.4	Loop Unrolling, Loop Fission, and Loop Reordering	177
6.5.5	Optimizing <i>Shared Memory</i> Usage	177
6.5.6	Tuning Kernel Parameters	178
6.5.7	Decreasing Memory Stalls by Increasing Work Per Thread	178
6.5.8	Minimizing <i>Shared Memory</i> Bank Conflicts	179

6.5.9	Optimizing Register Usage	179
6.5.10	Removing Ternary Operations	181
6.5.11	CUDA Graphs Utilization	181
6.5.12	Profiling and Performance Analysis	184
6.5.13	Multi-GPU Implementation	187
6.5.14	Improving Resource Utilization by harnessing MIG and MPS	188
6.5.15	Scalability on Modern GPUs	192
6.6	Summary	193
Chapter 7: Conclusions and Future Directions		195
7.1	Conclusion	195
7.2	Future Directions	197
7.2.1	Multi-Path Point-to-Point Communication	197
7.2.2	Performance Modeling for Multi-Path Communication	199
7.2.3	Heterogeneous Collective Communication	200
7.2.4	Optimization Strategies for Combinatorial Problems	200
References		202
Appendices		222
Appendix A: Configuring Multi-Instance GPU (MIG) on NVIDIA GPUs		223
A.1	MIG Configuration	223
A.2	MIG on a Multi-GPU Node	225
A.2.1	Compute Instances (CI)	226
A.2.2	Running CUDA Applications on MIG	227
A.3	Enabling MPS on MIG	228

List of Figures

2.1	An abstract view of the High-Performance Computing ecosystem. Higher layers may bypass the lower layers to access hardware/software resources directly.	9
2.2	Top500 supercomputers, accelerators and interconnect family share in June 2025 [145]	10
2.3	Traditional GPU-to-GPU communication vs. GPUDirect RDMA.	14
2.4	Simple graph with 2 copies and 3 kernels	21
2.5	RSA Allreduce: Reduce-Scatter followed by All-Gather in a Ring topology with 4 processes	27
2.6	Reduce-Broadcast Allreduce for 8 processes	28
2.7	Recursive Doubling Allreduce for 8 processes	28
2.8	MPI Alltoall Bruck algorithm for 4 processes. In this example, the <i>send</i> and <i>receive</i> buffers are used as temporary storage for data rotation steps to avoid data corruption without additional allocations.	30
2.9	Open MPI architecture [103]	31
2.10	OpenMPI Intra-Node Allreduce Tuning Table	32
2.11	UCX architecture and some of its components	33
3.1	(a) A typical four-GPU node with NVLink (two sub-links) per GPU pair (Similar to Beluga nodes), (b) A communication between GPU-0 and GPU-1 is split and transferred through multiple paths	42
3.2	Integration of our multi-path communication framework with the UCX library. The internal details of the UCX library are not shown for simplicity. CUDA Graph related modules are in dark green.	48

3.3	A simplified view of 2-D pipelined communication from GPU-0 to GPU-1 using the available NVLink and PCIe. Staging GPUs' timelines are not shown for simplicity.	49
3.4	A CUDA Graph-based multi-path communication from GPU-0 to GPU-1 using the available NVLinks and <i>Peripheral Component Interconnect Express (PCIe)</i>	51
3.5	Narval node architecture.	56
3.6	UCX Unidirectional Put Bandwidth comparison of our multi-path framework with and without CUDA Graph against default UCX (UCT::CUDA-IPC), on Beluga (top) and Narval (bottom)	58
3.7	Multi-Path OMB Unidirectional MPI Bandwidth (BW) comparison with and without CUDA Graph on Beluga	59
3.8	Multi-Path OMB Unidirectional MPI Bandwidth (BW) comparison with and without CUDA Graph on Narval	60
3.9	Multi-Path OMB Bidirectional MPI Bandwidth (BIBW) comparison with and without CUDA Graph on Beluga	61
3.10	Multi-Path OMB Bidirectional MPI Bandwidth (BIBW) comparison with and without CUDA Graph on Narval	62
3.11	Jacobi communication pattern (a) without multi-path, and (b) with multi-path (two paths per communication) on a four-GPU node	64
3.12	Jacobi runtime speedup over default UCX (UCT::CUDA-IPC) using four MPI ranks on Beluga and Narval clusters	65
3.13	Measurement of various CUDA Graph operations during the first iteration of OMB Latency benchmark on Narval for dual-path communication from message sizes 2MB to 512MB.	69
3.14	Overhead of the subsequent CUDA Graph launches for various node counts during OMB Latency benchmark on Narval for dual-path communication from message sizes 2MB to 512MB.	70
3.15	Contribution of each graph phase to the total latency (first iteration) during OMB Latency benchmark on Narval for dual-path communication from message sizes 2MB to 512MB.	70

3.16	Contribution of each graph phase to the total latency (subsequent iterations) during OMB Latency benchmark on Narval for dual-path communication from message sizes 2MB to 512MB.	71
3.17	Comparison of subsequent CUDA Graph phases with the CUDA Graph enabled multi-path transfer during OMB Latency benchmark on Narval for dual-path communication from message sizes 2MB to 512MB.	71
4.1	How a single transfer from GPU-0 to GPU-1 is split into four parts and assigned to different paths, some of which may send data with multiple chunks. Here, the startup and synchronization overheads are exaggerated for clarity.	90
4.2	The high-level design and implementation of the communication model in the UCX library.	95
4.3	θ distribution when utilizing different number of paths for OMB MPI unidirectional bandwidth tests on Beluga.	100
4.4	Unidirectional MPI bandwidth tests on Beluga. Comparing the predicted and two measured configurations (dynamic- and static-tuned) with the baseline (Direct Path) using various number of paths and <i>window</i> sizes. . .	101
4.5	Unidirectional MPI bandwidth tests on Narval. Comparing the predicted and two measured configurations (dynamic- and static-tuned) with the baseline (Direct Path) using various number of paths and <i>window</i> sizes. . .	102
4.6	Unidirectional MPI bandwidth tests on Daisy with host staging. Comparing predicted and measured bandwidths (dynamic- and static- tuned) for different <i>window</i> sizes.	103
4.7	Bidirectional MPI bandwidth tests on Beluga. Comparing the predicted and two measured configurations (dynamic- and static-tuned) with the baseline (Direct Path) using various number of paths and <i>window</i> sizes. . .	104
4.8	Bidirectional MPI bandwidth tests on Narval. Comparing the predicted and two measured configurations (dynamic- and static-tuned) with the baseline (Direct Path) using various number of paths and <i>window</i> sizes. . .	105
4.9	Bidirectional MPI bandwidth tests on Daisy with host staging. Comparing predicted and measured bandwidths (dynamic- and static- tuned) for different <i>window</i> sizes.	106

4.10	Latency speedup of MPI Alltoall tests on Beluga and Narval against MPI+UCC+UCX configuration.	108
4.11	Latency speedup of MPI Allreduce tests on Beluga and Narval against MPI+UCC+UCX configuration.	109
5.1	A typical target topologies for multi-path <i>Allreduce</i> framework that utilizes NVLink (any number of pairs) and PCIe paths between two GPUs of the system.	121
5.2	Multi-path Heterogeneous <i>Pairwise Exchange Allreduce</i> on two GPUs using NVLink and PCIe.	123
5.3	A summarized NVIDIA Nsight Systems timeline profile of the multi-path heterogeneous <i>Allreduce</i> (128 MB) on one of the GPU on Daisy cluster. Grey and blue boxes are reduction operations on the host and the device, respectively. Brown boxes are data transfers on NVLink, purple boxes are D2H transfers, and green boxes are H2D transfers on PCIe.	128
5.4	Different cases with and without pipelined communication/computation on device and host paths. Note that the startup latency (α), synchronization cost (ϵ), and kernel launch overhead terms (κ) are different for each path, but for simplicity, we have used the same notation in the figure.	134
5.5	Illustration of how pipelining changes the load-balance condition. The dashed lines represent the single-chunk per-path times, intersecting at θ'_d . With pipelining, the effective slopes and offsets of each path change, shifting the balance point to θ_d^*	136
5.6	Speedup of various Allreduce algorithms (including MPI_UCC_UCP, MPI_UCC_NCCL) against standalone NCCL on Daisy, Narval, and Mist nodes, using two GPUs.	143
5.7	Comparison of predicted and measured execution times for various Allreduce algorithms on Daisy nodes using two GPUs. The results demonstrate the accuracy of our performance model, with an average error of 8%	144
6.1	General Legendre Pairs (LPs) Algorithm Design and Generation Process: generating LPs through the iterative construction of sequences A and B and finding the common members.	157

6.2	Speedup of the OpenMP implementation of LP-45 algorithm on a single node of Daisy cluster comparing with the single-core execution.	164
6.3	Efficiency of the OpenMP implementation of LP-45 algorithm, executed on a single node of Daisy cluster.	164
6.4	Speedup of the OpenMP implementation of LP-117 algorithm on a single node of Daisy cluster comparing with the single-core execution.	164
6.5	Parallel efficiency of the OpenMP implementation of LP-117 algorithm, executed on a single node of Daisy cluster.	165
6.6	Speedup of the MPI implementation of LP-117 algorithm, executed on a single node of Daisy cluster, comparing with the single-core execution. . .	168
6.7	Parallel efficiency of the MPI implementation of LP-117 algorithm, executed on a single node of Daisy cluster.	168
6.8	Speedup of the MPI implementation of LP-117 algorithm, executed on four nodes of Daisy cluster, comparing with the single-core execution.	169
6.9	Parallel efficiency of the MPI implementation of LP-117 algorithm, executed on four nodes of Daisy cluster, comparing with the single-core execution. .	169
6.10	Communication and computation time for the MPI implementation of LP-45 and LP-117 algorithms, executed on a single node and four nodes of Daisy cluster.	170
6.11	Utilizing CUDA Graphs to encapsulate the entire sequence of kernel launches and memory operations (four passes of pipelined execution) in the LP-117 algorithm (Sequences A).	184
6.12	Occupancy of the LP-117 CUDA kernel on the A30 GPU with respect to <i>shared memory</i> and <i>register</i> utilization. The current implementation is marked with a red dot.	187
6.13	Weak scaling of the LP-117 multi-GPU implementation on a single node with up to 16 NVIDIA A30 GPUs. Each rank has a constant search space to sweep.	188
6.14	Application throughput per GPU (iterations/sec) for varying number of MPI processes running LP-117. While the <i>Ideal</i> case maps one rank to one GPU, all other configurations oversubscribe a single physical device. Higher values indicate better resource utilization and cost-efficiency	190

6.15	Speedup of multi-GPU implementation on a single NVIDIA A30 GPU relative to the <i>Multi-rank per GPU</i> baseline, using different configurations of MPI processes, MIG instances, and MPS. Each rank has a constant search space to sweep.	191
------	---	-----

List of Tables

2.1	CUDA Memory Types	19
2.2	CUDA Concurrency Mechanisms Comparison	23
4.1	Notations used in the model.	81
5.1	A summary of implemented <i>Allreduce</i> algorithms and their variants in the user-level. While the main versions rely on device DMA engines for memcpy operations with no overlap, the pipelined versions use chunking to overlap communication steps, and the kernel-only versions rely on SM-driven copies with fully integrated fine-grained overlap.	122
5.2	Extended notation including computation parameters.	130
5.3	Experimental Setup	141
6.1	Comparison of LP-45 and LP-117 algorithms.	161
6.2	Some of the relevant specifications of the NVIDIA A30 GPU	173
6.3	Performance of the LP-117 CUDA algorithms tested on a single NVIDIA A30 GPU, compared with the baseline CPU implementation on a single core of Intel Xeon Gold 6338 CPU.	185
6.4	Comparison of key specifications between NVIDIA H100 and A30 GPUs. .	193

List of Algorithms

3.1	CUDA Graph Creation and Launch for Multi-Path UCX Transfers	52
3.2	CUDA Graph Nodes Creation and Configuration	53
4.1	Compute topology constants to approximate number of chunks per path. . .	96
4.2	Optimal Path Configuration $O(paths_count)$	97
5.1	Multi-path Pipelined Pair Exchange Allreduce	125
5.2	Device-side and host-side exchanges in Pipelined Pair Exchange	127
5.3	Finding the Optimal Pipelined Dual-Path Split (Method 1).	140
6.1	Legendre Pairs (length 45): Sweeping the search space and looking for valid sequences A	158
6.2	Legendre Pairs (length 45): Sequence A construction.	159
6.3	Legendre Pairs (length 45): Compute DFT and filter based on PSD thresholds.	160
6.4	Legendre Pairs (length 45) - Work distribution on CPU cores using OpenMP	163
6.5	Legendre Pairs (length 45) with MPI: Sequence generation and workload distribution	166
6.6	LP Algorithm with MPI, the listener thread from the leader rank: receiving and flushing results from worker processes in a background thread.	167
6.7	LP-45, CUDA-v1: Naive GPU Kernel Implementation of finding and storing A sequences	172

6.8	LP-45, CUDA-v2: Recursively generate the masks for GPU threads with <i>bits</i> -many 0s in a 9-bit field	175
6.9	LP-45, CUDA-v2: Sequence <i>A</i> construction.	176
6.10	Finding Optimal Shared Memory Padding to Minimize Bank Conflicts . . .	180
6.11	Utilizing CUDA Graphs to encapsulate the entire sequence of kernel launches and memory operations in the LP algorithm (Sequences <i>A</i>).	183
6.12	Predicting GPU Occupancy based on device specifications, as well as Shared Memory and Register Usage	186

List of Code Listings

2.1	A simplified pseudocode of creating, and launching a CUDA Graph	21
2.2	Capturing a CUDA Graph over 2 streams	21
6.1	A Maple snippet to show the 9-compression formula	153
A.1	Enabling MIG on a GPU	224
A.2	Querying available MIG profiles	224
A.3	Placement information of MIG profiles	224
A.4	Creating GPU instances with different specifications	224
A.5	Destroying MIG instances	225
A.6	Listing GPUs before enabling MIG	225
A.7	Creating and listing MIG instances	225
A.8	Creating Compute Instances within MIG	226
A.9	Executing CUDA applications on MIG devices	227
A.10	Enabling MPS on MIG instances	228
A.11	Running CUDA applications on MIG instances with MPS	229
A.12	Disabling MPS on MIG instances	229

Glossary

AI Artificial Intelligence 2, 13, 16	DMA Direct Memory Access 13, 20, 117, 121
ANL Argonne National Laboratory 31	DSL Domain-Specific Language 45
API Application Programming Interface 2, 14–17, 19, 20, 32, 43, 49, 67, 116, 123, 154	FLOPS Floating-point Operations Per Second 9
BTL Byte Transfer Layer 32	GDR GPUDirect RDMA 13
C2C Chip-to-Chip 120	GI GPU Instance 190, 225
CAF Co-Array Fortran 15	GPU Graphics Processing Unit 1–7, 11–14, 17–20, 22, 25, 29, 31, 33, 34, 36–50, 52, 54–60, 63, 64, 66, 68, 72–82, 88–90, 94, 95, 98, 99, 106–108, 110–120, 122–124, 126, 128, 129, 131, 141–144, 148–150, 154–156, 170–174, 176–179, 181, 182, 184, 185, 187–201, 223–227
CI Compute Instance 190, 225, 226	H2D Host-to-Device 19, 124, 126, 128, 139, 183
CTS Clear To Send 24	HBM High Bandwidth Memory 11
CUDA Compute Unified Device Architecture 2, 4, 6, 15, 17–22, 29, 34, 38–46, 48–52, 54–58, 61–64, 66–68, 70–73, 77, 79, 94, 98, 108, 123–128, 139, 140, 148, 149, 155, 156, 173, 181–183, 188, 192, 194, 196, 197, 201, 226	HIP Heterogeneous-compute Interface for Portability 15, 17
D2H Device-to-Host 19, 124, 126, 127, 139, 183	HPC High-Performance Computing 1–9, 11, 12, 14–16, 23, 34–37, 39, 40, 43–46, 74, 75, 77, 112, 113, 115–118, 145, 146, 148–151,
DAG Directed Acyclic Graph 20	
DFT Discrete Fourier Transform 152, 157, 158, 160	
DL Deep Learning 2, 3, 8, 11, 12, 34, 57, 145	

153–155, 193, 195, 197, 198

HPL High-Performance Linpack 9, 34

IB InfiniBand 3, 10, 11, 13, 16, 34

IPC Inter-Process Communication 14, 40, 41, 48, 57, 77, 94

LLM Large Language Model 2, 57, 112

LP Legendre Pair 3, 5–7, 146–154, 156–165, 167–174, 176–179, 182–185, 190–193, 196, 200, 201

MIG Multi-Instance GPU 5, 22, 149, 189–192, 194, 223–225, 227–229

ML Machine Learning 16, 198

MMU Memory Management Unit 14

MoE Mixture of Experts 198

MPI Message Passing Interface 2, 4, 6, 16, 22–26, 29, 31–34, 37–40, 55–57, 59, 60, 63, 72–74, 76, 77, 98, 99, 113–117, 123, 140–142, 149, 150, 154–156, 165, 167, 187–190, 193, 194, 196, 197

MPP Massively Parallel Processors 9

MPS Multi-Process Service 5, 22, 149, 189–192, 194, 223, 228, 229

NCCL NVIDIA Collective Communications Library 3, 5, 34, 45, 74, 113–116, 121, 140–142, 144, 155, 196

NIC Network Interface Card 11, 13, 199

NPB NAS Parallel Benchmark suit 34

NUMA Non-Uniform Memory Access 57, 98, 103, 106, 141–143, 163, 169, 193

NVSHMEM NVIDIA OpenSHMEM Library 3

OMB OSU Micro-Benchmarks 34, 38, 55, 56, 59, 67, 72, 75, 98, 99

OMPI Open MPI 32

OPAL Open Portable Access Layer 31

OpenACC Open Accelerators 16

OpenCL Open Computing Language 17

OpenMP Open Multi-Processing 6, 15, 16, 29, 124, 126, 131, 139, 149, 150, 154, 156, 162, 165, 167, 193, 196

OpenSHMEM Open Symmetric Hierarchical Memory 15

OS Operating System 11, 24, 31, 35

P2P Point-to-Point 2, 4, 5, 13, 23, 24, 26, 32, 33, 37, 38, 41, 44, 46, 55, 72, 76, 80, 81, 98, 107, 108, 111, 139, 195, 196

PAF Periodic Autocorrelation Function 151–153

PCIe Peripheral Component Interconnect Express xvii, 4, 13, 37–39, 44, 51, 58, 63, 72, 75–77, 79, 80, 110, 113, 115–117, 119, 120, 122, 127, 142, 143, 195, 200

PGAS Partitioned Global Address Space 15

PRQ Posted Receive Queue 25

PRRTE Process Management Interface (PMIx) Reference RunTime Environment 31

PSD Power Spectral Density 151–153, 157, 158, 160, 161, 181

Pthreads POSIX Threads 15, 29

PTX Parallel Thread Execution 156

QE Queue Element 25

QoS Quality of Service 189

RDMA Remote Direct Memory Access 24, 33, 34, 40

RMA Remote Memory Access 25

RoCE RDMA over Converged Ethernet 11, 34

ROCm Radeon Open Compute Platform 2, 17

RSA Reduce Scatter Allgather 26, 115, 121, 122

RTS Request To Send 24

SASS Streaming Assembler 156

SHOC Scalable Heterogeneous Computing benchmark suite 34

SIMD Single Instruction Multiple Data 9, 16, 18, 156, 174

SIMT Single Instruction Multiple Thread 11

SM Streaming Multiprocessor 12, 13, 18, 22, 36, 121, 177, 178, 185, 188, 189, 192

SMP Symmetric Multi-Processor 9

SYCL Single-source C++ for OpenCL 15

TBB Threading Building Blocks 16

TGCS Task Graph Computing System 45

UCC Unified Communication Collectives 3, 45, 77, 98, 107, 113–116, 140, 142, 200

UCP Unified Communication Protocols 33, 107

UCS Unified Communication Services 33

UCT Unified Communication Transports 33, 40, 57, 64

UCX Unified Communication X 3–6, 32, 33, 37, 38, 40, 41, 44, 46, 47, 51, 55, 57, 60, 63, 64, 66, 71–77, 94, 98, 108, 114–117, 140, 195, 196

UMQ Unexpected Message Queue 25

UPC Unified Parallel C 15

UPI Ultra Path Interconnect 106

UVA Unified Virtual Addressing 19

Chapter 1

Introduction

The advent of supercomputers is considered the herald of a new era in which achieving ultra-scale computational power is no longer a dream. In this era, supercomputers have achieved unprecedented processing power by harnessing *High-Performance Computing* (HPC) platforms and techniques, mainly based on parallel and distributed processing. Processing techniques fundamentally consist of communication and computation phases, which are inter-dependently woven into the very fabric of HPC.

Over the past decades, the processing power of computational infrastructures has been drastically improved mainly due to introducing modern multi-core and many-core processors, and equipping computer clusters with accelerators like *Graphics Processing Units* (GPU) [50, 128, 145]. These computational enhancements have increased theoretical computation power, but achieving the expected performance requires efficient communication and computation mechanisms as well as performant communication runtime and system software. However, despite extensive research, communication and synchronization are still the main bottlenecks of HPC systems, and the gap between computation and communication performance is widening [122, 148].

1.1 Motivation

HPC software systems provide a set of functionalities as *Application Programming Interface* (API) to the user-level applications to let them utilize the clusters' computational resources. These functionalities need to be efficient to comply with performance requirements. However, the drastic increase of cores per node, the introduction of modern accelerators and interconnects, and emerging computation-intensive HPC applications, distributed *Artificial Intelligence* (AI) and *Deep Learning* (DL) frameworks, and *Large Language Models* (LLM) have significantly stressed the runtime libraries [1, 63, 78].

Message Passing Interface (MPI), as one of the most common runtime libraries in HPC systems, provides a set of APIs to application developers to design and implement their applications in a distributed environment [80]. Since the MPI programming model is specifically evolved to perform efficiently in distributed environments, application developers tend to combine other programming models with MPI, known as MPI+X programming models, to help them further utilize hardware resources, such as *Compute Unified Device Architecture* (CUDA) [92] or *Radeon Open Compute Platform* (ROCm) [7]. As will be discussed later, tuning such applications that utilize hybrid programming paradigms, like MPI+CUDA, is highly complex and requires extra care and attention by the application developers, and usually, the solutions are neither hardware-aware nor portable to other clusters. Consequently, these issues have given birth to the hybrid MPI runtimes, which natively support hybrid programming models with less required effort from application developers.

One of the primary challenges in this landscape is the optimization of intra-node GPU communication. Traditionally, *Point-to-Point* (P2P) communication between GPUs has relied on single-path transfers, which often become a critical bottleneck. While some multi-path communication strategies exist in different contexts, such as inter-node communication

over *InfiniBand* (IB) and Ethernet (by channel bonding), they are often unable to provide truly concurrent data staging across both host and device channels and lack a robust analytical model for multi-path transfers. This necessitates an exhaustive search to find the optimal configuration for each unique topology, which is expensive and impractical. Furthermore, this communication gap extends to collective operations, such as *Allreduce* and *Alltoall*, which are essential to many HPC and DL applications. Existing state-of-the-art communication libraries, including *NVIDIA Collective Communications Library* (NCCL) [99], *NVIDIA OpenSHMEM Library* (NVSHMEM) [102], *Unified Communication Collectives* (UCC) [149], and *Unified Communication X* (UCX) [150] do not fully exploit the potential of heterogeneous computing and multi-path communication to accelerate these operations, leaving significant performance under-utilized.

Beyond the challenges in the mentioned communication libraries, many specific scientific and mathematical problems pose unique computational hurdles. For example, in the study of certain mathematical objects, the *Legendre Pair* (LP) problem, a mathematically sound solution has been found, but its implementation is computationally impractical [41]. This presents a critical opportunity to apply modern HPC strategies, including parallelization on both CPU and GPU architectures, to transform an impractical problem into a high-throughput one, thereby enabling new avenues for mathematical discovery. The core motivation of this dissertation is to address these multifaceted computational and communication challenges by developing and evaluating novel HPC methodologies and runtime frameworks that bridge the performance gap between hardware capabilities and application requirements.

1.2 Problem Statement

The primary and most significant issue that this dissertation seeks to elaborate and analyze in depth is the increasing gap between the theoretical performance capabilities of modern HPC systems and the actual, practical performance levels that are realized by various applications, particularly those that depend heavily on effective communication protocols and the utilization of hybrid programming paradigms which combine different computational models and techniques. This complex challenge presents itself in several distinct research domains, leading to the following questions that will guide our study.

- How can a multi-path communication framework be designed and implemented within an existing runtime library like UCX to enhance intra-node GPU-to-GPU P2P transfers? How can multiple heterogeneous communication paths (NVLink and PCIe) be exploited concurrently to improve such transfers? Can CUDA Graph be effectively used to encapsulate and reuse multi-path communication workflows, thereby minimizing launch overhead and improving overall communication efficiency? What are the performance implications of this approach across different hardware topologies and configurations?
- What is a suitable analytical performance model for multi-path intra-node GPU communication that can accurately predict transfer times across heterogeneous channels under pipelined, staged data transfers? Can this predictive model be integrated into the MPI+UCX communication stack to enable automatic runtime tuning and select the optimal communication strategy without relying on exhaustive search or a prior knowledge of the topology? How accurate can be this model in predicting optimal configurations across various message sizes and hardware setups? Can we design a predicting model that can be used dynamically at runtime to select the best configuration for multi-path communication?

- How can a collaborative Allreduce collective communication framework be designed to leverage heterogeneous computing and multi-path communication in multi-GPU systems? What are the key strategies for optimizing both data transfer and host system computing power to achieve performance improvements? How does this approach perform across different hardware configurations and message sizes, and what are the trade-offs involved in terms of complexity and scalability? Can this type of communication be mathematically modeled and optimized effectively? What are the performance gains achievable through this method compared to state-of-the-art libraries like [NCCL](#)?
- What are the challenges and opportunities in [HPC](#) and scientific computing that can be addressed through advanced parallelization strategies? How can we apply parallelization techniques to complex problems, such as combinatorial optimization problems like the construction of [LPs](#)? What are the best practices for profiling, analyzing, and optimizing the performance of these parallel implementations on modern [HPC](#) architectures? What are the performance gains achievable through these methods, and how do they compare to traditional serial implementations? What optimization techniques (memory layout, kernel tuning, occupancy balancing) are most effective for this problem on modern [GPU](#) hardware? Can we leverage modern [GPU](#) features like *Multi-Instance GPU (MIG)* and *Multi-Process Service (MPS)* to improve resource utilization and throughput for this problem?

1.3 Contributions

This dissertation makes the following key contributions to the field of [HPC](#), particularly in the context of [GPU](#)-aware communication and computation:

1. **A Multi-path Communication Framework:** We propose a multi-path communication framework within the [UCX](#) library, designed to enhance [P2P](#) communication

between intra-node GPUs in Chapter 3. This framework includes a static 2-D pipelining engine and a dynamic CUDA Graph-based design that constructs and reuses communication workflows to minimize launch overhead. We also provide a comprehensive performance characterization of CUDA Graphs in this context, providing insights into their benefits and limitations.

2. **A Predictive Performance Model:** We develop a novel analytical performance model for multi-path intra-node GPU communication in Chapter 4. This model extends the classical *Hockney's* framework [48] to the multi-channel GPU setting and is integrated into the MPI +UCX stack for automatic and runtime tuning. This integration enables the selection of optimal communication strategies without exhaustive search or prior topology knowledge.
3. **A Heterogeneous Collective Framework:** In Chapter 5, we propose and implement a collaborative framework that leverages heterogeneous computing and GPU-aware multi-path communication to accelerate collective operations. We also provide the first known analytical performance model for heterogeneous Allreduce operations to guide optimizations and configurations.
4. **High-throughput LP Design and Implementation:** In Chapter 6, we develop and evaluate a series of HPC strategies for the construction of LPs, including comprehensive CPU parallelization (*Open Multi-Processing* (OpenMP) [105] and MPI), advanced GPU acceleration (CUDA), and a scalable hybrid MPI+CUDA solution. This work transforms a mathematically sound but computationally impractical solution into a high-throughput one, enabling new mathematical discoveries.

1.4 Dissertation Organization

The remainder of this dissertation is organized as follows. Chapter 2 presents the necessary background information. Chapter 3 presents our design and implementation of a multi-path communication framework for intra-node GPU transfers. Building on this work, Chapter 4 develops a predictive performance model for this multi-path communication and evaluates its effectiveness in runtime auto-tuning. Chapter 5 explores the application of these concepts to collective operations by proposing a collaborative, heterogeneous framework for *Allreduce*. Chapter 6 addresses the challenges of developing and accelerating the LP problem on diverse parallel architectures using HPC strategies. Finally, Chapter 7 summarizes the contributions of this dissertation and discusses avenues for future work.

Chapter 2

Background

The term **HPC** mainly refers to parallel computing techniques that utilize dedicated hardware such as high-performance processors, accelerators, and interconnection networks to tackle time-consuming problems with high performance. As abstractly depicted in Figure 2.1, **HPC** ecosystem includes computation-intensive **HPC** or distributed **DL** applications that utilize common runtime libraries and middleware to access high-performance processors and accelerators connected via low-latency and high-bandwidth interconnection networks. In more detail, **HPC** covers these three categories: (1) Architectures, such as interconnects and multi-core processors; (2) Software solutions, runtimes, programming models, libraries, and profilers; (3) Theories, abstract models, and performance analysis.

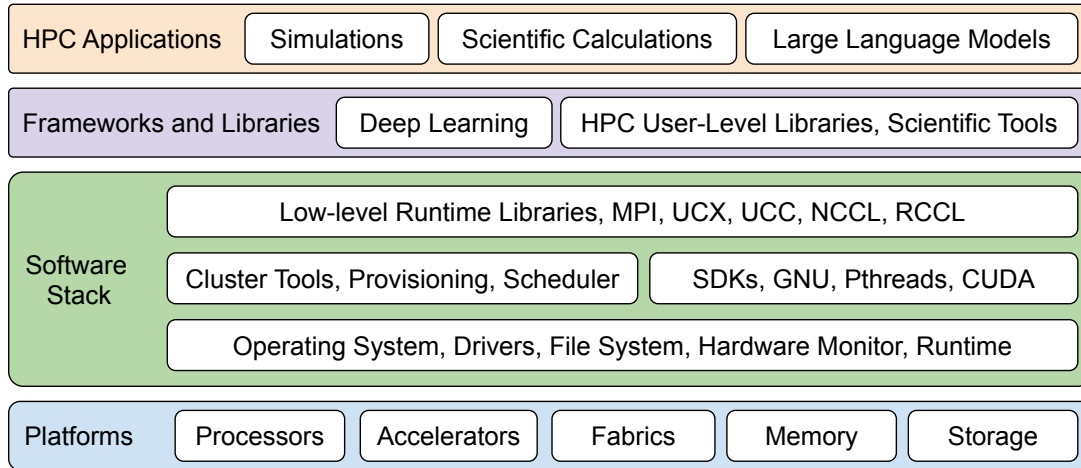


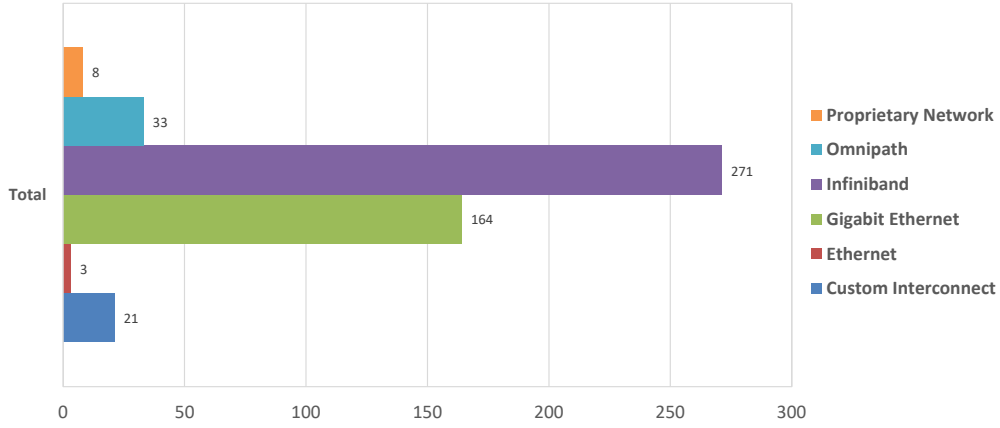
Figure 2.1: An abstract view of the High-Performance Computing ecosystem. Higher layers may bypass the lower layers to access hardware/software resources directly.

2.1 High-Performance Computing

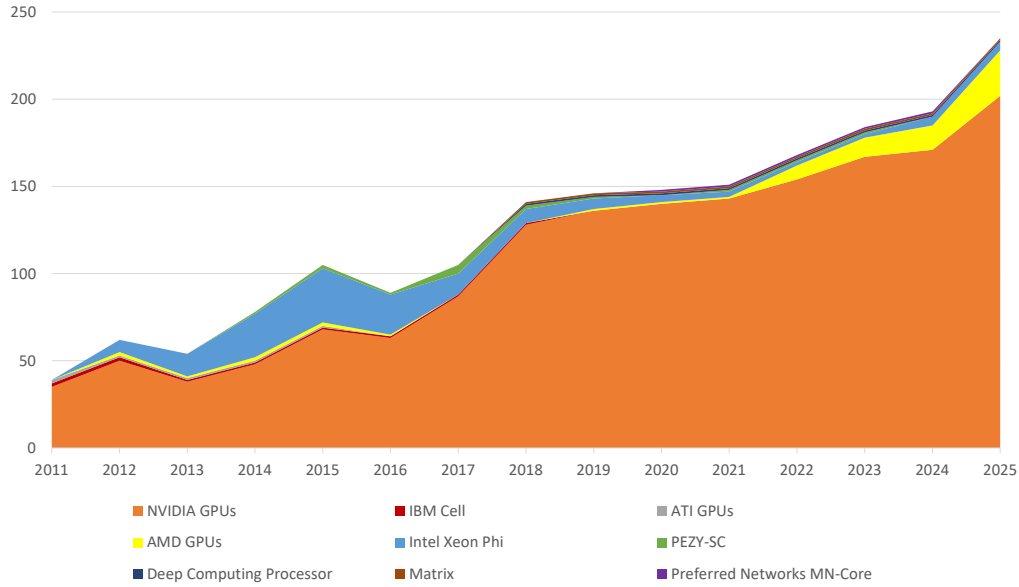
HPC architectures and supercomputers refer to computing platforms with a higher performance level than general-purpose systems. Supercomputer performance is commonly measured in *Floating-point Operations Per Second* (**FLOPS**), and the current first rank supercomputer in the world is El Capitan that hits 1.742 exa-**FLOPS** (10^{18} **FLOPS**) running *High-Performance Linpack* (**HPL**) benchmark [145]. At the time of writing this document, 86.2% of the top 500 supercomputers in the world are clusters. Previously, *Symmetric Multi-Processor* (**SMP**), *Massively Parallel Processors* (**MPP**), and *Single Instruction Multiple Data* (**SIMD**) architectures were popular, but the trend demonstrates how clusters have been dominating architecture share.

2.1.1 High-Performance Interconnects

Supercomputers utilize a large number of processing elements to perform high-performance computations, in particular, El Capitan is equipped with 11,039,616 cores. This massive number of cores requires an efficient and flexible interconnection network to provide



(a) Interconnect Family share in Top500 Supercomputers



(b) Number of Supercomputers utilizing Accelerators

Figure 2.2: Top500 supercomputers, accelerators and interconnect family share in June 2025 [145]

a suitable communication and utilization level. Modern clusters have utilized various approaches ranging from *fat trees* to *torus* and *Dragonfly* interconnects [145]. Figure 2.2a demonstrates the interconnect family share among the top 500 supercomputers, in which the top two interconnection networks are Gigabit Ethernet and IB.

InfiniBand: As one of the most commonly used interconnects in clusters, IB is a computer network communication standard that is characterized by its high bandwidth and low latency [54]. Currently, NVIDIA manufactures IB host bus adapters and network switches [100]. In traditional networks such as TCP/IP/Ethernet, *Operating System (OS)* owns and manages the network resources such as TCP/IP stack and *Network Interface Card (NIC)*, and applications communicate through OS to access network resources. However, IB provides messaging services that user-level applications can directly access network resources to drive communications. Another worth mentioning interconnect is *RDMA over Converged Ethernet (RoCE)* that is an IB over Ethernet protocol [113]. In this interconnect, IB messaging services are implemented on top of Ethernet link layer to provide high-performance communication over Ethernet networks.

2.1.2 GPU-enabled Clusters

Equipping HPC machines with GPUs has provided high-performance solutions to parallel applications. Although GPUs were originally designed to perform graphical operations, their *Single Instruction Multiple Thread (SIMT)* architecture and their *High Bandwidth Memory (HBM)* is remarkably fruitful for computation-intensive and data-parallel HPC and DL applications. These accelerators have been steadily enhanced over the past decade, and employing them is undeniably worthwhile for computer clusters. Figure 2.2b demonstrates how accelerator utilization in top 500 supercomputers have been steadily growing over the past years. At the time of writing this document, nine out of the top ten supercomputers worldwide utilize NVIDIA GPUs [145].

2.1.3 Modern GPU Features and Technologies

GPGUs contain several features and technologies that enhance their performance and usability in HPC clusters. In the following, we briefly describe some of the modern GPU

features and technologies that are relevant to this dissertation. While we focus on NVIDIA **GPUs** and their technologies, other vendors such as AMD and Intel also provide similar features in their **GPU** products.

Computational architecture of GPUs: **GPUs** are built upon an array of *Streaming Multiprocessors (SM)*, which are the fundamental processing units within the **GPU**. Each **SM** equipped with its own set of registers, shared memory, and cache hierarchies, consists of multiple cores capable of executing thousands of threads concurrently. Over successive generations, NVIDIA has introduced various enhancements to the **SM** architecture, such as increased core counts, improved memory bandwidth, advanced scheduling capabilities, *Tensor Cores* for matrix operations, and *dynamic parallelism* for more flexible execution models. Recent architectures, like Hopper and Blackwell, provide distributed shared memory and enhanced inter-**SM** communication.

Modern GPU Interconnects: NVIDIA has developed several high-performance interconnects for **GPUs** and CPUs, including NVLink, NVLink-SLI, NVSwitch, and the more recent NVLink-C2C. NVLink provides a bidirectional, link-based communication fabric, first introduced with Pascal **GPUs** and later expanded across generations. NVLink 2.0 (Volta **GPUs**) increased aggregate bandwidth to 300 GB/s per **GPU**, NVLink 3.0 (Ampere **GPUs**) to 600 GB/s, and NVLink 4.0 (Hopper **GPUs**) to 900 GB/s while also enabling heterogeneous CPU-**GPU** integration through NVLink-C2C to their Grace CPU. The latest NVLink 5.0 (Blackwell **GPUs**) doubles throughput further, reaching 1.8 TB/s per **GPU**. At system scale, NVSwitch extends NVLink to full all-to-all connectivity within nodes, and the NVLink Switch System (NVLS) enables rack-level fabrics such as NVL72, which interconnects 72 **GPUs** with uniform high-bandwidth, low-latency communication for large-scale **DL** and **HPC** workloads. Another noteworthy technology is NVLink-Fusion, which works by licensing the NVLink high-speed interconnect technology, including its port designs and

memory atomic protocols, to third-party chip makers so they can natively integrate it into their custom CPUs or accelerators. This allows non-NVIDIA chips to connect to NVIDIA GPUs and the rest of the NVIDIA AI infrastructure [101].

GPUDirect Technologies: Traditionally, in GPU-accelerated applications, the CPU was responsible for coordinating GPU computation and network communication, with messages between third-party devices staged through the system’s main memory. The introduction of *GPUDirect RDMA (GDR)* [97] removed this bottleneck by enabling intra-node or inter-node GPUs to directly exchange data without host-memory involvement, allowing third-party devices such as storage devices, NICs, and IB adapters to access GPU memory directly over PCIe [15, 27, 39]. Figure 2.3 illustrates the difference between traditional GPU-to-GPU communication and GDR-enabled transfers. Moreover, *GPUDirect_P2P* extends this capability to intra-node GPU-to-GPU transfers, bypassing the host memory altogether. While both GDR and *GPUDirect_P2P* reduce data-movement overheads, *GPUDirect_Async* further advances this model by offloading the communication control path onto the GPU itself [2, 3]. In this mode, the GPUs can autonomously initiate communication operations and handle synchronization events directly over the PCIe interconnect, effectively removing the CPU from the critical communication path and enabling greater overlap of computation and communication.

Data transfer methods: NVIDIA GPUs support several data transfer methods for moving data between the host and device memory, as well as between GPUs [93]. The most common methods are *cudaMemcpy*, and *cudaMemcpyAsync*, which allow for synchronous and asynchronous data transfers, using the *GPU Direct Memory Access (DMA)* engines. For asynchronous transfers, data should be *pinned* (page-locked) in the host memory to enable direct memory access by the GPU. Moreover, *SM copy* operations allow GPU threads to perform data transfers using SM resources, enabling fine-grained overlap of

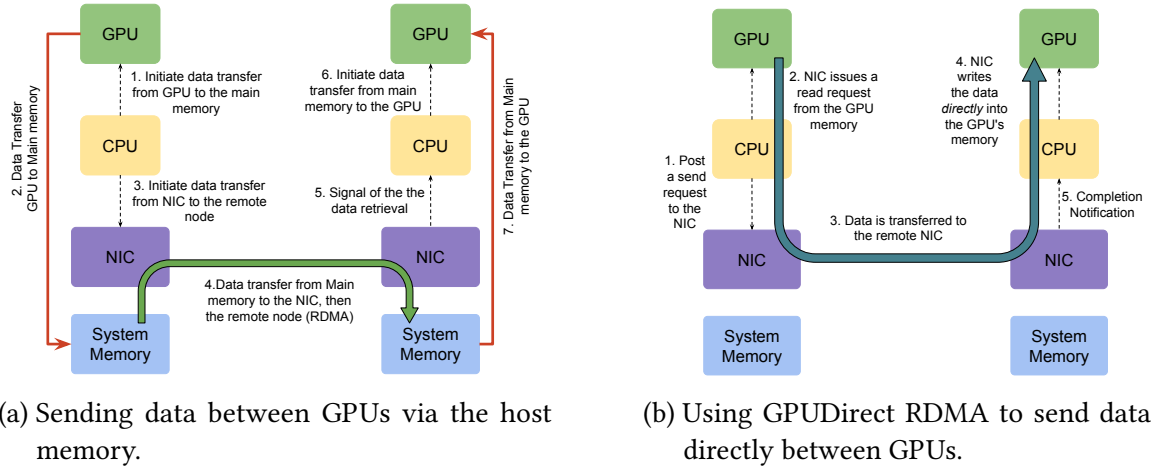


Figure 2.3: Traditional GPU-to-GPU communication vs. GPUDirect RDMA.

communication and computation. This method is particularly useful for small data transfers or when overlapping multiple communication steps within a kernel. One other method is *GDRCopy* library [96], which creates a user-space mapping of GPU memory in the host address space, allowing for zero-copy access to GPU memory from the host and third-party devices using *Memory Management Unit (MMU)*.

2.1.4 Parallel Programming Paradigms

As depicted in Figure 2.1, HPC runtime libraries provide a set of APIs, as one or a combination of parallel programming models, to the user-level applications. More specifically, parallel programming paradigms refer to the abstract types of computing approaches where several calculations are carried out by multiple processing elements simultaneously. In terms of *Inter-Process Communication (IPC)*, parallel programming models can be classified into *shared-memory* and *message-passing* categories, and in terms of problem decomposition, they are classified as *task parallelism* and *data parallelism* approaches. A task-parallel model focuses on processes or threads that execute distinctly, thus emphasizing the need for communication. On the other hand, the data-parallel model focuses on performing

equivalent operations on a data set, typically a regularly structured array.

Beside *shared-memory* and *message-passing* models, a notable programming model is *Partitioned Global Address Space* (PGAS), which provides a global address space that is logically partitioned, with a portion of it local to each process, allowing programmers to combine shared-memory style access with distributed-memory efficiency. Examples of PGAS programming models include *Unified Parallel C* (UPC) [67], *Co-Array Fortran* (CAF) [115], and *Open Symmetric Hierarchical MEMory* (OpenSHMEM) [106]. Recently, hybrid models that combine PGAS with *task-based runtime systems* have gained traction in modern HPC systems, enabling dynamic scheduling, heterogeneous hardware utilization, and improved load balancing. Additionally, with the rise of GPUs and other accelerators, programming frameworks like CUDA [92], *Heterogeneous-compute Interface for Portability* (HIP) [6], *Single-source C++ for OpenCL* (SYCL) [140], and Kokkos [64] extend traditional shared-memory and message-passing models to efficiently exploit fine-grained data parallelism on many-core devices.

2.2 Shared-Memory Programming Models

In a shared-memory programming model, processes may share a global address space where they can transfer data with each other concurrently. Two of the most widely used libraries among shared-memory models are *POSIX Threads* (Pthreads) and OpenMP, whereby the main thread of a process creates/forks a specified number of helping threads to perform a task cooperatively [105, 142].

Pthreads API is one of the earliest and most widely supported models for shared-memory parallel programming. It provides fine-grained control over thread creation, synchronization (mutexes, condition variables, barriers), and scheduling. Unlike OpenMP, which abstracts much of the complexity through compiler directives, Pthreads is a low-level

C library interface that requires explicit management of threads and synchronization, offering maximum flexibility but at the cost of higher programming effort.

[OpenMP](#) is a portable, scalable programming model that provides a simple and flexible interface for developing parallel applications on shared-memory architectures. The [API](#) supports multi-platform shared-memory multiprocessing programming in C/C++ and Fortran and is specified and maintained by the [OpenMP Architecture Review Board](#). [OpenMP](#) programs express parallelism using a combination of compiler directives, runtime library routines, and environment variables to define parallel regions, work-sharing constructs, tasking, synchronization, and data-sharing attributes. Modern [OpenMP](#) releases have extended the model well beyond loop parallelism: recent versions add tasking semantics, [SIMD](#) or vectorization directives, explicit device offloading (the *target* model) including memory management for accelerators, and various performance-portability features aimed at heterogeneous and [AI/Machine Learning \(ML\)](#) workloads. Other worth mentioning shared-memory programming models include Intel *Threading Building Blocks* ([TBB](#)) [[151](#)], Cilk [[79](#)], and *Open Accelerators* ([OpenACC](#)) [[143](#)].

There are some shared-memory mechanisms that are not to be confused with shared-memory programming models. For instance, XPMEM [[51](#)] is a Linux kernel module that enables zero-copy data sharing between processes on the same node by mapping memory segments into each other's address spaces. It is often used in conjunction with [IB](#) to facilitate high-performance intra-node communication in [HPC](#) applications. Another example is KMEM [[55](#)], a Linux kernel module that provides a mechanism for high-performance data transfers between processes on the same node, particularly useful for [MPI](#) implementations to optimize intra-node communication by allowing direct memory access between processes.

2.3 GPU Programming Models

GPU vendors provide developers with general-purpose models, APIs, and platforms, like CUDA [92], HIP [6], ROCm [7], and *Open Computing Language* (OpenCL) [104], to enable application acceleration. CUDA is a parallel computing platform and programming model developed by NVIDIA for general-purpose computing on NVIDIA GPUs. It provides a C/C++ based programming interface that allows developers to write programs that can execute on the GPU, leveraging its massive parallel processing capabilities.

2.3.1 CUDA Execution Model

Any CUDA application consists of two parts: the host code that runs on the CPU and the device code that runs on the GPU. The host code is responsible for managing memory, launching kernels, and coordinating data transfers between the host and device. The device code is executed in parallel by multiple threads on the GPU. All interactions between the host and device are managed through the CUDA runtime and driver APIs.

When the CPU thread(s) interact with the GPU, they do so through CUDA contexts. A CUDA context is an execution environment that encapsulates the state of the GPU, including memory allocations, kernel launches, and other resources like CUDA events and CUDA streams. Each process that uses the GPU has its own CUDA context, which is created when the process initializes the CUDA runtime. The context is associated with a specific GPU device and is used to manage all interactions between the host and device. A process can have multiple CUDA contexts for a single GPU device, but each context is related to a single device and process.

CUDA streams are a CUDA programming model feature in which an application submits different work to independent queues that can be processed independently by the GPU. Processes may utilize multiple CUDA streams, but the streams cannot be shared. CUDA

streams do not offer much isolation (i.e., the address space, the **SM**, the **GPU** memory bandwidth, caches, and capacity are all shared). **GPUs** have a feature called *Hyper-Q* to allow multiple **CUDA** streams to be executed concurrently on the **GPU**.

CUDA events are synchronization primitives that can be used to coordinate the execution of **GPU** operations. They can be used to measure the time taken by a specific operation or to synchronize the execution of different **CUDA** streams. **CUDA** events can be created and recorded in a **CUDA** stream, and the host can wait for an event to complete before proceeding with further operations.

The **CUDA** device-side programming model is based on the concept of *kernels*, which are functions that are executed on the **GPU** in parallel by multiple threads. Each thread has its own unique identifier, which can be used to determine which portion of the data it should operate on. The threads are organized into *blocks*, which are further organized into a *grid*. Recent NVIDIA **GPU** architectures introduced *Thread Clusters* to group multiple blocks together for improved scheduling and resource utilization. This hierarchical organization allows for efficient management of large numbers of threads and enables the **GPU** to handle complex data structures and algorithms.

GPU threads execute in a **SIMD** fashion, meaning that a group of threads (called a *warp* in NVIDIA terminology) execute the same instruction simultaneously on different data elements. In this model, the best practice is to minimize branch divergence within a warp, as divergent branches can lead to serialization of execution and reduced performance. The **CUDA** programming model also provides a rich set of libraries and tools for optimizing performance, including memory management, synchronization, and profiling tools.

Table 2.1: CUDA Memory Types

Memory Type	Scope	Lifetime	Access Speed
Global	All threads	Application lifetime	Slow
Shared	Threads in a block	Block lifetime	Fast
Constant	All threads	Application lifetime	Fast (cached)
Texture	All threads	Application lifetime	Fast (cached)
Local	Individual thread	Thread lifetime	Slow
Register	Individual thread	Thread lifetime	Very fast

2.3.2 CUDA Memory Model

As the GPU devices have independent memory spaces, data must be explicitly transferred between the host (CPU) and device (GPU) memory using CUDA API functions such as `cudaMemcpy()` to copy data between the host and device in both directions, *Device-to-Host* (D2H) and *Host-to-Device* (H2D). However, with the introduction of *Unified Virtual Addressing* (UVA) and *unified memory*, CUDA provides a virtual single memory space that abstracts away the explicit data management between the host and device. This feature simplifies programming by allowing the CUDA runtime to automatically manage data transfers between the host and device as needed, but it may not always provide optimal performance. Additionally, CUDA provides various memory types, such as *global*, *shared*, *constant*, and *texture* memory, each with different performance characteristics and access patterns. Table 2.1 summarizes the main types of CUDA memory.

Data transfers from the host to the device and vice versa can introduce significant overhead, especially for large data sets. To mitigate this, CUDA provides several techniques for optimizing memory transfers, such as *pinned memory*, which allows for faster data transfers by using page-locked memory on the host. In non-pinned memory, the operating system can move pages between the main memory and disk storage, which can introduce latency during data transfers. *Pinned memory*, on the other hand, is locked in the main memory,

allowing for direct access by the GPU's DMA engine, resulting in faster transfer times.

Both *Shared memory* and *L1 cache* are on-chip memory types that provide similar low-latency access to data. However, while L1 cache is automatically managed by the GPU hardware, *shared memory* is explicitly managed by the programmer and is shared among threads within a block. This allows for fine-grained control over data sharing and communication between threads, enabling efficient parallel algorithms.

Accesses to *shared memory* are managed through *banks* to allow multiple threads to access different memory locations simultaneously without conflicts. However, if multiple threads access the same memory bank simultaneously, a *bank conflict* occurs, leading to serialized access and reduced performance. To optimize performance, programmers must carefully design their memory access patterns to minimize bank conflicts.

2.3.3 CUDA Graphs

CUDA Graphs provide a more efficient model of task submission to GPUs as a *Directed Acyclic Graph* (DAG), which consist of a series of operations, such as memory copies and kernel launches [94]. These operations are defined as the graph nodes which are connected based on user-defined interdependencies. With this feature, the GPU workloads can be defined first and launched all at once to decrease the latency overhead imposed by synchronization between each step. CUDA Graphs can be created via two mechanisms: explicit API and stream capture. Figure 2.4 demonstrates a sample workflow graph, with two memory copies and three kernels, which is implemented using explicit CUDA Graphs API in Listing 2.1 and stream capturing in Listing 2.2. As demonstrated in Listing 2.1, the CUDA Graph object is created first (Line 1), and then nodes are added to the graph with their dependencies (Lines 2-6). After defining the graph, it is instantiated (Line 8) into an executable graph object that can be launched multiple times (Line 11). In the stream capture method

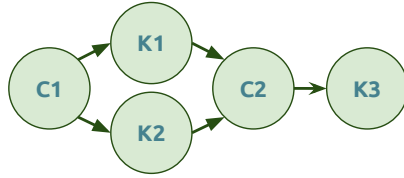


Figure 2.4: Simple graph with 2 copies and 3 kernels

```

0 // Define graph and dependencies
1 cudaGraphCreate(&graph);
2 cudaGraphAddNode(graph, C1, {});
3 cudaGraphAddNode(graph, K1, {C1});
4 cudaGraphAddNode(graph, K2, {C1});
5 cudaGraphAddNode(graph, C2, {K1,K2});
6 cudaGraphAddNode(graph, K3, {C2});
7 /* Instantiate graph and apply
   optimizations */
8 cudaGraphInstantiate(&inst, graph);
9 // Launch executable graph 100 times
10 for(int i = 0; i < 100; i++) {
11     cudaGraphLaunch(inst, stream);
12 }
13

```

Listing 2.1: A simplified pseudocode of creating, and launching a CUDA Graph

```

0 // Create the graph
1 cudaGraphCreate(&graph);
2 // stream1 is the main stream
3 cudaStreamBeginCapture(stream1);
4 cudaMemcpyAsync(..., stream1); // C1
5 // Fork into stream2
6 cudaEventRecord(event1, stream1);
7 cudaStreamWaitEvent(stream2, event1);
8 kernel_K1<<<..., stream1>>>(...);
9 kernel_K2<<<..., stream2>>>(...);
10 // Join stream2 back to stream1
11 cudaEventRecord(event2, stream2);
12 cudaStreamWaitEvent(stream1, event2);
13 cudaMemcpyAsync(..., stream1); // C2
14 kernel_K3<<<..., stream1>>>(...);
15 // End capture in the origin stream
16 cudaStreamEndCapture(stream1, &graph);
17 /* Instantiate and optimize graph */
18 cudaGraphInstantiate(&instance, graph);
19 // Launch executable graph 100 times
20 for(int i = 0; i < 100; i++) {
21     cudaGraphLaunch(instance, stream);
22 }
23

```

Listing 2.2: Capturing a CUDA Graph over 2 streams

(Listing 2.2), the graph is created by recording operations issued to one or more **CUDA** streams between `cudaStreamBeginCapture()` and `cudaStreamEndCapture()` (Line 3 and Line 16). When a **CUDA** stream is in capture mode, the operations issued to that stream are not executed, and instead, they are recorded into the graph. Then, the captured operations and their dependencies are automatically converted into a **CUDA** Graph when `cudaStreamEndCapture()` is called. After capturing, the graph is instantiated similarly to the explicit method (Line 18).

2.3.4 Other CUDA Advanced Features

Multi-Process Services: [MPS](#) allows cooperative multi-process applications to share computation resources on the [GPU](#), which is commonly used by [MPI](#) jobs. While [SMs](#) used by each [MPS](#) client can be optionally limited to a percentage of all [SMs](#), the scheduling hardware is still shared. Memory bandwidth, caches, and capacity are all shared between [MPS](#) clients [83].

Multi-Instance GPU: Since Ampere architecture, [MIG](#) allows [GPUs](#) to be securely partitioned into up to seven or four separate [GPU](#) instances for A100 and A30 models, respectively. This feature provides processes/users with separate [GPU](#) resources for optimal [GPU](#) utilization [98]. [MIG](#) is particularly beneficial for workloads that do not fully saturate the [GPU](#)'s compute capacity. Therefore, users may run different workloads in parallel to maximize [GPUs](#) utilization. [MIG](#) has been designed to be transparent to [CUDA](#) applications so that the [CUDA](#) programming model remains unchanged to minimize programming effort. Table 2.2 demonstrates how [CUDA](#) streams and [MPS](#) technologies compare to [MIG](#). Note that streams and [MPS](#) are parts of the [CUDA](#) programming model. Thus they can work together, potentially achieving even higher utilization levels for certain workloads. Section A.3 and Section A.1 provide more practical details on [MPS](#) and [MIG](#), respectively.

	Streams	MPS	MIG
Partition Type	Single Process	Logical	Physical
Max Partitions	Unlimited	48	7
SM Performance Isolation	No	Yes (by percentage)	Yes
Memory Protection	No	Yes	Yes
Memory Bandwidth QoS	No	No	Yes
Error Isolation	No	No	Yes
Cross-Partition Interop	Always	IPC	Limited IPC
Reconfigure	Dynamic	Process Launch	When Idle

Table 2.2: CUDA Concurrency Mechanisms Comparison

2.4 Message Passing Interface

MPI is a portable message-passing standard for parallel programming in **HPC** which is maintained by **MPI** Forum [80]. This standard defines the syntax, semantics, and specifications of the core routines of a library to enable the development of portable message-passing programs. The **MPI** interface provides virtual topologies in addition to synchronization and communication functionalities between a group of processes. These processes cooperate through a virtual communicator in which every process possesses a unique *id*. Communications can occur *inter-node* or *intra-node* through five main communication types: **P2P**, *one-sided*, *partitioned P2P*, *collectives*, and *Neighborhood collectives*.

In terms of timing and synchronization, **MPI** communication can be classified into *blocking* and *non-blocking* operations. In blocking operations, the process that initiates the communication is stalled until the operation is completed, ensuring that the data has been sent or received before proceeding. In contrast, non-blocking operations allow the initiating process to continue executing while the communication operation is still in progress. This enables overlapping of computation and communication, which can lead to improved performance in parallel applications. Non-blocking operations typically require

explicit synchronization (e.g., using `MPI_Wait` or `MPI_Test` functions) to ensure that the communication has completed before accessing the data.

Other notable features of `MPI` include support for *derived data types*, which allow users to define complex data structures for communication, as well as *virtual topologies* and *Neighborhood Collectives* that facilitate communication patterns in structured applications.

2.4.1 Point-to-Point Communication

In `P2P` communication, a message is transferred between two processes by a *send* and its corresponding *receive* operation. In further detail, the *source* process issues a *send* operation for which the *destination* process posts a matching *receive*. A *send* operation can be blocking/non-blocking, synchronous/asynchronous, and can utilize persistent, buffered, ready, or partitioned modes. On the other hand, A *receive* operation can be blocking/non-blocking and can utilize persistent or partitioned modes.

In `P2P` communication, `MPI` libraries may use either *eager* or *rendezvous* protocols for message transfers. *Eager* protocol enables `MPI` library to internally store the communication message to let the sender process re-use its send buffer instantly after returning from the `MPI` function. However, as the message size grows, `MPI` libraries cannot afford to store the messages internally and after a specific message size, called *eager threshold*, the `MPI` library switches to *rendezvous* protocol. In this protocol, sender and receiver processes communicate the transfer parameters by *Request To Send (RTS)* and *Clear To Send (CTS)* signals. After receiving `CTS` signal, the sender may utilize the interconnect's *Remote Direct Memory Access (RDMA)* feature to transfer data to the receiver's user-level buffer directly without `OS` involvement.

Message Matching Problem: In two-sided `P2P` communication, whether using *eager* or *rendezvous* protocols, it is almost impossible for the sender and receiver processes to reach

the communication point together. **MPI** has provided the *message matching* mechanism that handles out-of-order communications by *Unexpected Message Queue (UMQ)*, for storing the messages that the receiver was not ready to receive yet, and *Posted Receive Queue (PRQ)* for storing the receiver's requests. In both of these queues, each *Queue Element (QE)* corresponds to a unique message that is specified by its *envelope*: *source/receiver rank id*, *communicator id*, and *message tag*. Two preliminary designs of message queues are linked lists and arrays, which provide better memory usage and faster search time, respectively.

2.4.2 One-Sided Communication

This communication model decouples data movement from process synchronization. In this communication type, a process (called *origin*) transfers a message to/from another process (called *target*) without involving it in the communication process. The *origin* process provides all required communication parameters for both sending and receiving sides, and the *target* can either actively or passively participate in synchronization. This type of communication is also known as *Remote Memory Access (RMA)* since it enables the *origin* process to remotely read or write some parts of the *target* process's memory. In this context, sending data to the *target* memory is called *put* operation, while receiving data from the *target* memory is called *get* operation. This communication pattern is used in this dissertation (Chapter 3, Chapter 4, and Chapter 5) to design and implement **GPU**-aware algorithms via *put* and *get* operations.

2.4.3 Collective Communication

This communication mode provide a mechanism to exchange messages among a group of processes rather than just two. **MPI** defines three classes of *collective* communication: one-to-many, many-to-one, and many-to-many. Some of the commonly used *collectives* are *Allreduce*, *Reduce*, *Broadcast*, *Allgather*, and *Alltoall*. Each of these *collectives* may be

designed and implemented using a specific algorithm, considering the number of ranks, message size, and topology. Also, *collectives* can be hierarchical or non-hierarchical (*flat*). In *flat* algorithms, communication cost between any pair of processes is considered to be the same as others. Some of common *Allreduce* algorithms include: *Linear*, *Recursive Doubling*, *Ring*, *Segmented Ring*, and *Rabenseifner* (or *Reduce Scatter Allgather (RSA)* [110]). Moreover, *MPI* specifications define *barrier* functionalities to perform process synchronization, which is a type of *collective* communication. Lastly, all the *collectives* may be built based on *P2P* communication.

Allreduce Collective: To provide more details on *Allreduce* algorithms as they are used in this dissertation, we present three common algorithms in Figure 2.5, Figure 2.6, and Figure 2.7. In Figure 2.5, the flat *RSA* algorithm is demonstrated for four processes. In this algorithm, the *Allreduce* operation is divided into two phases: *Reduce-Scatter* and *All-Gather*, each with three steps. In the first phase, each process sends and receives data to/from its neighbors in a ring topology while performing the reduction operation on the received data. After completing the first phase, each process holds a segment of the reduced result (Figure 2.5.(e)). In the second phase, processes pass their segments to each other to gather the complete reduced result. This algorithm is efficient for large message sizes and scales well with the number of processes.

Next, Figure 2.6 illustrates the *Reduce-Broadcast* algorithm for eight processes. In this method, the *Allreduce* operation is performed in two main phases: *Reduce* and *Broadcast*. During the *Reduce* phase, processes are paired, and within each pair one process receives its peer data and performs the reduction operation. This pairing continues in a tree-like structure until one process holds the final reduced result. In the subsequent *Broadcast* phase, the reduced result is transferred back to all processes through a similar tree structure, ensuring that every process receives the final result (Figure 2.6.(g)). This algorithm

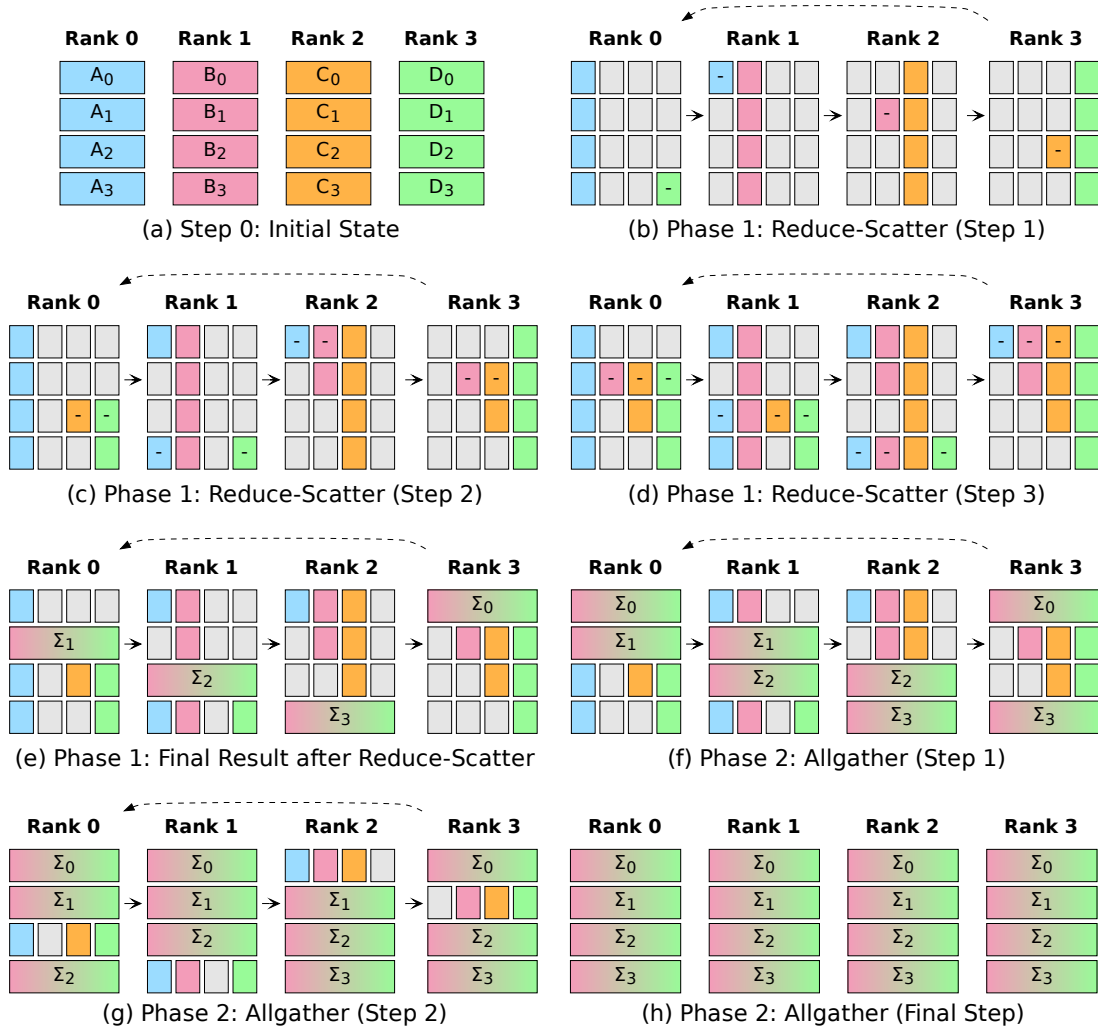


Figure 2.5: RSA Allreduce: Reduce-Scatter followed by All-Gather in a Ring topology with 4 processes

is particularly effective for small to medium message sizes and benefits from reduced communication costs due to its hierarchical approach.

Finally, Figure 2.7 depicts the *Recursive Doubling* algorithm for eight processes. In this approach, each process exchanges data with another process at increasing distances in each step. In the first step, processes exchange data with their immediate neighbors (distance of 1), performing the reduction operation on the received data. In subsequent steps, the

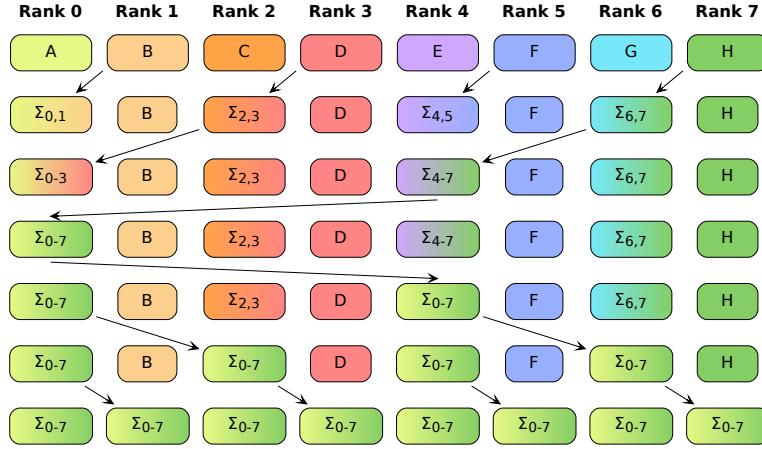


Figure 2.6: Reduce-Broadcast Allreduce for 8 processes

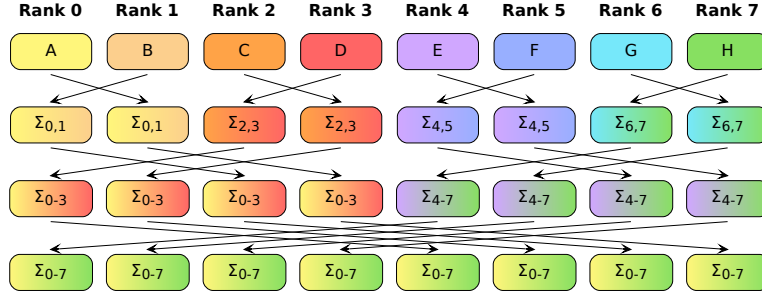


Figure 2.7: Recursive Doubling Allreduce for 8 processes

distance doubles (2, 4, etc.), and processes continue to exchange the data. The reductions after exchange is performed on both peers redundantly. This save the need for a separate broadcast phase, as each process accumulates the reduced result. This algorithm is efficient for small message sizes and provides logarithmic scaling with respect to the number of processes.

Alltoall Collective: Among *collectives*, *Alltoall* is known to be one of the most communication-intensive operations, as every process sends distinct data to all other processes. One of the well-known algorithms for *Alltoall* is the *Bruck* algorithm [20], which is particularly efficient for small messages on large-scale systems. Figure 2.8 illustrates this algorithm

for four processes. In this method, the communication occurs in $\log_2(P)$ steps with P processes. In the first step, the processes perform a local data rotation to prepare for the communication. In subsequent steps, each process exchanges data with another process at increasing distances (doubling the distance in each step). After completing all steps, a final local data rotation is performed to arrange the data in the correct order. Due to the number of data copies involved, this algorithm is more suitable for small to medium message sizes to avoid excessive overhead, making it less ideal for very large messages. However, newer variants and optimizations of the Bruck algorithm have been proposed to address these limitations and improve its performance for larger messages [123].

2.4.4 Hybrid MPI Programming Models

As stated in Section 1.1, it is common and sometimes essential to combine two or more programming models to utilize hardware resources better. For example, scientific application developers usually combine the programming models **CUDA**, **OpenMP** or **Pthreads** with **MPI**, such that they are used for parallelism within a node (with accelerators) while **MPI** is used for parallelism across the nodes. These models combine the strengths of each programming model to achieve better performance and scalability. For instance, in **MPI+CUDA** applications, **MPI** is used for inter-node communication, while **CUDA** is used for intra-node parallelism on the **GPUs**.

CUDA-aware **MPI** implementations can directly handle **GPU** memory buffers, eliminating the need for explicit data transfers between the host and device by the user. This feature simplifies programming and can lead to performance improvements by reducing data movement overhead. This capability is particularly beneficial in applications that require frequent communication between **GPU**-accelerated processes, and when the **GPUs** support GPUDirect technology, which allows direct data transfers between **GPUs** (even across

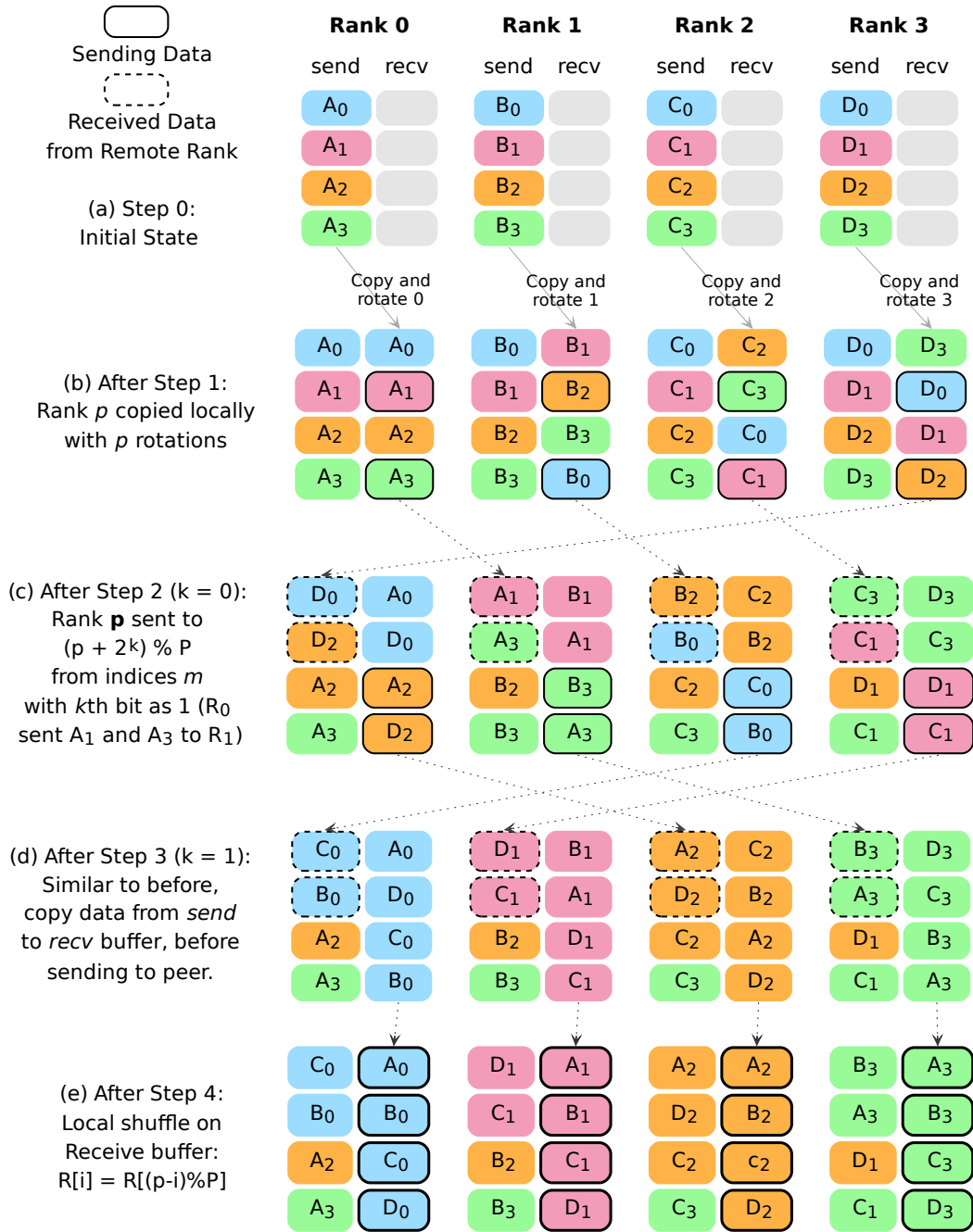


Figure 2.8: MPI Alltoall Bruck algorithm for 4 processes. In this example, the *send* and *receive* buffers are used as temporary storage for data rotation steps to avoid data corruption without additional allocations.

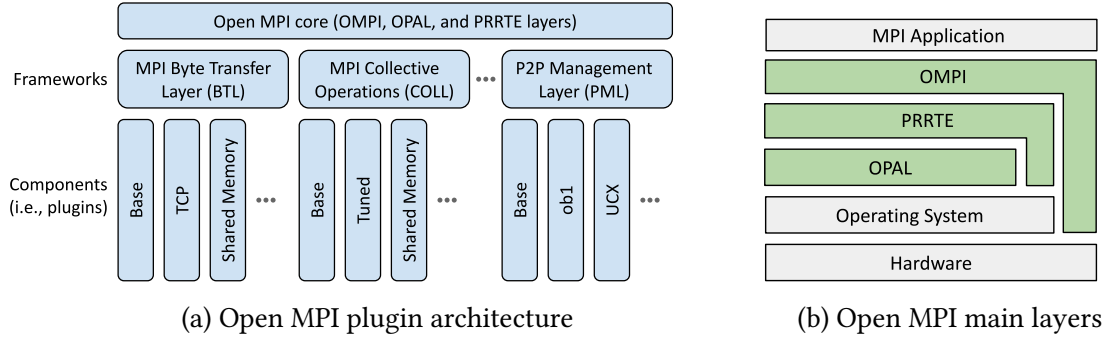


Figure 2.9: Open MPI architecture [103]

different nodes) without involving the host CPU.

2.4.5 Message Passing Interface Implementations

Popular implementations of MPI include MPICH [81] by Argonne National Laboratory (ANL), MVAPICH [85], and Open MPI [103]. Each of these libraries has evolved and has adopted various designs and architectures to implement MPI specifications. For instance, MPICH and MVAPICH store *message matching* data in linked lists, while Open MPI utilizes arrays of linked lists. Moreover, the MPI libraries that are GPU-aware can handle explicit data movement from and to the GPUs memory without user involvement.

Open MPI: As all of the experiments in this dissertation are conducted using Open MPI, we provide more details about its architecture and design here. However, similar concepts may apply to other MPI implementations as well. As one of the most popular open-source MPI implementations, Open MPI was designed around the central idea of grouping similar functionalities in distinct abstraction layers, which can be configured at run-time with loadable plugins to determine the library’s behavior. So, as Figure 2.9b demonstrates, Open MPI provides three main abstraction layers: (1) *Open Portable Access Layer (OPAL)*, which encompasses anything regarding individual processes and OS-specific functionalities; (2) *Process Management Interface (PMIx) Reference RunTime Environment (PRRTE)*, where all

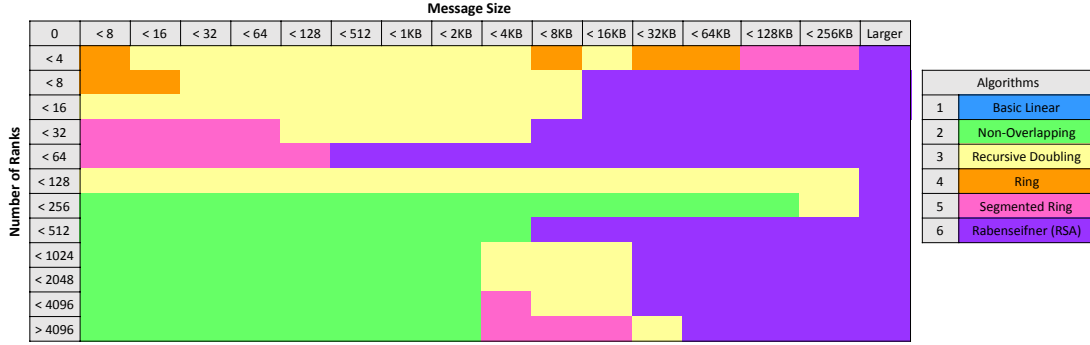


Figure 2.10: OpenMPI Intra-Node Allreduce Tuning Table

the functionalities related to launch, monitor, and kill parallel jobs exist, and (3) *Open MPI (OMPI)*, in which the main *MPI API* is implemented. As Figure 2.9a demonstrates, Open *MPI* plugin architecture provides the component concept throughout all three layers, in which several components are available to utilize. Each of these components belongs to exactly one framework, like *Byte Transfer Layer (BTL)*, and each framework supports exactly one kind of component.

Like other *MPI* implementations, Open *MPI* provides several algorithms for each *collective* communication. These algorithms may perform differently based on the message size, number of processes, and hardware topology. Open *MPI* provides a *Tuned* component that automatically selects the best algorithm according to tuning tables based on the number of processes and message size. For instance, as Figure 2.10 demonstrates, the *Tuned* component in *collective* framework utilizes a tuning table to decide which intra-node *Allreduce* algorithm to use at runtime [103].

2.5 Unified Communication X (UCX) Framework

We introduce *UCX* as it is used in this dissertation to implement and optimize *MPI* communication algorithms. *MPI* libraries, such as Open *MPI* and *MPICH*, can leverage *UCX* as a communication layer to enhance their performance, particularly for *P2P* communication

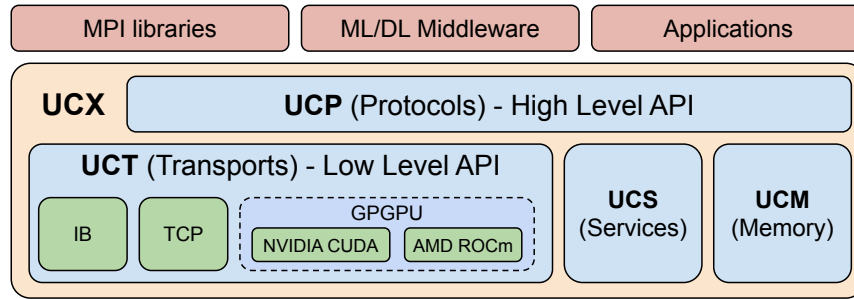


Figure 2.11: UCX architecture and some of its components

[124]. UCX has three major components: *Unified Communication Protocols (UCP)*, *Unified Communication Transports (UCT)*, and *Unified Communication Services (UCS)*. These layers provide a set of abstract communication primitives in order to utilize hardware resources, including *RDMA*, *GPUs*, *shared memory*, and network operations. Recently, this framework has been broadly utilized in development environments, like Open *MPI+UCX* or *MPICH+UCX*, to improve the performance of *MPI* libraries *P2P* communication [124]. The UCX library is an open-source, modular communication framework intended to enable low-latency, high-bandwidth, and efficient communication through multiple hardware interconnects, including shared memory, InfiniBand, RoCE, PCIe, NVLink, etc. [54, 100, 150]. Figure 2.11 displays the UCX hierarchy with a focus on important pieces, including the UCT layer.

2.5.1 Unified Communication Transport (UCT)

The UCT layer provides low-level communication operations, including *put*, *get*, and *active message* semantics, which can be used to build higher-level communication protocols and libraries. This layer is designed to be extensible, allowing new transport modules to be added as needed to support emerging hardware technologies.

The UCT layer comprises several core abstractions: (1) **Memory Domain** handles memory registration and memory access management specific to each hardware transport; (2)

Interface represents a communication resource tied to a specific transport on a given device, encapsulating the capabilities and constraints of the transport; and (3) **Endpoint** represents a logical communication link to a remote peer, managing connection state and message flow.

2.6 NVIDIA Collective Communications Library

NCCL (pronounced "Nickel") is an open-source library maintained by NVIDIA that provides **MPI**-like *collectives* on multiple **GPU** clusters, including: *AllReduce*, *AllGather*, *Reduce*, *Broadcast*, and *ReduceScatter* [99]. **NCCL** is broadly used by **HPC** and **DL** frameworks and its operations can be utilized within **CUDA** Graphs via stream capturing method. Moreover, it supports **IB** and **RoCE** interconnects to fully utilize **RDMA** on **GPUs** for inter-node communications, which is introduced as **GPUDirect_RDMA** technology [97].

2.7 Performance Measurement and Benchmarks

Although there is no consensus on how an application performance should be measured and reported, **HPC** developers employ various performance metrics to evaluate applications performance, such as *execution time*, *speed*, *speedup*, *parallel efficiency*, and *utilization*. To measure these parameters accurately, **HPC** developers utilize a benchmark application that captures and usually reports the application's processing and data movement characteristics. Some of the popular benchmarks are **HPL** [145], *Scalable Heterogeneous Computing benchmark suite* (**SHOC**) [154], *OSU Micro-Benchmarks* (**OMB**) [141], *NAS Parallel Benchmark suit* (**NPB**) [86], **Graph500** [44], and **Deep500** [33].

2.7.1 Performance Metrics

The performance of HPC applications is typically evaluated using a variety of metrics. The choice of performance metrics depends on the specific application and the goals of the performance evaluation. In the following, we briefly describe some of the common performance metrics used in HPC.

Execution Time: The execution time is the most common performance metric that indicates how long an application takes to complete its execution. The execution time can be measured using various tools and techniques, such as OS-provided timers, profiling tools, or functions/libraries provided by programming languages or frameworks.

Speedup: Speedup is a measure of how much faster an application runs on a parallel system compared to a serial system. The speedup is further classified into *strong scaling* and *weak scaling*. *Strong scaling* refers to the speedup achieved by increasing the number of processors while keeping the problem size fixed, while *weak scaling* refers to the speedup achieved by increasing both the number of processors and the problem size proportionally.

Strong scaling can be calculated as:

$$S = \frac{T_1}{T_p} \quad (2.1)$$

where T_1 is the execution time on a single processor, and T_p is the execution time on p processors. If we take into account the parallelizable fraction of the code, according to Amdahl's law [8], the speedup can be calculated as:

$$S = \frac{1}{(1 - f) + \frac{f}{p}} \quad (2.2)$$

where f is the parallelizable fraction of the code, and p is the number of processors. This formula shows that the speedup is limited by the serial fraction of the code, and as the

number of processors increases, the speedup approaches a maximum value of $\frac{1}{1-f}$. On the other hand, weak scaling is a measure of how well an application can handle larger problem sizes as the number of processors increases. Weak scaling, or Gustafson's law [46], can be calculated as:

$$S_w = \frac{T_1(n)}{T_p(n)} = \frac{(1-f) + f \cdot p}{(1-f) + f} = (1-f) + f \cdot p \quad (2.3)$$

Parallel Efficiency: *Parallel efficiency* is a measure of how effectively a parallel application utilizes the available processors as more processors are added. It is defined as the ratio of the speedup to the number of processors used. The efficiency can be calculated as:

$$E = \frac{S}{p} = \frac{T_1}{p \cdot T_p} \quad (2.4)$$

where S is the speedup, and p is the number of processors.

Utilization: Utilization is a measure of how much of the available resources are being used by an application. It is usually measured in terms of *CPU utilization*, *GPU utilization*, or *bandwidth utilization*. While CPU utilization is the percentage of time the CPU spends executing the application's code, GPU utilization might refer to various metrics, such as *compute utilization*, *memory utilization*, *SM utilization*, or *power utilization* (performance per cost). Also bandwidth utilization refers to the percentage of the available bandwidth being used by the application in data transfers.

In this chapter, we covered the necessary background on HPC architectures, GPU clusters, parallel programming paradigms, and performance measurement and benchmarks that are relevant to the rest of the dissertation. In the next chapter, we present the design and implementation of our work on "multipath point-to-point communication".

Chapter 3

Multipath Point to Point Communication

3.1 Introduction

The integration of GPU into HPC infrastructures has attracted much attention over the past decade, coupled with the advent of many applications across various domains [17]. In order to efficiently utilize the computational power of GPUs, distributed applications need to allow for efficient GPU communication. In particular, data exchange between GPUs on the same node with the help of MPI communication protocols, the de facto standard for distributed systems [80], plays a critical role in reaching optimal performance. Traditionally, P2P communication in GPU-accelerated systems relies on direct data transfers through a single NVLink or PCIe communication path, which often becomes a bottleneck and limits overall performance [138]. While several studies have attempted to improve P2P communication by message striping across multiple paths, such as in the UCX library, these approaches still lack the ability to provide concurrent data staging through both host and device [88, 138]. Our earlier work demonstrated that leveraging host-staged transfers alongside other GPU-staged communication could significantly enhance P2P performance [138, 139]. Building upon this foundation, we propose a unified multi-path communication

framework with two sub-designs: a static runtime-based pipelining engine and a dynamic **CUDA** Graph-based communication execution [133].

CUDA Graph enables capturing and reusing the workflow of execution that can be represented as a directed graph. Utilizing **CUDA** Graph leads to reducing the launch overhead, and has gained popularity in recent studies [26, 169]. In our proposed approach, we dynamically construct **CUDA** Graphs to encapsulate **P2P** communication workflows, allowing efficient execution of data transfers with minimal launch overhead [134]. Additionally, by caching **CUDA** Graphs in the **UCX** library and reusing them for repetitive communication patterns, we improve efficiency and reduce unnecessary synchronization costs. With this approach, the CPU is relieved from managing and orchestrating the flow, enabling more efficient data transfers between **GPUs**. Our key contributions are as follows:

1. We propose a multi-path communication framework within the **UCX** library, specifically designed to enhance **P2P** communication between two intra-node **GPUs**, by leveraging concurrent path utilization, including available NVLinks and **PCIe** through the host. We design and implement a 2-D pipelining engine to statically scatter the data across the communication channels. Our experimental evaluation on a four-**GPU** node demonstrates that this approach achieves up to a 2.8 $\times$ increase in **GPU-to-GPU OMB MPI** bandwidth test compared to traditional methods.
2. We extend the framework with a **CUDA** Graph-based design that dynamically constructs and reuses communication workflows to minimize launch and intra-node communication overhead. This integration is, to the best of our knowledge, the first of its kind within the **UCX** framework, and the evaluations demonstrate its effectiveness by increasing the communication bandwidth by up to 2.9 $\times$.
3. We show the benefits of our proposed framework in a real-world application, Jacobi,

where we achieve up to $1.28\times$ improvement in overall execution time compared to the traditional single-path communication approach, when we enable dual-path communication.

4. We provide a comprehensive analysis of the performance trade-offs and overheads associated with [CUDA](#) Graphs in the context of multi-path communication, offering insights and guidelines into their practical implications for [HPC](#) applications.

The remainder of this chapter is organized as follows. Section [3.2](#) provides the necessary background for this work, and Section [3.3](#) provides a review of the related work. Section [3.4](#) describes the design and implementation of our multi-path communication framework, including both static and dynamic models. Section [3.5](#) presents the experimental setup, performance evaluation, and analysis of the results. Finally, Section [3.6](#) concludes the chapter, summarizing the contributions.

3.2 Background and Motivation

3.2.1 GPU-enabled Systems, MPI, and UCX

Efficient utilization of modern [GPU](#)-enabled systems is a common challenge in [HPC](#) frameworks and applications. These systems typically feature multiple accelerators per node, connected via high-bandwidth interconnects such as NVLink, and to the host CPUs through [PCIe](#).

One of the most widely adopted communication standards in [HPC](#) is [MPI](#), which provides an extensive suite of communication primitives suitable for parallel and distributed applications. Among the various implementations, Open [MPI](#) [[103](#)] and [MPICH](#) [[81](#)] are notable for their flexibility and widespread use in both traditional [HPC](#) and emerging AI workloads.

To address the evolving communication needs in applications that leverage GPU acceleration, MPI implementations are often reliant on high-performance communication libraries, for example, the UCX library [124]. The UCX library is an open-source, modular communication framework intended to enable low-latency, high-bandwidth, and efficient communication through multiple hardware interconnects, including shared memory, InfiniBand, RoCE, PCIe, NVLink, etc. [54, 100, 150]. Figure 2.11 displays the UCX hierarchy with a focus on important pieces, including the UCT layer. The UCT layer enables a uniform interface for communication primitives while at the same time providing hardware-specific implementations through the separate transport modules.

3.2.2 CUDA, UCX-CUDA Modules, and Direct Communication

CUDA is a widely used platform and programming model developed by NVIDIA to harness the massive parallelism offered by GPUs [92]. Over the past decade, several middleware solutions have emerged to integrate GPUs into distributed HPC applications [153, 23, 132, 24]. While this dissertation primarily targets NVIDIA GPUs and CUDA, many of the underlying concepts are extensible to other accelerator platforms such as those from AMD and Intel.

The UCX library offers built-in support for CUDA through its `uct_cuda` transport module [150]. This module consists of a shared `base` and three distinct submodules:

- `cuda_copy` : Optimized for intra-process communication, supporting device-to-device memory transfers within the same process.
- `gdr_copy` : Utilizes GPUDirect_RDMA to allow direct memory access for small messages.
- `cuda_ipc` : Enables zero-copy inter-process communication via CUDA IPC handles, making it well-suited for large messages and intra-node MPI workloads. In our

framework, we primarily target the `cuda_ipc` submodule, which is critical for enabling efficient large message exchanges across GPUs residing on the same node.

Typically, when message sizes exceed 64KB, communication in UCX defaults to a *rendezvous* protocol. Depending on which process, sender or receiver, initiates the transfer and on internal configuration options, UCX chooses between a *put* or a *get* operation. In both scenarios, successful communication requires that the source and destination memory buffers reside within a shared CUDA context, accomplished through the exchange of CUDA IPC memory handles.

A CUDA context encapsulates a process's view of the GPU, including memory allocations, execution resources, and synchronization primitives such as CUDA streams and CUDA events. Each CUDA context is associated with a single device but can be shared across multiple threads or processes. The maintenance of a context is essential to coordinate memory accesses and manage ownership.

Historically, coordinating computation and communication between GPUs required explicit staging through host memory, increasing latency and memory overhead. With the advent of GPUDirect technologies, direct communication became feasible, including GPUDirect_P2P, which enables direct peer-to-peer data movement between intra-node GPUs, bypassing the host.

Figure 3.1 illustrates both the hardware topology and a common communication scenario. In Figure 3.1(a), we show a standard four-GPU node with bidirectional NVLink interconnects, and in Figure 3.1(b), we demonstrate how a data transfer between GPU-0 and GPU-1 can be split across multiple paths, utilizing idle bandwidth on alternative links.

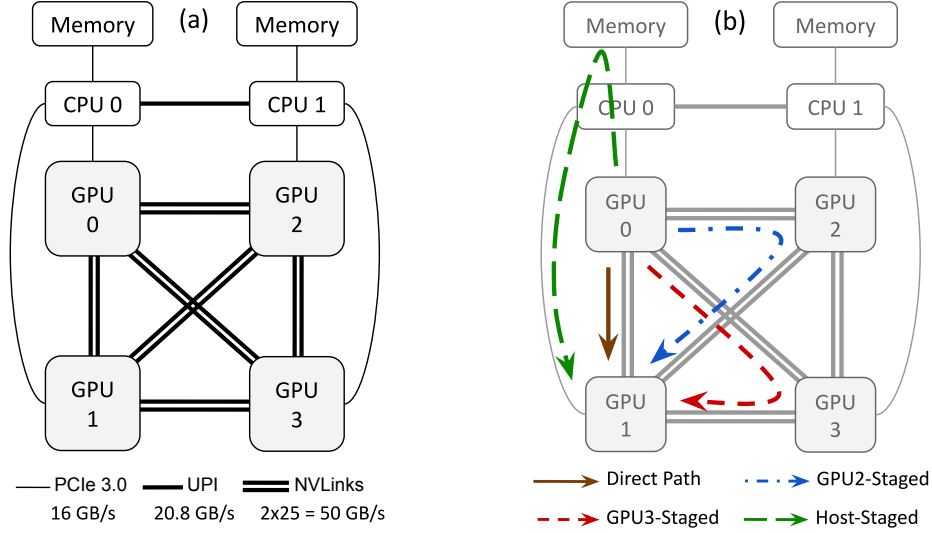


Figure 3.1: (a) A typical four-GPU node with NVLink (two sub-links) per GPU pair (Similar to Beluga nodes), (b) A communication between GPU-0 and GPU-1 is split and transferred through multiple paths

3.2.3 CUDA Graphs and Advanced Runtime Integration

Despite the potential for high bandwidth, managing concurrent communication paths introduces overhead in terms of coordination, memory management, and synchronization. As message sizes grow and communication becomes more complex, these overheads can limit scalability and performance.

To address this, we integrate [CUDA Graph](#) into our multi-path communication framework. [CUDA Graph](#) is a [CUDA](#) feature that allows users to capture a sequence of [GPU](#) operations and replay them as a single executable graph [94, 26, 169]. This reduces launch overhead and enables better scheduling and optimization by the [CUDA](#) runtime.

As discussed in Section 2.3.3, [CUDA Graph](#) consists of two main components:

- **Nodes:** Represent individual [GPU](#) operations such as memory copies or kernel launches.
- **Dependencies:** Specify ordering constraints between nodes, forming a directed

graph.

CUDA Graphs can be created either via explicit API calls or by capturing a stream of GPU operations. By encapsulating multi-path communication as a CUDA Graph, we enable efficient execution of complex workflows that involve multiple memory transfers, staging buffers, and synchronization points.

Furthermore, CUDA Graphs can be cached and reused for repetitive patterns, reducing overhead for recurring communication tasks. This is particularly advantageous in HPC applications with regular communication patterns, such as stencil computations or iterative solvers.

3.2.4 Motivation

As illustrated in Figure 3.1(b), traditional communication via a single path (e.g., between GPU-0 and GPU-1) can saturate available bandwidth, leading to performance bottlenecks. Splitting large data transfers into smaller chunks and routing them across multiple available paths can help mitigate this issue and fully exploit the system’s communication capabilities.

While earlier approaches such as host-staging or manual pipelining can improve throughput, they often require careful tuning and can introduce additional CPU overhead [138, 139]. Additionally, ensuring data integrity, minimizing synchronization costs, and maximizing overlap between transfers are non-trivial challenges.

Incorporating CUDA Graph into our design enables an automated, low-overhead solution to manage and schedule these complex workflows efficiently. It provides an elegant mechanism to orchestrate fine-grained transfers across multiple paths, thereby improving scalability and reducing CPU involvement.

Together, these technologies form the foundation of our proposed framework: a multi-path communication model that supports both static runtime scheduling and dynamic, **CUDA** Graph-enabled execution for optimized intra-node **GPU** communication in **HPC** environments.

3.3 Related Work

3.3.1 Multi-Path Communication

Tatsugi and Nukada [136] propose a method to enhance the performance of a data transfer from a **GPU** to host, by utilizing the idle **GPUs**. Their framework targets single-**GPU** applications running on a multi-**GPU** node, while our approach is designed for multi-**GPU** applications.

As part of our prior studies [138, 139], we enhance **P2P** communication within the **UCX** library by utilizing host-staging multi-path communication to devise more performant collective communication for Deep Learning applications. However, in this dissertation, we utilize both **NVLinks** and **PCIe** paths to further enhance the performance of **P2P** communication between intra-node **GPUs**. In contrast, our previous work only utilized **PCIe** paths, which limited the performance gains.

In [88], Nukada proposes a method that utilizes the **PCIe** path to accelerate *Allreduce* on a multi-**GPU** system. Although their method involves collectives, they follow a similar approach to our previous work to enhance each **P2P** communication. Again, our proposed approach utilizes both **NVLinks** and **PCIe** paths, while their method is limited to **PCIe**.

3.3.2 CUDA Graphs

Numerous studies have demonstrated that **CUDA** Graphs effectively reduce CPU overhead by consolidating multiple kernel launches into a single execution unit, thereby enhancing **GPU** utilization and overall application performance [169, 168, 70, 36, 109]. For example, frameworks have reported substantial speedups, such as up to 3.47× in kernel execution [168] and over 1.4× through kernel batching in iterative **HPC** applications [36]. Beyond reducing launch overhead, **CUDA** Graphs also provide the opportunity for more efficient resource management, including notable reductions in **GPU** memory consumption via metadata optimizations [169]. These improvements underscore the role of **CUDA** Graphs in achieving high performance in **HPC** workloads.

To facilitate the adoption of this feature, several frameworks and programming models have emerged that simplify **CUDA** Graph development [52, 70, 109]. Lin et al.[70] proposed a lightweight programming framework that abstracts the complexity of constructing **CUDA** Graphs for **GPU** computation. Similarly, Huang et al.[52] introduced a task-graph programming model designed to aid the development of *Task Graph Computing System* (**TGCS**), where function calls and their inter-dependencies are represented as task graphs; these are then submitted using **CUDA** Graphs on supported **GPUs**. Qiao et al. [109] leveraged **CUDA** Graphs within a source-to-source compiler for image processing *Domain-Specific Languages* (**DSL**), achieving improved workflow optimization through graph-based execution.

Libraries like **NCCL** and **UCC** have also integrated **CUDA** Graph support to optimize collective communication operations [99, 149]. **NCCL** utilizes **CUDA** Graphs to capture and replay collective operations, reducing overhead, and **UCC** uses **CUDA** Graphs to group multiple data transfers between intra-node **GPUs**. At the time of writing this dissertation, **UCC** does not put computations on the **CUDA** Graphs, and it only uses them

for communication operations. Our work is different from both of these libraries, as we focus on optimizing P2P communication between intra-node GPUs by leveraging multiple paths (with staging) and utilizing CUDA Graphs to encapsulate the entire communication workflow.

Despite these advances, challenges remain in fully exploiting CUDA Graphs, particularly for dynamic workloads and complex control flows [95]. Many implementations struggle with applications featuring varying input sizes or data-dependent execution paths, often requiring intricate compiler or runtime support and incurring higher memory overhead [169]. Additionally, optimizing dynamic and irregular communication/computation patterns using CUDA Graphs paradigm continues to be challenging. Addressing these limitations is essential for extending the applicability of CUDA Graphs to a broader spectrum of modern HPC and scientific computing workloads.

3.4 Design and Implementation

To enable efficient and scalable communication between GPUs in the same node, we have designed a system that builds upon the existing functionality of `uct_cuda` transport module in UCX. This is built by adding both a multi-path communication engine, which supports the use of multiple communication paths simultaneously, and a runtime acceleration mechanism based on CUDA Graph, which minimizes the synchronization costs. Our overall design philosophy is to prioritize maintaining compatibility with the internal structure of UCX, introducing innovative features on top of it, and utilizing the advantages of concurrent paths, with minimal overhead additions. In this section, we will present a detailed description of our architecture, describe the pipelining mechanism that we have used, discuss several strategies for runtime tuning, and ensure that we discuss the guarantees pertaining to data integrity in detail.

3.4.1 Design Objectives

Our design addresses the following core goals to support high-performance communication:

1. **Multi-GPU awareness:** Automatically detect and utilize all available GPUs and their interconnects for efficient communication path construction within UCX instances.
2. **Path selection:** Dynamically select the most suitable and available paths between source and destination GPUs, including direct and indirect routes via staging GPUs or host.
3. **Communication scheduling:** Efficiently distribute data along the selected paths using fine-grained pipelining.
4. **Path optimization:** Maximize throughput and minimize idle time by overlapping transfers across paths.
5. **Data integrity:** Maintain correctness by ensuring ordered, synchronized, and contention-free transfers.
6. **Low overhead:** Keep the framework's runtime and memory overhead negligible compared to the overall communication cost.
7. **Seamless integration:** Ensure compatibility with existing UCX applications, requiring minimal changes to leverage the new capabilities.
8. **Dynamic adaptability:** Support dynamic graph construction and caching to optimize repetitive communication patterns.

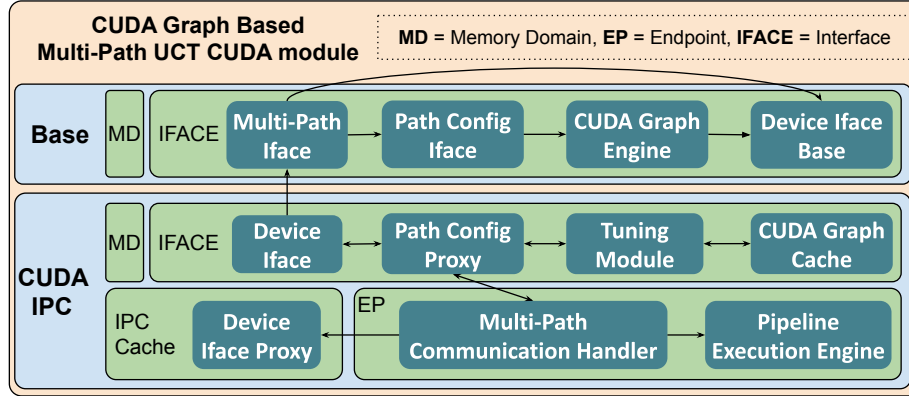


Figure 3.2: Integration of our multi-path communication framework with the UCX library. The internal details of the UCX library are not shown for simplicity. CUDA Graph related modules are in dark green.

3.4.2 Framework Architecture

Figure 3.2 presents a simplified overview of our framework. The major components are described below:

- **Base Module:** This module is responsible for probing the system's hardware topology and initializing internal structures for the framework. It detects the available GPUs and their interconnects, and creates the necessary runtime resources for each GPU, including CUDA streams, CUDA events, device buffers, path metadata, and the communication proxy objects that bridge CUDA IPC and the graph engine. Some of these functionalities are inherited from the original `uct_cuda` module, and we use the already available functionalities to detect the GPUs and their interconnects. However, the rest of the explained functionalities are newly implemented as part of our framework.
- **CUDA IPC Module:** This module handles incoming communication requests, selecting the suitable paths, handling configurations from environment variables, and scheduling the communication along the selected paths with the 2-D pipelining

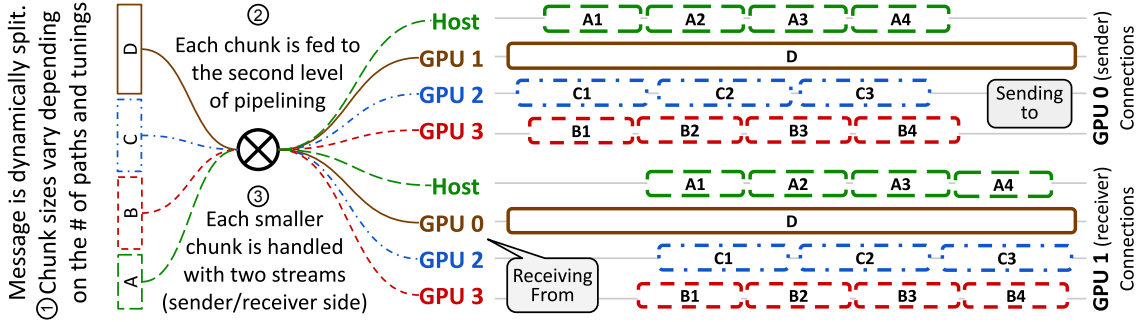


Figure 3.3: A simplified view of 2-D pipelined communication from GPU-0 to GPU-1 using the available NVLink and PCIe. Staging GPUs' timelines are not shown for simplicity.

engine. The connection between the `base` and `cuda_ipc` modules is established through proxy entities.

- **CUDA Graph Engine:** This engine constructs reusable `CUDA` Graphs in a lazy fashion using the explicit `CUDA` Driver `API`. It abstracts the pipelined multi-path workflow into a single `CUDA` Graph, reducing launch overhead and synchronizations. Each graph represents a unique communication configuration between two `GPUs`, with varying numbers of paths and chunks.
- **CUDA Graph Cache:** The cache stores instantiated `CUDA` Graphs based on the communication configuration, including source/destination buffer addresses and message size. It uses a configurable least-recently-used (LRU) eviction policy and is implemented as a fixed-size hash table, tunable via environment variables.

3.4.3 2-D Pipelining Engine and CUDA Graph Execution

Our 2-D pipelining engine is the core of the data distribution mechanism. As shown in Figure 3.3, a large message is first partitioned according to the number of selected paths (horizontal split). Then, each path is further subdivided into smaller chunks (vertical split) that are transferred in a pipelined fashion.

Each data chunk is processed using two separate **CUDA** streams: one for the source-to-staging transfer and another for the staging-to-destination transfer. This parallelization allows communication along multiple routes to proceed concurrently, while also enabling fine-grained overlap within each route. The number of chunks per path is a tunable parameter that ranges from 1 to 16, depending on message size and the capabilities of each link. More chunks typically improve concurrency but incur additional synchronization overhead.

When **CUDA** Graphs are enabled, the entire communication workflow, including chunk partitioning, staging transfers, synchronization, and final commit, is encapsulated into a DAG, as shown in Figure 3.4. Each node in the DAG represents a distinct memory copy or kernel operation, and edges represent dependencies. As demonstrated in Figure 3.4, the message is divided into four parts, each transferred via a different path. The data along the *direct* path is copied directly from **GPU-0** to **gpu-1**, while the other three parts are staged through intermediate **GPUs** or host memory with various number of chunks. This abstraction reduces per-transfer launch overhead, improves reuse for repetitive patterns, and allows for better optimization by the **CUDA** runtime scheduler.

Algorithm 3.1 and Algorithm 3.2 outline the algorithms for creating and launching these **CUDA** Graphs. As shown in Algorithm 3.1, the `CreateGraph` function is responsible for constructing a new **CUDA** Graph instance based on the communication parameters, while the `LaunchGraph` function retrieves and launches a **CUDA** Graph instance.

In more detail, Algorithm 3.1 starts by determining the source and destination device IDs from their respective **CUDA** *contexts* (Line 4). It then generates a unique graph ID based on the source/destination addresses and message size (Line 5). The path configuration is obtained using the `GetPathConfig` function, which considers factors such as message size and available paths (Line 6). This configuration is set statically during the `uct_cuda`

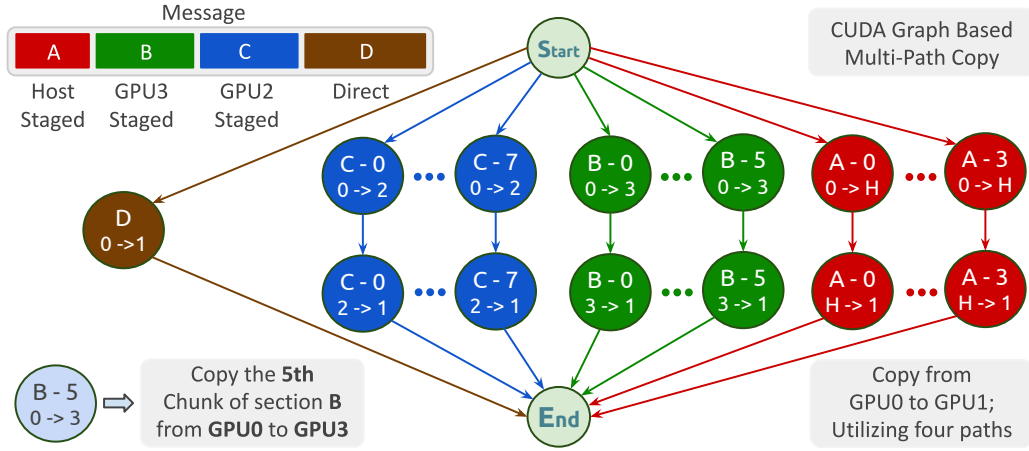


Figure 3.4: A CUDA Graph-based multi-path communication from GPU-0 to GPU-1 using the available NVLinks and PCIe.

module initialization and can be modified at runtime via environment variables. Then, temporary device IDs for staging are updated according to the source and destination devices and the configuration (Line 7). The message is then partitioned into chunks for each path, and the appropriate copy functions are invoked based on whether the path is direct or staged (Lines 9-23). In our implementation, we use a round-robin approach to distribute chunks across paths one-by-one, but we provide the simplified version here for clarity. In the simplified version, the chunks for each path are scheduled together and consecutively. Finally, the constructed graph instance is stored for future use (Line 24).

In the `LaunchGraph` function, the graph instance is retrieved using the unique graph ID (Line 27) from the cache. If the instance does not exist, it is created using the `CreateGraph` function (Lines 28-30). The graph is then launched on the appropriate stream (Line 31), and a `CUDA` event is recorded on the `CUDA` stream and passed to `UCX` for further processing (Line 32).

In Algorithm 3.2, we detail the helper functions used to create specific types of memory copy nodes, including direct peer-to-peer copies (Lines 1-9) and staged transfers via

Algorithm 3.1: CUDA Graph Creation and Launch for Multi-Path UCX Transfers

```
1 Input: src, dst, size, ctxsrc, ctxdst, stream
2 Output: A CUDA event to signal completion of the transfer
3 Function CreateGraph(src, dst, size, ctxsrc, ctxdst, stream):
4   idsrc, iddst  $\leftarrow$  get device IDs from ctxsrc, ctxdst
5   graph_id  $\leftarrow$  get unique graph ID based on src, dst, size
6   config  $\leftarrow$  GetPathConfig(size)
7   UpdateTemplIDs(config, idsrc, iddst)
8   sent_data  $\leftarrow$  0
9   foreach path p in config do
10    chunk_size[p]  $\leftarrow$   $\lceil (size \times share[p]) / max\_chunks \rceil$ 
11    srcp  $\leftarrow$  OffsetPtr(src, sent_data)
12    dstp  $\leftarrow$  OffsetPtr(dst, sent_data)
13    sent_data  $+=$  chunk_size[p]
14    if tmp_dev[p] is direct then
15      | PeerToPeerCopy(srcp, dstp, chunk_size[p], idsrc, iddst, streams[p], graph)
16    end
17    else if tmp_dev[p] is host then
18      | StageHostCopy(srcp, dstp, chunk_size[p], idsrc, iddst, streams[p], graph)
19    end
20    else if tmp_dev[p] is GPU then
21      | StageGPUCopy(srcp, dstp, chunk_size[p], idsrc, iddst, tmp_dev[p], streams[p], graph)
22    end
23  end
24  store constructed graph instance with graph_id in cache
25 end
26 Function LaunchGraph(ctxsrc, ctxdst, stream, graph_id):
27   instance  $\leftarrow$  get graph instance with graph_id from cache
28   if instance = NULL then
29     | instance  $\leftarrow$  CreateGraph(src, dst, size, ctxsrc, ctxdst, stream)
30   end
31   launch instance on appropriate stream
32   record event on stream and return to UCX
33 end
```

host (Lines 10-19) or intermediate **GPUs** (Lines 20-29). In each function, we create the necessary **CUDA** Graph nodes and configure them with the appropriate parameters, such as source/destination addresses, sizes, and device IDs (Lines 6, 16, and 26). These nodes are then added to the graph instance passed by reference. In Lines 17 and 27, we add the necessary dependencies to ensure the correct ordering of operations for staged transfers

Algorithm 3.2: CUDA Graph Nodes Creation and Configuration

```
1 Input:  $src, dst, size, id_{src}, id_{dst}, streams, graph$ 
2 Output: Updated  $graph$  with new nodes and dependencies
3 Function PeerToPeerCopy( $src, dst, size, id_{src}, id_{dst}, streams, graph$ ):
4   for  $s \leftarrow 0$  to  $streams - 1$  do
5     create memcpy node: device  $\rightarrow$  device
6     config node with  $size$  and  $(id_{src}, id_{dst})$ 
7     add memcpy node from  $src[s]$  to  $dst[s]$  on  $graph$ 
8   end
9 end
10 Input:  $src, dst, size, id_{src}, id_{dst}, streams, graph$ 
11 Output: Updated  $graph$  with new nodes and dependencies
12 Function StageHostCopy( $src, dst, size, id_{src}, id_{dst}, streams, graph$ ):
13   for  $s \leftarrow 0$  to  $streams - 1$  do
14     create memcpy node: device( $id_{src}$ )  $\rightarrow$  host (staging)
15     create memcpy node: host  $\rightarrow$  device( $id_{dst}$ )
16     config nodes with  $size$  and device ids
17     add dependency between staging and completion nodes
18   end
19 end
20 Input:  $src, dst, size, id_{src}, id_{dst}, id_{stage}, streams, graph$ 
21 Output: Updated  $graph$  with new nodes and dependencies
22 Function StageGPUCopy( $src, dst, size, id_{src}, id_{dst}, id_{stage}, streams, graph$ ):
23   for  $s \leftarrow 0$  to  $streams - 1$  do
24     create memcpy node: device( $id_{src}$ )  $\rightarrow$  staging GPU( $id_{stage}$ )
25     create memcpy node: staging GPU( $id_{stage}$ )  $\rightarrow$  device( $id_{dst}$ )
26     config nodes with  $size$  and device ids
27     add dependency between staging and completion nodes
28   end
29 end
```

of chunks.

3.4.4 Runtime Tuning and Configuration

To accommodate diverse hardware topologies and workloads, our framework supports both user-defined and auto-tuned configurations.

- **Environment Configuration:** Users can configure the framework via environment variables to enable or disable specific paths (e.g., host-staging), specify the number of

concurrent paths, and define the number of chunks per path. These configurations allow flexibility for both expert users and general-purpose deployment.

- **Topology Tuning:** We provide the offline tuning mechanism which exhaustively searches for optimal communication configurations based on the number of GPUs, available links, and transfer sizes. The tuned parameters are used as defaults unless overridden by the user. Note that this tuning should be performed separately for CUDA Graph-enabled and non-CUDA Graph modes, as the optimal configurations may differ due to the reduced overheads and improved scheduling capabilities of CUDA Graphs.

3.4.5 Ensuring Data Integrity and Correctness

To preserve correctness, our design ensures strict data consistency and avoids race conditions by leveraging CUDA’s synchronization semantics. The following guarantees are enforced with both the non-CUDA Graph and CUDA Graph-based implementations:

- **Contention Avoidance:** For each directional path (e.g., GPU-2 to GPU-3), only one transfer is allowed at a time. This is achieved by mapping all transfers for a specific direction to the same CUDA stream and synchronizing using CUDA events.
- **Dependency Handling:** For paths involving staging buffers (e.g., via host or intermediate GPU), the second transfer (staging to destination) is issued only after the first completes.
- **Ordering Guarantee:** Transfers for each chunk within a message are scheduled in a well-defined order and completed before subsequent messages are handled.
- **Final Synchronization:** All CUDA streams are synchronized at the end of a message to ensure that the transfer has completed and that the data in the destination buffer is

consistent and ready for use. As explained earlier, this is done by recording a `CUDA` event on each stream within each path and instructing the main `CUDA` stream to wait for all events to complete before returning control to the caller. This is done in a non-blocking manner, allowing `UCX` to continue processing other requests while waiting for the communication to complete.

These mechanisms ensure that pipelined and concurrent communications do not result in corrupted or out-of-order data, making the framework suitable for latency-sensitive and bandwidth-bound applications.

3.5 Evaluation

To assess the performance benefits and practical effectiveness of our proposed framework, we evaluate both the pipelined multi-path communication engine and the `CUDA` Graph-based runtime under a diverse set of benchmarks. These include synthetic micro-benchmarks as well as an application use case. The micro-benchmarks consist of `UCX` Put Bandwidth, `OMB MPI` Bandwidth (`OMB_BW`), and `OMB MPI` Bidirectional Bandwidth (`OMB_BIBW`) [21]. For application evaluation, we use the Jacobi iterative solver [82], a representative stencil computation pattern. Stencil communications, such as 2D-5 and 3D-27, are common in scientific computing and involve frequent `P2P` data exchanges between neighboring `GPUs`, making them suitable for evaluating our multi-path communication framework.

We provide results for both `UCX` and `MPI` benchmarks to highlight the performance differences between low-level communication operations and higher-level `MPI` abstractions. The `UCX Put` benchmark directly measures the raw communication capabilities of the `UCX` library, while the `MPI` benchmarks assess how these capabilities translate into real-world application scenarios using `MPI`. However, as will be discussed, the performance differences

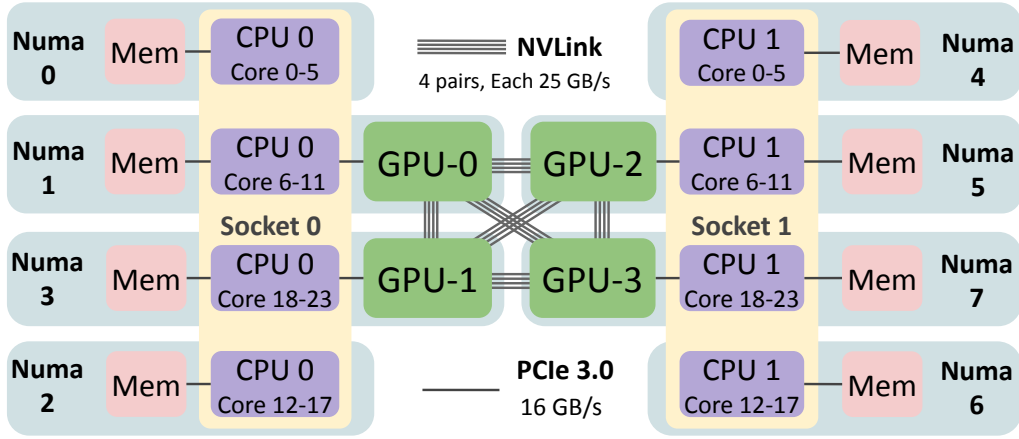


Figure 3.5: Narval node architecture.

for large message sizes are minimal between the two layers due to the dominance of data transfer times over overheads.

Moreover, to analyze the behavior and impact of various configurations of [CUDA](#) Graphs in our multi-path framework, we performed a series of experiments using [OMB MPI Latency](#) tests. These tests help us understand how different graph structures and sizes affect performance, particularly in terms of latency reduction and overhead minimization in various stages of a [CUDA](#) Graph lifecycle, including creation, construction, instantiation, and launch.

3.5.1 Experimental Setup

All experiments were conducted on two multi-[GPU](#) compute nodes from the Digital Research Alliance of Canada clusters: Beluga and Narval, as illustrated in [Figure 3.1](#) and [Figure 3.5](#), respectively. The specifications of these nodes are as follows:

- **Beluga:** A dual-socket node with two NVIDIA V100 [GPUs](#) per socket, connected via two bidirectional NVLink links in a full-mesh topology. Each socket houses an Intel Xeon Gold 6248R processor with 24 cores and hyper-threading enabled.

- **Narval:** An eight-*Non-Uniform Memory Access* (NUMA) node equipped with four NVIDIA A100 GPUs, with a full mesh topology and four pairs of NVLink between any two GPUs. The CPUs are two AMD EPYC 7413 with total of 24 cores with hyper-threading disabled. On this system, each GPU is connected to a single NUMA node, which consists a single memory channel and only 6 cores of one of the CPUs.

Both systems run UCX version 1.14.0. For MPI support, we use Open MPI version 5.0.4 for both the non-CUDA Graph and CUDA Graph-based evaluations. In all tests, unless otherwise noted, we use pinned host memory and CUDA IPC for all device memory allocations and transfers. We also perform these experiments 1000 times and report the average results. The error bars are not shown in the figures as they are negligible comparing to the average values.

3.5.2 UCX Put Bandwidth Evaluation

Figure 3.6 shows the performance of UCX Put operations measured with two ranks, one on each GPU, on both Beluga (top) and Narval (bottom) clusters. We compare the baseline performance of default UCX (UCT::CUDA-IPC) against our pipelined multi-path and CUDA Graph-based framework. The key observations include:

1. When using three GPU paths and optionally including a host-staging path, our approach achieves up to 2.85× and 2.75× bandwidth improvements over the baseline on Beluga and Narval, respectively, for message sizes larger than 32MB, which is important for bandwidth-bound applications such as LLM and DL workloads.
2. The integration of CUDA Graphs further enhances performance, especially for larger data sizes, due to the increased number of nodes/operations in the CUDA Graph comparing to the small data sizes. The results show up to 2.95× and 2.8× speedup on Beluga and Narval, respectively, for the same size range. The reason for more

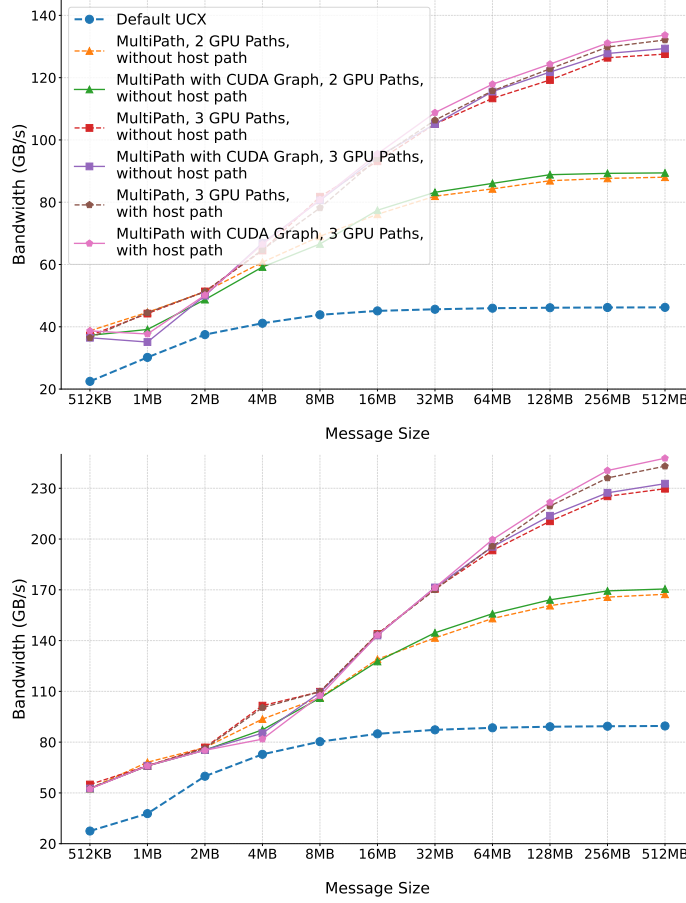


Figure 3.6: UCX Unidirectional Put Bandwidth comparison of our multi-path framework with and without CUDA Graph against default UCX (UCT::CUDA-IPC), on Beluga (top) and Narval (bottom)

improvement for larger message sizes is due to the fact that the number of operations in the [CUDA](#) Graph is larger. For instance, for a 128MB message size, where we use three [GPU](#) paths and four chunks per path (on average) the created [CUDA](#) Graph has 16 memory copy nodes. Packing more operations into a single [CUDA](#) Graph reduces the launch overhead and synchronization costs on the CPU side, leading to better performance.

3. Including the host path contributes marginally to the bandwidth improvement (up to 15%), which is expected given the lower performance of [PCIe](#) relative to NVLink.

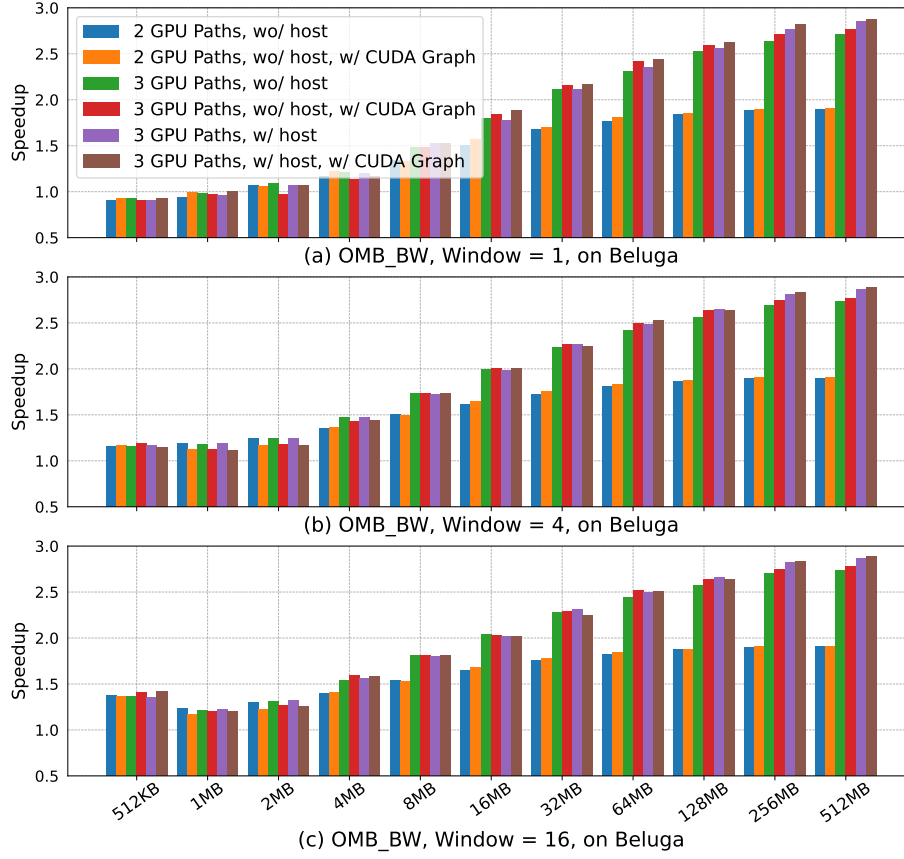


Figure 3.7: Multi-Path OMB Unidirectional MPI Bandwidth (BW) comparison with and without CUDA Graph on Beluga

In many scenarios, this contribution is negligible compared to enabling additional direct GPU paths.

4. To avoid clutter in the figure, intermediate configurations (e.g., one or two GPU paths with host path enabled) are omitted, since they show similar patterns to the configurations with three GPU paths.

3.5.3 MPI Micro-benchmark Results

Figure 3.7 and Figure 3.8 present OMB MPI unidirectional bandwidth (BW) results on Beluga and Narval, respectively. Also, Figure 3.9 and Figure 3.10 depict the bidirectional

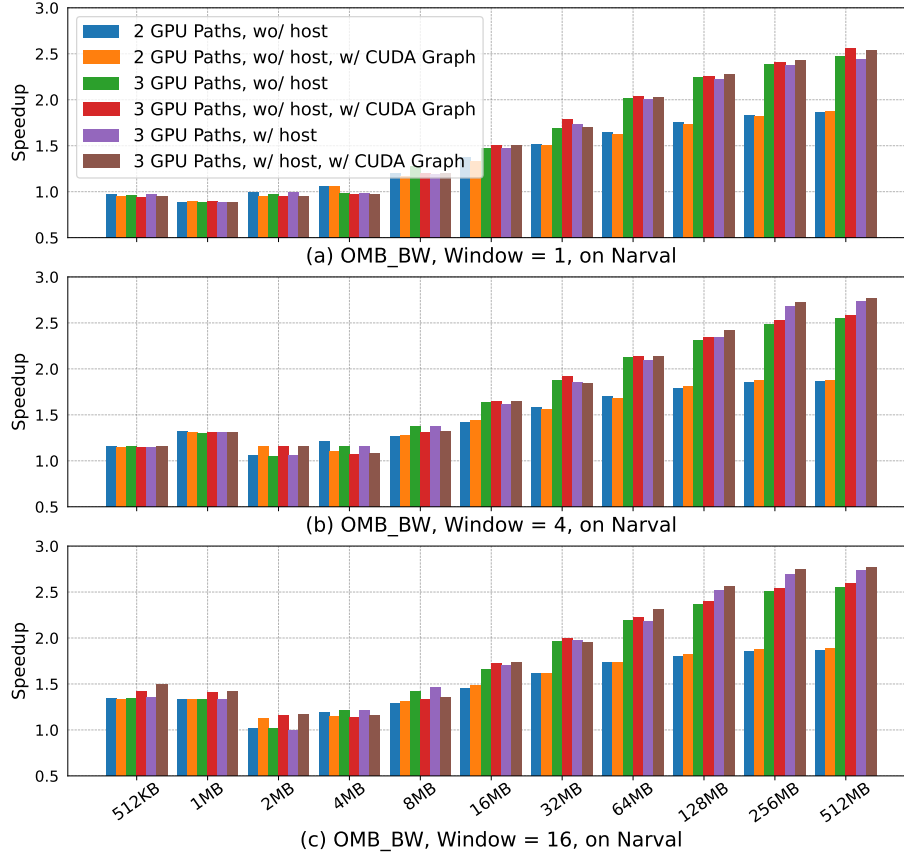


Figure 3.8: Multi-Path OMB Unidirectional MPI Bandwidth (BW) comparison with and without CUDA Graph on Narval

bandwidth (BIBW) tests for both nodes. These tests were conducted with two [MPI](#) ranks, each assigned to one [GPU](#), and various message sizes ranging from 1MB to 64MB. All tests were conducted across different message sizes and *window* sizes (1, 4, and 16). The *window* size defines how many messages can be posted without waiting for prior ones to complete.

We observe the following key results:

1. On Beluga, results are consistent with those observed in the [UCX](#) Put test. On Narval, however, we see less improvement for window size one, which suggests that its four NVLinks do not saturate as quickly as Beluga's two NVLinks. This pattern is

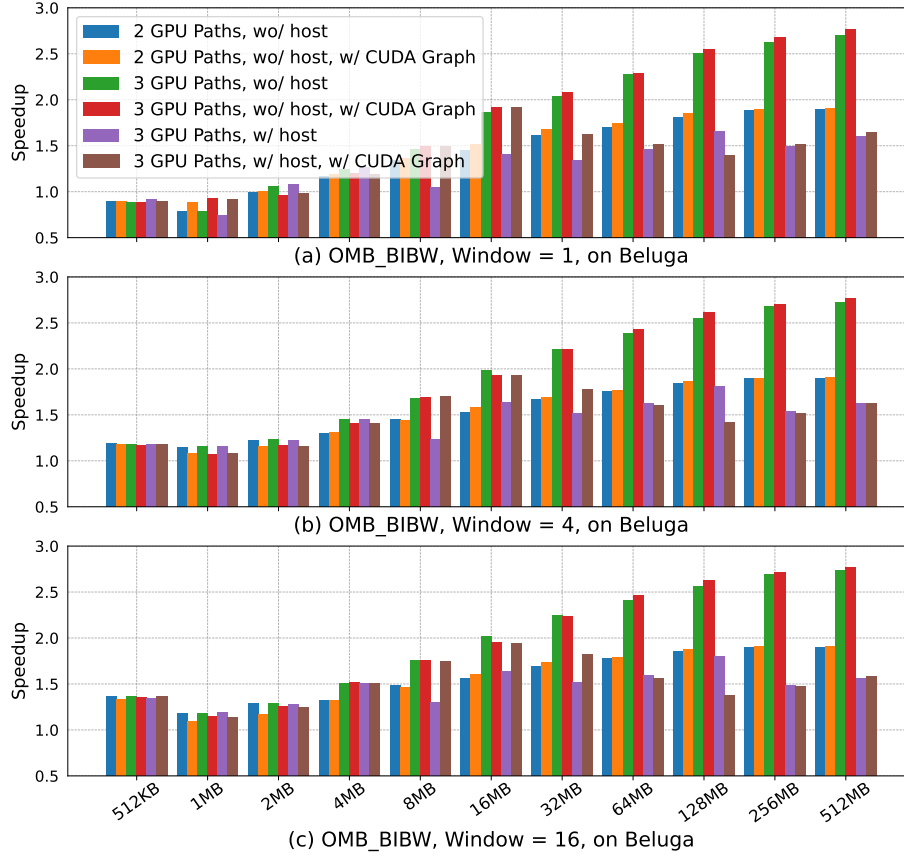


Figure 3.9: Multi-Path OMB Bidirectional MPI Bandwidth (BIBW) comparison with and without CUDA Graph on Beluga

observable in both classic and [CUDA](#) Graph-enhanced versions.

2. For both platforms, bandwidth utilization increases with higher *window* sizes, particularly for message sizes between 8MB and 64MB. This trend is visible in both unidirectional and bidirectional bandwidth plots. This is because larger *window* sizes allow more messages to be in-flight, better utilizing the available bandwidth and overlapping communication with computation.
3. For messages larger than 8MB, enabling [CUDA](#) Graph enhances performance even further, especially when the *window* size is increased to 4 or 16. [CUDA](#) Graph allows for overlapping transfers and reduced launch overhead, leading to better performance.

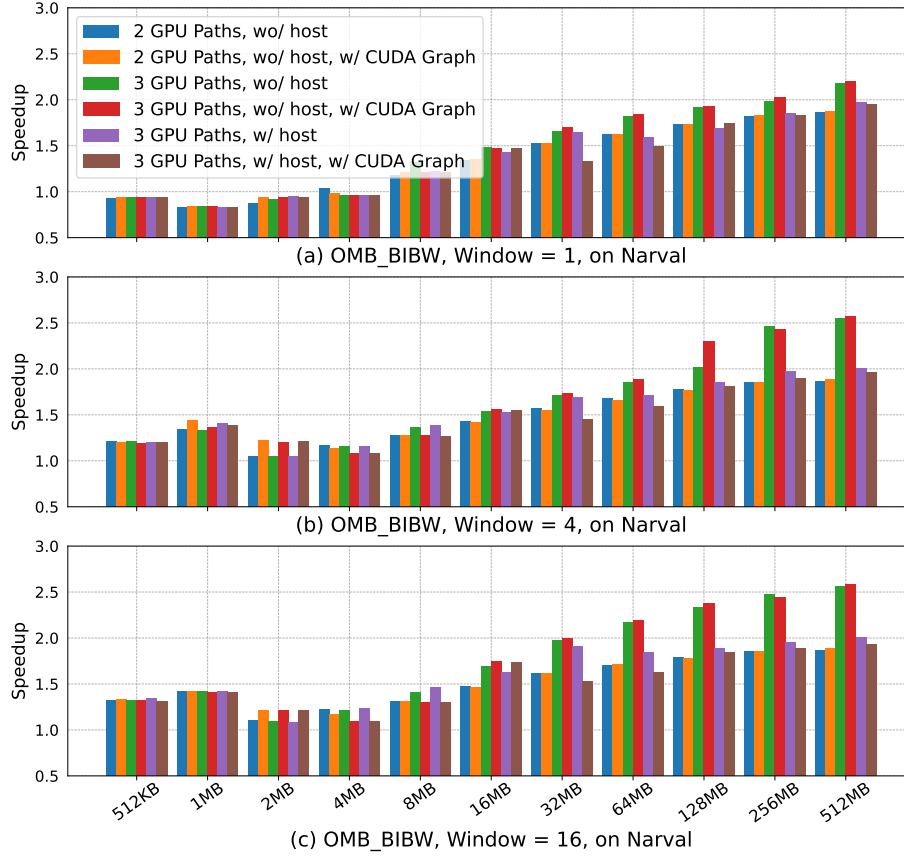


Figure 3.10: Multi-Path OMB Bidirectional MPI Bandwidth (BIBW) comparison with and without CUDA Graph on Narval

This observation suggests that applications that issue multiple outstanding/non-blocking requests (i.e., larger *window* sizes) can benefit more from [CUDA](#) Graph integration. This means that multiple [CUDA](#) Graphs can be launched concurrently, allowing the [CUDA](#) runtime to better schedule and overlap operations.

4. In contrast, for messages smaller than 8MB, the number of operations in the [CUDA](#) Graph is small, and the graph launch overhead becomes significant, negating its benefits. This can be seen in the performance drop for message sizes 512MB and 4MB in Figure 3.7(a), Figure 3.7(b), and Figure 3.7(c), where we have only one or two copy nodes in the [CUDA](#) Graph. This pattern is also observable in Narval results

and in bidirectional tests.

5. Enabling host paths generally does not improve performance significantly, especially on Narval with window size one (see Figure 3.8(d) and Figure 3.10(c)). On Narval, the difference in bandwidth between NVLink and the host-staging path is more prominent than Beluga, and the host path contribution to overall bandwidth is minimal.
6. In bidirectional tests, enabling the host path consistently degrades performance, especially in CUDA Graph versions. This is because both directions share the same PCIe link to the host, creating contention and bottlenecks. This pattern is more pronounced with higher *window* sizes, as more messages are in-flight. See Figure 3.9(c) and Figure 3.10(c) for instance.

We also observe performance drops for message sizes of 1MB and 2MB, which we suspect are caused by internal algorithm or protocol switches within UCX or MPI. These observations are consistent across both clusters and in both unidirectional and bidirectional tests, especially with larger *window* sizes.

3.5.4 Jacobi Iterative Solver

To validate the framework in an application scenario, we evaluated the performance of the Jacobi iterative solver using MPI with one rank per GPU (a total of four ranks per node). Each rank computes a subregion of the domain and exchanges boundary data with its immediate neighbors using a 2-D halo exchange pattern, as shown in Figure 3.11(a), which is basically a *ring*. Considering the unutilized diagonal links, and the NVLink’s bidirectional feature, 2/3 of the total available bandwidth is unused in this pattern. Therefore, we can enable multi-path communication for all of these data exchanges, and select the staging GPUs in a way that there is no contention on the NVLinks. Figure 3.11(b) demonstrates

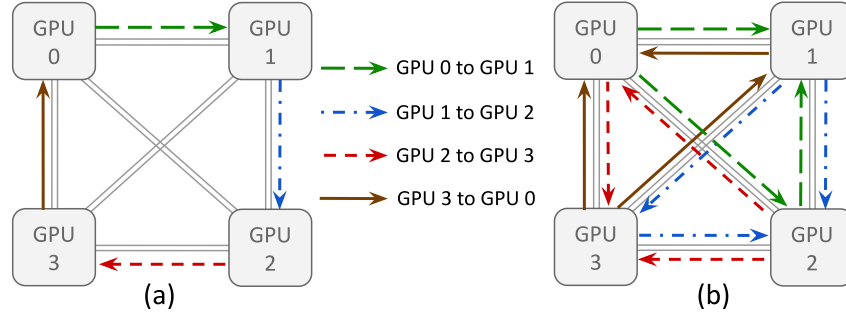


Figure 3.11: Jacobi communication pattern (a) without multi-path, and (b) with multi-path (two paths per communication) on a four-GPU node

how each communication is split into two paths, and the data is transferred concurrently through the NVLinks.

We varied the problem size by fixing the vertical dimension to 8 and increasing the horizontal dimension from 2^{23} to 2^{30} . This means that for the total application data size of 8GB on four GPUs, each rank exchanges 256MB of boundary data with its two neighbors in each iteration. We ran the solver for 1000 iterations and measured the total execution time. We also disabled host staging for this evaluation, as it consistently degraded performance in previous tests. The results, shown in Figure 3.12, are compared against the default UCX (UCT::CUDA-IPC) configuration. The key findings indicate:

1. Using two concurrent paths per communication improves application runtime by up to $1.26\times$ and $1.15\times$ on Beluga and Narval, respectively.
2. Integration of CUDA Graphs results in further performance gains, achieving up to $1.28\times$ and $1.16\times$ speedup on Beluga and Narval, respectively, for larger problem sizes.
3. As discussed earlier, the benefits of CUDA Graphs are more pronounced for larger messages due to the increased number of operations in the graph, which helps amortize the launch overhead. For smaller problem sizes, the overhead of CUDA

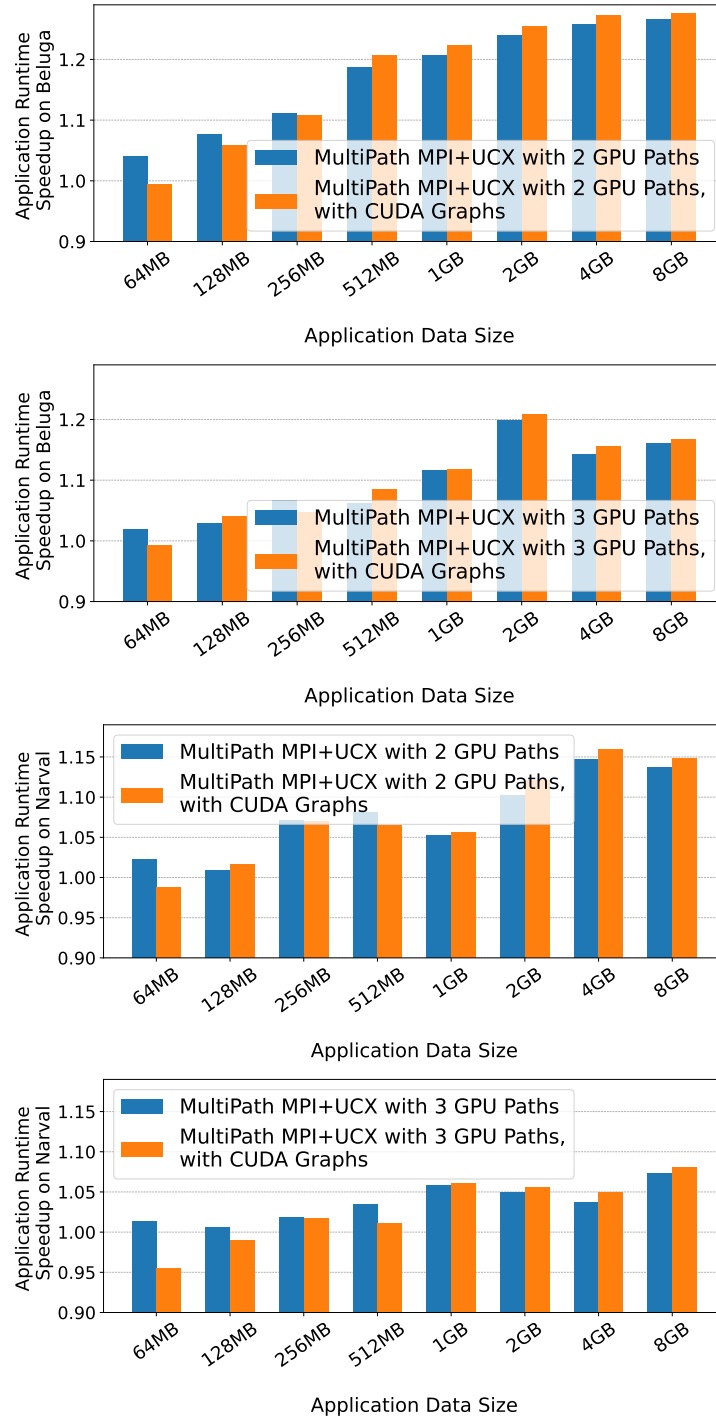


Figure 3.12: Jacobi runtime speedup over default UCX (UCT::CUDA-IPC) using four MPI ranks on Beluga and Narval clusters

Graph creation and launch can outweigh its benefits, leading to marginal or no improvement: for application data sizes smaller than 1GB (message size = 32MB), in which the number of operations in the **CUDA** Graph is insufficient to amortize the overhead. This is also consistent with our observations in the bidirectional micro-benchmarks (Section 3.5.3).

4. Even with three **GPU** paths per transfer, potentially incurring additional contention, our framework still improves the runtime. Although the gains are smaller than with two paths, we still observe improvements of up to 1.2× and 1.08× on Beluga and Narval, respectively. This suggests that the benefits of increased concurrency do not outweigh the contention costs in this scenario, and two paths may be the optimal choice.
5. It is important to note that in this application we are only using the **CUDA** Graphs for enabling multi-path transfers, and none of the other operations (e.g., computation kernels) are included in the **CUDA** Graph. This is because our framework is designed in the **UCX** layer, which is agnostic to higher-level application operations, and we do not have control over the application kernels to include them in the **CUDA** Graph. The communication patterns created and launched in this test map to a graph which has a very small number of vertices along each path (only two nodes per chunk per path). This limits the potential for optimization from **CUDA** Graph utilization. These results show that in scenarios with more complex communication patterns, the advantages of **CUDA** Graphs could be more significant, especially when kernel executions are involved.
6. Importantly, Jacobi’s numerical convergence remains unaffected, demonstrating the correctness of our pipelined and concurrent transfer mechanisms.

3.5.5 CUDA Graph Overhead Analysis

In this section, we characterize the overheads associated with various **CUDA** Graph operations in our multi-path framework, including creation, construction, instantiation, and launch. This analysis helps us understand the trade-offs involved in using **CUDA** Graphs, particularly in terms of latency reduction and overhead minimization for different message sizes and number of nodes in the graph. These insights can guide users in deciding when and how to effectively utilize **CUDA** Graphs in applications with multi-path communication patterns.

We collected timing data during the **OMB Latency** micro-benchmark on Narval for dual-path communication with message sizes between 2MB and 512MB (note that multi-pathing starts at 2MB with only two nodes in the graph). In more detail, these are the various phases:

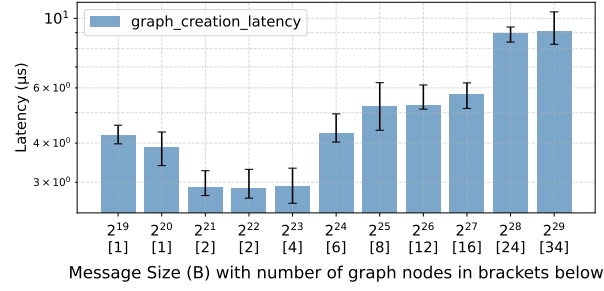
1. **Creation** involves allocation and initialization of the **CUDA** Graph object during the first time execution (`cudaGraph_t`). For subsequent executions, this step is skipped, and the existing **CUDA** Graph object is fetched from a hash table and reused.
2. **Construction** includes defining and creating the graph nodes, which denotes the data movements, as well as their inter-dependencies. We use the **CUDA** Graph explicit **API** to construct the **CUDA** Graph design and update the existing **CUDA** Graph object. Similar to *creation*, this step is only performed during the first execution, and the constructed graph is reused in subsequent executions.
3. **Instantiation** is a one-time expensive operation that allocates resources for the **CUDA** Graph execution and optimizing it. After this step, the *instance* object is created, which can be launched multiple times (`cudaGraphExec_t`). This step is synchronous to the main CPU thread, and its overhead can vary based on the

complexity of the graph.

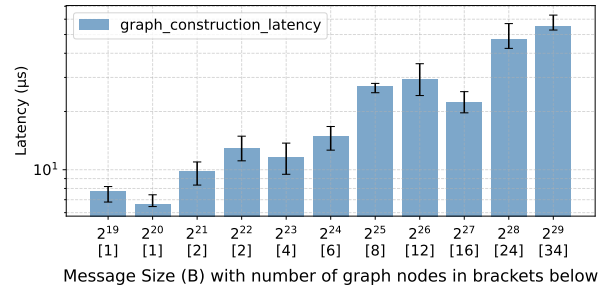
4. **Launch** refers to the initiation of the actual execution of the **CUDA** Graph on the **GPU**. Similar to a **CUDA** kernel launch, this process is asynchronous and expensive, and its overhead might vary depending on the complexity of the graph.

Figure 3.13, Figure 3.14, Figure 3.15, Figure 3.16, and Figure 3.17 demonstrate how the overhead of various **CUDA** Graph operations changes according to the number of nodes within the graph when they are called for the first/subsequent times. Analyzing the results, we observe the following:

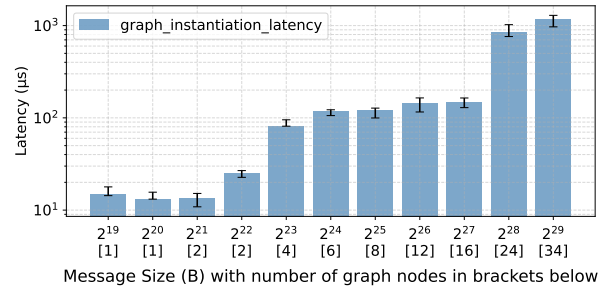
- **Observation 1:** In all the graph phases during the first iteration, the overhead increases with the number of nodes. See Figure 3.13(a), Figure 3.13(b), Figure 3.13(c), and Figure 3.13(d) for *creation*, *construction*, *instantiation*, and *launch*, respectively. This is expected, as more nodes imply more operations and dependencies to manage. Notably, *instantiation* depicts the highest overhead, reaching up to 3ms for 34 nodes (512MB message size with dual-path communication).
- **Observation 2:** During the subsequent calls, the overhead stays constant and low for all the cases, except for the launch phase. Figure 3.14 shows how the launch overhead increases with the number of nodes (figures of other phases are not provided here for brevity).
- **Observation 3:** Contribution of each graph phase to the total latency varies significantly between the first and subsequent iterations. While *instantiation* is the most dominant operation during the first iteration, the *launch* phase becomes more significant in subsequent iterations. Compare Figure 3.15 and Figure 3.16.
- **Observation 4:** The overall latency benefits from the multi-path optimization become more pronounced as the message size increases. Although the overhead of



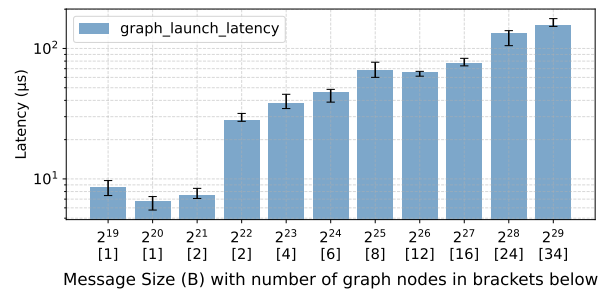
(a) Overhead of the first creation.



(b) Overhead of the first construction.



(c) Overhead of the first instantiation.



(d) Overhead of the first launch.

Figure 3.13: Measurement of various CUDA Graph operations during the first iteration of OMB Latency benchmark on Narval for dual-path communication from message sizes 2MB to 512MB.

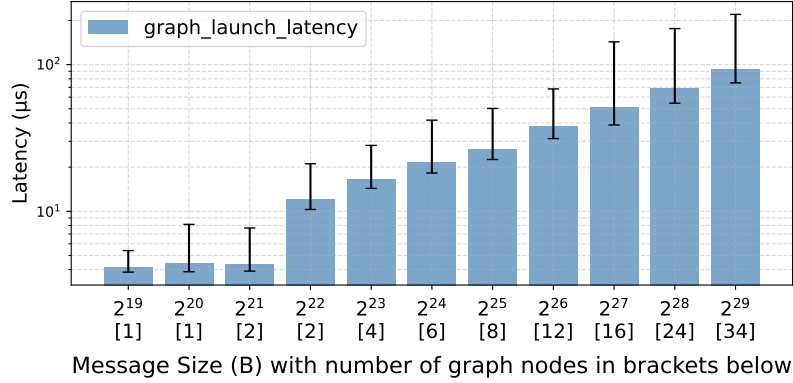


Figure 3.14: Overhead of the subsequent CUDA Graph launches for various node counts during OMB Latency benchmark on Narval for dual-path communication from message sizes 2MB to 512MB.

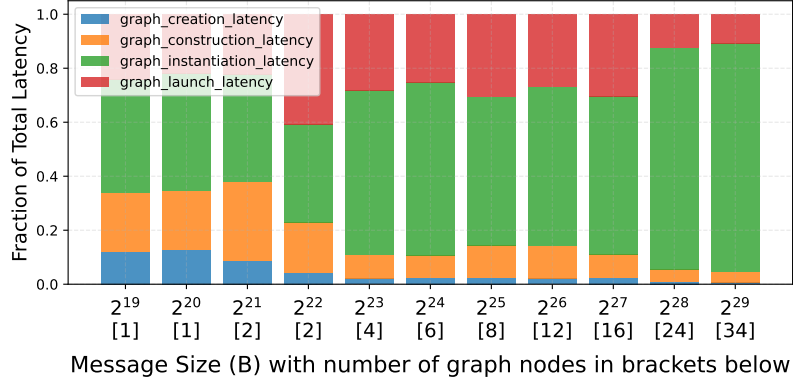


Figure 3.15: Contribution of each graph phase to the total latency (first iteration) during OMB Latency benchmark on Narval for dual-path communication from message sizes 2MB to 512MB.

CUDA Graph operations increases with node count (and message size), the relative impact on latency decreases (See Figure 3.17). These results suggest that CUDA Graphs provide scalable performance improvements as the complexity of the communication pattern increases.

Overall, these results suggest that while CUDA Graph operations introduce some overhead, especially during the first iteration, the benefits of multi-path communication outweigh these costs for larger messages. As we saw, the overhead of *creation*, *construction*, and

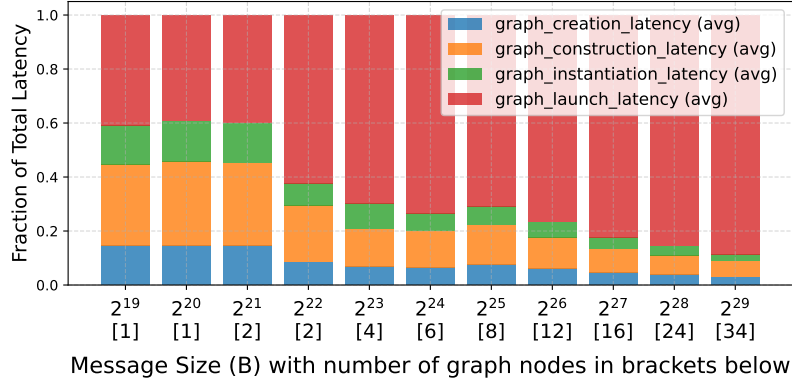


Figure 3.16: Contribution of each graph phase to the total latency (subsequent iterations) during OMB Latency benchmark on Narval for dual-path communication from message sizes 2MB to 512MB.

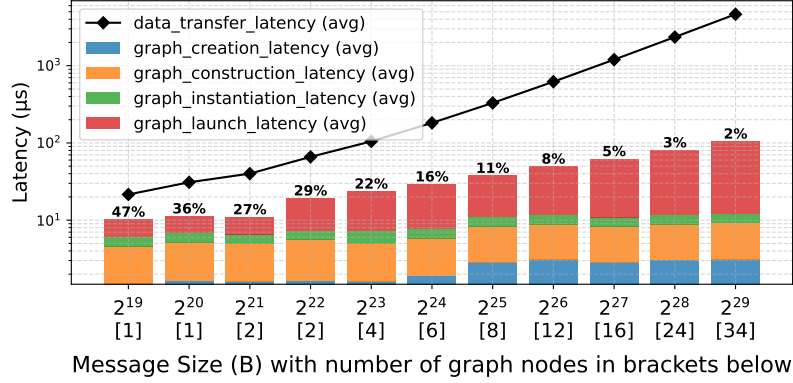


Figure 3.17: Comparison of subsequent CUDA Graph phases with the CUDA Graph enabled multi-path transfer during OMB Latency benchmark on Narval for dual-path communication from message sizes 2MB to 512MB.

instantiation is amortized over multiple launches, making them negligible for long-running applications. This is why libraries such as [UCX](#) can benefit from [CUDA](#) Graphs, as they can create and instantiate the graphs once and reuse them multiple times for recurring communication patterns.

3.6 Summary

We proposed a multi-path communication framework implemented within the [UCX](#) library. By leveraging available communication channels, our 2-D pipelining engine scatters [P2P](#) communication across both NVLink and [PCIe](#) channels to maximize communication bandwidth between intra-node [GPUs](#). We also provide end users with tuning capabilities for scheduling parameters, ensuring adaptability to diverse communication patterns and hardware configurations.

In our experiments, we observed up to 2.85x and 2.75x improvement in bandwidth tests for Beluga and Narval clusters, respectively, when using the host path and two [GPU](#) paths. We also observed that besides utilizing unused interconnects, harnessing NVLink’s bidirectional features will also improve communication performance. However, the lack of the same feature in [PCIe](#) communication channels may lead to contention, and consequently, performance degradation. Finally, we showed that our multi-path communication framework can improve the performance of the Jacobi iterative solver by up to 1.26x.

We extended our framework with a [CUDA](#) Graph-based design that constructs and reuses communication workflows to minimize launch overhead and improve intra-node communication efficiency. Our experimental evaluation on a four-[GPU](#) node of Beluga and Narval demonstrates that this approach achieves up to a 2.9x increase in [GPU-to-GPU OMB MPI](#) bandwidth test compared to traditional methods. We showed that the benefits of [CUDA](#) Graphs were marginal relative to the non-[CUDA](#) Graph multi-path framework, due to the limited number of operations in the graph.

Furthermore, we discussed the overheads associated with various [CUDA](#) Graph operations in the context of multi-path communication, including creation, construction, instantiation, and launch. Our analysis indicates that while these operations introduce some overhead,

particularly during the first iteration, the benefits of **CUDA** Graph-enabled multi-path communication outweigh these costs for larger messages, and the overheads are amortized over multiple launches, making them negligible for long-running applications.

In the next chapter, we present an analytical performance model for multi-path intra-node **GPU** communication that predicts data transfer times under pipelined, staged intra-node **GPU** transfers. We then integrate this model into the **MPI + UCX** communication stack for automatic runtime tuning and validate its accuracy and effectiveness.

Chapter 4

Accelerating Intra-Node GPU Communication: A Performance Model for Multi-Path Transfers

4.1 Introduction

In modern HPC systems, GPU accelerators form the core of computational resources, and efficient intra-node GPU-to-GPU communication is critical for achieving high overall performance [122, 148, 133]. Multi-GPU nodes are now equipped with advanced interconnects (e.g., NVIDIA NVLink/NVSwitch) to meet the demand for low-latency, high-bandwidth data exchange [100]. For example, recent top-ranked supercomputers feature up to 8 GPUs per node connected by multiple NVLink channels with aggregate bandwidths in the terabits-per-second range [145, 122]. Optimizing data transfers in the intra-node context, by fully exploiting all available communication paths, is therefore essential to achieve the full compute potential of modern HPC applications.

Despite the availability of multiple GPU communication paths, utilizing them for intra-node GPU communication is a complex problem. In practice, communication libraries, such as MPI [103], UCX [150], and NCCL [99], default to a single link for intra-node GPU

communication. Moreover, while some studies propose GPU multi-path communication strategies, like our work in Chapter 3, they lack analytical modeling for multi-path intra-node transfers, and they perform exhaustive search to find the best configuration for each topology [133, 88, 138, 87, 139]. The challenge is further compounded by the fact that the communication paths are heterogeneous, with different latencies and bandwidths, and the need to manage data staging and synchronization across these paths.

The importance of communication modeling in HPC is well-established, and significant efforts have been made to model and optimize data transfers, including CPU-GPU transfers, inter-node multi-GPU communication, and various collective operations [43, 71, 126, 116, 18, 144, 75]. Classical latency-bandwidth models (such as the *Hockney's* model) assume a single transfer path and do not capture concurrent use of channels [48, 4, 31]. More advanced models, address heterogeneity, or middleware costs but they do not target the specific problem of intra-node multi-path communication [10, 120, 157, 62, 125, 22].

In this dissertation, we address these gaps by proposing a predictive performance model for multi-path intra-node GPU communication [129]. Our model extends the classical *Hockney's* framework to the multi-channel GPU setting, incorporating the latencies and bandwidths of each NVLink and PCIe channel/link as well as the effects of pipelined, staged transfers. Given hardware parameters and message size, the model analytically computes end-to-end transfer time for candidate multi-path configurations, enabling dynamic selection of the optimal strategy without exhaustive search. We show that this model accurately predicts the best (or near-best) configuration, enabling auto-tuning of intra-node GPU communication.

Furthermore, we integrate our model into the UCX framework, based on the multi-path engine introduced in [133, 134]. We validate our model using a set of tests from OMB [21] on a multi-GPU node with NVLink and PCIe interconnects. Our results show that

this model can accurately predict the optimal configuration for intra-node GPU P2P communication, achieving significant speedup over single-path methods. Our contributions are as follows:

1. We propose a performance model for multi-path intra-node GPU communication, enabling the dynamic configuration of data transfers across multiple path [129]. To the best of our knowledge, this is the first analytical model for multi-path intra-node GPU communication, predicting data transfer times under pipelined, staged intra-node GPU transfers.
2. We prove that the optimal multi-path schedule (minimizing overall transfer time) is achieved when the per-path transfer times are equal (or nearly equal).
3. We integrate the performance model into the MPI +UCX communication stack for automatic runtime tuning over the pipeline engine developed in Chapter 3 [133].
4. We validate our model using a set of P2P and collective micro-benchmarks on a multi-GPU node with NVLink and PCIe interconnects, achieving a mean error below 6% in predicting the optimal configuration for messages larger than 4MB. We demonstrate the effectiveness of the model-enabled multi-path communication strategy in both P2P and collective operations (`MPI_Allreduce` and `MPI_Alltoall`), achieving up to 2.9× and 1.4× speedup, respectively, over the single-path methods.

The rest of this chapter is structured as follows. Section 4.2 covers relevant background and related work. Section 4.3 presents our performance model for multi-path intra-node GPU communication, and Section 4.4 includes its design and integration into the MPI+UCX stack. Section 4.5 presents experimental results and performance analysis, and Section 4.6 concludes the chapter.

4.2 Background and Related Work

4.2.1 GPU-aware MPI Communication

MPI is one of the most widely used communication libraries in **HPC**, which provides an extensive set of communication primitives for distributed applications [80]. Configured with **UCX** and **UCC**, **MPI** implementations, such as Open **MPI** and **MPICH**, provide a performant communication framework for **GPU**-accelerated applications [103, 81, 150, 149, 153, 124].

The **UCX** library handles intra-node **CUDA IPC** by its internal module, named `cuda_ipc`, which is responsible for managing **CUDA IPC** transfers and caching the **CUDA IPC** handles translations [150, 124]. This module is designed to perform **GPU-to-GPU** data transfers between **MPI** processes as one-sided **PUT** or **GET** operations. In this way, **UCX** can efficiently transfer data between **GPUs** without involving the **CPU**, which is particularly beneficial for applications with large data transfers. Although this research is developed using **NVIDIA GPU** hardware and their **CUDA** platform, the proposed model can be applied to any other **GPU** architecture with similar multi-path communication capabilities.

Figure 3.1(a) demonstrates a typical four-**GPU** node with **NVLink** interconnects. In this configuration, each pair of **GPUs** has two **NVLink** sub-links, and the **GPUs** are connected to the **CPUs** through **PCIe**. As discussed in Chapter 3, if a communication occurs between **GPU-0** and **GPU-1** of this setup, the data transfer can be split into smaller chunks and routed through multiple available paths concurrently, as shown in Figure 3.1(b). This approach allows for better utilization of the available bandwidth and reduces the overall transfer time when the direct link between the two **GPUs** is saturated. As will be discussed in Section 4.5, harnessing host-side available paths (**PCIe** lanes) may have an adverse effect on performance when they are already used by other processes or threads.

Previous studies demonstrated that when a communication channel (e.g., GPU-0 to GPU-1) reaches its bandwidth limit, the data transfer can be divided into smaller chunks and routed through multiple available paths concurrently to gain considerable improvements [133, 134, 138, 139]. However, managing such concurrent communication requires careful handling of data staging, synchronization, and ordering to maintain data integrity and achieve optimal performance. As the number of paths and chunks per path increases, the complexity of managing these aspects also increases, and finding the optimal configuration becomes a challenge.

4.2.2 Communication Performance Modeling

Communication performance modeling provides analytical models for the evaluation of communication costs based on various platform-specific parameters. Over the past decades, numerous models were invented, ranging from homogeneous to heterogeneous clusters. These models aim to predict the communication time accurately, guiding algorithm design, and informing architectural decisions. Some of the notable models include: *Hockney* [48], *LogP* [31], *LogGP* [49], *LogSC* [158], *Max-rate* [45], and τ -*Lop* [117].

One of the foundational models in communication performance is the *Hockney*'s linear model [48], which is a good starting point for understanding communication costs due to its simplicity and effectiveness. Due to its simplicity, it has played an important role in several research directions, such as the evaluation and optimization of collective algorithms, improving collective performance in switched networks, and studying collectives across different topologies in homogeneous/heterogeneous clusters [62, 90, 89, 107]. The *Hockney*'s model is also used, in slightly less related areas to this study, as the network model in simulation tools like SimGrid, and it forms the foundation for scalability analysis in benchmarks like High Performance Linpack (HPL) [116].

4.2.3 GPU Communication Modeling and Optimization

The invention of GPU accelerators has initiated several research efforts into modeling and optimizing data transfers. Many of these studies initially were focused on CPU-GPU data transfers, often occurring over PCIe links. These transfers are frequently identified as performance bottlenecks, and modeling efforts have aimed to predict PCIe transfer times and develop strategies for overlapping computation and communication [43, 159]. Asynchronous data transfers using mechanisms like CUDA streams have been modeled to estimate performance improvements through overlap. Studies have also explored optimal data partitioning and granularity for CPU-GPU transfers [34, 19, 71]. Models like *mHLogGP* have been introduced specifically for CPU/GPU heterogeneous computing environments, abstracting communication and memory access to estimate program running time and guide optimization [72]. Analytical models, sometimes combined with machine learning approaches, have been used for predicting CPU-GPU transfer times, highlighting the importance of accurate predictions [114].

More recently, introduction of technologies like GPUDirect by NVIDIA has enabled direct GPU-to-GPU communication in multi-GPU nodes and clusters, which is faster than PCIe-based transfers [97]. Studies have evaluated GPU interconnect topologies and quantified the benefits of these technologies, through various performance analyses [18, 60]. Several studies have evaluated distributed deep learning frameworks on multi-GPU and multi-node systems, highlighting the impact of communication overhead on training efficiency [126, 62]. Efforts to optimize communication in multi-GPU systems also include model-based approaches for auto-tuning and scheduling in domains like linear algebra (BLAS) and Conjugate Gradient (CG), addressing data transfer bottlenecks in multi-GPU systems [9, 10, 74, 22].

Optimizing communication by leveraging multiple available paths or network interfaces

is commonly studied in the context of inter-node communication, in which techniques like multi-HCA-aware designs and rail binding/sharing strategies are proposed to enhance collective communication performance [146]. These approaches aim to improve resource utilization by effectively using multiple network interfaces simultaneously, sharing a similar goal to our work of exploiting all available communication paths. However, these studies focus on inter-node communication, whereas our work targets intra-node communication and the distribution of a P2P transfer. Similarly, libraries like TCCL focus on optimizing collective communication over PCIe-dependent GPU clusters by finding optimal paths for collective algorithms [62], which differs from our focus on distributing a P2P transfer.

While the evaluated studies explore various aspects of multi-GPU communication, including inter-node scenarios and collective operations, studies on analytical performance models specifically designed for optimizing an intra-node GPU-to-GPU transfer by distributing it across multiple, heterogeneous paths (NVLink and PCIe) have not done before. Our work directly addresses this gap by providing the first analytical model for multi-path intra-node P2P GPU communication that explicitly considers heterogeneous NVLink and PCIe links for distributing a data transfer.

4.3 Performance Model for Multi-path Intra-node GPU Communication

In this section, we present our performance model based on the *Hockney's* model. Then, we extend it to account for multi-path communication and derive an optimization problem to find the optimal path configuration.

In this study, we first focus on intra-node communications that are isolated and not affected by other processes or threads. This assumption helps us to simplify the model and find

Notation	Description
T	Total communication time
n	Message size
α	Startup Latency
β	Bandwidth
p	Number of paths
T_i	Communication time of path i
α_i, β_i	Model parameters of path i
θ_i	Fraction of message on path i
ϵ_i	Synchronization overhead at staging device i
α'_i, β'_i	Parameters for the second link in staged transfer
Δ_i	$\alpha_i + \alpha'_i + \epsilon_i$
Ω_i	$1/\beta_i + 1/\beta'_i$
ϕ_i	Topology-specific constant for path i
k_i	Number of chunks used along the path i
$T_i(c)$	Time for transfer chunk c on path i

Table 4.1: Notations used in the model.

the highest achievable performance. However, in practice, intra-node GPU interconnects are often shared among multiple processes, which may lead to contention and reduced performance, but as we will discuss later, our approach still accelerates concurrent intra-node communication, including collectives, if there are any under-utilized paths. In this way, the underlying P2P communication is distributed across multiple paths, consequently improving the overall performance of the collective operation. In such cases, the model can still be applied, but the optimal configuration may vary based on the specific workload and contention patterns, and if the communication pattern (determined by the application workload or the collective communication algorithm) can be known ahead of time, unused paths can be extracted and utilized for multi-path communication more effectively.

4.3.1 Extending the Hockney's Model for Multi-Path

Communication

As discussed in Section 4.2.2, the *Hockney's* model is widely used for message-passing communication, defined as:

$$T = \alpha + \frac{n}{\beta} \quad (4.1)$$

where the parameters are defined in Table 4.1. Considering a system with p distinct available communication paths between two GPUs, we can classify these paths as:

1. **Direct GPU-to-GPU** path: A direct link connects the source and destination GPUs.
2. **GPU-Staged** path: Data can be staged through an intermediate GPU before reaching the destination.
3. **Host-Staged** path: Data can be staged through the host memory before reaching the destination GPU.

While the direct path time can be formulated using Equation (4.1), the staged paths are a combination of two direct transfers. Each path i has an associated latency α_i and bandwidth β_i . Therefore, the time to transfer a fraction θ of the message n on path i can be specified as (covering both the direct and staged paths):

$$T_i = \alpha_i + \frac{\theta_i n}{\beta_i} + \epsilon_i + \alpha'_i + \frac{\theta_i n}{\beta'_i}, \quad (4.2)$$

where α' and β' represents the parameters of the second link in a staged transfer, and ϵ_i is the overhead of synchronization on the staging device i . Note that:

$$0 \leq \theta_i \leq 1 \quad \text{and} \quad \sum_{i=1}^p \theta_i = 1 \quad (4.3)$$

Therefore, the overall time required for communication using p paths in parallel can be formulated as:

$$T = \max_{i \in \{1, \dots, p\}} T_i, \text{ where } T_i = \alpha_i + \frac{\theta_i n}{\beta_i} + \epsilon_i + \alpha'_i + \frac{\theta_i n}{\beta'_i} \quad (4.4)$$

4.3.2 Finding The Optimal Path Configuration

To find the optimal path configuration, we need to minimize the overall communication time T by adjusting the fraction θ_i of the message size n on each path i , given that we find the other parameters α_i , β_i , and ϵ_i from the system. This can be formulated as a constrained optimization problem:

$$\min_{\theta_i} T = \max_{i \in \{1, \dots, p\}} T_i, \text{ subject to } \sum_{i=1}^p \theta_i = 1 \text{ and } 0 \leq \theta_i \leq 1 \quad (4.5)$$

This optimization problem can be solved analytically, and we will prove that the optimal solution is when the communication time of all paths are equal, i.e., $T_i = T_j$ for all $i, j \in \{1, \dots, p\}$. To simplify the notation, we assume that every communication path is a direct path, i.e., the communication time of path i can be expressed as:

$$T_i = \alpha_i + \frac{\theta_i n}{\beta_i} \quad (4.6)$$

This assumption does not affect the generality of the proof, as the same principle applies to staged paths. The only difference is that the time consumption of path i will be a combination of two direct paths, as shown in Equation (4.2).

Theorem 1. Let $T_i = \alpha_i + \frac{\theta_i n}{\beta_i}$ be the communication time of path i for a message of size n . For the optimization problem (4.5), if the optimal solution is such that for any $i, j \in \{1, \dots, p\}$, $T_i \neq T_j$, we have $\alpha_i < \alpha_j + \frac{n\theta_j}{\beta_j}$, then the optimal solution is when the time consumption of

each path is equal, i.e., $T_i = T_j$ for all $i, j \in \{1, \dots, p\}$.

Proof. We prove this theorem by contradiction. Suppose that the optimal solution exists when there are paths that their times are not equal, i.e., $T_i \neq T_j$ for some $i, j \in \{1, \dots, p\}$. Without loss of generality, assume $T = T_1$ and $T_1 > T_2$. Then, we can increase θ_2 and decrease θ_1 such that $T_1 = T_2$. In more detail, to balance the communication times T_1 and T_2 , we can adjust θ_1 and θ_2 by introducing γ (the adjustment amount):

$$\theta_1^* = \theta_1 - \gamma, \quad \theta_2^* = \theta_2 + \gamma,$$

substituting into the Equation (4.6):

$$T_1^* = \alpha_1 + \frac{(\theta_1 - \gamma)n}{\beta_1}, \quad T_2^* = \alpha_2 + \frac{(\theta_2 + \gamma)n}{\beta_2}.$$

Setting $T_1^* = T_2^*$, we have:

$$\alpha_1 + \frac{(\theta_1 - \gamma)n}{\beta_1} = \alpha_2 + \frac{(\theta_2 + \gamma)n}{\beta_2}.$$

We can solve for γ :

$$\gamma = \frac{\beta_1 \beta_2}{n(\beta_1 + \beta_2)} (T_1 - T_2).$$

Now, we show that $\theta_1^\star \geq 0$ and $\theta_2^\star \leq 1$:

$$\begin{aligned}
\theta_1^\star &= \theta_1 - \gamma, \\
&= \theta_1 - \frac{\beta_1\beta_2}{n(\beta_1 + \beta_2)} (T_1 - T_2), \\
&= \theta_1 - \frac{\beta_1\beta_2}{n(\beta_1 + \beta_2)} \left(\alpha_1 - \alpha_2 + \frac{n\theta_1}{\beta_1} - \frac{n\theta_2}{\beta_2} \right), \\
&> \theta_1 - \frac{\beta_1\beta_2}{n(\beta_1 + \beta_2)} \left(\frac{n\theta_2}{\beta_2} + \frac{n\theta_1}{\beta_1} - \frac{n\theta_2}{\beta_2} \right), \\
&= \theta_1 - \frac{\beta_1\beta_2}{n(\beta_1 + \beta_2)} \left(\frac{n\theta_1}{\beta_1} \right), \\
&= \theta_1 - \frac{\beta_2}{\beta_1 + \beta_2} \theta_1, \\
&= \theta_1 \left(\frac{\beta_1}{\beta_1 + \beta_2} \right) \\
&> 0,
\end{aligned} \tag{4.7}$$

where we have used the assumption that $\alpha_i < \alpha_j + \frac{n\theta_j}{\beta_j}$ for all $i, j \in \{1, \dots, p\}$ for the inequality in the 4th line. Now for $\theta_2^\star$:

$$\begin{aligned}
\theta_2^\star &= \theta_2 + \gamma, \\
&= \theta_2 + \frac{\beta_1\beta_2}{n(\beta_1 + \beta_2)} (T_1 - T_2), \\
&= \theta_2 + \frac{\beta_1\beta_2}{n(\beta_1 + \beta_2)} \left(\alpha_1 - \alpha_2 + \frac{n\theta_1}{\beta_1} - \frac{n\theta_2}{\beta_2} \right), \\
&< \theta_2 + \frac{\beta_1\beta_2}{n(\beta_1 + \beta_2)} \left(\frac{n\theta_2}{\beta_2} + \frac{n\theta_1}{\beta_1} - \frac{n\theta_2}{\beta_2} \right), \\
&= \theta_2 + \frac{\beta_1\beta_2}{n(\beta_1 + \beta_2)} \left(\frac{n\theta_1}{\beta_1} \right), \\
&= \theta_2 + \frac{\beta_2}{\beta_1 + \beta_2} \theta_1, \\
&< \theta_2 + \theta_1, \\
&< 1.
\end{aligned} \tag{4.8}$$

This shows that $\theta_1^* \geq 0$ and $\theta_2^* \leq 1$.

Now, we compute the new time consumption of path 1 and path 2:

$$\begin{aligned}
T_1^* &= \alpha_1 + \frac{\theta_1^* n}{\beta_1} = \alpha_1 + \frac{\theta_1 n}{\beta_1} - \frac{\gamma n}{\beta_1} \\
&= T_1 - \frac{\beta_2}{\beta_1 + \beta_2} (T_1 - T_2) \\
&= \frac{\beta_1 T_1 + \beta_2 T_2}{\beta_1 + \beta_2},
\end{aligned} \tag{4.9}$$

and

$$\begin{aligned}
T_2^* &= \alpha_2 + \frac{\theta_2^* n}{\beta_2} = \alpha_2 + \frac{\theta_2 n}{\beta_2} + \frac{\gamma n}{\beta_2} \\
&= T_2 + \frac{\beta_1}{\beta_1 + \beta_2} (T_1 - T_2) \\
&= \frac{\beta_1 T_1 + \beta_2 T_2}{\beta_1 + \beta_2}.
\end{aligned} \tag{4.10}$$

This shows that $T_1^* = T_2^*$ and $T^* < T_1$ which contradicts the assumption that T_1 is the optimal solution. Therefore, the optimal solution is when the time consumption of each path is equal, i.e., $T_i = T_j$ for all $i, j \in \{1, \dots, p\}$. $\square$

Remark 1. The assumption in the theorem is indeed valid. Without this assumption, there would exist an optimal solution in which, for some $i, j \in \{1, \dots, p\}$, the inequality $\alpha_i > \alpha_j + \frac{n\theta_j}{\beta_j}$ holds. This implies that path i is less efficient than path j to the extent that no data would be transferred through path i in the optimal solution. In such a case, path i can be excluded from the set of available paths, and the optimal solution computed over the reduced set.

Now, we can find the optimal values of θ_i that minimizes the overall communication time T . According to Theorem 1, the optimal solution is achieved when the communication

time across all paths is equal, i.e., $T_i = T_j$ for all $i, j \in \{1, \dots, p\}$. Therefore, we have:

$$\alpha_1 + \frac{\theta_1 n}{\beta_1} = \alpha_i + \frac{\theta_i n}{\beta_i},$$

for all $i \in \{1, \dots, p\}$. This implies that:

$$\theta_i = \frac{\beta_i(\alpha_1 - \alpha_i)}{n} + \frac{\beta_i}{\beta_1} \theta_1. \quad (4.11)$$

Substituting Equation (4.11) into the constraint $\sum_{i=1}^p \theta_i = 1$, we get:

$$\sum_{i=1}^p \left(\frac{\beta_i(\alpha_1 - \alpha_i)}{n} + \frac{\beta_i}{\beta_1} \theta_1 \right) = 1$$

Summing up the terms, we get:

$$\theta_1 = \frac{\beta_1}{\sum_{i=1}^p \beta_i} \left(1 - \frac{\alpha_1}{n} \sum_{i=1}^p \beta_i + \frac{1}{n} \sum_{i=1}^p \alpha_i \beta_i \right)$$

Therefore, the optimal values of θ_i that minimizes the overall communication time T is:

$$\theta_i = \frac{\beta_i}{\sum_{j=1}^p \beta_j} \left(1 - \frac{\alpha_i}{n} \sum_{j=1}^p \beta_j + \frac{1}{n} \sum_{j=1}^p \alpha_j \beta_j \right) \quad (4.12)$$

Equation (4.12) indicates that if path i has a higher bandwidth, it is assigned a larger fraction of the total message. This is intuitively correct, as a higher-bandwidth path can transmit more data within the same amount of time. Conversely, if path i exhibits higher latency (i.e., larger α_i), it receives a smaller portion of the message, since its fixed startup cost makes it less efficient for data transfer.

4.3.3 Incorporating Paths with Staged Transfers

Staged transfers can be either GPU-staged or host-staged. In a GPU-staged transfer, the data is first transferred to an intermediate GPU before being sent to the destination GPU, and in a host-staged transfer, the data is first transferred to the host memory before being sent to the destination GPU. In either case, the time for a staged transfer can be modeled as a combination of two direct transfers. As discussed in Section 4.3.1, the time for a staged transfer can be modeled in Equation (4.2). By using the same constraint as before, we can determine the optimal values of θ_i that minimizes the overall communication time T . The only difference is that we need to consider both links and the associated overhead of synchronization on the staging devices. Therefore, for all $i \in \{1, \dots, p\}$, we have:

$$\alpha_1 + \frac{\theta_1 n}{\beta_1} + \epsilon_1 + \alpha_{1'} + \frac{\theta_1 n}{\beta_{1'}} = \alpha_i + \frac{\theta_i n}{\beta_i} + \epsilon_i + \alpha'_i + \frac{\theta_i n}{\beta'_i}$$

This implies that:

$$\theta_i = \frac{1}{n \left(\frac{1}{\beta_i} + \frac{1}{\beta'_i} \right)} \left[\alpha_1 + \alpha_{1'} + \epsilon_1 - (\alpha_i + \alpha'_i + \epsilon_i) + \theta_1 n \left(\frac{1}{\beta_1} + \frac{1}{\beta_{1'}} \right) \right] \quad (4.13)$$

To simplify the notations, for each i , we define the following terms:

$$\Omega_i = \frac{1}{\beta_i} + \frac{1}{\beta'_i} \text{ and } \Delta_i = \alpha_i + \alpha'_i + \epsilon_i$$

Therefore, we can rewrite Equation (4.13) as:

$$\theta_i = \frac{\Delta_1 - \Delta_i + \theta_1 n \Omega_1}{n \Omega_i} \quad (4.14)$$

Once again, by substituting the above equation into the mentioned constraint $\sum_{i=1}^p \theta_i = 1$, we get:

$$\sum_{i=1}^p \frac{\Delta_1 - \Delta_i + \theta_1 n \Omega_1}{n \Omega_i} = 1,$$

which implies:

$$\begin{aligned} 1 &= \sum_{i=1}^p \frac{\Delta_1 - \Delta_i}{n \Omega_i} + \sum_{i=1}^p \frac{\theta_1 n \Omega_1}{n \Omega_i} \\ &= \frac{1}{n} \sum_{i=1}^p \frac{\Delta_1 - \Delta_i}{\Omega_i} + \theta_1 \Omega_1 \sum_{i=1}^p \frac{1}{\Omega_i} \end{aligned}$$

Solving for θ_1 , we get:

$$\begin{aligned} \theta_1 &= \frac{1}{\Omega_1 \sum_{i=1}^p \frac{1}{\Omega_i}} \left(1 - \frac{1}{n} \sum_{i=1}^p \frac{\Delta_1 - \Delta_i}{\Omega_i} \right) \\ &= \frac{1}{\Omega_1 \sum_{i=1}^p \frac{1}{\Omega_i}} \left(1 - \frac{\Delta_1}{n} \sum_{i=1}^p \frac{1}{\Omega_i} + \frac{1}{n} \sum_{i=1}^p \frac{\Delta_i}{\Omega_i} \right) \end{aligned}$$

Therefore, the optimal values of θ_i that minimizes the overall communication time T including staged transfers are:

$$\theta_i = \frac{1}{\Omega_i \sum_{j=1}^p \frac{1}{\Omega_j}} \left(1 - \frac{\Delta_i}{n} \sum_{j=1}^p \frac{1}{\Omega_j} + \frac{1}{n} \sum_{j=1}^p \frac{\Delta_j}{\Omega_j} \right) \quad (4.15)$$

Note that Equation (4.15) generalizes Equation (4.12) to account for staged transfers. In the special case where path i is a direct path, we have $\Omega_i = 1/\beta_i$ and $\Delta_i = \alpha_i$. Substituting these parameters into Equation (4.15) leads to Equation (4.12), confirming it as a specific instance of the more general formulation.

Figure 4.1 illustrates how a message from GPU-0 to GPU-1 can be transferred using four distinct paths: (A) direct path, (B) GPU-staged transfer through GPU-2, (C) GPU-staged

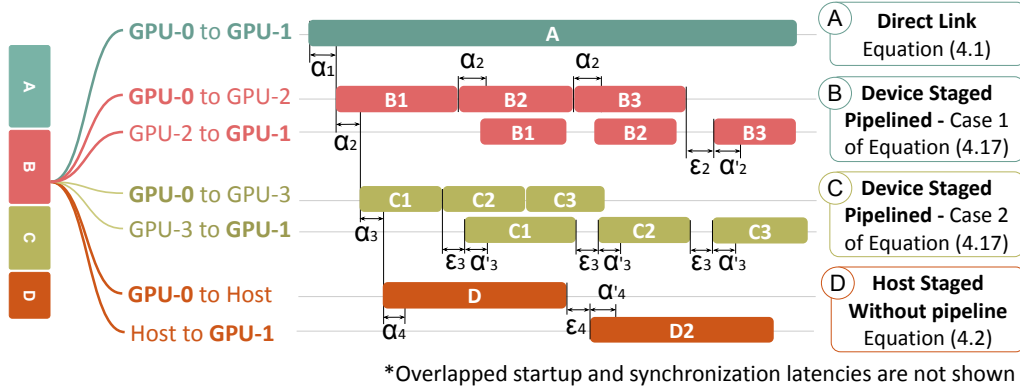


Figure 4.1: How a single transfer from GPU-0 to GPU-1 is split into four parts and assigned to different paths, some of which may send data with multiple chunks. Here, the startup and synchronization overheads are exaggerated for clarity.

transfer through GPU-3, and (D) host-staged transfer. So far, we have discussed staging without pipelining, which is path (D) in Figure 4.1.

4.3.4 Incorporating Pipelining for Staged Transfers

Now, we introduce a pipelining model in which the message is divided into k chunks to enable pipelining. Communication along each staged path proceeds through a repetitive three-step process for all the chunks:

1. Initiate the transfer of a chunk from the source GPU to the staging location.
2. Insert a synchronization point to ensure the chunk has arrived at the staging location.
3. Initiate the transfer of the chunk from the staging location to the destination GPU.

If the path i handles k_i chunks, the time required to transfer a single chunk c through this path is given by:

$$T_i(c) = \alpha_i + \frac{\theta_i n / k_i}{\beta_i} + \epsilon_i + \alpha'_i + \frac{\theta_i n / k_i}{\beta'_i} \quad (4.16)$$

With pipelining, the overall communication time is determined by the slower of the two

links in the staged path, since both segments operate concurrently and must remain synchronized for continuous data flow. Therefore, we can differentiate between two cases based on the bandwidth of the two links along the path:

$$T_i = \begin{cases} k_i \left(\alpha_i + \frac{\theta_i n / k_i}{\beta_i} \right) + \epsilon_i + \alpha'_i + \frac{\theta_i n / k_i}{\beta'_i}, & \text{if } \beta_i < \beta'_i, \\ \alpha_i + \frac{\theta_i n / k_i}{\beta_i} + k_i \left(\epsilon_i + \alpha'_i + \frac{\theta_i n / k_i}{\beta'_i} \right), & \text{if } \beta_i \geq \beta'_i. \end{cases} \quad (4.17)$$

Paths (B) and (C) from Figure 4.1 depicts the first and the second cases of Equation (4.17), respectively. To determine the optimal number of chunks to minimize the data transfer, we analyze the total communication time for both scenarios in the following.

Case 1: If the first link (source to the staging device) is the bottleneck, the total communication time is given by:

$$T_i^{\text{Case1}} = k_i \alpha_i + \frac{\theta_i n}{\beta_i} + \epsilon_i + \alpha'_i + \frac{\theta_i n}{k_i \beta'_i}.$$

To minimize T_i^{Case1} , we take its derivative with respect to k_i and set to zero. This approach is commonly used in scenarios with overlapping communication (or computation) [34, 43, 159]:

$$\frac{dT_i^{\text{Case1}}}{dk} = \alpha_i - \frac{\theta_i n}{k_i^2 \beta'_i} = 0.$$

This gives us:

$$k_i = \sqrt{\frac{\theta_i n}{\alpha_i \beta'_i}}. \quad (4.18)$$

Case 2: When the second link (staging device to the destination) is the bottleneck, the optimal chunk count is:

$$k_i = \sqrt{\frac{\theta_i n}{\beta_i (\epsilon_i + \alpha'_i)}}. \quad (4.19)$$

We can merge the two cases into one equation by defining a new variable $\lambda_i = \lfloor \beta'_i / \beta_i \rfloor$, which is 1 if the first link is the bottleneck, and 0 otherwise. This allows us to express the T_i as:

$$T_i = \lambda_i T_i^{\text{Case1}} + (1 - \lambda_i) T_i^{\text{Case2}}. \quad (4.20)$$

However, for more simplicity and without loss of generality, we can analyze the two cases separately. For the first case, we can express T_i as:

$$\begin{aligned} T_i^{\text{Case1}} &= \left(\sqrt{\frac{\theta_i n}{\alpha_i \beta'_i}} \right) \alpha_i + \frac{\theta_i n}{\beta_i} + \epsilon_i + \alpha'_i + \frac{\theta_i n}{\sqrt{\frac{\theta_i n}{\alpha_i \beta'_i}} \beta'_i} \\ &= \sqrt{\frac{\theta_i n \alpha_i}{\beta'_i}} + \frac{\theta_i n}{\beta_i} + \epsilon_i + \alpha'_i + \sqrt{\frac{\theta_i n \alpha_i}{\beta'_i}} \\ &= 2\sqrt{\frac{\theta_i n \alpha_i}{\beta'_i}} + \frac{\theta_i n}{\beta_i} + \epsilon_i + \alpha'_i. \end{aligned} \quad (4.21)$$

For comparison, the results for the second case would be:

$$T_i^{\text{Case2}} = 2\sqrt{\frac{\theta_i n (\epsilon_i + \alpha'_i)}{\beta_i}} + \frac{\theta_i n}{\beta'_i} + \alpha_i. \quad (4.22)$$

To find the optimal values for θ_i , we can use the same approach as before. So, we set the communication time of each path to be equal, i.e., $T_1 = T_i$; then, we can isolate θ_i to find its optimal value. However, this process will be much more complex for Equation (4.21) and Equation (4.22) than the previous equations, as these ones are not linear anymore. Therefore, we cannot isolate θ_i directly, and numerical methods are required to proceed further.

However, as our goal is to find the optimal values of θ_i in real-time, we need to avoid

using numerical methods. Instead, we can approximate Equations (4.18) and (4.19) with linear alternatives. Finding the topology-related constants (in the form of $c.f(n)$) for either cases helps us to approximate k_i values. As will be explained later in more detail, we can compute the topology-related constants (in the form of $c.f(n)$) for either cases, then approximate k_i values. Now, we can express k_i values for both cases as linear functions by using the topology constants ϕ_i :

$$k_i = \begin{cases} \phi_i^1 \cdot \frac{\theta_i n}{\alpha_i \beta_i'} & \text{if } \beta_i < \beta_i', \\ \phi_i^2 \cdot \frac{\theta_i n}{(\epsilon_i + \alpha_i') \beta_i} & \text{if } \beta_i \geq \beta_i'. \end{cases} \quad (4.23)$$

Therefore, we can express T_i as:

$$T_i = \begin{cases} \theta_i n \left(\frac{1}{\beta_i} + \frac{\phi_i^1}{\beta_i'} \right) + \epsilon_i + \alpha_i' + \frac{\alpha_i}{\phi_i^1}, & \text{if } \beta_i < \beta_i', \\ \theta_i n \left(\frac{\phi_i^2}{\beta_i} + \frac{1}{\beta_i'} \right) + \alpha_i + \frac{\epsilon_i + \alpha_i'}{\phi_i^2}, & \text{if } \beta_i \geq \beta_i'. \end{cases} \quad (4.24)$$

With some simplifications, we can express T_i as:

$$T_i = \theta_i n \Omega_i + \Delta_i, \quad (4.25)$$

where Ω_i and Δ_i are defined as:

$$\begin{cases} \Omega_i = \frac{1}{\beta_i} + \frac{\phi_i^1}{\beta_i'}, \Delta_i = \epsilon_i + \alpha_i' + \frac{\alpha_i}{\phi_i^1} & \text{if } \beta_i < \beta_i', \\ \Omega_i = \frac{\phi_i^2}{\beta_i} + \frac{1}{\beta_i'}, \Delta_i = \alpha_i + \frac{\epsilon_i + \alpha_i'}{\phi_i^2} & \text{if } \beta_i \geq \beta_i'. \end{cases} \quad (4.26)$$

Now, with the same approach of equating T_1 and T_i , we can isolate θ_i .

$$\theta_i = \frac{\Delta_1 - \Delta_i + \theta_1 n \Omega_1}{n \Omega_i}. \quad (4.27)$$

Similar to the Section 4.3.3, we can substitute the above equation into the constraint $\sum_{i=1}^p \theta_i = 1$ and solve for θ_1 . Then, we can formulate the general equation for θ_i as the following, which is similar to Equation (4.15):

$$\theta_i = \frac{1}{\Omega_i \sum_{j=1}^p \frac{1}{\Omega_j}} \left(1 - \frac{\Delta_i}{n} \sum_{j=1}^p \frac{1}{\Omega_j} + \frac{1}{n} \sum_{j=1}^p \frac{\Delta_j}{\Omega_j} \right). \quad (4.28)$$

4.4 Design and Implementation of the Proposed

Communication Model

To implement the proposed communication model, we have integrated our design into the [UCX](#) library. As shown in Figure 4.2, the performance model parameters are extracted once per system topology and stored on each compute node (**Step 1**). At program startup, the model is loaded into the [UCX](#) context (**Step 2**). Subsequently, for each data transfer that reaches the [CUDA IPC](#) module of [UCX](#), the model is invoked to compute the optimal transfer configuration (**Step 3**). By default, all available paths between the source and destination [GPUs](#) are considered. However, this behavior can be modified via environment variables to selectively include or exclude paths. In **Step 4**, the model determines the optimal configuration, including the number of chunks and the size of each chunk per path. Note that any path, except the direct one, may be excluded as a result of the optimization process. Finally in **Step 5**, the computed configuration is forwarded to the pipeline engine, which performs the actual communication scheduling based on [133].

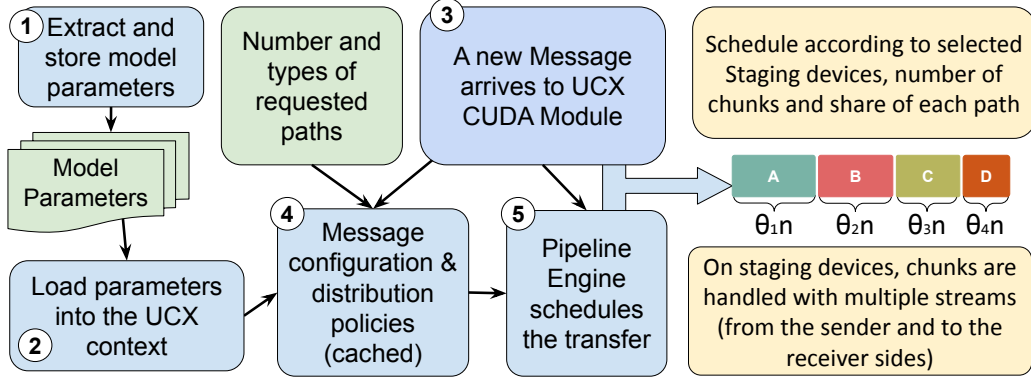


Figure 4.2: The high-level design and implementation of the communication model in the UCX library.

As depicted in Algorithm 4.1, we can compute the topology-related constants (in the form of $c.f(n)$) for either cases, then approximate k_i values. In this algorithm, we selected $min_ \theta$ and $max_ \theta$ as the minimum and maximum values of θ_i , which can be determined based on the number of available paths (ranging from 0 to 0.5). Then, we compute the corresponding minimum, maximum, and average values of k_i for both cases (Lines 6-8, and Lines 11-13). Finally, we compute the topology constant by dividing the average value of k_i by the corresponding term from Equations (4.18) and (4.19) (Line 17).

Algorithm 4.2 demonstrates the process of computing the optimal path configuration, given the source and destination GPUs, the data size, and an initial list of available paths. The algorithm begins by checking whether a cached configuration exists for the specified source-destination pair and the available paths (Lines 4-6). If no cached entry is found, it proceeds to compute the path configuration. For each path, the model calculates the parameters Ω_i and Δ_i using the performance model parameters α , β , and ϵ (Lines 7-18). Based on these computed parameters, and the topology-specific constants, the algorithm then determines the appropriate data size to assign to each path in the rest of the algorithm (Lines 20-30).

Algorithm 4.1: Compute topology constants to approximate number of chunks per path.

```

1 Input: path, data_size, path_config
2 Output: topology_constant
3 Function compute_topology_constant:
4   if path  $\neq$  direct then
5     if  $\beta < \beta_p$  then
6        $\min_{k_i} \leftarrow \sqrt{(\min_{\theta} \cdot \text{data\_size}) / (\alpha \cdot \beta_p)}$ 
7        $\max_{k_i} \leftarrow \sqrt{(\max_{\theta} \cdot \text{data\_size}) / (\alpha \cdot \beta_p)}$ 
8        $\text{avg}_{k_i} \leftarrow (\min_{k_i} + \max_{k_i}) / 2$ 
9        $\text{temp} \leftarrow (\text{avg}_{\theta} \cdot \text{data\_size}) / (\alpha \cdot \beta_p)$ 
10    else
11       $\min_{k_i} \leftarrow \sqrt{(\min_{\theta} \cdot \text{data\_size}) / (\epsilon + \alpha_p) \cdot \beta}$ 
12       $\max_{k_i} \leftarrow \sqrt{(\max_{\theta} \cdot \text{data\_size}) / (\epsilon + \alpha_p) \cdot \beta}$ 
13       $\text{avg}_{k_i} \leftarrow (\min_{k_i} + \max_{k_i}) / 2$ 
14       $\text{temp} \leftarrow (\text{avg}_{\theta} \cdot \text{data\_size}) / (\epsilon + \alpha_p) \cdot \beta$ 
15    end
16  end
17   $\text{topology\_constant} \leftarrow \text{avg}_{k_i} / \text{temp}$ 
18  return topology_constant
19 end

```

An important detail is the sequential nature of transfer initiation across the paths. To account for this, the latency α from previously scheduled paths must be accumulated and included in the computation of Δ_i for subsequent paths. This adjustment is performed in Line 18 of Algorithm 4.2.

Algorithm 4.2: Optimal Path Configuration $O(paths_count)$

```
1 Input: src_dev_id, dst_dev_id, data_size, paths[]
2 Output: configs[], shares[]
3 Function populate_path_config:
4   if cached config exists then
5     | return cached config
6   end
7   for  $i \leftarrow 0$  to paths_count do
8     | if paths[i] = direct then
9       |  $paths[i] \leftarrow get\_link(src, dst)$ 
10    | else if paths[i] = host_staged then
11      |  $paths[i] \leftarrow get\_link(src, host), get\_link(host, dst)$ 
12    | else
13      |  $paths[i] \leftarrow get\_link(src, paths[i]), get\_link(paths[i], dst)$ 
14    | end
15  end
16  for  $i \leftarrow 0$  to paths_count do
17    | Compute  $\Omega_i, \Delta_i$  Based on  $\alpha, \beta, \epsilon$ 
18    | Add previous  $\alpha$  value to  $\Delta_i$ 
19    | compute_topology_constant()
20    | Approximate number of chunks  $k_i$ 
21  end
22   $inv\_sum \leftarrow \sum 1/\Omega_i$  and  $delta\_sum \leftarrow \sum \Delta_i/\Omega_i$ 
23  for  $i \leftarrow 0$  to paths_count do
24    |  $\theta_i \leftarrow$  normalized formula using  $\Omega, \Delta$ 
25    |  $shares[i] \leftarrow \theta_i \cdot data\_size$ 
26  end
27  if remaining > 0 then
28    | Give leftover to direct
29  end
30  store config in cache
31  return config, shares
32 end
```

4.5 Performance Evaluation and Analysis

To evaluate the performance of our framework, we conducted a series of tests to compare three different configurations:

- **Static Path Distribution:** Refers to a fixed, predetermined configuration across available paths, chosen statically (offline), where the distribution strategy is extracted by

exhaustive search, similar to the non-CUDA Graph version of the work in Chapter 3 [133].

- **Model-Driven Prediction:** This is our proposed approach in this chapter [129], where we report the prediction of the model of the bandwidth, not the actual measurement.
- **Dynamic Path Distribution:** This is the implementation of our model in the runtime library to dynamically compute the model’s parameters and to find the best distribution policy.

To assess the performance of our framework powered by the performance model, we conducted our evaluations based on the OMB MPI unidirectional (BW) and bidirectional (BIBW) P2P bandwidth tests [21]. We also performed MPI_Allreduce and MPI_Alltoall tests to demonstrate the effectiveness of our model-driven multi-path communication strategy in collective operations. In all of the test cases, the runtime overhead of the model-driven framework is negligible for large message sizes (less than 0.1% of the total execution time).

4.5.1 Experimental Setup

We developed our model-driven framework by extending the UCX library and the multi-path engine. We utilized Open MPI v5.0.4 configured with UCC v1.3, UCX v1.14.0, and CUDA v12.6 for the experiments, which were performed on three different multi-GPU node configurations: Daisy, Beluga, and Narval. **Daisy** is a local cluster at Queen’s University which has two-socket computational nodes equipped with two NVIDIA A30s on each socket. Each GPU pair is connected by four pairs of NVLink-V3, and with one of the CPUs, they are configured as a single NUMA node. The CPUs are Intel Xeon Gold 6338 CPUs, each having 32 cores and supporting two threads per core, resulting in a total of 64 physical cores and 128 logical threads per node. Moreover, each node is equipped with 256

GB of DDR4 memory, and the NVIDIA A30 GPUs have 24 GB of GDDR6 memory each. The configuration of Beluga and Narval nodes are described in Section 3.5.1

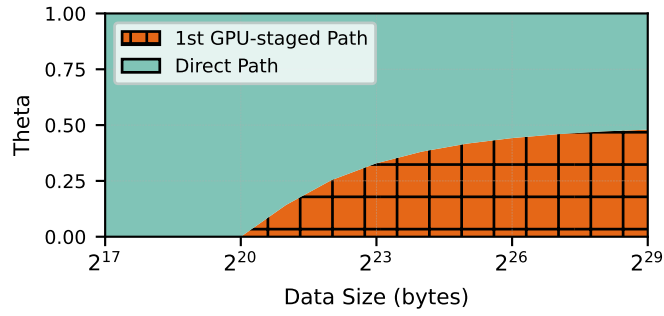
4.5.2 Unidirectional and Bidirectional Bandwidth Evaluation

We conducted the OMB unidirectional and bidirectional *bandwidth* tests using three different *window* sizes: 1, 4, and 16, for message sizes ranging from 2MB to 512MB. Performance is measured in terms of aggregate bandwidth, and we report prediction error as a percentage deviation from the observed optimal performance. For unidirectional tests, the results are shown in Figure 4.4, Figure 4.5, and Figure 4.6, and for bidirectional tests, the results are shown in Figure 4.7, Figure 4.8, and Figure 4.9. In these figures, 2_GPUs refers to a transfer using 2 paths, the direct NVLink plus 1 GPU-staged paths. Similarly, 3_GPUs refers to the direct and 2 GPU-staged paths, and finally, 3_GPUs_w_host refers to a transfer with 4 paths: the direct, 2 GPU-staged, and 1 host-staged paths.

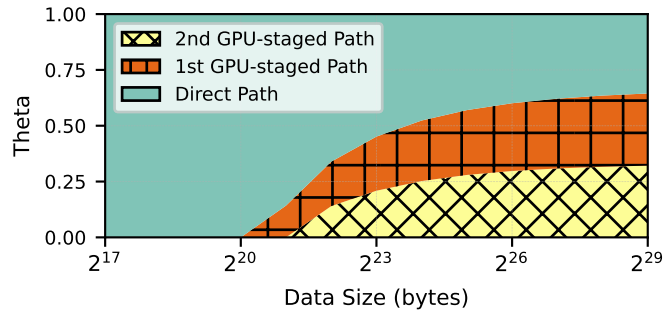
Figure 4.3 shows the distribution of the θ (message fraction) values across multiple paths for OMB MPI unidirectional bandwidth tests on Beluga. The results reveal how much each path contributes to the overall bandwidth for each message size. Figure 4.3(c) shows the distribution across four paths: direct, two GPU-staged, and one host-staged. For instance, for the last data size (512MB), the θ values are distributed as follows: 0.33 for the direct path, 0.299 for the first GPU-staged path, 0.297 for the second GPU-staged path, and 0.075 for the host-staged path.

Analyzing the results across different configurations, the following observations can be made from the evaluations:

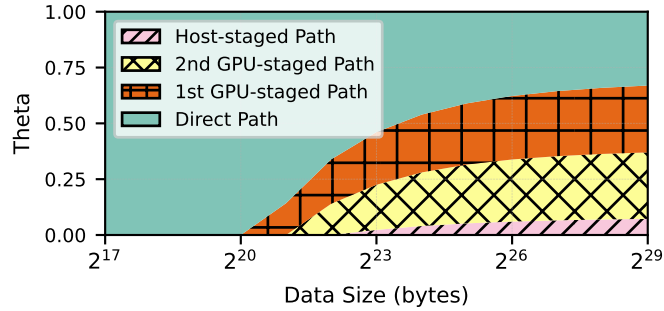
- **Observation 1:** For message sizes greater than 8MB, the model-driven prediction closely matches the observed optimal performance in BW test, with an average prediction error of less than 6% across all configurations and clusters. In contrast, the prediction



(a) 2 Paths: direct and 1 GPU-staged



(b) 3 Paths: direct and 2 GPU-staged

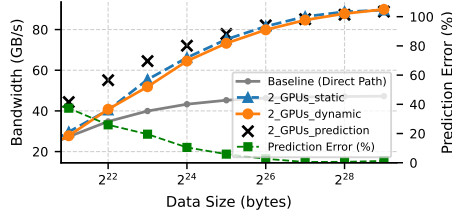


(c) 4 Paths: direct, 2 GPU-, and 1 Host-staged

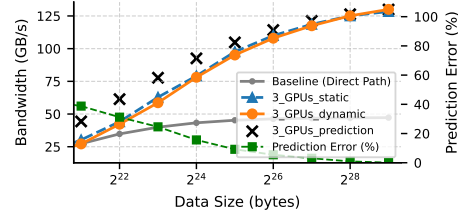
Figure 4.3: θ distribution when utilizing different number of paths for OMB MPI unidirectional bandwidth tests on Beluga.

error for **BIBW** tests is higher, averaging around 8% for non-host-staged across all cluster nodes and *window* sizes.

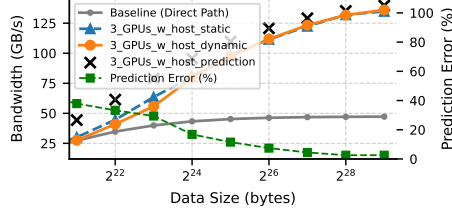
- **Observation 2:** As the number of *windows* increases, the performance gap between the static and dynamic path distributions narrows. Similarly, the prediction errors decrease with higher number of *windows*. This is due to the fact that larger *window* sizes allow



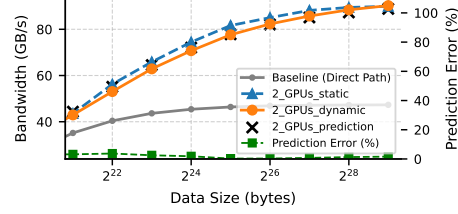
(a) BW on Beluga; 2 GPU paths, $win=1$



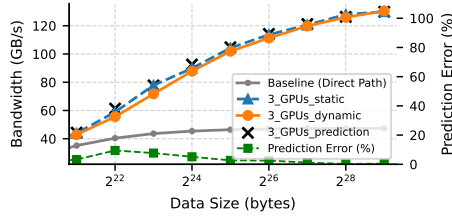
(b) BW on Beluga; 3 GPU paths, $win=1$



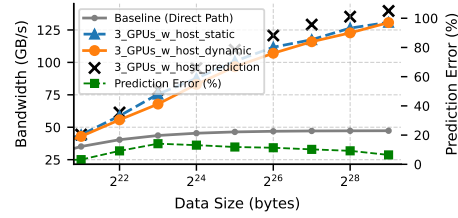
(c) BW on Beluga; 3 GPU paths & host, $win=1$



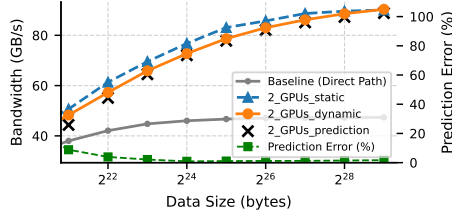
(d) BW on Beluga; 2 GPU paths, $win=4$



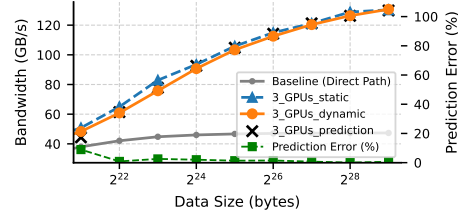
(e) BW on Beluga; 2 GPU paths, $win=4$



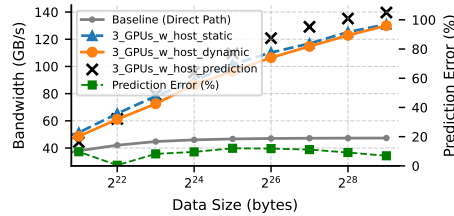
(f) BW on Beluga; 3 GPU paths & host, $win=4$



(g) BW on Beluga; 2 GPU paths, $win=16$

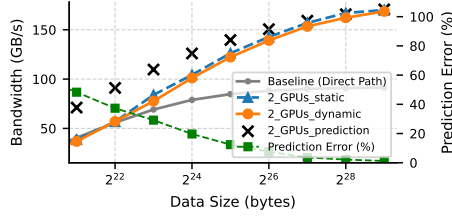


(h) BW on Beluga; 3 GPU paths, $win=16$

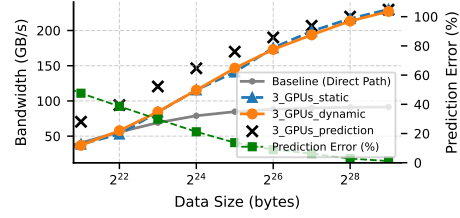


(i) BW on Beluga; 3 GPU paths & host, $win=16$

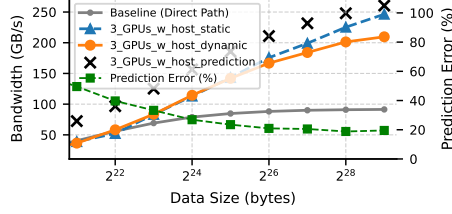
Figure 4.4: Unidirectional MPI bandwidth tests on Beluga. Comparing the predicted and two measured configurations (dynamic- and static-tuned) with the baseline (Direct Path) using various number of paths and $window$ sizes.



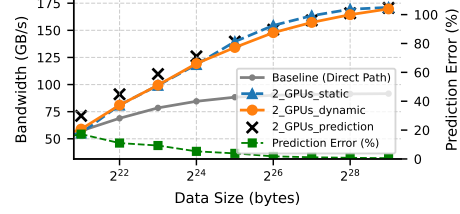
(a) BW on Narval; 2 GPU paths, $win=1$



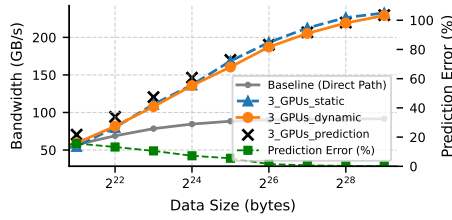
(b) BW on Narval; 3 GPU paths, $win=1$



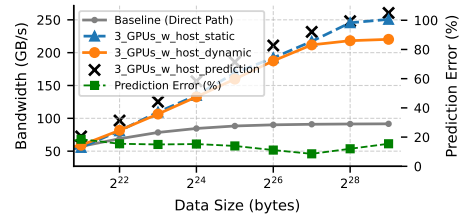
(c) BW on Narval; 3 GPU paths & host, $win=1$



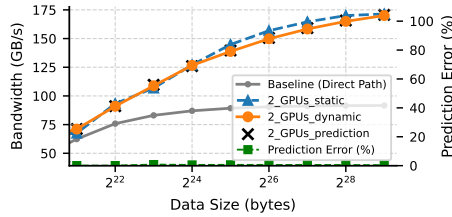
(d) BW on Narval; 2 GPU paths, $win=4$



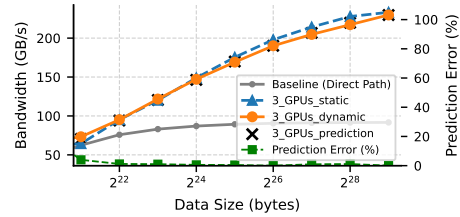
(e) BW on Narval; 2 GPU paths, $win=4$



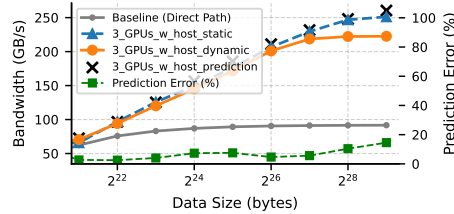
(f) BW on Narval; 3 GPU paths & host, $win=4$



(g) BW on Narval; 2 GPU paths, $win=16$



(h) BW on Narval; 3 GPU paths, $win=16$



(i) BW on Narval; 3 GPU paths & host, $win=16$

Figure 4.5: Unidirectional MPI bandwidth tests on Narval. Comparing the predicted and two measured configurations (dynamic- and static-tuned) with the baseline (Direct Path) using various number of paths and $window$ sizes.

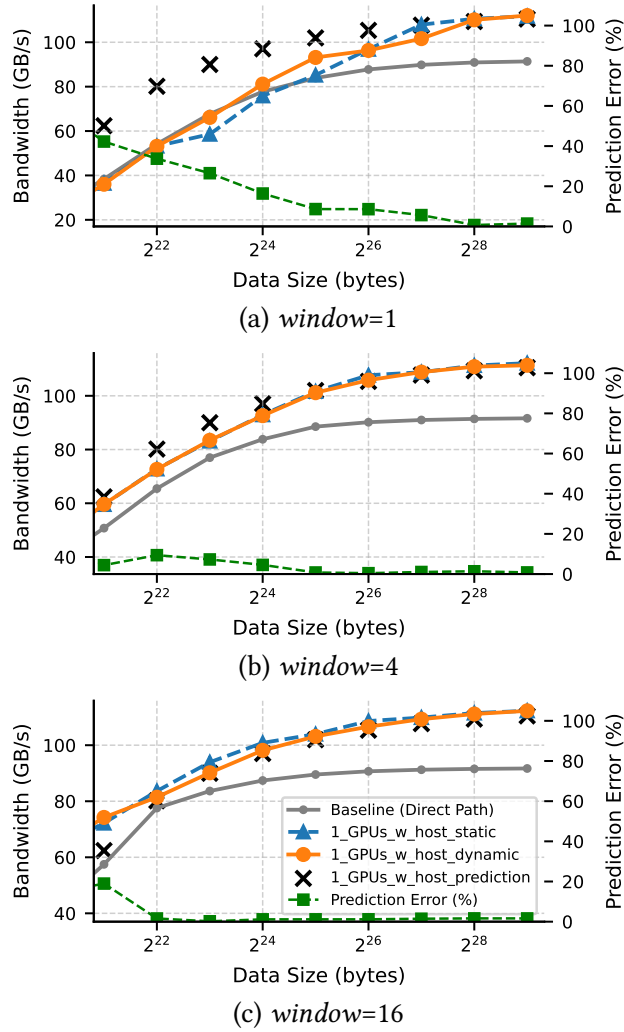
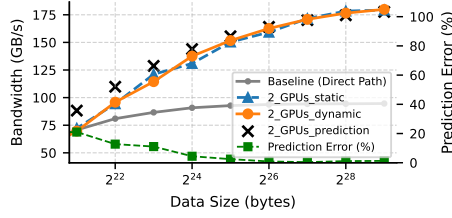


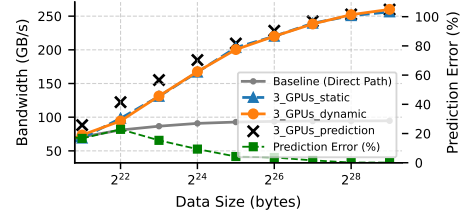
Figure 4.6: Unidirectional MPI bandwidth tests on Daisy with host staging. Comparing predicted and measured bandwidths (dynamic- and static- tuned) for different *window* sizes.

for more concurrent transfers, reducing the impact of latency and bandwidth variations. For instance, compare Figure 4.5(b) with Figure 4.5(h), Figure 4.4(b) with Figure 4.4(h), and Figure 4.6(a) with Figure 4.6(c).

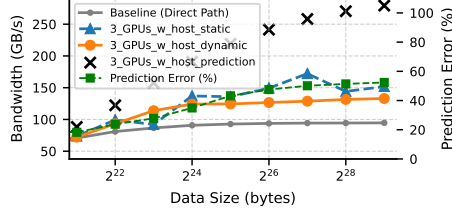
- **Observation 3:** The prediction errors are generally higher when the host-staged path is enabled in the configuration for `BW` tests. This issue is particularly present on Narval, where the host-staged path is less efficient due to the `NUMA` configuration (compare



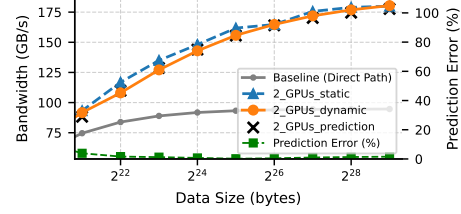
(a) BIBW on Beluga; 2 GPU paths, $w=1$



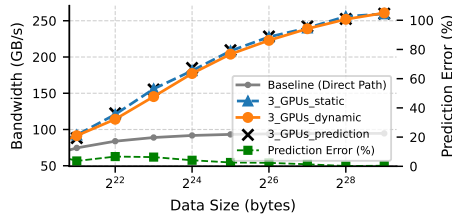
(b) BIBW on Beluga; 3 GPU paths, $w=1$



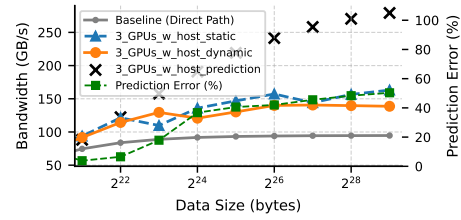
(c) BIBW on Beluga; 3 GPU paths & host, $w=1$



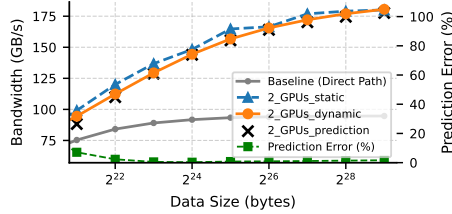
(d) BIBW on Beluga; 1 GPU paths, $w=4$



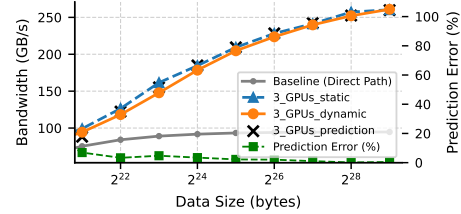
(e) BIBW on Beluga; 2 GPU paths, $w=4$



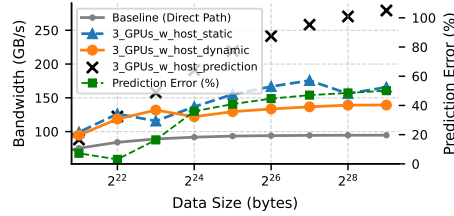
(f) BIBW on Beluga; 3 GPU paths & host, $w=4$



(g) BIBW on Beluga; 2 GPU paths, $w=16$

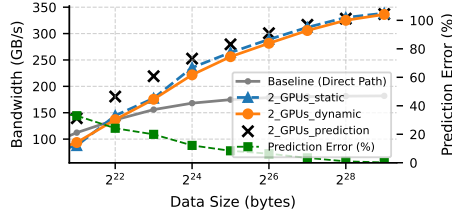


(h) BIBW on Beluga; 3 GPU paths, $w=16$

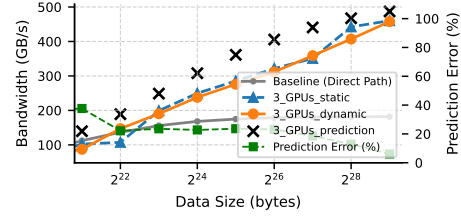


(i) BIBW on Beluga; 3 GPUs & host, $w=16$

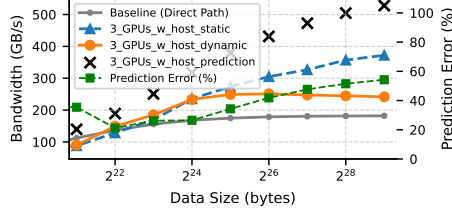
Figure 4.7: Bidirectional MPI bandwidth tests on Beluga. Comparing the predicted and two measured configurations (dynamic- and static-tuned) with the baseline (Direct Path) using various number of paths and $window$ sizes.



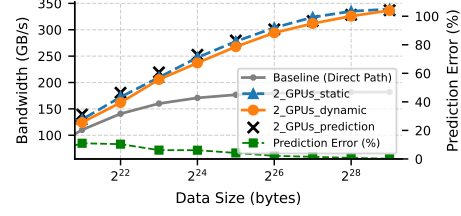
(a) BIBW on Narval; 2 GPU paths, $w=1$



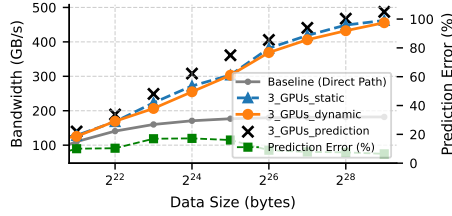
(b) BIBW on Narval; 3 GPU paths, $w=1$



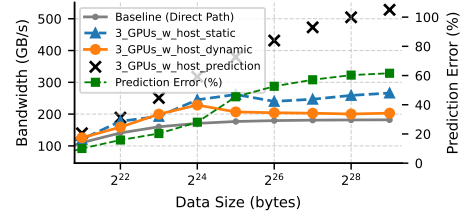
(c) BIBW on Narval; 3 GPU & host, $w=1$



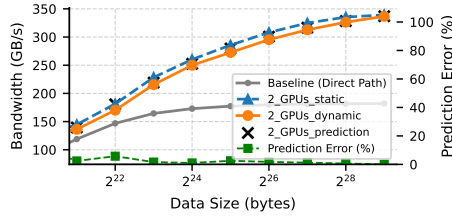
(d) BIBW on Narval; 1 GPU paths, $w=4$



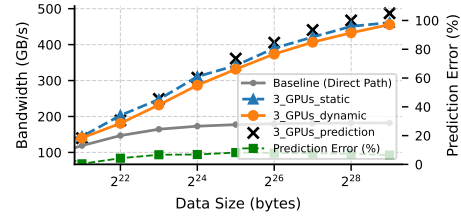
(e) BIBW on Narval; 2 GPU paths, $w=4$



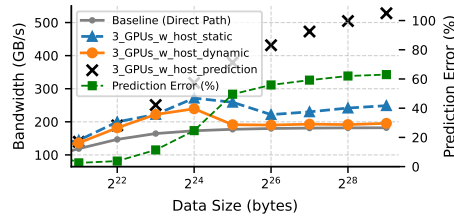
(f) BIBW on Narval; 3 GPU & host, $w=4$



(g) BIBW on Narval; 2 GPU paths, $w=16$



(h) BIBW on Narval; 3 GPU paths, $w=16$



(i) BIBW on Narval; 3 GPUs & host,
 $w=16$

Figure 4.8: Bidirectional MPI bandwidth tests on Narval. Comparing the predicted and two measured configurations (dynamic- and static-tuned) with the baseline (Direct Path) using various number of paths and $window$ sizes.

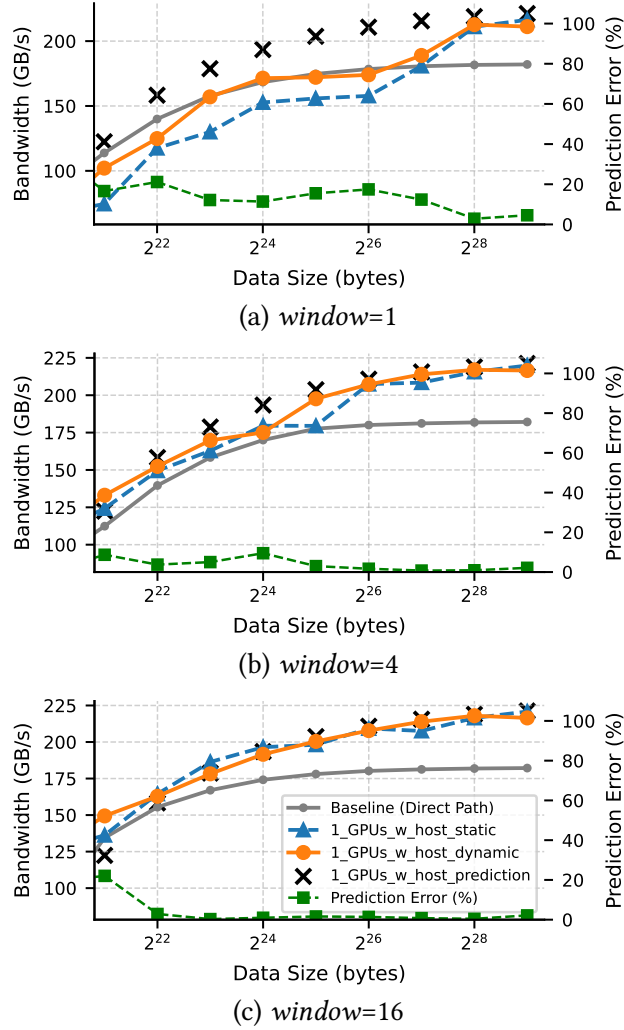


Figure 4.9: Bidirectional MPI bandwidth tests on Daisy with host staging. Comparing predicted and measured bandwidths (dynamic- and static- tuned) for different *window* sizes.

Figure 4.5(c) and Figure 4.4(c)). As can be inferred from Figure 3.5, host-staged transfers on Narval includes an extra transfer (through *Ultra Path Interconnect (UPI)* or equivalent), because the memory channels are not shared between the GPUs. Moreover, this issue is less pronounced for Daisy nodes due to the NUMA configuration (see Figure 4.6).

- **Observation 4:** The model-driven predictions are less accurate for smaller message sizes on all cluster nodes, especially for small *window* sizes. This is due to the linear

nature of the *Hockney*'s model and the way the asymptotic bandwidth is calculated. This issue is more pronounced for `BW` tests compared to `BIBW` tests. As Figure 4.4(a), Figure 4.4(b), Figure 4.5(a), Figure 4.5(b), and Figure 4.6(a) show, the model tends to overestimate the performance for smaller message sizes.

- **Observation 5:** The `BIBW` tests demonstrate worse performance with the presence of the host-staged path on all clusters, as shown in Figure 4.7(c), Figure 4.7(f), Figure 4.8(c), and Figure 4.8(f). This is due to the contention between the bidirectional transfers along the host-staged path, which is not considered in our performance model. This observation suggests that while host staging still increases the overall bandwidth comparing to single-path, the contention between the bidirectional transfers degrades the performance, and the model should be extended to account for host-staged contention. It is worth noting that this issue is less pronounced on Daisy nodes, as shown in Figure 4.9, due to the fact that there is only host-staged path available (other than the direct path), and an anomaly in the host-staged path does not affect the performance on Daisy nodes as adversely as other clusters (compare Figure 4.9(a) and Figure 4.7(c)).

4.5.3 Collective Communication Evaluation

Although the primary focus of the work in this chapter is on `P2P` communication, we also evaluate the performance of collective communication operations, specifically `MPI_Allreduce` and `MPI_Alltoall`. These operations are essential for many parallel applications and can benefit from the intra-node `GPU` communication optimizations discussed in this work.

We evaluated the performance of the collective operations on both Beluga and Narval clusters, using the same configurations as for the `P2P` tests. We utilized the `UCC` library and its `UCP` module for each collective operation. For `MPI_Allreduce` and large message sizes, `UCP` uses the *recursive K-nomial scatter-reduce followed by K-nomial allgather* algorithm,

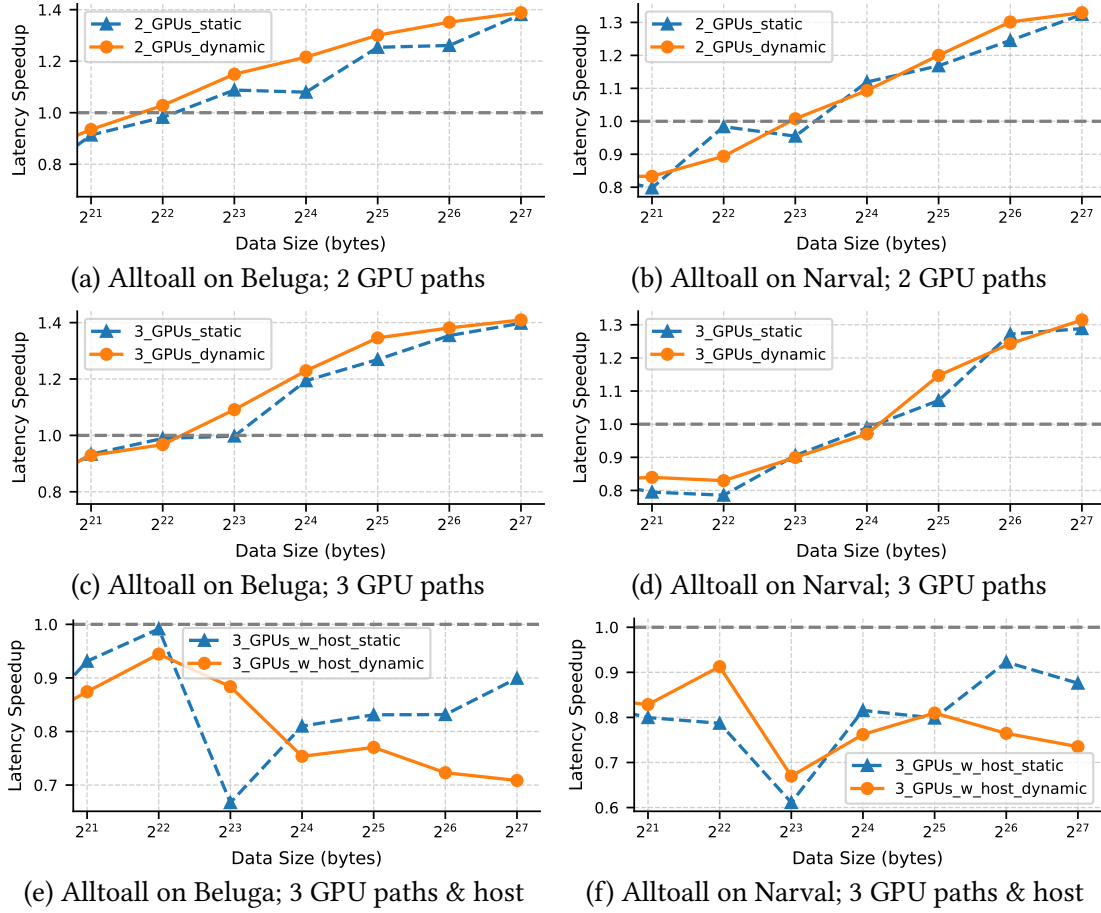


Figure 4.10: Latency speedup of MPI Alltoall tests on Beluga and Narval against MPI+UCC+UCX configuration.

and for `MPI_Alltoall`, it uses the *Bruck* algorithm. These algorithms consist of multiple non-blocking `P2P` communication steps which are handled by `UCX` under the hood. With this setup, we can enforce the utilization of the `UCX cuda_ipc` module for intra-node communication, where our framework is integrated.

We compared the performance of our model-driven path distribution and the static multi-path configuration (non-`CUDA` Graph version of the work in Chapter 3) against the default `MPI+UCC+UCX` stack, with the results presented in Figure 4.10, and Figure 4.11 (x-axis is the size of message per rank/GPU). The following observations can be made:

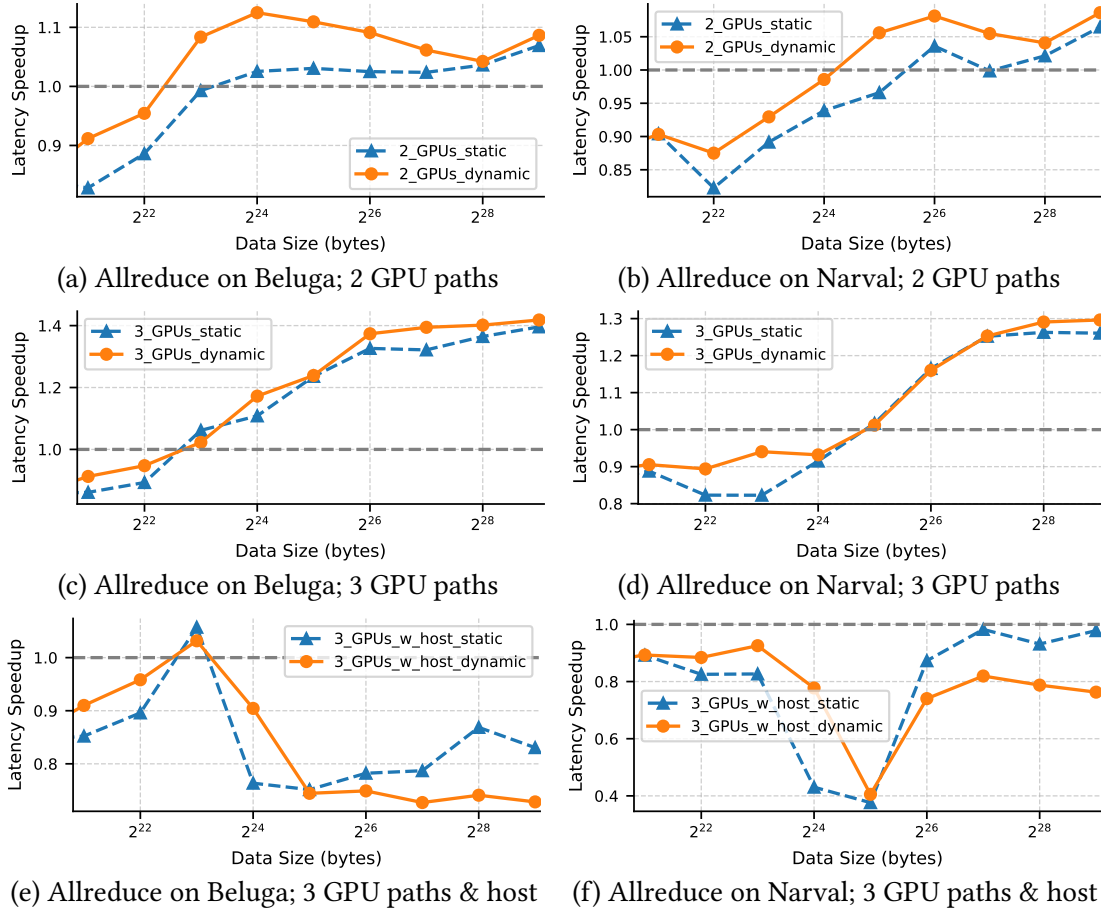


Figure 4.11: Latency speedup of MPI Allreduce tests on Beluga and Narval against MPI+UCC+UCX configuration.

- **Observation 1:** Enabling multi-path transfers improves the performance of both collective operations on both clusters, but the improvements are more significant on Beluga due to the lower bandwidth of its NVLink connections compared to Narval. This characteristic makes Beluga more prone to benefit from multi-path transfers, as the additional paths help to alleviate the bottleneck caused by the limited bandwidth of individual NVLink connections.
- **Observation 2:** Overall, the results indicate that our model-driven approach outperforms the statically-tuned multi-path configuration for both collective operations on both

clusters.

- **Observation 3:** The performance improvement is higher for `MPI_Alltoall` compared to `MPI_Allreduce`, which is due to the computation overhead in `MPI_Allreduce` (compare Figure 4.10(a) with Figure 4.11(a), and Figure 4.10(b) with Figure 4.11(b)). However, with three GPU paths, the performance difference between the two collective operations is lower (compare Figure 4.10(c) with Figure 4.11(c)).
- **Observation 4:** For `MPI_Allreduce`, the performance improvement is more pronounced with three GPU paths compared to two GPU paths (compare Figure 4.11(a) with Figure 4.11(c), and Figure 4.11(b) with Figure 4.11(d)). However, for `MPI_Alltoall`, the performance improvement with three GPU paths is marginal compared to two GPU paths (compare Figure 4.10(a) with Figure 4.10(b), and Figure 4.10(b) with Figure 4.10(d)). This is due to the different communication patterns of the two collective operations and how adding more paths does not lead to significant gains in this case.
- **Observation 5:** Similar to the BIBW tests, the presence of the host-staged path degrades the performance of `MPI_Allreduce` and `MPI_Alltoall` operations. Again, this is due to the contention between the bidirectional transfers along the host-staged path, which is not considered in our performance model. This is evident in Figure 4.10(e) and Figure 4.10(f) for `MPI_Alltoall`, and Figure 4.11(e) and Figure 4.11(f) for `MPI_Allreduce`.

4.6 Summary

In this chapter, we designed and evaluated a performance model for multi-path intra-node GPU communication, considering the latencies and bandwidths of each NVLink and PCIe link as well as the effects of pipelined, staged transfers. We showed that the optimal multi-path schedule (minimizing overall transfer time) is achieved when the per-path transfer times are equal (or nearly equal). We demonstrated the effectiveness of the model-enabled

multi-path communication strategy in both P2P and collective operations, achieving up to 2.9x and 1.4x speedup, respectively, over the single-path methods. We evaluated the accuracy of our model, achieving less than 6% error in predicting the optimal configuration for messages larger than 4MB in unidirectional transfers.

In the next chapter, we build upon this work by proposing a model-driven collaborative *Allreduce* collective communication framework that utilizes heterogeneous computing and multi-path communication for multi-GPU systems. We also introduce the first performance model for heterogeneous *Allreduce* operations, which accurately predicts execution times and guides our design and optimization.

Chapter 5

Collaborative Bandwidth-Efficient Intra-Node Allreduce

5.1 Introduction

In the fast-paced world of [HPC](#), achieving efficient utilization of computational and communication resources remains a critical challenge. The growing demand for processing vast datasets and running complex simulations necessitates innovations that can deliver higher performance and scalability. The two prominent paradigms, heterogeneous computing and multi-path communication, have emerged as key contributors to addressing these demands [[167](#), [148](#)].

Heterogeneous computing enables developers to utilize a variety of computing or communication units at the same time. As demonstrated in previous studies, utilizing computation or communication heterogeneity, can lead to significant performance improvements in [HPC](#) applications [[53](#), [148](#), [121](#)]. For instance, in training [LLMs](#), when the model size exceeds the memory capacity of the [GPUs](#), model states can be spilled to the host memory in a pipeline fashion to remedy the memory constraints [[73](#), [118](#)]. However, leveraging this potential

requires careful analysis of the underlying hardware architecture and the applications' requirements. Moreover, with multi-path communication, developers optimize the data movement by utilizing available communication paths. By balancing traffic and reducing congestion, this approach can significantly enhance bandwidth utilization and minimize communication latency. Previous studies have shown that multi-path communication even for single messages can improve the performance of HPC applications when applied with explicit consideration of the underlying hardware architecture [136, 138].

Collective communication operations, such as *Allreduce*, are fundamental building blocks in many HPC applications, including machine learning, scientific simulations, and numerical computations. These operations often dominate the overall execution time, especially in GPU-heavy workloads where data exchange between multiple GPUs can become a bottleneck [5, 14, 139]. As such, optimizing collective communication for multi-GPU systems is a critical area of research. Exploring heterogeneity in the context of collective communication has been shown to improve the performance of these operations [166]. However, existing state-of-the-art solutions, such as NCCL [99], UCC [149], and Open MPI [103], do not exploit the potential of heterogeneous computing and multi-path communication. These libraries are designed to work with homogeneous systems and do not consider the potential benefits of utilizing both CPU and GPU resources for collective communication.

Furthermore, communication modeling in HPC has enabled many researchers to analyze and optimize data transfers, including CPU-GPU transfers, inter-node multi-GPU communication, and various collective operations [10, 120, 157, 62, 125, 22]. Classical latency-bandwidth models (such as the *Hockney's* model) assume a single transfer path and do not capture concurrent use of NVLink and PCIe channels, especially they lack consideration of computation [48, 4, 31]. More advanced models, address heterogeneity, or

middleware costs but they do not target the specific problem of heterogeneous computation in intra-node multi-path communication [43, 71, 126, 116, 18, 144, 75].

In this chapter, we investigate the integration of GPU-aware multi-path communication and heterogeneous computing to enhance the performance of *Allreduce* collective operations in multi-GPU systems. We propose a collaborative framework that leverages the high memory bandwidth and parallelism of GPUs alongside the computational and interconnect resources of the host system. Our approach is designed to address the challenges in existing state-of-the-art solutions, such as NCCL, UCC, and MPI with UCX. We introduce a design and implementation of a novel *Allreduce* framework on a multi-GPU node and evaluate its performance against state-of-the-art libraries and several algorithms [130]. Our results demonstrate that our proposed framework can achieve up to 1.45x speedup in *Allreduce* compared to NCCL for very large messages, highlighting its potential for further optimizations and extensions. Our contributions in this chapter are summarized as follows:

1. We present a novel framework that integrates heterogeneous computing and multi-path communication for *Allreduce* operations in multi-GPU systems. Our design effectively utilizes both CPU and GPU resources to optimize data transfer and computation.
2. We provide a performance model that predicts the performance of our proposed framework based on system parameters, and enables identifying optimal configurations for different message sizes and system architectures. The results show that our performance model can accurately predict the performance of our framework with an average error of less than 8%.
3. We conduct a comprehensive performance evaluation of our proposed framework

against state-of-the-art libraries, including [NCCL](#), [UCC](#), and [MPI](#) with [UCX](#). Our experiments demonstrate significant performance improvements, particularly for large message sizes where our framework achieves up to 1.45x speedup compared to [NCCL](#).

4. We provide detailed analysis and insights into the benefits and challenges of leveraging heterogeneous computing and multi-path communication for collective operations. Our findings contribute to the understanding of how these paradigms can be effectively utilized in [HPC](#) applications.

The remainder of this chapter is organized as follows. Section [5.2](#) provides the necessary background and related work, and Section [5.3](#) describes the design, implementation, and optimizations of our heterogeneous multi-path collective framework. Section [5.4](#) presents the performance model for our proposed framework. Section [5.5](#) presents the experimental setup, performance evaluation, and analysis of the results. Finally, Section [5.6](#) concludes the chapter, summarizing the contributions.

5.2 Background and Related Work

5.2.1 Collective Communication Algorithms and GPU-awareness

Although classic collective algorithms (like ring, tree/binomial, recursive-doubling, Bruck, and [RSA](#) [[20](#), [110](#)]) are foundational and still used today, they are abstract and do not consider [GPU](#)-specific characteristics. Implementations targeting [GPUs](#) must account for device memory semantics, [PCIe](#)/[NVLink](#) performance characteristics, and [GPU](#) kernel overlap. Prior works propose [GPU](#)-shared-buffer algorithms, hierarchical designs, and pipelined reductions that combine [GPU](#) and host-mediated stages to maximize overlap and use all available bandwidths [[39](#), [138](#), [28](#)].

NVIDIA’s [NCCL](#) [99], [UCC](#) [149], and [MPI+UCX](#) [103] represent the dominant software stacks for distributed [GPU](#)-aware collectives. [NCCL](#) is topology-aware and highly optimized for NVIDIA hardware but focuses on device-centric paths and assumes homogeneous [GPU](#)-centric resources. [UCC](#) exposes a flexible collective [API](#) intended to enable hierarchical and topology-aware implementations and has been adopted as a framework for building production collectives that can target both CPU- and [GPU](#)-oriented backends. Open [MPI](#) with [UCX](#) support enables [MPI](#) applications to use [UCX](#)’s low-level transports.

Early work on [GPU](#)-aware [MPI](#) and vendor libraries made it possible to initiate communication directly from device memory (avoiding extra copies), pushed collectives to the hardware/software boundary, and explored hierarchical algorithms that mix device-local and network-wide phases [103, 99, 57, 40, 39].

Recent research has demonstrated that explicitly using multiple intra-node paths in parallel (multi-path transfers) can significantly increase point-to-point throughput between [GPUs](#) and improve the performance of collectives when used as building blocks for hierarchical algorithms [138, 137, 133]. In particular, implementations that extend [UCX](#) to be [NVLink/PCIe](#)-aware and orchestrate concurrent transfers over distinct links achieve large bandwidth gains for large messages.

5.2.2 Heterogeneous Computing in HPC

Heterogeneous computing (coordinating CPUs, [GPUs](#), and other accelerators) is well studied in the [HPC](#) community, especially for workloads that exceed the memory or compute capacity of a single device [38, 161]. Several lines of work show that leveraging idle or underutilized CPU resources alongside [GPUs](#) can improve collective and reduction operations by (a) offloading parts of the reduction/aggregation to host cores, (b) pipelining data movement and computation across host and device, and (c) exploiting additional

memory capacity on the host to stage data that cannot fit on the device [1, 63, 78]. This approach is particularly useful when GPU memory limits require spilling state to host memory or when interconnect topology offers multiple distinct paths whose combined bandwidth can be exploited by a heterogeneous design [61]. However, comparing to our work in this chapter, these studies do not split a single collective operation across CPU and GPU resources; rather, they offload entire phases of the collective or segments of the application to the host.

The concept of multi-path communication, long used in network protocols, is increasingly being applied to HPC interconnects. Within a multi-GPU node, this involves concurrently using all available communication paths. As demonstrated by authors in [138], this can be achieved by striping messages across multiple NVLink channels within the UCX communication library. This multi-path mechanism was used as a building block for a hierarchical, pipelined `MPI_Allreduce` operation, utilizing in-GPU kernel reductions [138]. Extending this work, in Chapter 3 and Chapter 4, we developed a multi-path framework for MPI + UCX that enhances point-to-point GPU communication by concurrently leveraging both NVLink and PCIe, achieving significant bandwidth improvements [133].

Authors from [1] proposed a framework in which the *Allreduce* buffer is partitioned, and multiple processes share the same GPU, with each CPU core asynchronously progressing its unique portion of the operation. In this way, both CPU and GPU resources are utilized to accelerate the collective operation. However, unlike our approach, they do not utilize the CPU cores to perform parts of the reduction.

Another notable example is DeepSpeed ZeRO and its predecessors, ZeRO++ and ZeRO-infinity [111, 112, 156]. These works demonstrate spilling strategies that place optimizer states or parameter partitions in host memory and DMA when needed. The key insight for us to design the heterogeneous collectives is that staged transfers combined with CPU-

side computation can be cost-effective for very large messages when done with careful overlapping.

5.2.3 Performance Modeling of Collective Communications

Optimizing collective communications requires accurate performance models to select the best algorithm for a given message size, process count, and hardware topology [107]. Foundational work in this area extended classic point-to-point communication models, such as the *Hockney's* model [48] to collective operations.

The *Hockney's* model represents communication time (T) as a function of message size (m) using two parameters: latency (α) and bandwidth (β): $T = \alpha + n/\beta$ [48]. This simple model has been widely used to analyze and optimize collective algorithms by estimating their communication costs based on message size and system parameters [107, 110].

However, the complexity of modern HPC systems, with memory hierarchies and heterogeneous interconnects, poses significant challenges for these traditional analytical models. As noted by previous studies, *LogP* and its variants are "increasingly difficult to calibrate for modern heterogeneous systems" [164].

Recent empirical and analytical studies that profile GPU interconnects emphasize that the best data movement strategy depends strongly on message size, topology and the hardware generation [116, 32, 148]. This motivates performance models and tuning frameworks which predict splitting points in which host-assisted or multi-path approaches outperform device-only collectives. Such models are crucial for runtime selection of algorithms and for auto-tuning parameters like segment sizes, number of parallel paths, and CPU/GPU load split.

Modeling collectives in heterogeneous/multi-path settings extends classical latency-bandwidth ($\alpha - \beta$) models by adding per-path bandwidths, per-path startup costs, and contention

or serialization penalties when multiple transfers share links (or when the host is used concurrently by many transfers). Prior work typically augments the $\alpha - \beta$ model with pipeline stall terms and path-selection costs. These analytic models have proven useful for algorithm selection and for setting segment sizes for pipelining across multiple paths [129, 138, 32].

We can conclude that there is a substantial opportunity in combining heterogeneous CPU/GPU resources to accelerate collectives (especially for large messages), while benefiting from exploiting multiple intra-node physical paths when their bandwidths can be aggregated. However, none of state-of-the-art libraries automatically exploits heterogeneous multi-path opportunities inside a node or across host and device in a coordinated way. This gap motivates a collaborative framework that (a) integrates topology-aware multi-path transfers, (b) coordinates CPU and GPU resources for reduction and staging, and (c) uses a performance model to choose the optimal configuration per message size and topology (the topic of the present chapter).

5.3 Design and Implementation

Figure 5.1 illustrates a typical target topology for our multi-path *Allreduce* framework. In this topology two GPUs are directly connected by NVLink as well as PCIe interconnect to the system. In such topologies, and depending on the workloads, CPU cores are usually used for data staging and control operations, while the GPUs are used for the actual computation. As the application data resides on the GPUs, and the GPUs are directly connected, employing the CPUs to participate in the *Allreduce* might seem counter-intuitive. However, when CPU cores are idle during a blocking GPU collective operation, they could be utilized to accelerate the reduction process, when the data is large enough to saturate the NVLink.

Although this work focuses on NVIDIA GPUs and NVLink interconnect, the proposed design and implementation can be extended to other GPU vendors and interconnects, such as AMD GPUs with Infinity Fabric, or NVIDIA *Chip-to-Chip* (C2C) NVLink in which the main memory of the system is accessible by the GPUs and vice versa. The key requirement is having multiple communication paths between the GPUs, which can be exploited for multi-path communication, as well as having computational resources on both CPU and GPU sides to leverage heterogeneous computing.

To design and implement our multi-path heterogeneous *Allreduce* framework, we have considered the following key design goals:

- **Multi-path Communication:** Utilizing all available bandwidth to decrease data transfer overhead between the GPUs.
- **Leveraging Idle CPUs in the Data Path:** Concurrently utilizing the computational capabilities of both CPUs and GPUs to enhance collective communication.
- **Low Overhead:** Implementing a low-overhead pipelined communication scheme to overlap computation and communication tasks along both NVLink and PCIe paths.
- **Data Integrity:** Ensuring data integrity and consistency during data transfers and computations.
- **Reducing CPU Involvement in the Control Path:** Utilizing asynchronous communication primitives to offload communication and computation tasks from the CPUs to the GPUs asynchronously.

5.3.1 Framework Architecture

To select a performant base algorithm for our *Allreduce* design, we implemented and evaluated several well-known *Allreduce* algorithms: *Reduce Broadcast*, *Pairwise Exchange*

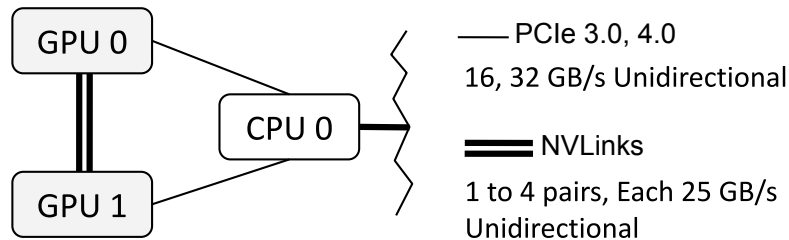


Figure 5.1: A typical target topologies for multi-path *Allreduce* framework that utilizes NVLink (any number of pairs) and PCIe paths between two GPUs of the system.

and *Segmented Ring*, including their pipelined and kernel-only versions. These algorithms are explained in detail in Section 2.4.3. In *Reduce Broadcast*, the data is first reduced to a root process using a binomial tree structure, and then broadcasted back to all processes. In *Pairwise Exchange*, each process exchanges data with every other process in a pairwise manner, performing local reductions at each step. For only two processes, *Pairwise Exchange* is similar to the *Recursive Doubling* algorithm. *Segmented Ring*, which is a ring-based [RSA](#), consists of two main phases: the *Scatter-Reduce* phase, where data segments are distributed and reduced in a ring fashion, followed by the *Allgather* phase, where the reduced segments are collected and distributed back to all processes.

In the pipelined versions of these algorithms, the data is split into multiple chunks to overlap the algorithm steps, while in the kernel-only versions, both computation and communication tasks are packed into a single kernel, similar to how [NCCL](#) performs. See Table 5.1 for a summary of the implemented algorithms and their variants, which utilize neither multi-path nor heterogeneous techniques. We have implemented these algorithms to demonstrate the differences between utilizing [DMA](#) engines and [SM](#)-driven copies, as well as the impact of pipelining and overlapping communication steps. This way, we can ensure a fair comparison between our framework and [NCCL](#) by highlighting the sources of performance gains and improvements.

#	Algorithm	Description	Copy Method	Overlap
1	<i>Reduce Broadcast</i> (Main)	Memcpy-based	DMA Engine	–
2	<i>Reduce Broadcast</i> (Pipelined)	Chunked memcpy	DMA Engine	✓
3	<i>Reduce Broadcast</i> (Kernel-only)	In-kernel copy	SM Copy	✓
4	<i>Pairwise Exchange</i> (Main)	Memcpy-based	DMA Engine	–
5	<i>Pairwise Exchange</i> (Pipelined)	Chunked memcpy	DMA Engine	✓
6	<i>Pairwise Exchange</i> (Kernel-only)	In-kernel copy	SM Copy	✓
7	<i>Segmented Ring</i> (Main)	Memcpy-based	DMA Engine	–
8	<i>Segmented Ring</i> (Pipelined)	Chunked memcpy	DMA Engine	✓
9	<i>Segmented Ring</i> (Kernel-only)	In-kernel copy	SM Copy	✓

Table 5.1: A summary of implemented *Allreduce* algorithms and their variants in the user-level. While the main versions rely on device DMA engines for memcpy operations with no overlap, the pipelined versions use chunking to overlap communication steps, and the kernel-only versions rely on SM-driven copies with fully integrated fine-grained overlap.

We use *Pairwise Exchange* as the base algorithm for our *Allreduce* design, due to its efficiency for small number of GPUs, and its extensibility to seamlessly support multi-path communication. Moreover, as will be further discussed in Section 5.5, we have observed marginal performance superiority from *Pairwise Exchange* compared to other algorithms for two GPUs. However, for larger number of GPUs, other algorithms such as *Segmented Ring* or *RSA* might be more bandwidth-efficient.

Figure 5.2 provides a simplified overview of our approach, illustrating how the multi-path heterogeneous *Pairwise Exchange Allreduce* works on two GPUs. The steps on each side of the figure, device (left) and host (right), represent the algorithm tasks that should occur with the specified order and may be concurrent to those of the other side. Our main goal is to maximize concurrency and overlap between the two sides to minimize latency. Therefore, data should be carefully divided into two parts at a certain threshold to be handled by each side. This threshold is determined based on the data size, the available bandwidth of the NVLink and PCIe paths, and the computational power of the GPUs and

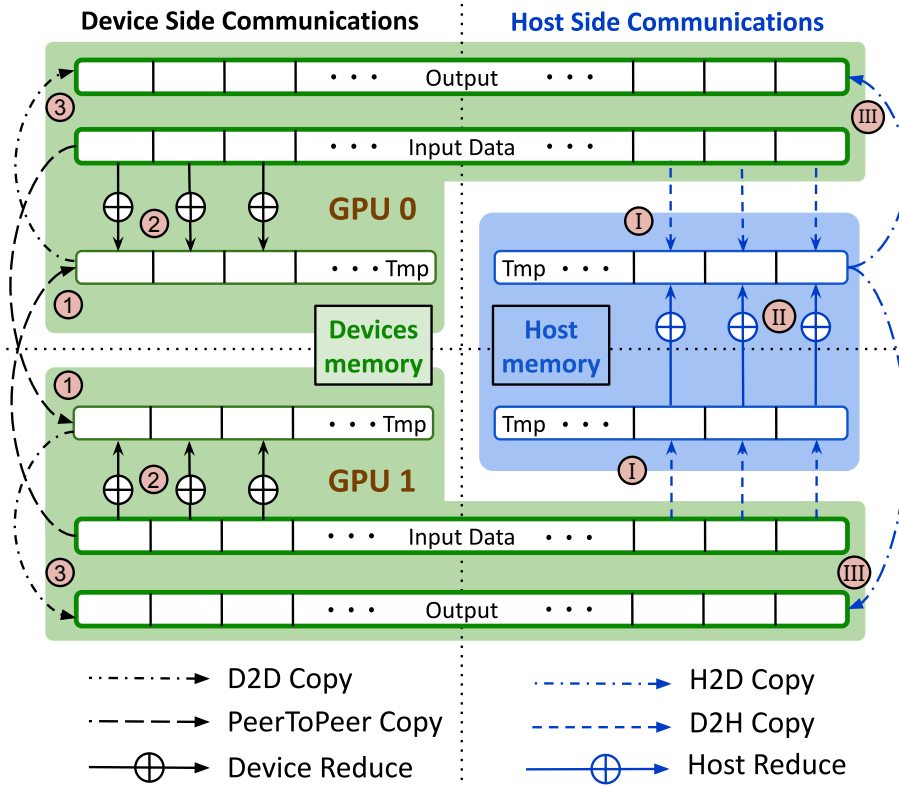


Figure 5.2: Multi-path Heterogeneous *Pairwise Exchange Allreduce* on two GPUs using NVLink and PCIe.

the host system. We have statically tuned this threshold for various message sizes and different systems, and we report the best results.

Additionally, we have implemented our proposed algorithm as well as the algorithms described in Table 5.1 outside the MPI library by hooking the *Allreduce* function inside our framework using CUDA Driver API, controlling the GPUs via the CUDA contexts. We have also utilized CUDA streams and CUDA events to achieve the desired concurrency and overlap between several simultaneous series of tasks on both sides. Algorithm 5.1 and Algorithm 5.2 present the simplified implementation of our multi-path pipelined pairwise *Allreduce* and the device/host exchange functions, respectively.

5.3.2 Multi-path Heterogeneous MPI_Allreduce

As Figure 5.2 depicts on the devices side (left), in **Step 1**, each GPU context initiates a series of Peer-to-Peer transfers (*get* operations) to retrieve the peer data through the *get* primitive, chunk by chunk, and on a specific stream. In **Step 2**, to apply reduction on the received chunk with the local data, a computation kernel is enqueued on the corresponding stream. Finally, in **Step 3**, the reduced data is stored in the output buffer. For further improvement, this step can be packed into the computation kernel to store the results directly into the output buffer from the temporary chunk.

The algorithm on the host side (right) performs differently. Instead of the *get* operations, a series of D2H *put* operations are initiated in **Step I** from each CUDA context to store their input data to a pre-allocated pinned memory location, chunk by chunk. Then, one of the GPU contexts (e.g., GPU 0) synchronizes its specific streams with the completion of the D2H *put* operations from both contexts through CUDA events. After that, for each pair of received chunks from both sides, a *callback* is enqueued into the corresponding streams with the `cudaHostFunction` type. When the data becomes ready, in **Step II**, the host function is called to reduce the data and store the results in the output buffer. For this step, we have utilized OpenMP threads to perform the reduction on the host side. Finally, in **Step III**, the reduced data is sent back to the devices through a series of H2D *get* operations by each CUDA context.

The reason for using *get* operations instead of *put* operations on the device side is that it requires one less synchronization point compared to *put* operations. On the device side, in *get* operations, each GPU can independently initiate the data transfer from the peer GPU without needing to coordinate with the peer for data availability. This way the receiving GPU can launch the reduction kernel as soon as the data is available, without waiting for the peer to signal that the data has been sent. On the other hand, the reason for using

Algorithm 5.1: Multi-path Pipelined Pair Exchange Allreduce

```
1 Input: dev_ids, data_buffers, data_size
2 Output: Reduced results in dsts[src], dsts[dst]
3 Function multipah_pipelined_hetero_allreduce:
    // Both ranks execute this function
4   if This is the first call then
5       Initialize CUDA contexts and other objects for all the devices
6       Create CUDA streams and CUDA events for all the devices
7       Allocate direct_temp_bufs, host_temp_bufs
8   end
9   Split data_size into direct_size (device path) and host_size (host path)
10  Compute chunk sizes and offsets for host and device paths based on data_size
11  Configure grid/block/thread parameters for device kernels
12  for each pipeline step p do
13      if p has device work then
14          Update starting index for the current device-side chunk
15          Device_Exchange(dev_ids, data_buffers, direct_chunk_size, index)
16      end
17      if p has host work and host_size > 0 then
18          if I am the leader (e.g., src_dev_id) then
19              Update starting index for the current host-side chunk
20              Host_Exchange(dev_ids, data_buffers, host_chunk_size, index)
21          end
22      end
23  end
24  Link and bind synchronize points for device and host via streams/events
25  Ensure completion of operations in utilized streams
26 end
```

put operations on the host side is that it allows for better overlap between host-side and device-side data transfers without recurring context switches between the CPU threads competing for resources. In fact in this way, there is no need to busy-wait for the data to be available on the host side, as the *put* operations can be enqueued on the **CUDA** stream to wake up the host function only when the data is ready.

In further detail, Algorithm 5.1 presents the simplified implementation of our multi-path pipelined pairwise *Allreduce*. The function `multipath_pipelined_hetero_allreduce` is called by both ranks to perform the *Allreduce* operation. It first initializes the necessary

CUDA objects and allocates temporary buffers if it is the first call (Lines 4-8). Then, it splits the data size into two parts for the device and host paths, computes chunk sizes and offsets, and configures grid/block/thread parameters for device kernels (Lines 9-11). The ideal chunk size on each path is statically determined based on the hardware characteristics and the data size. Next, it enters a loop over the pipeline steps, where it calls the device and host exchange functions as needed in a round-robin fashion (Lines 12-23). Finally, it links and binds synchronization points for device and host via streams and events, and ensures the completion of operations in the utilized streams before returning (Lines 24, 25).

Algorithm 5.2 demonstrates the device-side and host-side exchange functions used in the *Allreduce* algorithm. The function `Device_Exchange` is executed by both ranks asynchronously and concurrently (Lines 4-12). It first determines the peer and self device IDs, data pointers, and selects streams and events for the current chunk (Lines 4-6). Then, it performs a Peer-to-Peer transfer, followed by enqueueing a computation kernel to reduce the chunk, and storing the reduced chunk to the destination buffer (Lines 7-9). Finally, it records completion events, and returns the events for synchronization to the main method (Lines 10 and 11). On the other hand, the function `Host_Exchange` is executed only by the leader rank (e.g., GPU 0) (Lines 14-27). It follows a similar pattern as the device exchange function, but instead performs D2H and H2D transfers, and enqueues a host function to reduce the chunks (Lines 17-21). The host reduction function `Add_Host` performs an element-wise addition of two input buffers using OpenMP (Lines 29-32). Then, after the reduction, it transfers the reduced chunks back to both devices (Lines 23 and 24). Finally, it records completion events and returns them for synchronization (Lines 25 and 26).

Figure 5.3 presents a summarized profile of our multi-path heterogeneous *Allreduce* on one of the GPUs (the other GPU profile is very similar). The profile clearly illustrates how all interconnects are utilized, and how both CPU and GPU compute resources are leveraged

Algorithm 5.2: Device-side and host-side exchanges in Pipelined Pair Exchange

```
1 Input: dev_ids, data_buffers, direct_chunk_size, index
2 Output: events for synchronization
3 Function Device_Exchange:
    // Both ranks execute this function asynchronously and concurrently
4     Determine peer and self device IDs from dev_ids
5     Determine src and dst data pointers from data_buffers and index (chunk c)
6     Select streams and CUDA events for both devices for chunk c
7     Get chunk c from peer to self via cuMemcpyPeerAsync // Figure 5.2: Step 1
8     Put a CUDA kernel Add_Device for chunk c on stream // Figure 5.2: Step 2
9     Store reduced chunk c from temporary buffer to dst // Figure 5.2: Step 3
10    Record completion events for chunk c on stream
11    return events for synchronization
12 end
13 Function Host_Exchange:
    // Only the leader rank executes this function
14    Determine peer and self device IDs from dev_ids
15    Determine src and dst data pointers from data_buffers and index (chunk c)
16    Select streams and CUDA events for both devices for chunk c
17    Put chunk c from peer → host via cuMemcpyDtoHAsync // Figure 5.2: Step I
18    Put chunk c from self → host via cuMemcpyDtoHAsync // Figure 5.2: Step I
19    Record completion events for both transfers on streams
20    Put a wait for both transfers to complete
21    Enqueue cudaHostFunction Add_Host on stream to reduce c // Figure 5.2: Step II
22    Record completion event for host reduction on stream
23    Put reduced chunk c from host → peer via cuMemcpyHtoDAsync // Figure 5.2: Step III
24    Put reduced chunk c from host → self via cuMemcpyHtoDAsync // Figure 5.2: Step III
25    Record completion events for both transfers on streams
26    return events for synchronization
27 end
28 Function Add_Host:
29    Determine thread_count based on chunk_size and platform
30    for i ← 0 to chunk_size/sizeof(float) do in parallel
31        | dst_tmp[i] ← dst_tmp[i] + src_tmp[i]
32    end
33 end
```

for reduction. As depicted, the data transfers on NVLink start first, followed by the D2H transfers on PCIe (Step I). Once the data is available on the host, the host reduction starts while the NVLink transfers are still ongoing (Step II). After the host reduction is completed,

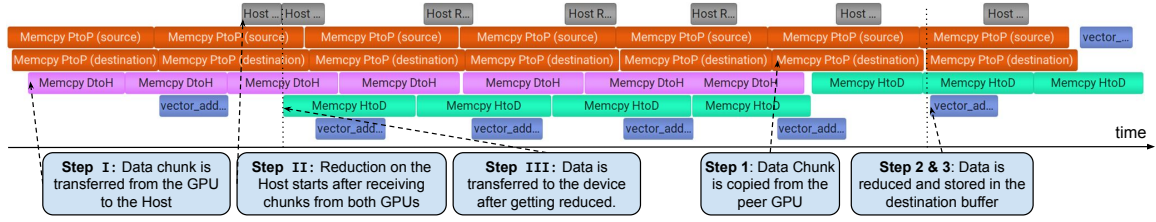


Figure 5.3: A summarized NVIDIA Nsight Systems timeline profile of the multi-path heterogeneous *Allreduce* (128 MB) on one of the GPU on Daisy cluster. Grey and blue boxes are reduction operations on the host and the device, respectively. Brown boxes are data transfers on NVLink, purple boxes are D2H transfers, and green boxes are H2D transfers on PCIe.

the reduced data is sent back to both GPUs via H2D transfers (Step III). Meanwhile, on the device side, as soon as each chunk is received from the peer GPU, the device reduction starts immediately, overlapping with both NVLink transfers and host-side operations. This profile demonstrates the effectiveness of our design in achieving concurrency and overlap between communication and computation tasks across both paths.

Note that all these operations are initiated by a CPU core in an asynchronous manner, and the GPU contexts are synchronized with the completion of data transfers and computations using CUDA events. Additionally, special cases such as the last chunk of data or data sizes that are not multiples of the chunk size are carefully handled to ensure data integrity.

5.4 Performance Model for Heterogeneous Multi-Path Allreduce

To better understand and predict the performance of our multi-path heterogeneous *Allreduce* framework, we have developed a simplified performance model based on the Hockney's model. In Chapter 4, we established a performance model for multi-path intra-node GPU communication, primarily focusing on communication time. In this section, we extend the model to incorporate the cost of computation that arises in a heterogeneous *Allreduce*

operation. Specifically, we consider the concurrent utilization of two paths: (i) a direct GPU-to-GPU path (e.g., NVLink), and (ii) a host-staged path, where the data is first staged through the host memory and subsequently reduced on the CPU before being transferred back to the GPUs.

The main idea is that the overall performance of the heterogeneous collective depends not only on communication, but also on the computation time of the reduction kernels on both host and device. Similar to our earlier discussion, the optimal configuration is achieved when both paths finish in approximately the same time, thereby maximizing concurrency. Moreover, as the computation on the host is performed by the leader rank, we will only consider the leader rank in our model. This will not affect the overall performance, as the non-leader rank will not be blocked by the host computation, and the bidirectional peer-to-peer transfers on the device can simultaneously occur.

Table 5.2 summarizes the symbols and notations used in our extended performance model. Compared to Table 4.1, we have added parameters to account for the computation time on both the host and the device. Moreover, we have included launch latency for device kernels and host functions.

5.4.1 Modeling Communication and Computation

We now write the total time of *Allreduce* on the two GPUs using the discussed Steps in Figure 5.2. If pipelining is not enabled, for each path (direct or host-staged), the total time can be expressed as the sum of communication and computation times including synchronization and kernel launch overheads. Note that on the device side, we have the data transfer time, synchronization overhead, kernel launch overhead, and computation time. On the other hand, for the host side, we have a data transfer, synchronization overhead, a kernel launch overhead, synchronization, and another data transfer back to

Notation	Description
T	Total time of the heterogeneous Allreduce
n	Message size in bytes
T_d, T_h	Total time along device and host paths
T_d^C, T_h^C	Communication time on the device and host paths
T_d^R, T_h^R	Reduction time on the device and host
θ_d, θ_h	Fraction of message size assigned to device and host path
α_d, β_d	Latency and bandwidth parameters for communication of device path
α_h, β_h	Latency and bandwidth parameters for communication of host path
κ_d, κ_h	Kernel (or host function) launch overhead
ϵ_d, ϵ_h	Synchronization overhead for device and host
γ_d, γ_h	Effective device- and host-side compute throughput [ops/sec]
λ_d, λ_h	Number of chunks on device and host paths if pipelining is enabled
f	Computational intensity (ops per element, e.g., 1 for sum)

Table 5.2: Extended notation including computation parameters.

the devices. Therefore, we can express the total time along each path as the following:

$$T_d = T_d^C + \epsilon_d + \kappa_d + T_d^R \quad (5.1)$$

$$T_h = T_h^C + \epsilon_h + \kappa_h + T_h^R + \epsilon_h + T_h^C \quad (5.2)$$

As we can recall from Chapter 4, the communication time along device (or host) path can be expressed using the Hockney’s model as:

$$T_d^C = \alpha_d + \frac{\theta_d n}{\beta_d}, \quad T_h^C = \alpha_h + \frac{\theta_h n}{\beta_h}, \quad (5.3)$$

where α_d and β_d are the latency and bandwidth parameters of the device path, respectively, and θ_d is the fraction of the total message size n assigned to the device path. Note that $\theta_d + \theta_h = 1$ and $0 \leq \theta_d, \theta_h \leq 1$.

The computation term can be approximated as:

$$T_d^R = \frac{f \cdot \theta_d n}{\gamma_d}, \quad T_h^R = \frac{f \cdot \theta_h n}{\gamma_h}, \quad (5.4)$$

where f is the number of floating-point (or integer) operations per element (for a simple vector addition or summation, $f = 1$), and γ_d is the effective throughput of the device kernel (in ops/sec), and γ_h is the effective throughput of the host function (in ops/sec). We have obtained γ_d and γ_h empirically through micro-benchmarks, via a GPU kernel performing vector addition and a multi-threaded OpenMP reductions for the host over large arrays.

5.4.2 Overall Time and Optimization

As before, the heterogeneous *Allreduce* operation completes when the slowest path finishes. Therefore, the overall time can be expressed as:

$$T = \max_{i \in \{\text{device(d)}, \text{host(h)}\}} \left(T_i^C + T_i^R \right). \quad (5.5)$$

The optimization problem is then:

$$\text{minimize } T = \max_i \left(T_i^C + T_i^R \right), \quad \text{subject to } \sum_i \theta_i = 1, \quad 0 \leq \theta_i \leq 1. \quad (5.6)$$

As we have only two paths, the optimal solution is achieved when both paths complete in equal time:

$$T_d = T_h \quad (5.7)$$

5.4.3 Analytical solution for the optimal split

Using Equations (5.3) and (5.4) and substituting into Equations (5.1) and (5.2), the per-path times become:

$$T_d = \underbrace{\alpha_d + \epsilon_d + \kappa_d}_{\Delta_d} + \theta_d n \left(\frac{1}{\beta_d} + \frac{f}{\gamma_d} \right), \quad (5.8)$$

$$T_h = \underbrace{2\alpha_h + 2\epsilon_h + \kappa_h}_{\Delta_h} + (1 - \theta_d) n \left(\frac{2}{\beta_h} + \frac{f}{\gamma_h} \right), \quad (5.9)$$

where we have used $\theta_h = 1 - \theta_d$. For compactness we define

$$\Omega_d = \left(\frac{1}{\beta_d} + \frac{f}{\gamma_d} \right), \quad \Omega_h = \left(\frac{2}{\beta_h} + \frac{f}{\gamma_h} \right) \quad (5.10)$$

Then Equation (5.8) and Equation (5.9) become:

$$T_d = \Delta_d + \theta_d n \Omega_d, \quad T_h = \Delta_h + (1 - \theta_d) n \Omega_h \quad (5.11)$$

As argued earlier, the optimal split minimizing the overall completion time $T = \max(T_d, T_h)$ (with two paths) is obtained when the two paths finish at the same time: $T_d = T_h$. Solving Equation (5.7) for θ_d gives:

$$\theta_d^* = \frac{\Delta_h - \Delta_d + n\Omega_h}{n(\Omega_d + \Omega_h)}, \quad \theta_h^* = 1 - \theta_d^* \quad (5.12)$$

5.4.4 Incorporating Pipelining

When pipelining is enabled, the message assigned to each path is divided into multiple chunks and the stages of transfer and reduction can be overlapped across chunks. Let

λ_d and λ_h denote the number of chunks on the device and host paths respectively. Then, considering Equation (5.1) and Equation (5.2), we can express the time required to process a single chunk c on each path as:

$$T_d(c) = T_d^C(c) + \epsilon_d + \kappa_d + T_d^R(c), \quad (5.13)$$

$$T_h(c) = T_h^C(c) + \epsilon_h + \kappa_h + T_h^R(c) + \epsilon_h + T_h^C(c), \quad (5.14)$$

where $T_d^C(c)$ and $T_h^C(c)$ are the communication times for chunk c on the device and host paths, respectively, and $T_d^R(c)$ and $T_h^R(c)$ are the computation times for chunk c on the device and host, respectively. Using Equation (5.3) and Equation (5.4), we can express these times as:

$$T_d^C(c) = \alpha_d + \frac{\theta_d n / \lambda_d}{\beta_d}, \quad T_h^C(c) = \alpha_h + \frac{\theta_h n / \lambda_h}{\beta_h} \quad (5.15)$$

$$T_d^R(c) = \frac{f \cdot (\theta_d n / \lambda_d)}{\gamma_d}, \quad T_h^R(c) = \frac{f \cdot (\theta_h n / \lambda_h)}{\gamma_h} \quad (5.16)$$

Now, for each path, there are two cases to consider: (1) when the computation is faster than communication, and (2) when communication is faster than computation. In the first case, the total time on each path is dominated by the communication time of all chunks, and the computations are hidden. This is shown in Figure 5.4 with the types **B** and **E**, for device and host, respectively. While in the second case, the process is dominated by the computation time of the chunks, as shown in Figure 5.4 with the types **C** and **F**, for device and host, respectively. Therefore, we can express the total time on each path depending on the dominant factor as follows:

$$T_d = \begin{cases} \lambda_d T_d^C(c) + \epsilon_d + \kappa_d + T_d^R(c), & \text{if communication is dominant (Case 1)} \\ T_d^C(c) + \lambda_d (\epsilon_d + \kappa_d + T_d^R(c)), & \text{if reduction is dominant (Case 2)} \end{cases} \quad (5.17)$$

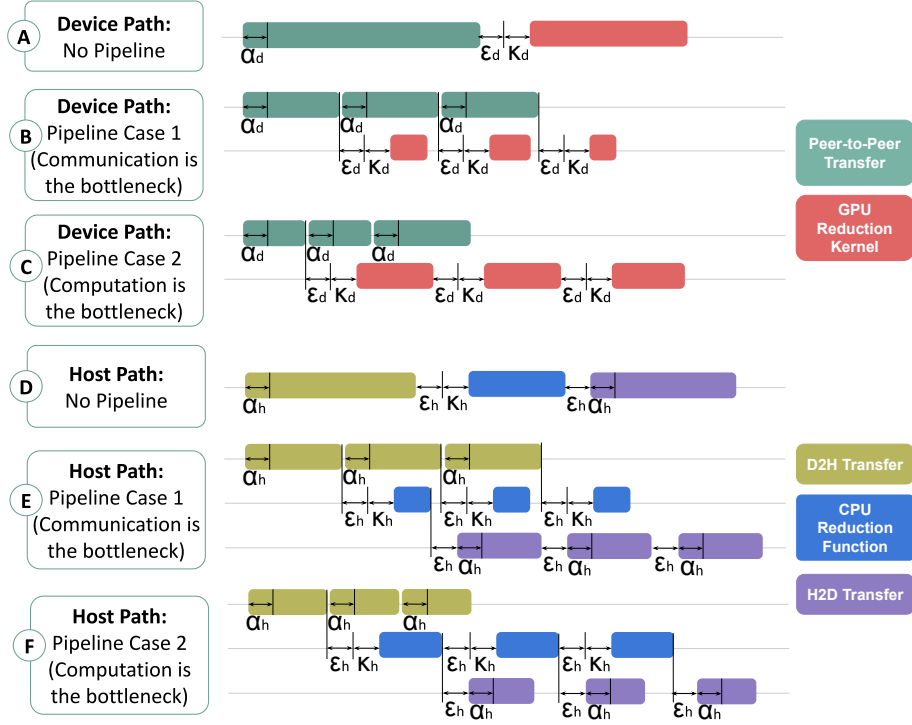


Figure 5.4: Different cases with and without pipelined communication/computation on device and host paths. Note that the startup latency (α), synchronization cost (ϵ), and kernel launch overhead terms (κ) are different for each path, but for simplicity, we have used the same notation in the figure.

Similarly for the host path:

$$T_h = \begin{cases} T_h^C(c) + \epsilon_h + \kappa_h + T_h^R(c) + \lambda_h (\epsilon_h + T_h^C(c)), & \text{if communication is dominant (Case 1)} \\ T_h^C(c) + \lambda_h (\epsilon_h + \kappa_h + T_h^R(c)) + \epsilon_h + T_h^C(c), & \text{if reduction is dominant (Case 2)} \end{cases} \quad (5.18)$$

We restrict the analysis to **Case 1** on both paths, for which the per-path completion times from Equations (5.17) and (5.18) reduce to:

$$T_d = \lambda_d T_d^C(c) + \epsilon_d + \kappa_d + T_d^R(c), \quad (5.19)$$

$$T_h = T_h^C(c) + \epsilon_h + \kappa_h + T_h^R(c) + \lambda_h (\epsilon_h + T_h^C(c)). \quad (5.20)$$

Substituting the chunk-level terms from Equations (5.15) and (5.16), we obtain the following simplified forms:

$$T_d = \lambda_d \alpha_d + \frac{\theta_d n}{\beta_d} + \epsilon_d + \kappa_d + \frac{f \cdot \theta_d n}{\lambda_d \gamma_d}, \quad (5.21)$$

$$T_h = (1 + \lambda_h) \alpha_h + (1 + \lambda_h) \frac{\theta_h n}{\lambda_h \beta_h} + (1 + \lambda_h) \epsilon_h + \kappa_h + \frac{f \cdot \theta_h n}{\lambda_h \gamma_h}. \quad (5.22)$$

Rearranging, we can write each path as:

$$T_d = \underbrace{\lambda_d \alpha_d + \epsilon_d + \kappa_d}_{\Delta_d^{\text{pipe}}} + \theta_d n \left(\frac{1}{\beta_d} + \frac{f}{\lambda_d \gamma_d} \right), \quad (5.23)$$

$$T_h = \underbrace{(1 + \lambda_h) \alpha_h + (1 + \lambda_h) \epsilon_h + \kappa_h}_{\Delta_h^{\text{pipe}}} + \theta_h n \left(\frac{1 + \lambda_h}{\lambda_h \beta_h} + \frac{f}{\lambda_h \gamma_h} \right). \quad (5.24)$$

For compactness, we define:

$$\Delta_d^{\text{pipe}} = \lambda_d \alpha_d + \epsilon_d + \kappa_d, \quad (5.25)$$

$$\Delta_h^{\text{pipe}} = (1 + \lambda_h) \alpha_h + (1 + \lambda_h) \epsilon_h + \kappa_h, \quad (5.26)$$

$$\Omega_d^{\text{pipe}} = \frac{1}{\beta_d} + \frac{f}{\lambda_d \gamma_d}, \quad (5.27)$$

$$\Omega_h^{\text{pipe}} = \frac{1 + \lambda_h}{\lambda_h \beta_h} + \frac{f}{\lambda_h \gamma_h}. \quad (5.28)$$

Therefore, we can express the per-path times as:

$$T_d = \Delta_d^{\text{pipe}} + \theta_d n \Omega_d^{\text{pipe}} \quad (5.29)$$

$$T_h = \Delta_h^{\text{pipe}} + \theta_h n \Omega_h^{\text{pipe}}. \quad (5.30)$$

Before we proceed to find the optimal split, we should note that number of chunks (λ_d

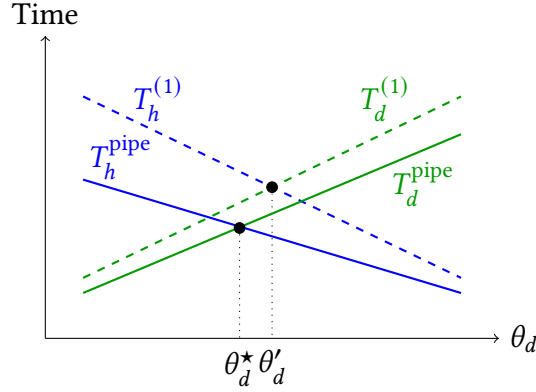


Figure 5.5: Illustration of how pipelining changes the load-balance condition. The dashed lines represent the single-chunk per-path times, intersecting at θ'_d . With pipelining, the effective slopes and offsets of each path change, shifting the balance point to θ_d^* .

and λ_h) are functions of θ_d and θ_h , respectively, and pipelining (chunking) and byte split interact. In other words, changing λ_d and λ_h changes the optimal split, which changes the load-balance between the two paths. This is conceptually illustrated in Figure 5.5. The dashed lines represent the single-chunk per-path times, intersecting at θ'_d . With pipelining, the effective slopes and offsets of each path change, shifting the balance point to θ_d^* .

Therefore, for finding the optimal split, we either can: (1) treat λ_d and λ_h as fixed parameters, find the split (θ'), then co-optimize the number of chunks based on the found split as a starting point, which can be done iteratively with a few iterations to converge to the optimal solution; or (2) treat λ_d and λ_h as variables and find the optimal number of chunks on each path, then find the optimal split accordingly.

Method 1: To find the optimal split with fixed number of chunks, we can directly use Equation (5.29) and Equation (5.30) and solve for θ_d using the condition in Equation (5.7).

So, we can solve for θ_d as:

$$\theta'_d = \frac{\Delta_h^{\text{pipe}} - \Delta_d^{\text{pipe}} + n\Omega_h^{\text{pipe}}}{n(\Omega_d^{\text{pipe}} + \Omega_h^{\text{pipe}})}, \quad \theta'_h = 1 - \theta'_d. \quad (5.31)$$

Then, we can iteratively test different values of λ_d and λ_h to find the optimal split and number of chunks. Our experiments show that a feasible window of (1, 32) for both λ_d and λ_h are sufficient to converge to the optimal solution. This local search is very fast and can be done in a few microseconds which is negligible compared to the overall *Allreduce* time. Also, this configuration can be cached for future calls with the same message size.

Note that Equation (5.31) gives the initial message split (θ'_d, θ'_h) between device and host paths when pipelining is enabled and communication dominates computation (Case 1). Similar formulation can be carried out for Case 2, when computation dominates. As we can see, if the number of chunks λ_d and λ_h are set to 1, the pipelined model generates the non-pipelined model in Equation (5.12).

Method 2: We treat λ_d and λ_h as continuous positive variables and minimize T_d and T_h by taking derivatives w.r.t. λ_d and λ_h , respectively. This gives us the optimal number of chunks on each path. Then we can find the optimal split point θ_d^* and θ_h^* by substituting λ_d^* and λ_h^* and solving Equation (5.7). We again restrict the analysis to Case 1.

For the device path, we had Equation (5.23), which we differentiate w.r.t. λ_d and set to zero:

$$\frac{dT_d}{d\lambda_d} = \alpha_d + \theta_d n \cdot \frac{d}{d\lambda_d} \left(\frac{f}{\lambda_d \gamma_d} \right) = \alpha_d - \theta_d n \frac{f}{\lambda_d^2 \gamma_d} = 0$$

Therefore, we can write λ_d as:

$$\lambda_d^* = \sqrt{\frac{\theta_d n f}{\alpha_d \gamma_d}} \quad (5.32)$$

Similarly for the host path, we had Equation (5.24), which we differentiate w.r.t. λ_h and set

to zero. Then we can find λ_h as:

$$\lambda_h^* = \sqrt{\frac{\theta_h n \left(\frac{1}{\beta_h} + \frac{f}{\gamma_h} \right)}{\alpha_h + \epsilon_h}} \quad (5.33)$$

Note that in both cases, the second derivatives are positive for $\lambda_d > 0$ and $\lambda_h > 0$, so the found extremums are minimum ($\frac{d^2 T_h}{d\lambda_h^2} > 0$ and $\frac{d^2 T_d}{d\lambda_d^2} > 0$).

Now, substituting Equation (5.32) and Equation (5.33) back into Equation (5.23) and Equation (5.24), we can update the per-path times:

$$T_d = 2\sqrt{\frac{\alpha_d \theta_d n f}{\gamma_d}} + \epsilon_d + \kappa_d + \frac{\theta_d n}{\beta_d} \quad (5.34)$$

$$T_h = 2\sqrt{(\alpha_h + \epsilon_h) \theta_h n \left(\frac{1}{\beta_h} + \frac{f}{\gamma_h} \right)} + \kappa_h + \theta_h n \left(\frac{1}{\beta_h} + \frac{f}{\gamma_h} \right) \quad (5.35)$$

Here, finding the optimal split gets very involved and may not provide a closed-form solution. It appears that either a numerical method should be used, or a polynomial equation of degree four (with many terms) should be solved. In either case, this solution is more complex and time consuming than the previous method. Therefore, to continue with this method, we should approximate the number of chunks to the closest linear equations, then solve for the optimal split. This is similar to the idea developed in Chapter 4 and Equation (4.23). However, we will not pursue this direction further in this dissertation.

5.4.5 Developing the Model in Our Framework

To use the model in our framework, we need to measure the parameters of the model (from Table 5.2) through various micro-benchmarks. Note that these parameters are measured once per system and reused for all applications. In more detail, we need to measure the

following parameters:

- α_d, β_d and α_h, β_h : micro-benchmarks for P2P transfers over the device path and the host-staged path (D2H and H2D) in each direction.
- γ_d, γ_h : running computation-only kernel (device) and multithreaded host reduction on arrays of various sizes to obtain effective throughput (ops/sec).
- κ_d, κ_h : measuring kernel-launch overheads (CUDA kernel launch, or host OpenMP thread initiations).
- ϵ_d, ϵ_h : measuring synchronization overheads for device and host (e.g. CUDA event synchronization and host callback dispatch/synchronization cost).

Then, we can use Algorithm 5.3 to find the optimal split and number of chunks for pipelining. The algorithm performs a local search over a feasible range of chunk counts (1 to 32) for both device and host paths, computes the corresponding split using Equation (5.31), and evaluates the total time for each configuration (Lines 11-22). The best configuration is cached for future calls with the same message size (Line 25).

Algorithm 5.3: Finding the Optimal Pipelined Dual-Path Split (Method 1).

```
1 Input:  $src\_dev\_id, dst\_dev\_id, data\_size$  ( $n$ ),  $\{\alpha, \beta, \epsilon, \kappa, \gamma, f\}$ 
2 Output:  $\theta_d, \theta_h, \lambda_d, \lambda_h$ 
3 Function  $find\_pipelined\_split$ :
4   if cached config exists for ( $src, dst, n$ ) then
5     return cached config
6   end
7    $best\_time \leftarrow \infty$ 
8    $best\_config \leftarrow \emptyset$ 
9   for  $\lambda_d \leftarrow 1$  to 32 do
10    for  $\lambda_h \leftarrow 1$  to 32 do
11      Compute  $\Delta_d$  // According to Equation (5.25)
12      Compute  $\Delta_h$  // According to Equation (5.26)
13      Compute  $\Omega_d$  // According to Equation (5.27)
14      Compute  $\Omega_h$  // According to Equation (5.28)
15      Solve for  $\theta_d$  // According to Equation (5.31)
16      Compute  $T_d$  // According to Equation (5.29)
17      Compute  $T_h$  // According to Equation (5.30)
18       $T \leftarrow \max(T_d, T_h)$ 
19      if  $T < best\_time$  then
20         $best\_time \leftarrow T$ 
21         $best\_config \leftarrow (\theta'_d, \theta'_h, \lambda_d, \lambda_h)$ 
22      end
23    end
24  end
25  store  $best\_config$  in cache
26  return  $best\_config$ 
27 end
```

5.5 Evaluation

To evaluate the performance of our framework, we compared it against [NCCL](#) (v2.24.3-1), [MPI](#) (v5.0.2) configured with [UCC](#) (v1.3), [UCX](#) (v1.17), and [CUDA](#) (v12.6), as well as three commonly used *Allreduce* algorithms: *Reduce Broadcast*, *Pairwise Exchange* and *Segmented Ring*, including their pipelined and kernel-only versions. In the pipelined versions of these algorithms, the data is split into multiple chunks to overlap the algorithm steps, while in the kernel-only versions, both computation and communication tasks are packed into a single kernel, similar to how [NCCL](#) performs. These implementations utilize neither

Table 5.3: Experimental Setup

Cluster	GPUs x4	CPU	GPUs Topology
Daisy	A30	Intel Xeon Gold 6338	2x2 with 4-pairs NVLink
Mist	V100	Power9 8335-GTH	2x2 with 3-pairs NVLink
Narval	A100	AMD EPYC 7413	Full Mesh (4 pairs)

multi-path nor heterogeneous techniques, thereby ensuring a fair comparison between our framework and **NCCL**, by highlighting the sources of performance gains and improvements. These algorithms as well as our framework are implemented in our standalone library that intercepts `MPI_Allreduce` calls using **MPI**'s *PMPI* interface.

5.5.1 Experimental Setup

We conducted a series of experiments on three different **GPU** configurations from these clusters: Daisy, Mist, and Narval. **Mist** is a two-socket node equipped with two NVIDIA V100 **GPUs** on each socket. Each **GPU** pair is connected by three pairs of NVLink-V2, and with one of the CPUs, they are configured as a single **NUMA** node. The CPUs are Power9 8335-GTH with 16 cores and four Hardware threads. The configuration of Daisy and Narval nodes are described in Section 3.5.1 and Section 4.5.1, respectively. Narval nodes with only 6 cores per **NUMA** node provide an example that is not ideal for our framework due to the **NUMA** configuration and the low number of cores available for computation per each **NUMA**.

We assessed the performance of our framework using two **GPUs** in each node in these clusters, resembling a two-GPU topology. Table 5.3 summarizes the specifications of the experimental setups and how the **GPUs** are connected.

5.5.2 Performance Analysis

Figure 5.6 illustrates the speedup of our proposed methodology, (I) *multi-path heterogeneous Pairwise Exchange*, as well as [MPI](#) and [UCC](#), against standalone [NCCL](#) on Daisy, Narval, and Mist nodes, for message sizes ranging from 32 MB to 512 MB. Evaluating the performance of various algorithms/solutions, we can observe that our proposed approach outperforms [NCCL](#) by up to 1.45x for very large messages on Daisy. Other observations include:

- **Observation 1:** By implementing and comparing similar *Allreduce* algorithms to [NCCL](#) and [UCC](#) (pipelined and kernel-based *Segmented Ring* and *Pairwise Exchange*), we can observe that the runtime libraries ([NCCL](#) and [MPI](#)) have a small but noticeable overhead compared to their standalone versions.
- **Observation 2:** Our proposed method outperforms all the other implemented algorithms, confirming that utilizing multiple paths as well as CPUs and [GPUs](#) for computation improves the performance of *Allreduce* for large messages, independent of the communication libraries overhead.
- **Observation 3:** Comparing the improvements from different nodes, we can observe that the performance gain is less significant on Narval. This is due to the [NUMA](#) configuration of this system, the low number of CPU cores per [NUMA](#) for computation, and the greater performance difference between the NVLink and [PCIe](#).
- **Observation 4:** Pipelined-based collectives outperform the kernel versions due to utilizing *copy engines*. Although not shown in the figure, this trend starts around message sizes larger than 16 MB. However, for smaller messages, the kernel versions are increasingly more efficient. This observation is consistent with previous studies [155].
- **Observation 5:** In some cases, *Segmented Ring* outperforms our proposed *Pairwise Exchange* due to the hardware specifics of the tested systems, where NVLink provides

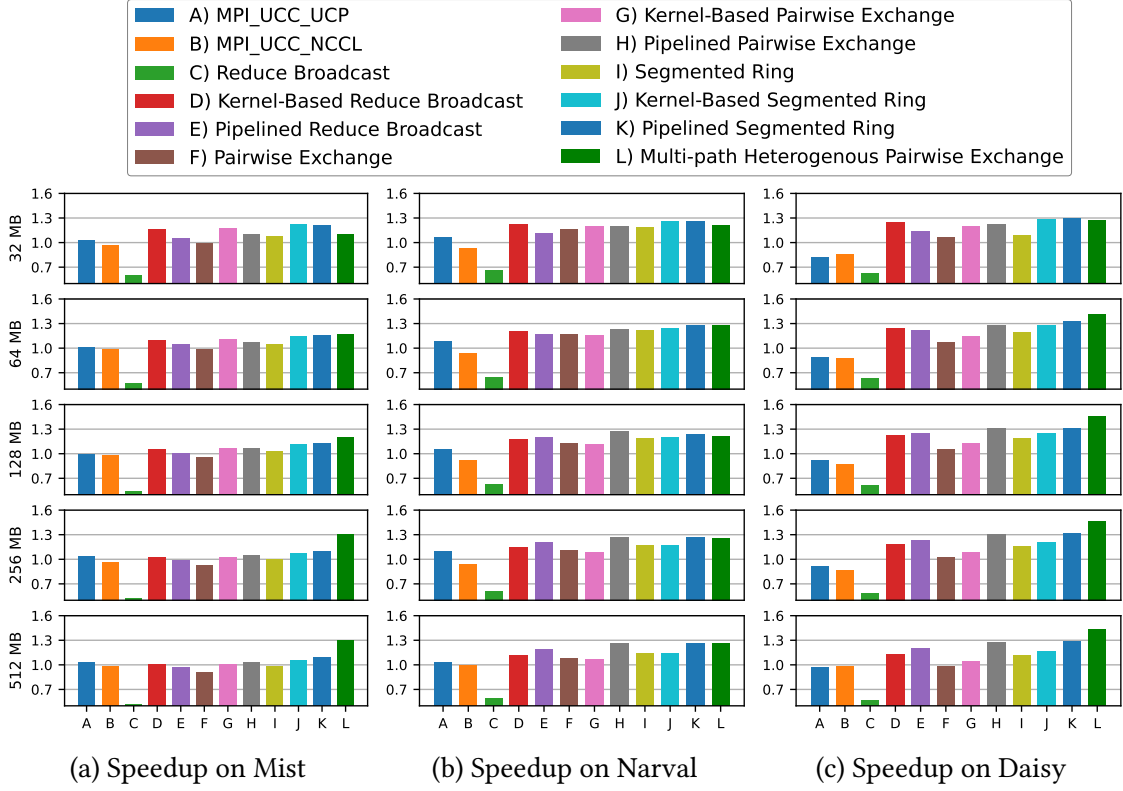


Figure 5.6: Speedup of various Allreduce algorithms (including MPI_UCC_UCP, MPI_UCC_NCCL) against standalone NCCL on Daisy, Narval, and Mist nodes, using two GPUs.

high bandwidth between any two GPUs compared to PCIe (see Figure 5.6(b)). Moreover, the reductions on the CPUs are particularly slower on Narval due to the low number of cores available per NUMA node for computation.

5.5.3 Performance Model Evaluation

To validate our performance model, we implemented it in our framework and compared the predicted execution times with the actual measured times for our proposed multi-path heterogeneous *Pairwise Exchange*, compared to the statically tuned method on Daisy node. Figure 5.7 presents the results of this comparison for message sizes ranging from 4 MB to 512MB. The results indicate that our performance model accurately predicts the

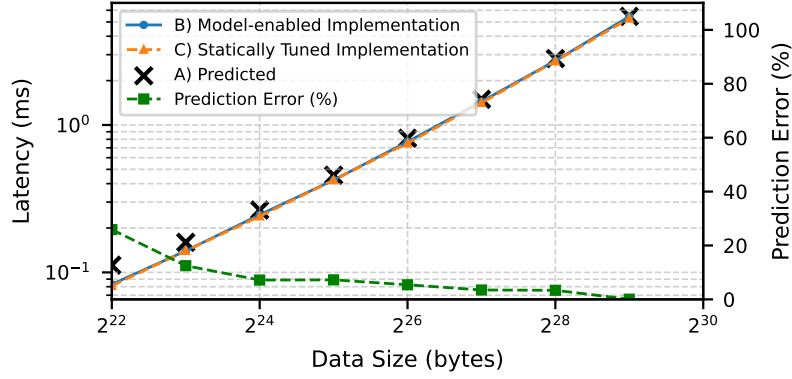


Figure 5.7: Comparison of predicted and measured execution times for various Allreduce algorithms on Daisy nodes using two GPUs. The results demonstrate the accuracy of our performance model, with an average error of 8%

execution times for our proposed algorithm, with an average error of 8% observed for the pipelined *Pairwise Exchange* on Daisy. The model’s accuracy is particularly notable for larger message sizes, where it closely aligns with the measured times. This validation confirms the reliability of our performance model in estimating the execution times of different *Allreduce* algorithms across diverse hardware configurations.

5.6 Summary

In this chapter, we presented a novel multi-path heterogeneous *Allreduce* framework for multi-GPU systems. By leveraging both GPU and CPU resources and interconnects, our approach effectively utilizes available bandwidth and computational power to enhance performance. Our experimental results demonstrate that our framework can achieve up to 1.45x speedup over NCCL. We also developed a performance model to find the optimal splitting ratio between CPU and GPU computations, and to predict the performance of our proposed *Allreduce* algorithms. Our evaluations confirm the effectiveness of our performance model, with an average error less than 8% in predicting execution times across different message sizes. The integration of multi-path communication and hetero-

geneous computing demonstrates to be a promising direction for improving the efficiency of *Allreduce* operations commonly used in HPC and DL.

In the next chapter, we will explore a different aspect of HPC systems, which is about optimizing a scientific application. Specifically, we will focus on the combinatorial problems and how to leverage HPC techniques to build a highly optimized solution for them.

Chapter 6

Legendre Pairs Algorithm Optimizations

6.1 Introduction

HPC is an essential tool in modern scientific computing, enabling researchers to tackle complex problems that are computationally intensive and require significant processing power. The ability to perform large-scale simulations, data analysis, and algorithmic computations has revolutionized various fields, including physics, chemistry, biology, and mathematics. In particular, **HPC** has become a critical enabler for exploring mathematical conjectures and constructing complex mathematical objects that would be infeasible to handle with traditional computing resources.

Legendre Pairs (**LP**) represent fundamental combinatorial entities, which are introduced in [41]. They play a crucial role in the systematic construction of Hadamard matrices, and their properties have been studied by recent research work [13, 65, 66, 147]. As stated in previous studies, the existence of **LPs** for every odd order is conjectured, a proposition that, if proven true, would imply the validity of the Hadamard conjecture. This conjecture has important theoretical and practical implications, with applications from image analysis, signal processing, to cryptography.

The construction of **LPs** generally follows two primary methodologies: for lengths that are odd prime numbers, they can be systematically constructed via the Legendre Symbol, with precise definitions [12]. For composite lengths, a more generalized approach involves the theory of compression of complementary sequences, as detailed in [35].

The compression methodology for **LPs** operates on the principle that for any divisor d of an order n , an $\text{LP}(n)$ can be derived from "seed" sequences of order n/d , and termed candidate pairs of compressed sequences. The generation of these candidate compressed sequences is generally a more tractable computational task than directly computing an $\text{LP}(n)$. However, the subsequent "decompression" process, which transforms each candidate pair into a full $\text{LP}(n)$, is computationally intensive. Despite this significant computational burden, the benefits of the compression/decompression method distinctly outweigh those of alternative direct construction methods. This inherent computational complexity associated with the decompression phase establishes a critical bottleneck, underscoring the necessity for advanced computational strategies to make the discovery of **LPs** for larger lengths feasible.

The construction of **LPs**, particularly for composite lengths, often presents an intractable computational challenge when approached via brute-force search or direct methods, primarily due to the immense size of the search space. The utility of the compression method is well-illustrated by its application in solving previously open problems. For instance, $\text{LP}(77)$ was successfully constructed in [147] using simultaneous decompression with divisors 7 and 11, notably resolving the smallest open order for **LPs** for over two decades. Similarly, the construction of $\text{LP}(87)$ in [66] involved decompression with divisor 3, necessitating the parallel decompression of several thousands of candidate 3-compressions of length 29. These examples highlight the practicality of the compression method, even with its high computational demands.

The extreme need for HPC becomes strongly evident when considering the remaining open problems in LPs. The list of odd integers less than 200 for which the existence of LPs remains unconfirmed, as stated in [66], includes several values, half of which are multiples of 5, indicating that the conjectures and methodologies discussed in [66] are directly applicable to a substantial subset of these outstanding challenges. The case of LP(117) exemplifies the extreme scale of the problem: its search space encompasses approximately 3.1×10^{18} independent test cases. Using sampling and extrapolation, we estimate that a brute-force search for LP(117) on a single CPU core requires an impractical 1.2×10^{10} years. This staggering computational requirement transforms the problem from merely slow to fundamentally infeasible within a practical timeframe without the application of advanced computational techniques. Therefore, the inquiry of these mathematical goals relies heavily on the development and application of efficient HPC strategies. It is worth mentioning that the given estimation applies to the the generic form of LP(117) algorithm. For instance, there is one pair found in the previous study [65]. However, in this chapter, we are looking for a particular LP(117) with a specific property, which will be explained later.

The computational bottleneck highlighted by problems with massive yet parallelizable search spaces necessitates the utilization of multi-core CPUs and GPUs. Although CPUs have traditionally been the primary target for HPC application distribution, the advent of GPUs created new opportunities for accelerating parallel workloads. These devices and their architecture is very well suited for workloads with high data-level parallelism, including our specific problem, the LP algorithm. The programming model for GPUs, CUDA [92], allows for the execution of thousands of threads in parallel, making it the ideal paradigm for tackling the LP problem. However, the CUDA programming model should be paired with a deep understanding of the GPU architecture to achieve optimal

performance. This includes managing memory hierarchies, minimizing branch divergence, and minimizing resource contention, all of which are critical for maximizing throughput and efficiency in GPU-accelerated applications.

This chapter addresses these challenges by developing and evaluating a series of HPC strategies for the construction of LPs. The specific contributions of this work [131] are outlined as follows:

- **Comprehensive CPU Parallelization:** We develop and evaluate both shared-memory, OpenMP [105], and distributed-memory, MPI [80], parallelization strategies for the LP problem. Our analysis demonstrates significant speedups and scalability on multi-core and multi-node architectures.
- **Advanced GPU Acceleration:** We design and develop a highly optimized CUDA-based solution, in a multi-layered approach to overcome common GPU performance bottlenecks. This includes techniques such as removing branch divergence, optimizing memory access patterns (constant and shared memory), applying loop transformations, tuning kernel parameters, utilizing CUDA Graphs, and minimizing bank conflicts and register pressure, resulting in substantial overall speedups.
- **Improve Resource Utilization:** We explore and use various GPU optimization strategies to improve resource utilization, including exploiting NVIDIA's MPS and MIG technologies. With these modern GPU features, our approach achieves a high degree of throughput, and more efficient resource management.
- **Hybrid MPI-CUDA Scalability:** We integrate MPI and CUDA to enable a multi-GPU, multi-node solution. Our weak scaling evaluation demonstrates the effectiveness of this hybrid approach in distribution, showcasing near-optimal scalability with minimal inter-GPU communication overhead.

- **In-depth Performance Analysis and Optimization Insights:** We provide a detailed empirical analysis of each parallelization and optimization step, offering quantitative data and practical insights into the performance characteristics of the LP algorithm across diverse HPC platforms. This includes profiling results and an analysis of GPU occupancy.
- **Enabling New Mathematical Discovery:** The methodologies and optimization techniques developed in this chapter significantly reduce the computational time required for LP, thereby paving the way for their discovery and contributing directly to the ongoing computational search for these challenging mathematical objects.

The rest of the chapter is organized as follows: Section 6.2 provides the necessary background and reviews the related work. Section 6.3 discusses the OpenMP and MPI designs and implementations, while Section 6.4 presents the GPU implementation and optimization strategies. We have covered the performance evaluation and analysis of each approach in their respective sections. Finally, Section 6.6 concludes the chapter.

6.2 Background and Related Work

6.2.1 Legendre Pairs: Definition and Properties

LPs represent a specific type of combinatorial sequence, distinct from Legendre's Conjecture related to prime numbers or Legendre polynomials. These pairs were introduced in [41] as a means to construct Hadamard matrices systematically. Some of their properties were recently studied in [13, 65, 66, 147]. It is well-known that LPs of order an odd prime number can be constructed via the Legendre Symbol. For the precise definition and properties of the Legendre Symbol see [12]. To construct LPs of order a composite number, one can use the general theory of compression of complementary sequences [35]. The general

idea of compression, as it applies to the case of **LPs**, is that for every divisor d of the order n of the **LP**, one can construct $\text{LP}(n)$ starting with some pair of *seed* sequences of order n/d , that we call candidate pairs of compressed sequences. The point is that the construction of candidate compressed sequences is generally a much easier task than computing $\text{LP}(n)$. Subsequently, each candidate pair of compressed sequences has to be decompressed, into a $\text{LP}(n)$.

The decompression process is computationally expensive, but its benefits clearly outweigh the alternative direct method of construction of **LPs**. An illustration of the usefulness of the compression method for **LPs** can be found in previous work [147, 66]. In [147], the authors construct $\text{LP}(77)$ using simultaneous decompression with the divisors $d = 7$ and $d = 11$. Note that $\text{LP}(77)$ was the smallest open order for **LPs** for more than 20 years. In [66], the authors construct $\text{LP}(87)$ using decompression with the divisor $d = 3$. The construction of $\text{LP}(87)$ required to construct several thousands of candidate 3-compressions of length 29 and subsequently decompress each one of them in parallel.

Given the quite fertile application of compression/decompression in the construction of **LPs**, it is reasonable to try to speed up the currently available C code [66], using modern technologies available in **HPC**. The currently available C code for decompression is automatically generated using meta-programming in Maple, which guarantees a correct and bug-free solution.

In order to define Legendre Pairs, we first mention the well-known definitions of *Periodic Autocorrelation Function* (**PAF**) and *Power Spectral Density* (**PSD**) [42].

Definition 1. The **PAF** of a finite sequence of length n , $A = [a_0, \dots, a_{n-1}]$ is defined as:

$$\text{PAF}(A, s) = \sum_{k=0}^{n-1} a_k a_{k+s}, \quad s = 0, \dots, n-1$$

where $(k + s \text{ is taken mod } n)$.

A well-known **PAF** symmetry property is that

$$PAF(A, s) = PAF(A, n - s), s = 1, \dots, n - 1$$

which essentially tells us that we need to know $\frac{n}{2}$ or $\frac{n-1}{2}$ **PAF** values, according to whether n is even or odd.

Definition 2. The **PSD** of a finite sequence of length n , $A = [a_0, \dots, a_{n-1}]$ is defined as:

$$PSD(A, k) = |\mu_k|^2 = \Re(\mu_k)^2 + \Im(\mu_k)^2$$

where the μ_k are the elements of the *Discrete Fourier Transform* (**DFT**) sequence associated to $[a_1, \dots, a_n]$, i.e.

$$DFT_{[a_0, \dots, a_{n-1}]} = [\mu_0, \dots, \mu_{n-1}], \text{ with } \mu_k = \sum_{i=0}^{n-1} a_i \omega^{ik}, \quad k = 0, \dots, n - 1$$

with $\omega = e^{\frac{2\pi i}{n}} = \cos\left(\frac{2\pi}{n}\right) + i \sin\left(\frac{2\pi}{n}\right)$ a primitive n -th root of unity.

Since the **PSD** values are sums of two squares, they are always nonnegative. In contrast, the **PAF** values may be positive or negative.

Definition 3. A Legendre Pair of (odd) order $n > 1$, denoted by **LP**(n), consists of two sequences $A = [a_0, \dots, a_{n-1}]$ and $B = [b_0, \dots, b_{n-1}]$, both of length n , with $\{-1, +1\}$ elements, that satisfy:

$$PAF(A, s) + PAF(B, s) = -2, \forall s = 1, \dots, \frac{n-1}{2}.$$

For an $\text{LP}(n)$, the normalizations $a_0 + \dots + a_{n-1} = 1$ and $b_0 + \dots + b_{n-1} = 1$ are typically imposed, which leads to the upper bound on the number of potential A,B sequences: $\binom{n}{\frac{n+1}{2}}$. This is because in each of the two sequences A, B , we must have $\left(\frac{n+1}{2}\right) + 1$ elements and $\left(\frac{n-1}{2}\right) - 1$ elements. Another important consequence of the definition of $\text{LP}(n)$ is the **PSD** constancy property:

$$\text{PSD}(A, s) + \text{PSD}(B, s) = 2n + 2, s = 1, \dots, \frac{n-1}{2}.$$

In summary, **LPs** are characterized by the constancy of their **PAF** and **PSD** values.

As mentioned earlier, we are interested in constructing an $\text{LP}(117)$ whose "9-compression" is given by the formula in Listing 6.1. For the specific details of `LegendreSymbol` and the mathematical background behind this formula, please refer to [12].

```

0 > with(NumberTheory):
1 > p:=13; q:=3;
2 > L:=q*[seq(LegendreSymbol(i,p),i=1..p-1)];
3 L:=[3, -3, 3, 3, -3, -3, -3, -3, 3, 3, -3, 3];
4
5 > a:=[1,op(L)];
6 > b:=[1,-op(L)];
7 a := [1, 3, -3, 3, 3, -3, -3, -3, -3, 3, 3, -3, 3]
8 b := [1, -3, 3, -3, -3, 3, 3, 3, 3, -3, -3, 3, -3]

```

Listing 6.1: A Maple snippet to show the 9-compression formula

6.2.2 Parallel Processing in Legendre Pairs Search Problem

To the best of our knowledge, there is no previous work that specifically addresses the parallelization of the **LP** search problem using **HPC** techniques. The existing literature primarily focuses on the mathematical properties and theoretical aspects of **LPs**, with limited attention to computational methods for their construction. The work by [66] discusses the use of compression techniques for constructing **LPs**, but does not provide

parallelization or optimization strategies.

However, beyond [LPs](#), the application of parallel processing to similar exhaustive search problems in combinatorial design, Boolean Satisfiability, and cryptographic key searches demonstrates the immense power of these techniques [[29](#), [162](#), [152](#), [37](#)]. Furthermore, maximizing performance in complex parallel and hybrid environments requires careful optimization, including data locality management, memory footprint reduction, and, in some cases, the development of hardware-aware methods [[127](#), [76](#)].

6.2.3 Parallel Programming Models, OpenMP, MPI, and CUDA

[OpenMP](#) is a widely adopted, directive-based [API](#) designed for parallelization on shared-memory architectures [[105](#)]. It utilizes a fork-join model in which a master thread creates a group of threads to execute some regions of the code in parallel. The ease of use of [OpenMP](#) has made it a popular choice for parallelizing applications, particularly those involving loop-centric tasks where data can be shared efficiently among the threads.

On the other hand, [MPI](#) provides the [API](#) for parallel programming on distributed memory systems, enabling communication between independent processes. It supports point-to-point and collective operations, with well-known implementations like [MPICH](#) [[81](#)], [MVAPICH](#) [[85](#)], and Open [MPI](#) [[103](#)].

[GPUs](#), initially created only for graphics purposes, have evolved into high throughput hardware chips for extreme computing power in [HPC](#) and machine learning applications. Their architecture, including thousands of cores, are very well suited for data-parallel tasks such as scientific simulations, machine learning, and data analysis. Over the past decade, this capability have allowed numerous studies to demonstrate significant speedups for highly parallelizable workloads, when using [GPUs](#) compared to the CPUs [[69](#), [56](#), [132](#), [92](#)].

Leveraging GPUs in HPC applications, often involves hybrid MPI+X models, such as MPI+CUDA and MPI+NCCL [99, 92]. These models combine the strengths of MPI for distributed memory parallelism with CUDA for intra-node parallelism, enabling efficient scaling across multiple nodes with multiple GPUs per node. This hybrid approach is particularly effective for large-scale problems, allowing for fine-grained parallelism within nodes while maintaining the ability to scale across the cluster [76, 127, 108]

GPU-Specific Optimization Strategies

Despite their computational power, GPUs optimization are very challenging due to their distinct architecture compared to CPUs. Key bottlenecks often stem from memory latency, resource contention, and the management of parallel execution flow [47]. Effective GPU utilization demands close management of the memory hierarchy, which includes global memory, shared memory, constant memory, and registers [163, 25, 16].

Minimizing costly data transfers between the host (CPU) and device (GPU) is important to the extent that is possible, and overlapping computation with communication for the cases where data transfer is unavoidable [68, 135, 59]. Code that transfers data for brief use by a small number of threads will see little or no performance benefit, emphasizing the need for substantial work per data element transferred [58]. Various loop transformations, such as loop unrolling, can also significantly enhance performance by reducing the overhead of loop control and increasing instruction-level parallelism [84].

Further challenges include thread divergence among warps, where conditional branches can result in serialized execution, or the careful resource utilization (e.g., registers and shared memory) in order to maximize occupancy [77]. Optimizing GPU kernels is a complex task requiring detailed understanding of architecture, extensive profiling, and iterative experimentation [160]. This inherent complexity is a consequence of the GPU's hierarchical

memory organization, and their SIMD execution model [165]. More interestingly, some recent studies have shown that manual utilization of GPU Streaming Assembler (SASS) or Parallel Thread Execution (PTX) can lead to better performance, underscores a persistent gap between high-level programming abstractions and low-level hardware efficiency [30, 119, 91]. Consequently, this intrinsic complexity of GPU architecture, has driven the continuous development of advanced profiling tools and, more recently, the emergence of AI-driven automated optimization frameworks designed to abstract away or automate portions of this low-level complexity [11].

Comparing to the previous work in this area, we apply and evaluate a variety of these GPU-specific optimization strategies to enhance the performance of the LP algorithm. In this chapter, we provide a comprehensive evaluation of these optimization techniques in the context of the LP problem, demonstrating their effectiveness in overcoming performance bottlenecks and achieving significant speedups for combinatorial search problems.

6.3 CPU-Based Approaches

6.3.1 Experimental Framework

As we provide various parallelization strategies and optimizations for the LP algorithms, we provide the evaluation and discussion of each approach in their respective sections. This helps to better understand the performance characteristics of each approach. All the experiments were conducted on the Daisy cluster at Queen’s University (Section 4.5.1). For the implementations, we utilized OpenMP v4.5, Open MPI v5.0.1, and CUDA v12.0.

6.3.2 Baseline: Single-Core Implementation

The main approach to generate the LPs is to find common members of the two groups of sequences, A and B . Figure 6.1 illustrates the general algorithm design and generation

process. The algorithm iteratively constructs sequences A and B , checking for common members to identify the **LPs**. The sequences are generated based on their specific initial parameters. This brute-force approach explores the parameter space, and for any candidate, its **DFT** is calculated and checked for **PSD** validation. If the **PSD** is valid, the candidate is saved as a potential **LP**. The algorithm continues until all possible combinations of the parameters are explored. It is important to note that we cannot predict where in the parameter space the **LPs** will be found, so we must explore all possible combinations before we can find the common members.

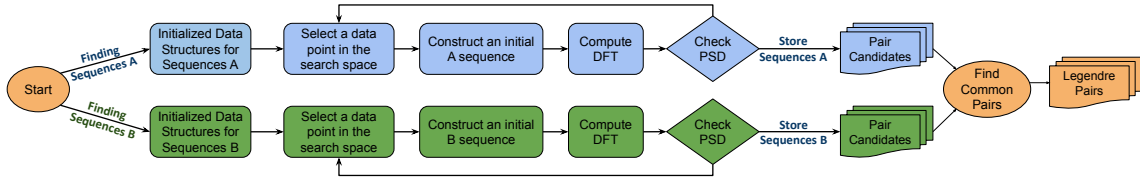


Figure 6.1: General Legendre Pairs (LPs) Algorithm Design and Generation Process: generating LPs through the iterative construction of sequences A and B and finding the common members.

Algorithms 6.1, 6.2, and 6.3 show the process of searching for sequences A in more detail. These algorithms are based on the ones described in [66] with minor modifications to improve data structure and locality. In Lines 2-5, the Algorithm 6.1 takes the input parameters to construct the initial data structures. For sequences B , the initial parameters are different (the code is available on its GitHub repository¹). As shown in Lines 5-9, the algorithm constructs the initial sequence using a series of nested loops, iterating through all possible combinations of the parameters. For length 45, there are about 6.27×10^9 independent test-cases for either sequences, which is computationally very expensive.

Moreover, as depicted in Algorithm 6.2, the algorithm constructs the sequences based on the input parameters through a series of switch cases (Lines 3-19). Then, based on

¹<https://github.com/amirsojoodi/LeGendre-Pairs-Algorithm>

Algorithm 6.1: Legendre Pairs (length 45): Sweeping the search space and looking for valid sequences A .

```

1 Function GenerateSequence():
2    $PairLength \leftarrow 45$ ,  $PSDCount \leftarrow 23$ 
3   Precompute  $\omega^k$  for  $k \in [0, PairLength)$ 
4    $Op1 \leftarrow 1$ ,  $Op3 \leftarrow \{2, 5\}$ ,  $Om3 \leftarrow \{3, 4\}$ 
5   for  $xp1 \leftarrow 1$  to 126 do
6     for  $xp3[0] \leftarrow 1$  to 84 do
7       for  $xp3[1] \leftarrow 1$  to 84 do
8         for  $xm3[0] \leftarrow 1$  to 84 do
9           for  $xm3[1] \leftarrow 1$  to 84 do
10             $A \leftarrow \text{ConstructA}(Op1, Op3, Om3, xp1, xp3, xm3)$ 
11             $dft \leftarrow \text{ComputeDFT}(A, \omega)$ 
12            if  $\text{CheckPSD}(dft, \omega\_powers)$  then
13              Save valid sequence
14            end
15          end
16        end
17      end
18    end
19  end
20 end

```

the properties and formulation described in 6.2.1, the algorithm computes the DFT of the sequence using the `ComputeDFT` function in Algorithm 6.3 (Lines 1-12). Then it checks if the computed DFT is valid by checking whether it satisfies the PSD conditions, in the `ComputePSDs` function (Lines 13-25). One improvement to the function `ComputePSDs` is to check for PSD_{15} first, as most of the sequences will fail this condition (Lines 17-20 of Algorithm 6.3). It is out of the scope of this study to provide the reason behind this phenomenon, and more details can be found in [66]. Once again, the conditions are different for sequences A and B .

Performance Analysis of the Baseline Implementation

While the LP-45 algorithm is computationally expensive, it is still feasible to run on a single-core CPU. However, the LP-117 algorithm is significantly more complex and requires a

Algorithm 6.2: Legendre Pairs (length 45): Sequence A construction.

```
1 Input:  $Op1[ ]$ ,  $Op3[ ]$ ,  $Om3[ ]$ ,  $xp1[ ]$ ,  $xp3[ ]$ ,  $xm3[ ]$ 
2 Output:  $A[ ]$ 
3 Function ConstructA():
4   Initialize array  $A[PairLength + 1]$ 
5   foreach  $i \in \{1, 2\}$  do // Update  $A$  based on  $xp3_i$ 
6     switch  $xp3_i$  do
7       case 1 do
8          $A[Op3s[i]] = -1$ 
9          $A[Op3s[i] + 5] = -1$ 
10         $A[Op3s[i] + 10] = -1$ 
11         $A[Op3s[i] + 15] = 1$ 
12         $A[Op3s[i] + 20] = 1$ 
13         $A[Op3s[i] + 25] = 1$ 
14         $A[Op3s[i] + 30] = 1$ 
15         $A[Op3s[i] + 35] = 1$ 
16         $A[Op3s[i] + 40] = 1$ 
17      end
18      // ... 83 more cases
19    end
20  foreach  $i \in \{1\}$  do // Update  $A$  based on  $xp1_i$ 
21    switch  $xp1_i$  do
22      // 126 different cases
23    end
24  foreach  $i \in \{1, 2\}$  do // Update  $A$  based on  $xm3_i$ 
25    switch  $xm3_i$  do
26      // 84 different cases
27    end
28  return  $A$ 
29 end
```

more efficient approach. The brute-force search for LP-117 on a single core is not practical due to the sheer number of combinations, which is in the order of 10^{18} different test cases. Table 6.1 provides a comparison of the two algorithms, highlighting the differences in their search space sizes, number of switch cases, and execution times, measured on a single core of the Intel Xeon Gold 6338 CPU. LP-117 algorithm is not provided here, but it follows

Algorithm 6.3: Legendre Pairs (length 45): Compute DFT and filter based on PSD thresholds.

```

1 Function ComputeDFT( $A, \omega\_powers, step, t$ ):
2    $dft \leftarrow A[0] \cdot \omega\_powers[0]$  // Initialize DFT sum
3    $k \leftarrow t$ 
4   for  $j \leftarrow 1$  to  $PairLength/step$  do
5      $dft += A[j] \cdot \omega\_powers[k]$ 
6      $k += t$ 
7     if  $k \geq PairLength$  then
8        $k -= PairLength$ 
9     end
10  end
11  return  $dft$ 
12 end
13 Function ComputePSDs( $A, \omega\_powers$ ):
14  for  $t \leftarrow 1$  to  $PSDCount - 1$  do
15     $dft \leftarrow \text{ComputeDFT}(A, \omega\_powers, 1, t)$ 
16     $PSD_t \leftarrow \text{Re}(dft)^2 + \text{Im}(dft)^2$ 
17    if  $t = 15$  then
18      if  $PSD_t \neq 28$  then
19        return False
20      end
21    else if  $PSD_t > 92$  then
22      return False
23    end
24  end
25  return True
26 end

```

a similar structure to the LP-45 algorithm, with different initial parameters, conditions, DFT/PSD computations, and more importantly, the size of the address space. The source code for the LP-117 algorithm is available at its GitHub repository ².

We profiled the execution time of the LP-45 using gprof and perf tools, and the results suggest that the algorithm is mainly compute-bound. While all the test cases are independent, the preparation of the initial sequences and their DFT computations are the most time-consuming parts of the algorithm (Lines 10-11, of Algorithm 6.1). For LP-45, very few

²<https://github.com/amirsojoodi/LeGendre-Pairs-Algorithm>

Table 6.1: Comparison of LP-45 and LP-117 algorithms.

Parameter	LP-45	LP-117
Sequence length	45	117
Size of the search space	6.27×10^9	3.1×10^{18}
Number of switch cases per test-case	462	2142
Number of PSDs to compute per test-case	23	59
Execution time (on a single CPU core)	≈ 38.8 minutes	$\approx 1.2 \times 10^{10}$ years
Lines of code in the algorithm	+7K	+13K
Unique valid sequences A	246651	Unknown
Unique valid sequences B	129651	Unknown
Unique pairs found	192	Unknown

test cases satisfy all the PSD condition to be a valid sequence. In fact, out of the 6.27×10^9 test cases, only 246651 and 129651 unique sequences are valid for sequences A and B, respectively. We expect that the LP-117 algorithm will have a similar behavior, but with a larger search space and more complex conditions.

As can be seen in Table 6.1, the LP-45 algorithm is computationally feasible, taking about 38.8 minutes to run on a single core. However, as mentioned earlier, the brute-force search algorithm for LP-117 is not practical, as it would take approximately 12 billion years to complete on a single core. This highlights the need for parallelization and optimization strategies to make the LP-117 algorithm more efficient. The LP-45 algorithm is a good starting point for understanding the structure and performance characteristics of the LPs algorithms, and as we progress, we will provide various parallelization strategies and optimizations to improve the performance of the both lengths.

For evaluations, we will provide the results for both lengths when they exhibit different behaviors, otherwise we will only provide the results for one of them. We also provide the performance evaluations of both sequences separately, as they have different initial parameters and conditions, which can lead to different performance characteristics. The

performance results for the entire application (both sequences) can be estimated by adding the execution times of the individual sequences when they are executed sequentially. Furthermore, to evaluate the optimizations of the LP-117 algorithm, we select a subset of the address space, which allows us to approximate the performance of the algorithms and optimizations without running them on the entire address space, which is unfeasible.

6.3.3 Utilizing OpenMP

The first step of the parallelization process is to identify the independent test cases, which in this case are every iteration of the inner-most loop of the algorithm. This means that each iteration can be executed independently without any data dependencies. However, to keep the number of forks/joins between the threads at the minimum, the strategy is to parallelize at the outermost loop level, as depicted in Algorithm 6.4. Line 5 illustrates the scheduling policy, which is set to `dynamic` to allow for dynamic load balancing among the threads. On Line 16, the `critical` section is used to ensure that only one thread at a time can save the valid sequence. As the number of valid sequences is very small compared to the total number of test cases, this critical section will not be a bottleneck. However, we optimized this section to temporarily store the valid sequences in a local array, which is then merged into the global array at the end of the computation. However, this optimization is not shown in the algorithm for simplicity, and the performance impact is negligible.

To evaluate the performance of the OpenMP implementation, we executed the algorithm on a single node of the Daisy cluster. The results are shown in Figure 6.2 and Figure 6.3. Speedup computation consists of dividing the execution time of the single-core version by that of the multi-core version. The efficiency metric or *parallel efficiency* of the application is the ratio of the speedup to the number of threads, mathematically represented as $E = \frac{S}{P}$,

Algorithm 6.4: Legendre Pairs (length 45) - Work distribution on CPU cores using OpenMP

```

1 Function GenerateSequence():
2    $PairLength \leftarrow 45$ ,  $PSDCount \leftarrow 23$ 
3   Precompute  $\omega^k$  for  $k \in [0, PairLength)$ 
4    $Op1 \leftarrow 1$ ,  $Op3 \leftarrow \{2, 5\}$ ,  $Om3 \leftarrow \{3, 4\}$ 
5   #pragma omp parallel for schedule (dynamic)
6     shared (Op1, Op3, Om3, omega_powers)
7     private (A, dft, xp1, xp3, xm3)
8     for  $xp1 \leftarrow 1$  to 126 do
9       for  $xp3[0] \leftarrow 1$  to 84 do
10        for  $xp3[1] \leftarrow 1$  to 84 do
11          for  $xm3[0] \leftarrow 1$  to 84 do
12            for  $xm3[1] \leftarrow 1$  to 84 do
13               $A \leftarrow \text{ConstructA}(Op1, Op3, Om3, xp1, xp3, xm3)$ 
14               $dft \leftarrow \text{ComputeDFT}(A, \omega)$ 
15              if  $\text{CheckPSD}(dft, \omega\_powers)$  then
16                #pragma omp critical
17                Save valid sequence
18              end
19            end
20          end
21        end
22      end
23    end
24 end

```

where S stands for speedup and P stands for the number of threads. As shown in Figure 6.2, the speedup increases with the number of threads, but the performance improvement degrades for 128 threads, which is due to the overhead of thread management and memory accesses on different NUMA nodes within the same process. The efficiency plot in Figure 6.3 suggests that the algorithm is highly parallelizable as the efficiency remains above 90% for up to 64 threads.

Results for LP-117 are provided in Figure 6.4 and Figure 6.5. The speedup and efficiency figures follow the same trend as the LP-45 algorithm, but the speedup is lower due to the more complex conditions and larger amount of data to be processed. The speedup is still

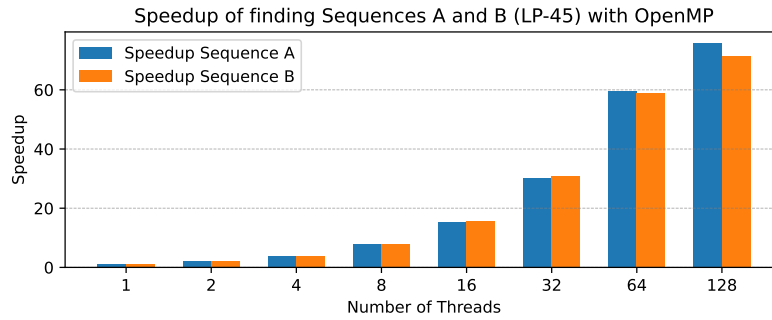


Figure 6.2: Speedup of the OpenMP implementation of LP-45 algorithm on a single node of Daisy cluster comparing with the single-core execution.

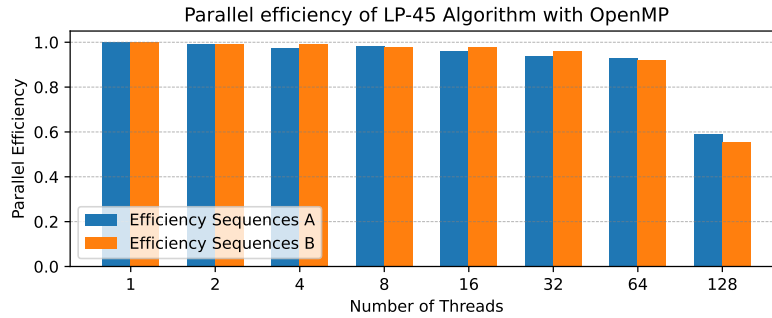


Figure 6.3: Efficiency of the OpenMP implementation of LP-45 algorithm, executed on a single node of Daisy cluster.

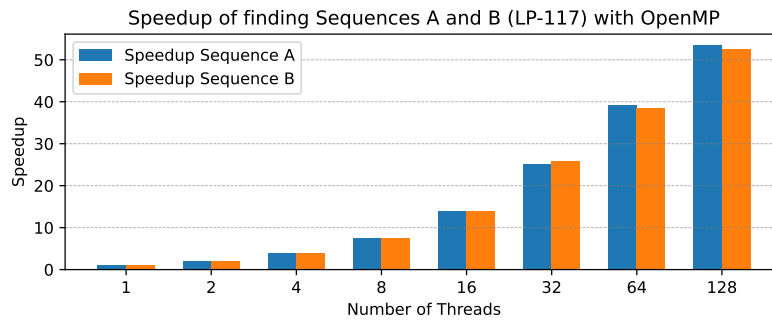


Figure 6.4: Speedup of the OpenMP implementation of LP-117 algorithm on a single node of Daisy cluster comparing with the single-core execution.

significant, reaching up to 40x for 64 threads, and the efficiency remains above 80% for up to 32 threads. This shows that the LP-117 is also highly parallelizable and scalable.

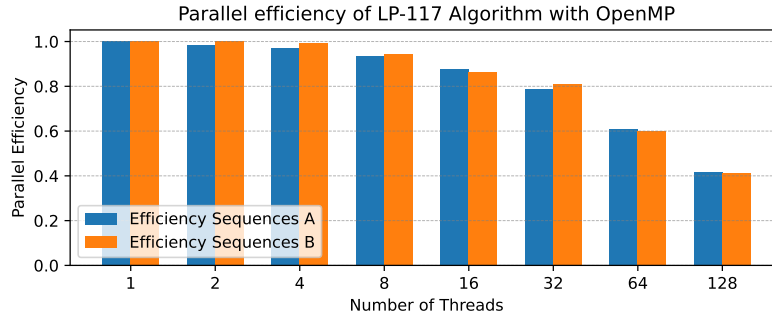


Figure 6.5: Parallel efficiency of the OpenMP implementation of LP-117 algorithm, executed on a single node of Daisy cluster.

6.3.4 Utilizing MPI

Now that we have a single-node parallel implementation of the LP algorithms, we can further scale the implementation to multiple nodes using MPI. The MPI implementation of the LP algorithm is similar to the OpenMP implementation, but it uses a master-worker model, where the leader process (rank 0) collects the results from the worker processes (Algorithm 6.5). As Algorithm 6.6 shows, the leader process utilizes a listener thread that communicates with the workers using `MPI_Iprobe` and `MPI_Recv` functions. The worker processes execute the same algorithm as the single-core implementation, and once their local buffers are full of valid sequences, they send the results back to the leader process by the `MPI_Ssend` function to make sure that the leader process has collected all the results successfully and to avoid extensive memory usage in the workers (Line 22 of the Algorithm 6.5).

Note that the leader process also participates in the computation, and it is not idle while waiting for the results from the worker processes. This is done by interleaved execution between the listener and the main thread to share the CPU resources. That is why the listener thread has a sleep time of 1ms (Line 10 of Algorithm 6.6) to avoid busy-waiting.

In the listener thread, the leader probes for any incoming messages from the worker

Algorithm 6.5: Legendre Pairs (length 45) with MPI: Sequence generation and workload distribution

```

1 Function GenerateSequence():
2   MPIInit() // Initialize MPI
3   if rank = 0 then // Start listener pthread on the leader
4     | Launch background thread to ListenForResults()
5   end
6   PairLength  $\leftarrow$  45 , PSDCount  $\leftarrow$  23
7   Precompute  $\omega^k$  for  $k \in [0, \text{PairLength})$ 
8   Op1  $\leftarrow$  1 , Op3  $\leftarrow$  {2, 5} , Om3  $\leftarrow$  {3, 4}
9   // Distribute the workload among the worker processes
10  total_tasks  $\leftarrow$  126 * 84
11  for task  $\leftarrow$  rank to total_tasks by process_count do
12    | xp1[0]  $\leftarrow$  task/84
13    | xp3[0]  $\leftarrow$  task%84
14    | for xp3[1]  $\leftarrow$  1 to 84 do
15      | for xm3[0]  $\leftarrow$  1 to 84 do
16        | for xm3[1]  $\leftarrow$  1 to 84 do
17          | A  $\leftarrow$  ConstructA(Op1, Op3, Om3, xp1, xp3, xm3)
18          | dft  $\leftarrow$  ComputeDFT(A,  $\omega$ )
19          | if CheckPSD(dft,  $\omega_{\text{powers}}$ ) then
20            | | SaveValidSequence(PSDs)
21          | end
22          | if Local buffer is full then
23            | | SendResults() // to the leader process
24          | end
25        | end
26      | end
27    | end
28  end
29  MPIBarrier() // Synchronize before shutdown
30  if rank = 0 then
31    | Signal listener thread to FlushFinalResults() and exit
32  end
33  MPIFinalize()

```

processes. If a message is received, it gets the message size and receives the results from the worker process (Lines 8-9 of Algorithm 6.6). The leader process then flushes the results to a file whenever the buffer is full or when the worker processes finish their computations

Algorithm 6.6: LP Algorithm with MPI, the listener thread from the leader rank: receiving and flushing results from worker processes in a background thread.

```

1 Function ListenForResults(received_results, run_flag):
2   while run_flag ≠ 0 do
3     MPIIprobe(AnySource, AnyTag, probe_flag, status)
4     if probe_flag = 0 then // No awaiting messages
5       Sleep for 1ms
6       continue
7     end
8     // A new message is available, getting count from the status
9     MPIGetCount(status, count)
10    // Receive results from the worker process
11    MPIRecv(received_results, status.source)
12    FlushToFile()
13    if status.tag = TAG_RESULT then
14      // Remaining results flushed from rank status.MPI_Source
15    else if status.tag = TAG_BUFFER_FULL then
16      // Buffer is full, from rank status.MPI_Source
17    end
18  end
19  // received the stop signal from the leader process
20  FlushRemaining() // Flush any remaining results to file
21  ExitThread()
22 end

```

(Line 10 of the Algorithm 6.6). At the end of the computation, the leader process signals the listener thread to flush any remaining results and exit (Lines 15-16 of Algorithm 6.6).

To evaluate the performance of the MPI implementation, we executed the algorithm in two different scenarios: (1) on a single node, and (2) four nodes of the Daisy cluster, with increasing number of ranks per node. The results for the single-node execution of LP-117 are depicted in Figure 6.6 and Figure 6.7. Similar to the OpenMP implementation, the speedup increases with the number of ranks, but the performance degrades for 128 ranks per node. The efficiency plot in Figure 6.7 suggests that the algorithm is highly parallelizable as the efficiency remains above 95% for up to 64 ranks.

Figure 6.8 and Figure 6.9 illustrate the results for the multi-node version of LP-117 executed

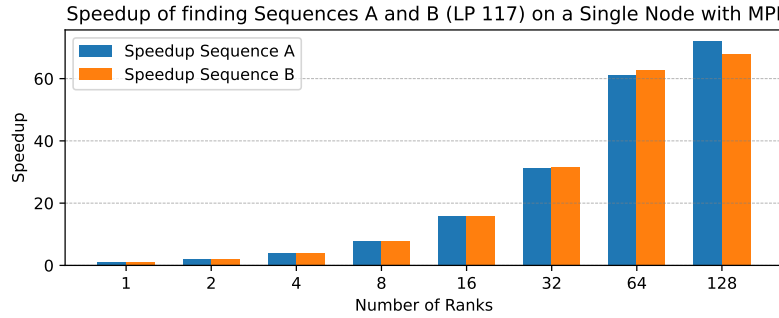


Figure 6.6: Speedup of the MPI implementation of LP-117 algorithm, executed on a single node of Daisy cluster, comparing with the single-core execution.

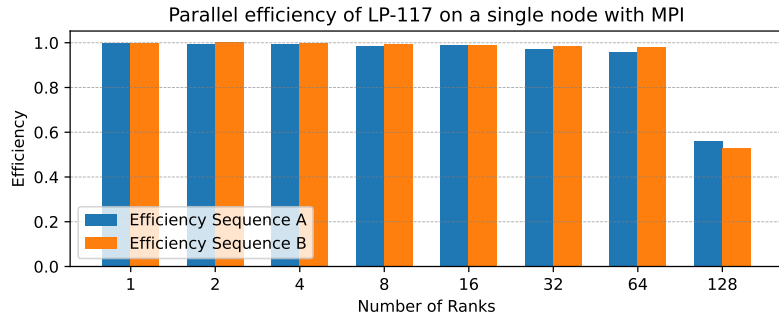


Figure 6.7: Parallel efficiency of the MPI implementation of LP-117 algorithm, executed on a single node of Daisy cluster.

on four nodes of the Daisy cluster. Similar to the single-node version, the results suggest that the algorithm parallel efficiency remains above 90% for up to 64 ranks per node, where the speedup reaches up to 230x for 256 ranks.

To analyze the ratio of the communication time to the computation time, we executed both LP-45 and LP-117 algorithms in four scenarios:

- 64 ranks on a single node (Figure 6.10.a and Figure 6.10.e)
- 128 ranks on a single node (Figure 6.10.b and Figure 6.10.f)
- 256 ranks on four nodes - 64 ranks per node (Figure 6.10.c and Figure 6.10.g)
- 512 ranks on four nodes - 128 ranks per node (Figure 6.10.d and Figure 6.10.h)

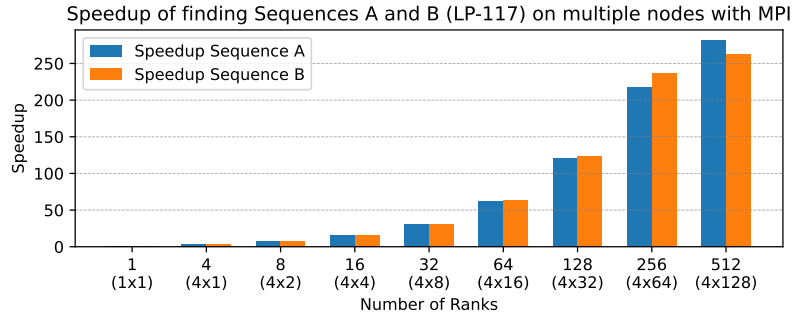


Figure 6.8: Speedup of the MPI implementation of LP-117 algorithm, executed on four nodes of Daisy cluster, comparing with the single-core execution.

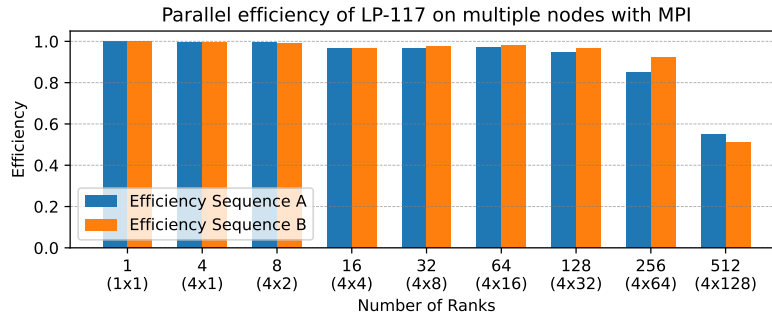


Figure 6.9: Parallel efficiency of the MPI implementation of LP-117 algorithm, executed on four nodes of Daisy cluster, comparing with the single-core execution.

As shown in Figure 6.10, communication time is negligible compared to the computation time for both algorithms, especially for LP-117. This is due to the increase of search space and computation time per rank, which makes the communication overhead less significant. For LP-45, the communication time is slightly more noticeable, but it remains below 10% of the total execution time in all scenarios except for the last one. We can also observe that communication overhead becomes slightly noticeable (15% of the execution time) for LP-45 on multiple nodes and 128 ranks per node (Figure 6.10.d). Comparing to Figure 6.10.c and Figure 6.10.b, we can infer that this noticeable overhead is due to contention on the intra-node interconnect between the NUMA nodes, not the inter-node interconnect.

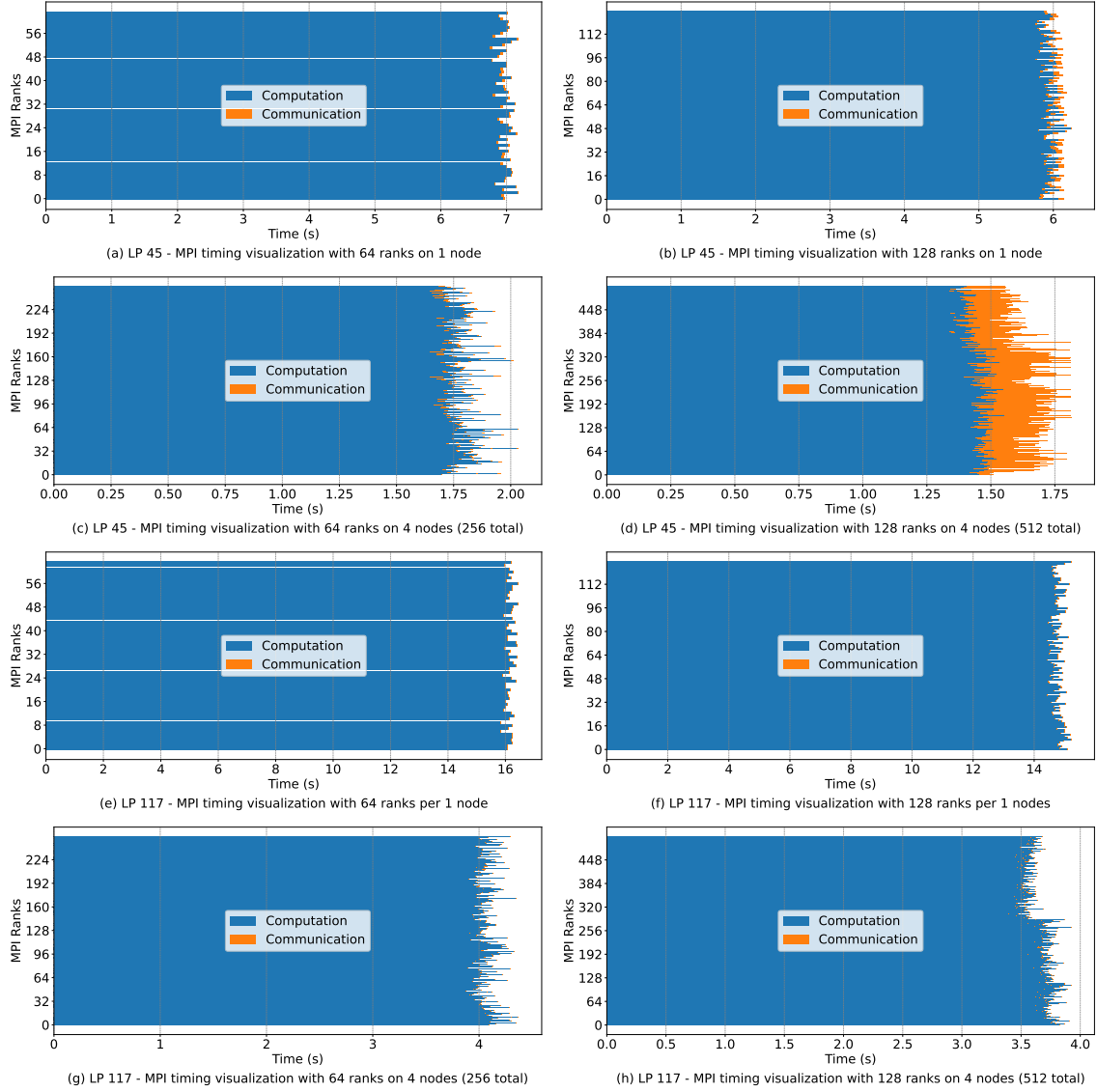


Figure 6.10: Communication and computation time for the MPI implementation of LP-45 and LP-117 algorithms, executed on a single node and four nodes of Daisy cluster.

6.4 GPU Acceleration - Design and Evaluation

As we discussed in Section 6.3, the LP algorithm is embarrassingly parallel, making it very suitable for GPU acceleration. In this section, we will explore the design and evaluation of a GPU-based implementation of the LP algorithm, focusing on performance optimizations

and scalability. Once again, for simplicity, we will only provide a high-level design and approach to the LP-45 version of the algorithm, but the same principles have been applied to the LP-117 version as well, and the results are presented for both versions.

6.4.1 Baseline GPU Implementation

We will start with a baseline GPU implementation of the LP algorithm, which serves as a foundation for further optimizations. The baseline implementation is designed to be straightforward, focusing on the core functionality of the LP algorithm while leveraging the parallel processing capabilities of GPU architectures. Algorithm 6.7 provides a high-level overview of the main function and the GPU kernel. The main function initializes the necessary data structures, allocates memory on the GPU, and launches the GPU kernel for the LP-45 version of the algorithm. The functions `ConstructA`, `ComputeDFT`, and `CheckPSD` in this kernel are the `__device__` versions of the same functions we discussed in Algorithm 6.1.

As depicted, in this version, every GPU thread handles a single data point, while the workload is distributed across 126 blocks of 84 threads each ($126 \times 84 = 10584$ threads). Once a thread finds a valid sequence, it atomically increments a counter to ensure thread safety when updating the results array (Line 10 of Algorithm 6.7). The results are then stored in a global array, which is later copied back to the host memory for further processing (Line 26 of Algorithm 6.7).

In comparison to the LP-45, the LP-117 version of the algorithm follows a similar design, with adjustments made to accommodate the increased complexity and size of the data structures involved. The LP-117 kernel encompasses a larger computational workload, and the launch itself is inside an eight-fold nested loop (the details are skipped for clarity).

Algorithm 6.7: LP-45, CUDA-v1: Naive GPU Kernel Implementation of finding and storing A sequences

```

1  global legendre_kernel(results, count, omega_powers, Op1s, Op3s, Om3s):
2      xp1[0] ← blockIdx.x // 126 blocks per grid
3      xp3[0] ← threadIdx.x // 84 threads per block
4      for xp3[1] ← 1 to 84 do
5          for xm3[0] ← 1 to 84 do
6              for xm3[1] ← 1 to 84 do
7                   $A \leftarrow \text{ConstructA}(Op1s, Op3s, Om3s, xp1, xp3, xm3)$ 
8                   $dft \leftarrow \text{ComputeDFT}(A, \omega\_powers)$ 
9                  if  $\text{CheckPSD}(dft)$  then
10                      $idx \leftarrow \text{atomicAdd}(count, 1)$ 
11                     Store sequence at  $results[idx]$ 
12                 end
13             end
14         end
15     end
16 end
17 Function main():
18     Initialize and precompute data // Similar to Algorithm 6.1
19     cudaMalloc() and cudaMemcpy() input data to device
20     cudaMalloc(results, count) // Allocate device memory to store results
21     cudaMemset(count, 0) // Initialize count to zero on device
22     grid_size ← 126 // First loop of Algorithm 6.1 assigned to 126 blocks
23     block_size ← 84 // Second loop is assigned to 84 threads per block
24     LaunchKernel(legendre_kernel, args)
25     cudaDeviceSynchronize()
26     cudaMemcpy(results, count) // from device to host
27     FlushToFile(results)
28 end

```

6.5 GPU-based Optimizations

The evaluations on single GPU were conducted on an NVIDIA A30 GPU, and the results are compared with the baseline CPU implementation on a single core of an Intel Xeon Gold 6338 CPU. The baseline GPU version of the LP-45 and LP-117 outperforms their CPU counterparts, achieving a speedup of 21.5 $\times$ and 18.95 $\times$, respectively. The same improvement is observed for both sequences A and B . Analyzing the baseline implementation, we

Table 6.2: Some of the relevant specifications of the NVIDIA A30 GPU

Specification	Value
CUDA Version	12.3
CUDA capability major/minor version number	8.0
GPU Max clock rate	1440 MHz (1.44 GHz)
L2 Cache Size	25165824 bytes
Maximum number of threads per block	1024
Memory clock rate	1215 MHz
Memory bus width	3072-bit
Multiprocessors, CUDA Cores/MP	56, 64 CUDA Cores/MP
Total amount of constant memory	65536 bytes
Total amount of global memory	24061 MBytes
Total amount of shared memory per block	49152 bytes
Total number of registers per block	65536
Warp size	32

can identify several opportunities for performance improvements, which we will explore in the following sections.

As we discussed in Section 6.3.2, the LP algorithm is highly parallelizable, but the observed speedup from the GPU version is relatively modest considering the potential of the A30 GPU. Although several factors contribute to performance improvements, the GPU specifications in Table 6.2 suggest that the naive GPU implementation is unable to fully utilize the available CUDA cores. As depicted, the NVIDIA A30 GPU has 56 multiprocessors, each with 64 CUDA cores, resulting in a total of 3584 CUDA cores, and the speedup of 18.95× is not even close to the potential peak performance of the GPU.

6.5.1 Removing Branch Divergence

One of the primary performance bottlenecks in the baseline GPU implementation is the presence of branch divergence, particularly in the `switch cases` where each thread might

take a different execution path based on the value of `Op1s`, `Op3s`, and `Om3s` variables.

In the `SIMD` execution model, threads execute in lockstep, meaning that if some threads take a different path due to a conditional statement, the `GPU` must serialize the execution of those divergent paths. In `GPUs`, this issue occurs when threads within a warp (a group of 32 threads) take different execution paths, leading to performance degradation as the `GPU` must wait for all threads in the warp to complete their execution.

To address this issue, we can refactor the code to utilize bit-wise operations inside the kernel to completely avoid the `switch cases`. As depicted in Algorithm 6.8 and 6.9, we can precompute the masks for all the threads, and based on their thread IDs, we can identify how the threads should initialize their variables without branching. This approach allows us to eliminate the `switch cases` and ensure that all threads follow the same execution path, thereby minimizing thread divergence and improving performance. The same approach is applied to the `LP-117` version of the algorithm as well with a relative speedup of 10.42× compared to the baseline implementation (Section 6.4.1).

6.5.2 Utilizing *Constant Memory*

Constant memory is a fast, limited, and read-only `GPU` memory space, especially designed for efficient access to broadcast data. When all threads in a half-warp access the same constant memory address, the value can be read in one operation, optimizing bandwidth and caching. Subsequent accesses by other half-warps to the same address utilize the constant cache, resulting in less memory traffic. This memory is ideal for storing small, uniformly accessed read-only values, like lookup tables, or configuration parameters.

Therefore, in addition to removing branch divergence, we can further optimize the `GPU` implementation by utilizing *constant memory* for frequently accessed data. As depicted in Table 6.2, the NVIDIA A30 `GPU` has a total of 64KB of *constant memory*. By storing variables

Algorithm 6.8: LP-45, CUDA-v2: Recursively generate the masks for GPU threads with *bits*-many 0s in a 9-bit field

```

1 Function intToBinarySequence(a):
2   switch a do
3     case 1: return 0b000000001
4     // ...
5     case 9: return 0b100000000
6   end
7 Function intToBinarySequenceNegated(a):
8   switch a do
9     case 1: return 0b111111110
10    // ...
11    case 9: return 0b011111111
12  end
13 Function populateMasks(masks, bits, depth, length, index, entry, count):
14   if depth ≥ bits then
15     masks[count] ← entry
16     count ← count + 1
17   return
18 end
19 for i ← index to length − (bits − depth) do
20   entry ← entry & intToBinarySequenceNegated(i + 1)
21   populateMasks(masks, bits, depth + 1, length, i + 1, entry, count)
22   entry ← entry | intToBinarySequence(i + 1)
23 end
24 end
25 Function main():
26   // ...
27   Initialize masks_84_3bits_h, masks_84_6bits_h, and masks_126_4bits_h arrays
28   mask ← 0b111111111 // 9-bit mask with all bits set to 1
29   mask_count ← 0
30   populateMasks(masks_126_4bits_h, 4, 0, 9, 0, mask, mask_count)
31   populateMasks(masks_84_3bits_h, 3, 0, 9, 0, mask, mask_count)
32   populateMasks(masks_84_6bits_h, 6, 0, 9, 0, mask, mask_count)
33   // Copy masks to global memory
34   // The rest of the main function, similar to Algorithm 6.7
35 end

```

including `Op1s`, `Op3s`, `Om3s`, and `omega_powers` as well as the created `masks` from the previous step, in *constant memory*, we can reduce *global memory* accesses and improve

Algorithm 6.9: LP-45, CUDA-v2: Sequence A construction.

```
1 __device__ ConstructA():
2   Initialize array  $A[PairLength + 1]$ 
3   foreach  $i \in \{1, 2\}$  do // Update  $A$  based on  $xp3_i$ 
4      $mask \leftarrow masks\_84\_6bits[xp3_i - 1]$ 
5     for  $j \leftarrow 0$  to 8 do
6        $A[Om3s[i] + j \times 5] \leftarrow (mask \& (1 \ll j)) ? 1 : -1$ 
7     end
8   end
9   foreach  $i \in \{1\}$  do // Update  $A$  based on  $xp1_i$ 
10     $mask \leftarrow masks\_126\_4bits[xp1_i - 1]$ 
11    for  $j \leftarrow 0$  to 8 do
12       $A[Op1s[i] + j \times 5] \leftarrow (mask \& (1 \ll j)) ? 1 : -1$ 
13    end
14  end
15  foreach  $i \in \{1, 2\}$  do // Update  $A$  based on  $xm3_i$ 
16     $mask \leftarrow masks\_84\_3bits[xm3_i - 1]$ 
17    for  $j \leftarrow 0$  to 8 do
18       $A[Om3s[i] + j \times 5] \leftarrow (mask \& (1 \ll j)) ? 1 : -1$ 
19    end
20  end
21  return  $A$ 
22 end
```

performance. This improvement leads to a relative speedup of $1.59\times$ for the LP-117 version of the algorithm, compared to Section 6.5.1.

6.5.3 Utilizing *Shared Memory*

Another common optimization in GPU computation is the utilization of *shared memory*, which is a fast, on-chip memory that is shared with threads within the same block. By storing frequently accessed data in *shared memory*, we can further reduce the number of accesses to *global memory*. However, as we have assigned each data point of the address space to a single thread, which is independent of other threads, utilizing *shared memory* seems counterintuitive. In the current version, the candidate sequences A and B are stored in each thread's local memory which is stored in the *global memory*. Also, as the memory

accesses are not cache friendly, the load and store operations are high-latency.

To alleviate the intensive *global memory* accesses, we can store the candidate sequences in the *shared memory* to force low-latency accesses for the active warps. This optimization demonstrate a relative speedup of $2.07\times$ for LP-117 compared to the previous optimization (Section 6.5.2).

6.5.4 Loop Unrolling, Loop Fission, and Loop Reordering

Loop unrolling is a common optimization technique that can improve performance by reducing the overhead of loop control and increasing instruction-level parallelism. In the context of the LP algorithm, we can apply loop unrolling to the nested loops in the GPU kernel and the `ConstructA` function (Algorithm 6.9). Moreover, due to the threads task assignment, we have several redundant iterations of the nested loops that can be moved to the outer loops, where the `xp3` and `xm3` variables are modified. Therefore, using *loop fission* and *loop reordering* techniques, we can integrate the `ConstructA` function into the GPU kernel at different levels of the nested loops. This optimization leads to a relative significant speedup of $1.52\times$ compared to the previous optimization (Section 6.5.3).

6.5.5 Optimizing *Shared Memory* Usage

Although utilizing *shared memory* is a great technique to improve GPU kernels performance, overusing it leads to lower occupancy. Occupancy is the ratio of the number of active warps to the maximum number of warps per SM. While higher occupancy does not necessarily translate to higher performance, lower occupancy often limits the device peak potential due to inability to hide latencies. As a rule of thumb, an occupancy of at least 50% is considered an efficient work distribution on the GPU [92]. Moreover, large differences between the theoretical and achieved occupancy shows that the workload is

highly imbalanced.

Similarly in **LP** algorithm, overusing *shared memory* leads to performance degradation due to decreased occupancy. Considering the amount of *shared memory* and the size of sequences *A* and *B* per thread, reducing the sequences data type from `int` to `short int`, or even to `char`, increases the occupancy thereby improving the performance by 1.26× compared to the previous version (Section 6.5.4). Note that this data type representation does not result in data loss, as the sequence indices are populated with either 1 or −1 (See Algorithm 6.9).

6.5.6 Tuning Kernel Parameters

Another attribute of **GPU** occupancy is the *thread block* size and dimension. While larger block size helps with low-latency synchronization among higher number of threads, it decreases the number of resident *thread blocks* per **SM** due to lack of enough resources for all the threads. Changing the *thread block* and *grid* size from $(1, 1, 84) \times (1, 32, 32)$ to $(84, 84, 84) \times (1, 1, 32)$ improves the performance by 1.21× compared to the previous version (Section 6.5.5).

6.5.7 Decreasing Memory Stalls by Increasing Work Per Thread

In the previous steps, we have optimized the **GPU** kernel to reduce memory stalls and improve performance. However, we can further enhance the performance by increasing the work done per thread to hide more memory stalls. In the **LP-117** version, we can achieve this by moving another loop inside the kernel, which allows each thread to perform more computations before waiting for memory accesses. This optimization leads to a relative speedup of 1.02× compared to the previous version (Section 6.5.6).

6.5.8 Minimizing *Shared Memory* Bank Conflicts

Profiling the GPU kernel reveals that the *shared memory* accesses are not fully optimized, because of the high number of bank conflicts. As discussed in Section 6.2.3, *shared memory* is divided into 32 banks, and each bank can be accessed by multiple threads simultaneously. However, if multiple threads within the same warp attempt to access different addresses that map to the same memory bank at the same time, a *bank conflict* occurs, leading to serialization of memory accesses and performance degradation.

One way to minimize bank conflicts is to ensure that threads access different banks by introducing padding in the *shared memory* arrays [16]. However, finding the correct padding size can be challenging, as it depends on the number of active threads, the size of the *shared memory* arrays, and the access patterns. In our case, Algorithm 6.10 finds the optimal padding size, considering the mentioned parameters. This approach can be applied on various lengths of LP algorithms with different configurations. The best padding size determined by Algorithm 6.10 is 7, and it is used as a `__shared__` buffer with the type and size in the GPU kernel `char A[blockSize][pairLength + padding]`. This optimization leads to a relative speedup of 1.14× compared to the previous version (Section 6.5.7).

6.5.9 Optimizing Register Usage

Registers are the fastest memory available on the GPU, and optimizing their usage is crucial for performance. However, excessive register usage can lead to register pressure, spilling registers to local memory, which is significantly slower. To optimize register usage, we first need to analyze the register usage of our GPU kernel. Utilizing compiler option `-Xptxas="-v"` allows us to see the number of registers used per thread. We can also utilize the `nvprof` or `Nsight Compute` tools to analyze the register usage and identify potential bottlenecks.

Algorithm 6.10: Finding Optimal Shared Memory Padding to Minimize Bank Conflicts

```
1 Function CalculateBankConflicts( $T, W, P, C, B$ ):  
    //  $T$ : threads per block,  $W$ : width,  $P$ : padding,  $C$ : accessed column,  $B$ : number of banks  
2 for  $b \leftarrow 0$  to  $B - 1$  do  
3      $bank\_conflicts[b] \leftarrow 0$   
4 end  
5  $W_p \leftarrow W + P$  // Padded width  
6 for  $t \leftarrow 0$  to  $T - 1$  do  
7      $address \leftarrow t \times W_p + C$   
8      $bank \leftarrow \lfloor address/4 \rfloor \bmod B$   
9      $bank\_conflicts[bank] \leftarrow bank\_conflicts[bank] + 1$   
10 end  
11 return  $bank\_conflicts$   
12 end  
13 Function FindBestPadding( $T, W, C, B$ ):  
14      $min\_conflicts \leftarrow T$   
15      $best\_padding \leftarrow 0$   
16 for  $P \leftarrow 0$  to  $B - 1$  do  
17      $conflicts \leftarrow \text{CalculateBankConflicts}(T, W, P, C, B)$   
18      $max\_conflict \leftarrow \max(conflicts)$   
19     if  $max\_conflict < min\_conflicts$  then  
20          $min\_conflicts \leftarrow max\_conflict$   
21          $best\_padding \leftarrow P$   
22     end  
23 end  
24 return  $best\_padding$   
25 end  
26 Function Main():  
27      $T \leftarrow 32, W \leftarrow 117, B \leftarrow 32$   
    // Traversing all cases for computing PSDs:  $13 \times 9 = 117$   
28 for  $col \leftarrow 0$  to 13 do  
29     for  $case \leftarrow 0$  to 9 do  
30          $C \leftarrow col + 13 \times case$  // Absolute accessed column  
31          $P^* \leftarrow \text{FindBestPadding}(T, W, C, B)$   
32         Append  $P^*$  to  $best\_paddings$   
33     end  
34 end  
    //  $best\_paddings$  now contains the optimal padding for each column  
35 end
```

In our case, we found that the GPU kernel uses a significant number of registers, mostly to store PSDs and their intermediate results (112 per thread). To reduce the register usage, we lowered the precision of the PSDs, and we also store the PSDs lazily once they are validated. Moving PSDs to *shared memory* helps with reducing the register usage, but it increases the memory usage and leads to lower occupancy. This optimization reduces the register usage to 32 per thread, and it improves the performance by 1.06× compared to the previous version (Section 6.5.8).

6.5.10 Removing Ternary Operations

Ternary operations in the Algorithm 6.9 (Lines 6, 12, and 18) can cause unnecessary stalls in the GPU pipeline, as they introduce additional branches that can lead to thread divergence. To optimize the code further, we can replace the ternary operations with simple arithmetic binary operations. More specifically, we can replace instructions like:

- `A1[local_id][Om3s_d[i]] = (mask&1)?1 : -1;` with:
- `A1[local_id][Om3s_d[i]] = -1 + (((mask >> 0)&1) << 1);`

This change introduces two additional bitwise operations plus one *add* operation. Our tests show that even with the added operations, the new instructions still outperform the ternary operations. Although this improvement is a relatively minor optimization, but it can still contribute to performance gains by reducing the number of branches and improving instruction-level parallelism and pipeline stalls. As a result, we can achieve a relative speedup of 1.01× compared to the previous version (Section 6.5.9).

6.5.11 CUDA Graphs Utilization

As discussed in Section 2.3.3, CUDA Graphs allow us to create a sequence of CUDA operations and execute them as a single entity. This can help reduce the overhead of launching

multiple kernels and improve performance. In the **LP** algorithm, we can utilize **CUDA** Graphs to encapsulate the entire sequence of kernel launches and memory operations, thereby reducing the overhead of multiple kernel launches. As depicted in Algorithm 6.11, we can create a **CUDA** Graph that includes all the memory operations and kernel launches required for the pipelined execution of the **LP** algorithm. Other unrelated parts of the code, including the **CUDA** kernels are omitted. Note that the same approach is applied to both sequences *A* and *B*.

In more detail, in the `SetupGraph` function, we create a new **CUDA** Graph and add the necessary memory operations and kernel launches (with dependencies) for each pass of the pipelined execution (Lines 2-10). Once the graph is set up, we can instantiate it and upload it to the device (Line 11 and Line 12). In the `LaunchGraph` function, we simply launch the graph on the corresponding stream and synchronize the device to ensure completion (Lines 15-18). Finally, in the `Main` function, we initialize the data structures and parameters, set up the graph, and instead of launching individual kernels, we launch the entire graph for execution (Lines 21-28).

Figure 6.11 illustrates the structure of a four-pass **CUDA** Graph for the **LP**-117 created by Algorithm 6.11. In this graph, the first nodes on top are the memory operations required to set up the algorithm. Then, on the next level, we have four kernel nodes, each representing one pass of the pipelined execution. Finally, we have the memory operations required to retrieve the results from the **GPU** back to the host.

We have evaluated various configurations of **CUDA** Graphs, including different numbers of passes up to 1024 resulting in a **CUDA** Graph of more than 3072 nodes. However, the performance improvement from utilizing **CUDA** Graphs is observed to be marginal, due to the fact that the overhead of launching multiple kernels is already low compared to the overall execution time of the **LP** algorithm. Moreover, the non-**CUDA** Graph version

Algorithm 6.11: Utilizing CUDA Graphs to encapsulate the entire sequence of kernel launches and memory operations in the LP algorithm (Sequences A).

```

1 Function SetupGraph():
2   Initialize a new CUDA Graph // cudaGraphCreate
3   Add memory initialization node to the graph // cudaGraphAddMemsetNode
4   Add H2D memory operations to transfer input data // cudaGraphAddMemcpyNode
5   for  $i \leftarrow 0$  to  $\text{num\_passes} - 1$  do
6     Create a kernel node for pass  $i$ , configured with appropriate parameters
7     Add the kernel node to the graph, as a successor to the initial memory operations.
        // cudaGraphAddKernelNode
8     Create D2H memcpy nodes for transferring results back to the host
9     Add the memcpy nodes as successors to the corresponding kernel nodes
        // cudaGraphAddMemcpyNode
10  end
11  Instantiate and store the graph // cudaGraphInstantiate
12  Upload the graph to the device // cudaGraphUpload
13  return the created graph
14 end
15 Function LaunchGraph(graph):
16   Launch the CUDA Graph on a corresponding stream // cudaGraphLaunch
17   Synchronize the device to ensure completion // cudaStreamSynchronize
18 end
19 Function Main():
20   Initialize data structures and parameters
21   graph  $\leftarrow$  SetupGraph()
        // ...
        // 8-fold nested loops for different configurations
22   Launch the graph for execution LaunchGraph(graph)
23   Save the results (similar to LP-45 in Algorithm 6.7)
24 end

```

of the **LP** algorithm is already designed to overlap computation and memory transfers using multiple streams, which further reduces the potential benefits of **CUDA** Graphs. However, this approach is not scalable with increasing number of passes, as managing multiple streams and overlapping operations becomes increasingly complex. Nevertheless, this optimization still contributes to performance gains by reducing the overhead of multiple kernel launches. As a result, we achieve a relative speedup of $1.01\times$ compared to the previous version (Section 6.5.10). We speculate if **CUDA** Graphs were applied in

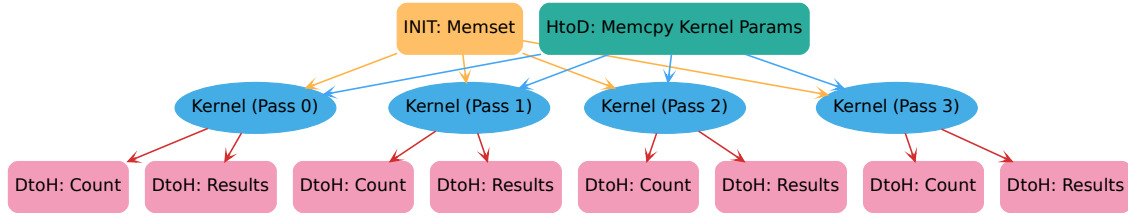


Figure 6.11: Utilizing CUDA Graphs to encapsulate the entire sequence of kernel launches and memory operations (four passes of pipelined execution) in the LP-117 algorithm (Sequences A).

earlier stages of the optimizations, they would have shown more noticeable performance improvement comparing to the previous steps.

6.5.12 Profiling and Performance Analysis

Until now, we have discussed various optimization techniques applied to the GPU implementation of the LP algorithm. Each optimization step is designed to address specific performance bottlenecks and improve the overall efficiency of the algorithm on the GPU architecture. Table 6.3 summarizes the performance improvements achieved through each optimization step for the LP-117 version of the algorithm, compared to the baseline CPU implementation on a single core of an Intel Xeon Gold 6338 CPU. The overall speedup achieved by the final optimized version (LP-117-v12) is approximately 1,927.53×

In order to ensure that there is no more room for performance improvements, we profiled the GPU kernel after each optimization to identify performance bottlenecks and understand how the GPU resources are utilized. We used NVIDIA *Nsight Compute* and *nvprof* to analyze the performance of the GPU kernel. Moreover, we developed an offline algorithm to predict GPU occupancy based on the *shared memory* and *register* usage, which is provided in Algorithm 6.12.

As shown in Figure 6.12, the occupancy of the LP-117 CUDA kernel on the A30 GPU

Table 6.3: Performance of the LP-117 CUDA algorithms tested on a single NVIDIA A30 GPU, compared with the baseline CPU implementation on a single core of Intel Xeon Gold 6338 CPU.

Version	Brief Description	Relative Speedup	Overall Speedup
LP-117-v1 (Section 6.4.1)	Naive CUDA implementation	18.95×	18.95×
LP-117-v2 (Section 6.5.1)	Remove branch divergence	10.42×	197.56×
LP-117-v3 (Section 6.5.2)	Utilizing <i>constant memory</i>	1.59×	314.27×
LP-117-v4 (Section 6.5.3)	Utilizing <i>shared memory</i>	2.07×	652.55×
LP-117-v5 (Section 6.5.4)	Loop unrolling and loop reordering	1.52×	997.6×
LP-117-v6 (Section 6.5.5)	Reducing <i>shared memory</i> usage	1.26×	1, 257.59×
LP-117-v7 (Section 6.5.6)	Tuning kernel parameters	1.21×	1, 523.92×
LP-117-v8 (Section 6.5.7)	Increasing work per thread	1.02×	1, 559.63×
LP-117-v9 (Section 6.5.8)	Minimize <i>shared memory</i> bank conflicts	1.14×	1, 781.73×
LP-117-v10 (Section 6.5.9)	Decreasing register pressure	1.06×	1, 900.36×
LP-117-v11 (Section 6.5.10)	Replacing ternary operations with binary	1.01×	1, 912.42×
LP-117-v12 (Section 6.5.11)	Utilizing CUDA Graphs	1.01×	1, 927.53×

is at 50% with respect to *shared memory* and *register* usage. A30 GPU can schedule up to 64 warps per SM, and as discussed before, aiming for an occupancy of at least 50% is a good practice to ensure that the GPU can hide latencies effectively. As Figure 6.12.a depicts, the occupancy cannot be further improved as the *shared memory* per block is the limiting factor; and as Figure 6.12.(b) shows, the threads can use more registers (up to

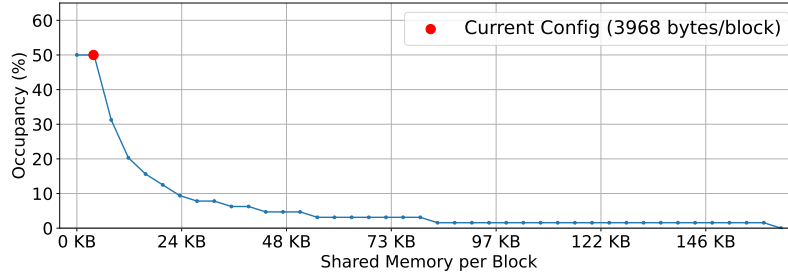
Algorithm 6.12: Predicting GPU Occupancy based on device specifications, as well as Shared Memory and Register Usage

```

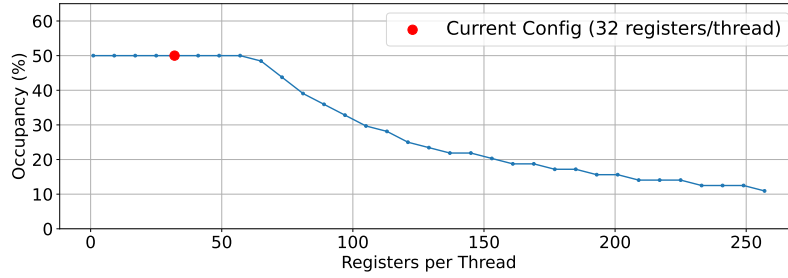
1 Function predictOccupancy(threadsPerBlock, sharedMemPerBlock, registersPerThread):
    // Constants for the A30 GPU extracted at runtime
2   threadsPerWarp  $\leftarrow$  32
3   maxWarpsPerSM  $\leftarrow$  64
4   maxThreadsPerSM  $\leftarrow$  2048
5   maxBlocksPerSM  $\leftarrow$  32
6   registersPerSM  $\leftarrow$  65536
7   maxRegistersPerThread  $\leftarrow$  255
8   maxRegistersPerBlock  $\leftarrow$  65536
9   maxSharedMemPerBlock  $\leftarrow$  167936
10  sharedMemPerSM  $\leftarrow$  167936
11  warpsPerBlock  $\leftarrow$   $\lceil \text{threadsPerBlock} / \text{threadsPerWarp} \rceil$ 
    // Warp per thread limits
12  maxBlocks  $\leftarrow$   $\min \left( \text{maxBlocksPerSM}, \frac{\text{maxThreadsPerSM}}{\text{threadsPerBlock}}, \frac{\text{maxWarpsPerSM}}{\text{warpsPerBlock}} \right)$ 
    // Register limit
13  regsPerBlock  $\leftarrow$  registersPerThread  $\times$  threadsPerBlock
14  if regsPerBlock > maxRegistersPerBlock then
15    | maxBlocks  $\leftarrow$  0
16  else
17    | maxBlocks  $\leftarrow$   $\min (\text{maxBlocks}, \text{registersPerSM} / \text{regsPerBlock})$ 
18  end
    // Shared memory limit
19  if sharedMemPerBlock > maxSharedMemPerBlock then
20    | maxBlocks  $\leftarrow$  0
21  else
22    | maxBlocks  $\leftarrow$   $\min \left( \text{maxBlocks}, \frac{\text{sharedMemPerSM}}{\text{sharedMemPerBlock}} \right)$ 
23  end
24  totalThreads  $\leftarrow$  maxBlocks  $\times$  threadsPerBlock
25  occupancy  $\leftarrow$  totalThreads / maxThreadsPerSM
26  return occupancy  $\times$  100
27 end

```

64) without affecting the occupancy. However, the current version of the algorithm and selected datatype cannot be altered in any ways to optimize the register usage further.



(a) Impact of Varying Shared Memory per Block on Occupancy
(Fixed: 32 threads/block, 32 regs/thread)



(b) Impact of Varying Registers per Thread on Occupancy
(Fixed: 32 threads/block, 4KB shared memory)

Figure 6.12: Occupancy of the LP-117 CUDA kernel on the A30 GPU with respect to *shared memory* and *register* utilization. The current implementation is marked with a red dot.

6.5.13 Multi-GPU Implementation

After optimizing the single-GPU version, we combined the GPU implementation with the MPI implementation to provide a multi-GPU version of the algorithm. The multi-GPU implementation uses the same MPI approach as the CPU-only version (discussed in Section 6.3.4), but with each MPI process assigned to a single GPU. To evaluate the scalability of the multi-GPU implementation, we conducted weak scaling evaluation, where the overall problem size increases proportionally with the number of GPUs to maintain a constant workload per device.

Figure 6.13 demonstrates how the algorithm scales: as the number of GPUs increases from 1 to 16, the total execution time increases slightly, indicating that the parallelization strategy effectively distributes the workload and minimizes inter-GPU communication

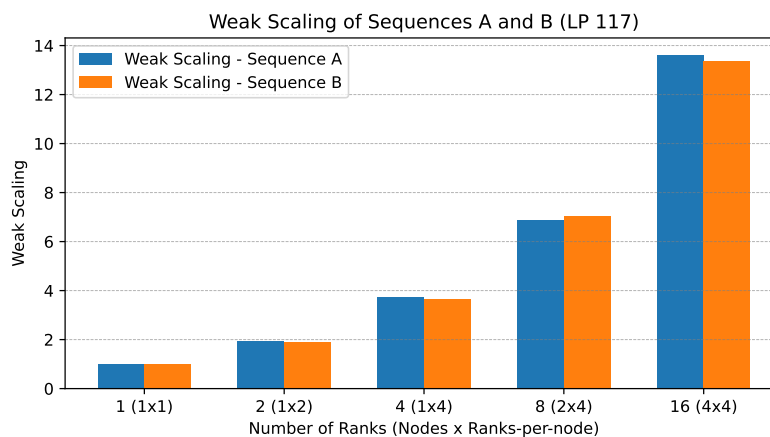


Figure 6.13: Weak scaling of the LP-117 multi-GPU implementation on a single node with up to 16 NVIDIA A30 GPUs. Each rank has a constant search space to sweep.

overhead. This confirms that the [MPI+CUDA](#) implementation is well-suited for large-scale deployments.

6.5.14 Improving Resource Utilization by harnessing MIG and MPS

Although we have achieved significant performance improvements through various optimizations so far, there is still room for further enhancements by saturating the [GPU](#) resources more effectively. This is especially true for modern [GPUs](#), which have a larger number of [SMs](#) and higher memory bandwidth compared to older models. A common pattern is to oversubscribe the [GPU](#), placing multiple [MPI](#) processes per [GPU](#), to fill the execution and memory resources more effectively. Moreover, oversubscribing the [GPUs](#) can improve the application-level throughput and save costs.

However, placing multiple [MPI](#) processes per [GPU](#) can lead to resource contention and increased overhead due to context switching. When multiple processes target the same [GPU](#), the driver creates a separate *context* for each. Because the [GPU](#) hardware can only execute one *context* at a time, the *contexts* must be time-sliced, which can introduce additional latency and reduce overall performance. This overhead can negate the benefits

of increased resource utilization if not managed properly.

To address this challenge, we can utilize NVIDIA MPS [83], which is designed to improve the performance of multiple processes sharing a single GPU. By utilizing NVIDIA MPS, we can manage the *contexts* more efficiently and reduce the overhead associated with context switching. MPS consists of a server process that manages multiple client processes, allowing them to share the same *context* on the GPU. This allows the MPI processes to connect to the MPS server instead of creating their own *contexts*. By using MPS, we can replace the driver-level time-slicing with a lightweight spatial-slicing. The details of configuring MPS are provided in Appendix A.3.

Another approach to improve resource utilization on modern GPUs is to use NVIDIA MIG [98], which allows partitioning a single physical GPU into multiple isolated instances. Depending on the slicing method, each MIG instance has its own dedicated resources, including SMs, memory, and cache, allowing multiple workloads to run concurrently on the same physical GPU without interfering with each other. By using MIG, we can create multiple smaller GPU instances that can be assigned to different MPI processes, thereby improving resource utilization and reducing contention. While MPS provides concurrency, MIG provides isolation by guaranteeing a predictable *Quality of Service (QoS)* for each MPI process. The details of configuring MIG instances are provided in Appendix A.1.

Furthermore, these two technologies can be combined to further enhance resource utilization. In this way, we can create multiple MIG instances on a single physical GPU, and then run multiple MPI processes on each instance using MPS. MIG provides a coarse-grained, hardware-level partitioning of the GPU, while MPS provides a fine-grained, software-level sharing of the GPU resources within each MIG instance. This allows us to take advantage of the benefits of both technologies, improving concurrency while maintaining isolation between workloads.

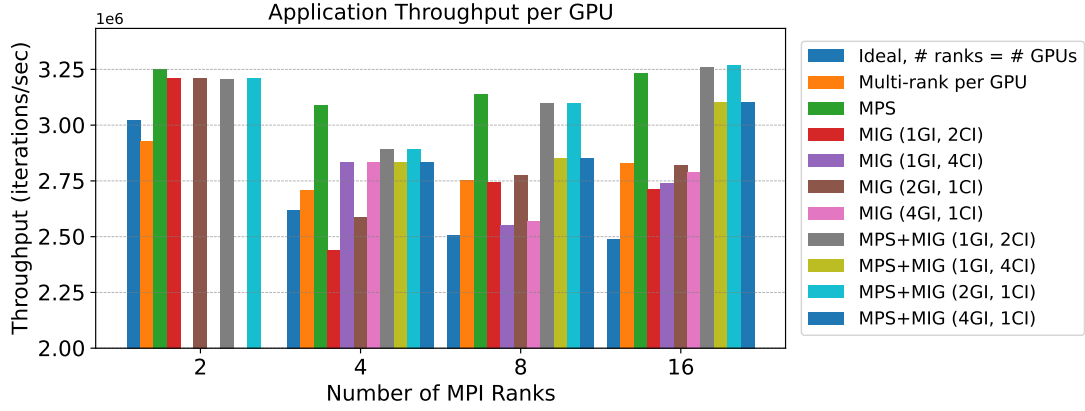


Figure 6.14: Application throughput per GPU (iterations/sec) for varying number of MPI processes running LP-117. While the *Ideal* case maps one rank to one GPU, all other configurations oversubscribe a single physical device. Higher values indicate better resource utilization and cost-efficiency

Figure 6.14 illustrates the application throughput per GPU (iterations/sec) for varying number of MPI processes running the LP-117 algorithm on a single NVIDIA A30 GPU. The *Ideal* case maps one rank to one GPU, while all other configurations oversubscribe a single physical device. The other configurations include: (1) *Multi-rank per GPU*, where multiple MPI processes share the same GPU without any special management; (2) *MPS*, where multiple MPI processes share the GPU using NVIDIA MPS; (3) *MIG*, where multiple MPI processes run on separate MIG instances without MPS; and (4) *MPS +MIG*, where multiple MPI processes run on separate MIG instances using MPS. Various configurations of MIG instances are included: 1-GI-2-CI (1 GPU Instance (GI) with 2 Compute Instance (CI)), 1-GI-4-CI, 2-GI-1-CI, and 4-GI-1-CI.

Moreover, Figure 6.15 presents the speedup of the multi-GPU implementation on a single NVIDIA A30 GPU relative to the *Multi-rank per GPU* baseline, using different configurations of MPI processes, MIG instances, and MPS. Based on the results, we can make the following observations:

- **Oversubscription significantly improves resource utilization.** A primary obser-

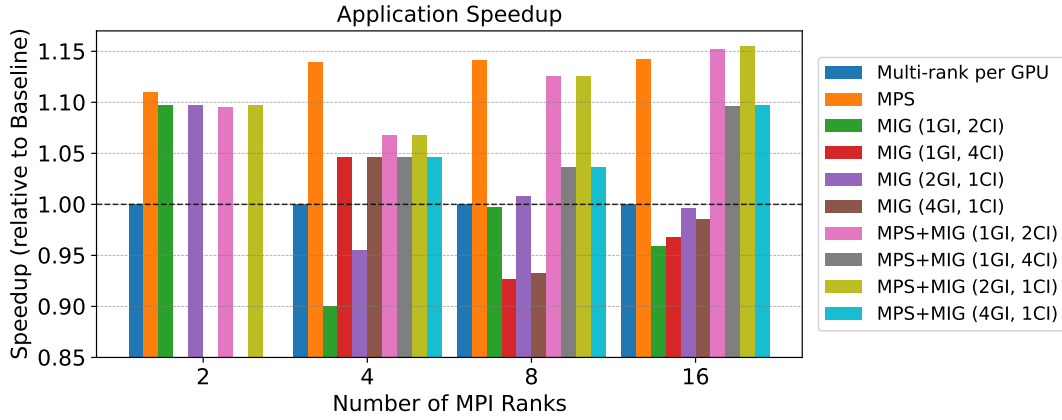


Figure 6.15: Speedup of multi-GPU implementation on a single NVIDIA A30 GPU relative to the *Multi-rank per GPU* baseline, using different configurations of MPI processes, MIG instances, and MPS. Each rank has a constant search space to sweep.

variation from Figure 6.14 is that oversubscribing the GPU is substantially more efficient than a 1:1 rank-to-GPU mapping. While the *Ideal* case yields the lowest per-GPU throughput, all 16-rank oversubscribed configurations achieve higher throughputs, confirming that the 1:1 mapping underutilizes the hardware.

- **MPS consistently mitigates context-switching overhead.** As shown in Figure 6.15, enabling MPS provides a consistent performance improvement of over the baseline. This illustrates the performance penalty of the default driver-level time-slicing and demonstrates that MPS effectively eliminates this overhead.
- **MIG-only configurations degrade performance when oversubscribed.** Conversely, employing MIG partitions without MPS (the MIG configurations) results in a consistent performance degradation relative to the baseline when number of ranks exceeds the number of MIG instances (Figure 6.15). This suggests that LP-117 is already efficient enough that the strict hardware partitioning of MIG, when combined with driver-level time-slicing within each instance, introduces more resource

contention or scheduling inefficiencies that are more detrimental than the baseline.

- The most effective configuration is the combination of **MPS** and **MIG**. The **MPS + MIG** configurations achieve the highest throughput (Figure 6.14), demonstrating that the two technologies complement each other well in our implementation. By leveraging the fine-grained resource management of **MPS** within the isolated environments provided by **MIG**, we can maximize the utilization of the underlying hardware when oversubscribing the **GPU**.

6.5.15 Scalability on Modern GPUs

To evaluate the scalability of our optimized **GPU** implementation on modern hardware, we tested the performance of the **LP-117** algorithm on a more recent **GPU** model, the NVIDIA H100, on Nibi cluster from Compute Canada. The H100 is based on the Hopper architecture and offers significant improvements in terms of computational power, memory bandwidth, and overall efficiency compared to the A30.

As shown in Table 6.4, the H100 has about 4.4× more **SMs** and **CUDA** cores, as well as 3.6× higher memory bandwidth compared to the A30. These enhancements are expected to yield substantial performance gains, and as expected, our evaluation shows a speedup of 3.4× when running the **LP-117** algorithm on the H100 compared to the A30. This means that the H100 implementation achieves an overall speedup of 6,569× compared to the baseline CPU implementation on a single core of Intel Xeon Gold 6338 CPU. This demonstrates that our optimized **GPU** design and implementation scales well with modern **GPU** technologies, making it suitable for future hardware platforms.

Table 6.4: Comparison of key specifications between NVIDIA H100 and A30 GPUs.

Feature	NVIDIA H100 (SXM)	NVIDIA A30
GPU Architecture	Hopper	Ampere
SM Count	132	56
CUDA Cores per SM	128	64
Total CUDA Cores	16896	3804
Transistors	80 Billion	54.2 Billion
Memory Size	80 GB HBM3	24 GB HBM2
Memory Bandwidth	3.35 TB/s	933 GB/s
Memory Bus Width	5120-bit	3072-bit
L2 Cache Size	50 MB	24 MB
Shared Memory per SM	228 KB	164 KB
Shared Memory per Block	48 KB	48 KB
Registers per Block	65536	65536

6.6 Summary

In this study, we highlighted the critical role of **HPC** in addressing the immense computational demands of constructing **LPs**. The infeasibility of solving large orders like **LP-117** on single-core systems, with estimates exceeding 10 billion years, necessitated advanced parallelization and optimization strategies.

We showed CPU-based approaches demonstrate strong scalability. **OpenMP** effectively utilized multi-core CPUs up to 64 threads, though diminishing returns were observed at 128 threads due to **NUMA** effects. The **MPI** implementation extended scalability across nodes, achieving over 90% parallel efficiency and up to 230× speedup (strong scaling) with 256 ranks, while analysis revealed that for over-subscribed configurations, intra-node contention was the main bottleneck, not inter-node bandwidth.

GPU acceleration yielded substantial performance gains by exploiting the algorithm’s parallelism. Key optimizations included:

- Removing branch divergence for 10.42× speedup.

- Leveraging constant/shared memory for 1.59× and 2.07× gains.
- Applying loop transformations for 1.52× improvement.
- Managing resources to achieve 1.26× speedup.
- Employing fine-tuning strategies (e.g., register optimization, shared memory bank conflict avoidance) for additional cumulative gains.
- Utilizing **CUDA** Graphs to reduce kernel launch overhead, achieving 1.01× speedup.
- Utilizing **MPS** and **MIG** for better resource utilization, showing 1.3× improvements in application throughput per **GPU**.

Collectively, **GPU** optimizations achieved over 6,500× and 1,900× speedup on NVIDIA H100 and A30 **GPU**, respectively, versus the single-core baseline executed on an Intel Xeon Gold 6338 CPU. Profiling indicated a 50% occupancy, bounded by shared memory. The hybrid **MPI+CUDA** solution confirmed significant weak scaling (close to 14× for 16 **GPU** devices) and minimal inter-**GPU** overhead, validating its scalability.

Chapter 7

Conclusions and Future Directions

7.1 Conclusion

This dissertation presents a comprehensive study on various techniques to enhance the performance of HPC systems, focusing on GPU-aware multi-path communication, heterogeneous computing, collective communication, and parallelization strategies for complex computational problems. Our work investigated these complex problems through distinct research directions, each contributing to a deeper understanding and optimization of GPU-aware communication and computation. The key contributions of this study were a novel multi-path communication framework (Chapter 3), a predictive analytical performance model for multi-path GPU-to-GPU communication (Chapter 4), a model-enabled heterogeneous collective framework (Chapter 5), and a high-throughput parallelization strategy for a complex combinatorial optimization problem (Chapter 6).

First, in Chapter 3, we designed and implemented a novel multi-path communication framework within the UCX library. This framework effectively utilized heterogeneous interconnects such as NVLink and PCIe to scatter P2P communication, achieving significant bandwidth improvements. Our experiments showed up to a 2.85× speedup on the Beluga

cluster and a $2.75\times$ speedup on the Narval cluster in bandwidth tests. By integrating this framework with a [CUDA](#) Graph-based design, we also demonstrated a substantial increase in communication efficiency by minimizing launch overheads, with performance gains up to $2.9\times$ in intra-node [GPU-to-GPU](#) communication tests.

Second, in Chapter [4](#), to address the challenge of dynamic optimization discussed in Chapter [3](#), we developed a novel predictive performance model for multi-path intra-node [GPU](#) communication. This model extended classical *Hockney's* model to account for multi-channel, pipelined transfers and was integrated into the [MPI + UCX](#) stack. This integration enabled the automatic selection of optimal communication strategies without relying on exhaustive searches, achieving an average prediction error of less than 6% for messages larger than 4MB. The effectiveness of this model-enabled approach was validated by speedups of up to $2.9\times$ in [P2P](#) and $1.4\times$ in collective operations.

Third, in Chapter [5](#), we proposed and implemented a novel model-enabled multi-path heterogeneous *Allreduce* framework, which leveraged both [GPU](#) and CPU resources and their respective interconnects to accelerate collective operations. Our approach demonstrated the benefits of combining heterogeneous computing and multi-path communication, achieving a notable speedup of up to $1.45\times$ over state-of-the-art libraries like [NCCL](#) for very large messages. We also provided the first known analytical performance model for such heterogeneous collectives, which guided our design and optimization efforts. The results showed that our model could predict performance with an average error of less than 8% for message sizes ranging from 4MB to 512MB.

Finally, in Chapter [6](#), we explored advanced parallelization strategies to address the immense computational demands of constructing [LPs](#). This work transformed a mathematically sound but computationally infeasible problem into a high-throughput solution. We demonstrated that CPU-based approaches, using [OpenMP](#) and [MPI](#), delivered strong

scalability with up to a $230\times$ speedup (using 256 ranks), while fine-tuned GPU optimizations provided even more dramatic performance gains, achieving over $6,500\times$ and $1,900\times$ speedup on NVIDIA H100 and A30 GPU, respectively, compared to a single-core baseline executed on an Intel Xeon Gold 6338 CPU. Our profiling confirmed that these methods effectively utilized modern HPC architectures, demonstrating the immense potential of hybrid MPI+CUDA solutions for complex combinatorial problems.

In summary, this dissertation successfully addressed the performance gap in modern HPC systems by providing a suite of innovative solutions, including a dynamic multi-path communication framework, a predictive performance model for multi-path GPU-to-GPU communication, a model-enabled heterogeneous collective framework, and a comprehensive parallelization methodology for complex scientific problems. We hope these contributions offer a path toward more efficient and scalable high-performance computing and communication, enabling new discoveries across various scientific domains.

7.2 Future Directions

Building upon the foundations discussed in this dissertation, several promising avenues for future research exist to further enhance HPC system performance and applicability.

7.2.1 Multi-Path Point-to-Point Communication

In Chapter 3, we introduced a multi-path communication framework that effectively utilizes multiple interconnects for intra-node GPU-to-GPU communication. While our approach improves the communication bandwidth, the performance gain in real-world scenarios is dependent on the concurrent communication pattern. A possible future direction is to dynamically adapt the multi-path communication configuration based on both the application's communication pattern and the hardware characteristics. This requires

extending our framework to monitor real-time communication metrics and adjust the data distribution across available paths accordingly. It is worth noting that users can currently manually tune the communication parameters based on their application's needs. However, this method is static during the life time of an application. Another method is utilizing `MPI_Info` hints to provide application-specific communication patterns to the multi-path framework at runtime, allowing for more informed decision-making regarding path selection and data distribution.

Also, this work can be extended to support other vendor GPUs, such as AMD and Intel. Furthermore, with the new hardware features, such as NVLink-Fusion, future work could explore how to integrate these capabilities into the multi-path communication framework. This would involve extending our framework to be hardware-agnostic and adaptable to emerging interconnect technologies. Also, modern HPC multi-GPU nodes are equipped with NVSwitch interconnects that often connect 8 or more GPUs with high-bandwidth links, such as in the NVIDIA DGX series [100].

Moreover, future work could explore the applicability of multi-path communication strategies to ML workloads, particularly in distributed training scenarios where large models are communicated across multiple GPUs. Although in ML models, message sizes are often smaller than the large messages we focused on for multi-path communication, the workloads can utilize frameworks like Horovod for distributed training, in which messages can be aggregated to larger sizes. In this way, other applications, such as PyTorch and TensorFlow, can benefit from multi-path communication strategies to enhance data transfer efficiency during training and inference phases. Also, in *Mixture of Experts* (MoE) models, where different experts are activated based on the input data, multi-path communication can be employed to efficiently route data to the appropriate experts across multiple GPUs.

Finally, another direction is to apply the similar multi-path communication strategies to inter-node GPU communication. This would involve extending our framework to handle data transfers across multiple nodes, leveraging inter-node communication channels/links. In more detail, on clusters with several NICs per node, or multi-port/multi-rail NICs, multi-path communication can be employed. This extension would require addressing additional challenges, such as considering latency variations across nodes and ensuring minimal contention on shared links in a loaded network.

7.2.2 Performance Modeling for Multi-Path Communication

In Chapter 4, we developed a predictive performance model for multi-path intra-node GPU communication. While our model effectively predicts optimal configurations for pipelined, staged transfers, future work could focus on extending the model to support more complex intra-node communication patterns, such as collective operations. This would involve incorporating additional factors into the model, such as synchronization overheads and contention on shared links in a loaded network.

In more detail, we aim to explore the possibilities of utilizing other performance models as the basis for our multi-path model, such as *MaxRate* [45], when considering contention on shared links in a loaded network. This would involve integrating these models into our existing framework and validating their effectiveness in predicting optimal configurations under various load conditions. Similarly, when extending our model to multi-node communication, the challenges of network contention and latency variations across nodes must be addressed. This would require incorporating network topology and traffic patterns into the model, enabling it to accurately predict performance in distributed environments.

7.2.3 Heterogeneous Collective Communication

In Chapter 5, we proposed a model-enabled multi-path heterogeneous *Allreduce* framework that leverages both GPU and CPU resources to accelerate collective operations. One of the primary directions for future work is to enhance the scalability of our framework. Currently, our implementation focuses on a limited number of GPUs within a single node. In future work, we will focus on extending our framework to support a larger number of GPUs and more complex topologies. This way, we can observe the applicability, scalability, and performance of our approach in multi-node systems where multiple data transfers compete for PCIe bandwidth.

Additionally, we plan to explore dynamic adaptation techniques to optimize communication patterns based on applications requirements and dynamic hardware conditions. We also aim to investigate the integration of our framework within UCC library to provide a more seamless experience for end-users.

Moreover, the discussed techniques and approaches are adaptable to other collective communication operations, such as `MPI_Reduce`, including various number of participants and topologies. Lastly, we will explore the applicability of our collective design in different domains such as machine learning and scientific simulations.

7.2.4 Optimization Strategies for Combinatorial Problems

In Chapter 6, we developed a high-throughput parallelization strategy for constructing LPs, achieving significant performance improvements. Future work could focus on applying the methodologies we developed to other LP lengths. This would require to review and adapt our algorithms to accommodate the unique characteristics of different lengths, ensuring that the performance gains observed in our current implementation are maintained.

Although our current implementation effectively utilizes CPU and GPU resources, future work could explore the integration of other modern hardware features, such as *Tensor Cores* to accelerate specific computations or *CUDA AsyncCopy* to improve data transfer to the device *shared memory*.

Another future direction of this work is to investigate various metrics, such as performance-per-watt and performance-per-dollar, to evaluate the efficiency of our parallelization strategies on different hardware platforms. This would enable us to identify the most cost-effective and energy-efficient solutions with guided optimizations.

Furthermore, we aim to investigate the applicability of our parallelization strategies to other combinatorial optimization problems. This would involve identifying problems with similar computational patterns and adapting our methodologies to address their specific challenges. By doing so, we can extend the impact of our work beyond LP construction, contributing to the broader field of high-performance computing for complex scientific problems.

References

- [1] M. Adams and A. Bienz, “Optimizing Allreduce Operations for Heterogeneous Architectures with Multiple Processes per GPU,” *arXiv*, vol. 1, no. 1, pp. 1–9, 2025. DOI: [10.48550/arXiv.2508.13397](https://doi.org/10.48550/arXiv.2508.13397). arXiv: [2508.13397v1](https://arxiv.org/abs/2508.13397).
- [2] E. Agostini, D. Rossetti, and S. Potluri, “GPUDirect Async: Exploring GPU synchronous communication techniques for InfiniBand clusters,” *Journal of Parallel and Distributed Computing*, vol. 114, pp. 28–45, 2018. DOI: [10.1016/j.jpdc.2017.12.007](https://doi.org/10.1016/j.jpdc.2017.12.007).
- [3] E. Agostini, D. Rossetti, and S. Potluri, “Offloading communication control logic in GPU accelerated applications,” in *Proceedings of the IEEE/ACM International Symposium on Cluster, Cloud, and Grid Computing (CCGRID)*, 2017, pp. 248–257. DOI: [10.1109/CCGRID.2017.29](https://doi.org/10.1109/CCGRID.2017.29).
- [4] A. Alexandrov, M. F. Ionescu, K. E. Schauser, and C. Scheiman, “LogGP: Incorporating Long Messages into the LogP Model,” *Journal of Parallel and Distributed Computing*, pp. 95–105, 1995. DOI: [10.1145/215399.215427](https://doi.org/10.1145/215399.215427).
- [5] P. Alizadeh, A. Sojoodi, Y. Hassan Temucin, and A. Afsahi, “Efficient Process Arrival Pattern Aware Collective Communication for Deep Learning,” in *Proceedings of the European MPI Users’ Group Meeting (EuroMPI)*, 2022, pp. 68–78. DOI: [10.1145/3555819.3555857](https://doi.org/10.1145/3555819.3555857).
- [6] AMD, *HIP*, 2025. [Online]. Available: <https://rocm.docs.amd.com/projects/HIP/> (visited on 10/01/2025).
- [7] AMD, *ROCm*, 2025. [Online]. Available: <https://rocmdocs.amd.com/> (visited on 10/01/2025).

- [8] G. M. Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” in *Proceedings of the April 18-20, 1967, Spring Joint Computer Conference*, ser. AFIPS ’67 (Spring), Atlantic City, New Jersey: Association for Computing Machinery, 1967, pp. 483–485. DOI: [10.1145/1465482.1465560](https://doi.org/10.1145/1465482.1465560).
- [9] P. Anastasiadis, N. Papadopoulou, G. Goumas, and N. Koziris, “CoCoPeLia: Communication-Computation Overlap Prediction for Efficient Linear Algebra on GPUS,” in *Proceedings of IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2021, pp. 36–47. DOI: [10.1109/ISPASS51385.2021.00015](https://doi.org/10.1109/ISPASS51385.2021.00015).
- [10] P. Anastasiadis, N. Papadopoulou, G. Goumas, N. Koziris, D. Hoppe, and L. Zhong, “PAR-ALiA: A Performance Aware Runtime for Auto-tuning Linear Algebra on Heterogeneous Systems,” *ACM Transactions on Architecture and Code Optimization*, vol. 20, no. 4, pp. 1–25, 2023. DOI: [10.1145/3624569](https://doi.org/10.1145/3624569).
- [11] M. Andrews and S. Witteveen, *Gpu kernel scientist: An llm-driven framework for iterative kernel optimization*, 2025. arXiv: [2506.20807](https://arxiv.org/abs/2506.20807) [cs.LG].
- [12] T. M. Apostol, *Introduction to analytic number theory* (Undergraduate Texts in Mathematics). Springer-Verlag, New York-Heidelberg, 1976, pp. xii+338.
- [13] K. T. Arasu, D. A. Bulutoglu, and J. R. Hollon, “Legendre G -array pairs and the theoretical unification of several G -array families,” *J. Combin. Des.*, vol. 28, no. 11, pp. 814–841, 2020. DOI: [10.1002/jcd.21745](https://doi.org/10.1002/jcd.21745).
- [14] A. A. Awan, A. Jain, C. H. Chu, H. Subramoni, and D. K. Panda, “Communication profiling and characterization of deep-learning workloads on clusters with high-performance interconnects,” *IEEE Micro*, vol. 40, no. 1, pp. 35–43, 2020. DOI: [10.1109/MM.2019.2949986](https://doi.org/10.1109/MM.2019.2949986).
- [15] D. S. Banerjee, K. Hamidouche, and D. K. Panda, “Designing high performance communication runtime for GPU managed memory: Early experiences,” in *Proceedings of the Workshop on General Purpose Processing using GPUs (GPGPU)*, 2016, pp. 82–91. DOI: [10.1145/2884045.2884050](https://doi.org/10.1145/2884045.2884050).

- [16] K. Berney and N. Sitchinava, “Eliminating bank conflicts in gpu mergesort,” in *Proceedings of the 37th ACM Symposium on Parallelism in Algorithms and Architectures*, ser. SPAA ’25, Portland, OR, USA: Association for Computing Machinery, 2025, pp. 158–170. DOI: [10.1145/3694906.3743337](https://doi.org/10.1145/3694906.3743337).
- [17] D. E. Bernholdt *et al.*, “A survey of MPI usage in the US exascale computing project,” *Concurrency and Computation: Practice and Experience (CCPE)*, no. 3, pp. 1–16, 2020. DOI: [10.1002/cpe.4851](https://doi.org/10.1002/cpe.4851).
- [18] A. Bienz, L. N. Olson, W. D. Gropp, and S. Lockhart, “Modeling Data Movement Performance on Heterogeneous Architectures,” in *Proceedings of the IEEE High Performance Extreme Computing Conference (HPEC)*, IEEE, 2021, pp. 1–7. DOI: [10.1109/HPEC49654.2021.9622742](https://doi.org/10.1109/HPEC49654.2021.9622742). arXiv: [2010.10378](https://arxiv.org/abs/2010.10378).
- [19] M. Boyer, J. Meng, and K. Kumaran, “Improving GPU performance prediction with data transfer modeling,” in *Proceedings of IEEE International Parallel and Distributed Processing Symposium Workshops and PhD Forum (IPDPSW)*, IEEE, 2013, pp. 1097–1106. DOI: [10.1109/IPDPSW.2013.236](https://doi.org/10.1109/IPDPSW.2013.236).
- [20] J. Bruck, C. T. Ho, S. Kipnis, E. Upfal, and D. Weathersby, “Efficient algorithms for all-to-all communications in multiport message-passing systems,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 8, no. 11, pp. 1143–1156, 1997. DOI: [10.1109/71.642949](https://doi.org/10.1109/71.642949).
- [21] D. Bureddy, H. Wang, A. Venkatesh, S. Potluri, and D. K. Panda, “OMB-GPU: A micro-benchmark suite for evaluating MPI libraries on GPU clusters,” in *Proceedings of the European MPI Users’ Group Meeting (EuroMPI)*, 2012, pp. 110–120. DOI: [10.1007/978-3-642-33518-1_16](https://doi.org/10.1007/978-3-642-33518-1_16).
- [22] J. Cámara, J. Cuenca, V. Galindo, A. Vicente, and M. Boratto, “An autotuning approach to select the inter-GPU communication library on heterogeneous systems,” *Journal of Supercomputing*, vol. 81, no. 1, pp. 1–16, 2025. DOI: [10.1007/s11227-024-06794-3](https://doi.org/10.1007/s11227-024-06794-3).

- [23] C. C. Chen *et al.*, “MPI-xCCL: A Portable MPI Library over Collective Communication Libraries for Various Accelerators,” in *Proceedings of the SC Workshops of The International Conference on High Performance Computing, Network, Storage, and Analysis (SC-W)*, 2023, pp. 847–854. DOI: [10.1145/3624062.3624153](https://doi.org/10.1145/3624062.3624153).
- [24] Y. Chen, B. Brock, K. Yelick, and J. D. Owens, “Scalable Irregular Parallelism with GPUs : Getting CPUs Out of the Way,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2022, pp. 1–16. DOI: [10.1109/SC41404.2022.00055](https://doi.org/10.1109/SC41404.2022.00055).
- [25] H. Choi and J. Lee, “Efficient Use of GPU Memory for Large-Scale Deep Learning Model Training,” *Applied Sciences*, pp. 1–17, 2021. DOI: [10.3390/app112110377](https://doi.org/10.3390/app112110377).
- [26] J. Choi, D. F. Richards, and L. V. Kale, “Improving Scalability with GPU-Aware Asynchronous Tasks,” in *Proceedings of the IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, IEEE, 2022, pp. 1–10. DOI: [10.1109/IPDPSW55747.2022.00097](https://doi.org/10.1109/IPDPSW55747.2022.00097). arXiv: [2202.11819](https://arxiv.org/abs/2202.11819).
- [27] C. H. Chu, K. Hamidouche, A. Venkatesh, A. A. Awan, and D. K. Panda, “CUDA Kernel Based Collective Reduction Operations on Large-scale GPU Clusters,” in *Proceedings of the IEEE/ACM International Symposium on Cluster, Cloud, and Grid Computing (CCGRID)*, 2016, pp. 726–735. DOI: [10.1109/CCGrid.2016.111](https://doi.org/10.1109/CCGrid.2016.111).
- [28] C.-H. Chu, P. Kousha, A. A. Awan, K. S. Khorassani, H. Subramoni, and D. K. Panda, “NV-Group: Link-Efficient Reduction for Distributed Deep Learning on Modern Dense GPU Systems,” in *Proceedings of the International Conference on Supercomputing (ICS)*, 2020, pp. 1–12. DOI: [10.1145/3392717.3392771](https://doi.org/10.1145/3392717.3392771).
- [29] J. Colmenares, A. Galizia, J. Ortiz, A. Clematis, and W. Rocchia, “A combined mpi-cuda parallel solution of linear and nonlinear poisson-boltzmann equation,” *BioMed Research International*, vol. 2014, no. 1, p. 560 987, 2014. DOI: <https://doi.org/10.1155/2014/560987>.

- [30] C. Cui, “Acceleration of tensor-product operations with tensor cores,” *ACM Transactions on Parallel Computing*, vol. 11, no. 4, pp. 1–24, 2024.
- [31] D. Culler *et al.*, “LogP: Towards a realistic model of parallel computation,” in *Proceedings of the ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPOPP)*, vol. Part F1296, 1993, pp. 1–12. DOI: [10.1145/155332.155333](https://doi.org/10.1145/155332.155333).
- [32] D. De Sensi, T. Bonato, D. Saam, and T. Hoefler, “Swing: Short-cutting Rings for Higher Bandwidth Allreduce,” *Proceedings of the USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pp. 1–19, 2024. arXiv: [2401.09356](https://arxiv.org/abs/2401.09356).
- [33] Deep500.org, *Deep500: An HPC Deep Learning Benchmark and Competition*, 2025. [Online]. Available: <https://www.deep500.org/> (visited on 10/01/2025).
- [34] F. Desprez, P. Ramet, and J. Roman, “Optimal grain size computation for pipelined algorithms,” *Lecture Notes in Computer Science*, pp. 165–172, 1996. DOI: [10.1007/3-540-61626-8_21](https://doi.org/10.1007/3-540-61626-8_21).
- [35] D. Z. Djokovic and I. S. Kotsireas, “Compression of periodic complementary sequences and applications,” *Des. Codes Cryptogr.*, vol. 74, no. 2, pp. 365–377, 2015. DOI: [10.1007/s10623-013-9862-z](https://doi.org/10.1007/s10623-013-9862-z).
- [36] J. Ekelund, S. Markidis, and I. Peng, “Boosting Performance of Iterative Applications on GPUs: Kernel Batching with CUDA Graphs,” in *Proceedings of Euromicro International Conference on Parallel, Distributed and Network-based Processing (PDP)*, 2025, pp. 1–8. DOI: [10.1109/PDP66500.2025.00019](https://doi.org/10.1109/PDP66500.2025.00019). arXiv: [2501.09398](https://arxiv.org/abs/2501.09398).
- [37] S. Eum, M. Song, S. Kim, and H. Seo, “Efficient gpu parallel implementation and optimization of aria for counter and exhaustive key-search modes,” *Electronics*, vol. 14, no. 10, 2025. DOI: [10.3390/electronics14102021](https://doi.org/10.3390/electronics14102021).
- [38] J. Fan, Y. Zhang, X. Li, and D. S. Nikolopoulos, “Parallel CPU-GPU Execution for LLM Inference on Constrained GPUs,” *arXiv*, pp. 1–13, 2025. arXiv: [2506.03296](https://arxiv.org/abs/2506.03296).

- [39] I. Faraji and A. Afsahi, “Design considerations for GPU-aware collective communications in MPI,” *Concurrency and Computation: Practice and Experience (CCPE)*, vol. 30, no. 17, pp. 1–24, 2018. DOI: [10.1002/cpe.4667](https://doi.org/10.1002/cpe.4667).
- [40] I. Faraji, S. H. Mirsadeghi, and A. Afsahi, “Topology-aware GPU selection on multi-GPU nodes,” in *Proceedings of the IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2016, pp. 712–720. DOI: [10.1109/IPDPSW.2016.44](https://doi.org/10.1109/IPDPSW.2016.44).
- [41] R. J. Fletcher, M. Gysin, and J. Seberry, “Application of the discrete Fourier transform to the search for generalised Legendre pairs and Hadamard matrices,” *Australas. J. Combin.*, vol. 23, pp. 75–86, 2001.
- [42] S. W. Golomb and G. Gong, *Signal design for good correlation: for wireless communication, cryptography, and radar*. Cambridge University Press, Cambridge, 2005, pp. xviii+438.
- [43] J. Gómez-Luna, J. M. González-Linares, J. I. Benavides, and N. Guil, “Performance models for asynchronous data transfers on consumer Graphics Processing Units,” *Journal of Parallel and Distributed Computing*, vol. 72, no. 9, pp. 1117–1126, 2012. DOI: [10.1016/j.jpdc.2011.07.011](https://doi.org/10.1016/j.jpdc.2011.07.011).
- [44] Graph500.org, *Graph 500*, 2025. [Online]. Available: <https://graph500.org/> (visited on 10/01/2025).
- [45] W. Gropp, L. N. Olson, and P. Samfass, “Modeling MPI communication performance on SMP nodes: Is it time to retire the ping pong test,” in *Proceedings of the European MPI Users’ Group Meeting (EuroMPI)*, 2016, pp. 41–50. DOI: [10.1145/2966884.2966919](https://doi.org/10.1145/2966884.2966919).
- [46] J. L. Gustafson, “Reevaluating amdahl’s law,” *Commun. ACM*, vol. 31, no. 5, pp. 532–533, 1988. DOI: [10.1145/42411.42415](https://doi.org/10.1145/42411.42415).
- [47] P. Hijma, S. Heldens, A. Sclocco, B. Van Werkhoven, and H. E. Bal, “Optimization Techniques for GPU Programming,” *ACM Computing Surveys*, vol. 55, no. 11, pp. 1–81, 2023. DOI: [10.1145/3570638](https://doi.org/10.1145/3570638).

- [48] R. W. Hockney, “The communication challenge for MPP: Intel Paragon and Meiko CS-2,” *Parallel Computing*, vol. 20, no. 3, pp. 389–398, 1994. DOI: [10.1016/S0167-8191\(06\)80021-9](https://doi.org/10.1016/S0167-8191(06)80021-9).
- [49] T. Hoefler, T. Schneider, and A. Lumsdaine, “LogGP in theory and practice - An in-depth analysis of modern interconnection networks and benchmarking methods for collective operations,” *Simulation Modelling Practice and Theory*, vol. 17, no. 9, pp. 1511–1521, 2009. DOI: [10.1016/j.simpat.2009.06.007](https://doi.org/10.1016/j.simpat.2009.06.007).
- [50] A. Hori *et al.*, “An international survey on MPI users,” *Parallel Computing*, pp. 1–13, 2021. DOI: [10.1016/j.parco.2021.102853](https://doi.org/10.1016/j.parco.2021.102853).
- [51] hpc (GitHub Community), *hpc/xpmem: Linux Cross-Memory Attach*, 2025. [Online]. Available: <https://github.com/hpc/xpmem> (visited on 10/01/2025).
- [52] T. W. Huang, D. L. Lin, C. X. Lin, and Y. Lin, “Taskflow: A Lightweight Parallel and Heterogeneous Task Graph Computing System,” *IEEE Transactions on Parallel and Distributed Systems*, pp. 1303–1320, 2021. DOI: [10.1109/TPDS.2021.3104255](https://doi.org/10.1109/TPDS.2021.3104255).
- [53] T. Ichimura and J. Wells, “Heterogeneous computing in a strongly-connected CPU-GPU environment : fast multiple time-evolution equation-based modeling accelerated using data-driven approach,” in *Proceedings of the workshops of the International Conference on High Performance Computing, Network, Storage, and Analysis (SC-W)*, 2024, pp. 1967–1978. DOI: [10.1109/SCW63240.2024.00246](https://doi.org/10.1109/SCW63240.2024.00246).
- [54] InfiniBand Trade Association, 2025. [Online]. Available: <https://www.infinibandta.org/> (visited on 10/01/2025).
- [55] Inria, *KNEM: High-Performance Intra-Node MPI Communication*, 2025. [Online]. Available: <https://knem.gitlabpages.inria.fr/> (visited on 10/01/2025).
- [56] U. Iqbal, T. Davies, and P. Perez, “A review of recent hardware and software advances in gpu-accelerated edge-computing single-board computers (sbcs) for computer vision,” *Sensors*, vol. 24, no. 15, 2024. DOI: [10.3390/s24154830](https://doi.org/10.3390/s24154830).

- [57] J. Jenkins, J. Dinan, P. Balaji, N. F. Samatova, and R. Thakur, "Enabling fast, noncontiguous GPU data movement in hybrid MPI+GPU environments," *Proceedings of the IEEE International Conference on Cluster Computing (CLUSTER)*, pp. 468–476, 2012. DOI: [10.1109/CLUSTER.2012.72](https://doi.org/10.1109/CLUSTER.2012.72).
- [58] R. Kaur, A. Asad, S. Al Abdul Wahid, and F. Mohammadi, "A survey of advancements in scheduling techniques for efficient deep learning computations on gpus," *Electronics*, vol. 14, no. 5, 2025. DOI: [10.3390/electronics14051048](https://doi.org/10.3390/electronics14051048).
- [59] A. H. Khan, M. Al-Mouhamed, M. Al-Mulhem, and A. F. Ahmed, "Rt-cuda: A software tool for cuda code restructuring," *International Journal of Parallel Programming*, vol. 45, no. 3, pp. 551–594, Jun. 2017. DOI: [10.1007/s10766-016-0433-6](https://doi.org/10.1007/s10766-016-0433-6).
- [60] H. Khetawat, N. Jain, A. Bhatele, and F. Mueller, "Predicting GPUDirect Benefits for HPC Workloads," *Proceedings of Euromicro International Conference on Parallel, Distributed and Network-Based Processing (PDP)*, pp. 88–97, 2024. DOI: [10.1109/PDP62718.2024.00020](https://doi.org/10.1109/PDP62718.2024.00020).
- [61] H. Kim, J. Ryu, and J. Lee, "TCCL: Discovering Better Communication Paths for PCIe GPU Clusters," in *International Conference on Architectural Support for Programming Languages and Operating Systems - ASPLOS*, 2024, pp. 1–17. DOI: [10.1145/3620666.3651362](https://doi.org/10.1145/3620666.3651362).
- [62] H. Kim, J. Ryu, and J. Lee, "TCCL: Discovering Better Communication Paths for PCIe GPU Clusters," in *International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, vol. 3, 2024, pp. 999–1015. DOI: [10.1145/3620666.3651362](https://doi.org/10.1145/3620666.3651362).
- [63] Y. Kim, H. Lim, and D. Han, "Scaling Beyond the GPU Memory Limit for Large Mixture-of-Experts Model Training," in *Proceedings of the International Conference on Machine Learning (PMLR)*, 2024, pp. 1–12. [Online]. Available: <https://proceedings.mlr.press/v235/kim24w.html>.
- [64] Kokkos Project, *Kokkos C++ Performance Portability Programming Ecosystem*, 2025. [Online]. Available: <https://kokkos.org/> (visited on 10/01/2025).

- [65] I. Kotsireas and C. Koutschan, “Legendre pairs of lengths $\ell \equiv 0 \pmod{3}$,” *J. Combin. Des.*, vol. 29, no. 12, pp. 870–887, 2021.
- [66] I. S. Kotsireas, C. Koutschan, D. A. Bulutoglu, D. M. Arquette, J. S. Turner, and K. J. Ryan, “Legendre pairs of lengths $\ell \equiv 0 \pmod{5}$,” *Spec. Matrices*, vol. 11, Paper No. 20230105, 15, 2023.
- [67] Lawrence Berkeley National Laboratory, *Berkeley Unified Parallel C (UPC) Project*, 2025. [Online]. Available: <https://upc.lbl.gov/> (visited on 10/01/2025).
- [68] S. Lee, J. Oh, J. Kim, S. Go, J. Park, and D. Mahajan, *Characterizing compute-communication overlap in gpu-accelerated distributed deep learning: Performance and power implications*, 2025. arXiv: [2507.03114 \[cs.DC\]](#).
- [69] B. Li *et al.*, “STAR: Sub-Entry Sharing-Aware TLB for Multi-Instance GPU,” *Proceedings of the International Symposium on Microarchitecture (MICRO)*, pp. 1–15, 2024. DOI: [10.1109/MICRO61859.2024.00031](#). arXiv: [2404.18361](#).
- [70] D. L. Lin and T. W. Huang, “Efficient GPU Computation Using Task Graph Parallelism,” in *Proceedings of the European Conference on Parallel Processing (Euro-Par)*, Springer International Publishing, 2021, pp. 435–450. DOI: [10.1007/978-3-030-85665-6_27](#).
- [71] B. Liu, W. Qiu, L. Jiang, and Z. Gong, “Software pipelining for graphic processing unit acceleration: Partition, scheduling and granularity,” *International Journal of High Performance Computing Applications*, vol. 30, no. 2, pp. 169–185, 2016. DOI: [10.1177/1094342015585845](#).
- [72] G. Liu, Y. Wang, T. Zhao, J. Gu, and D. Li, “mHLogGP: A Parallel Computation Model for CPU/GPU,” *Network and Parallel Computing - 9th IFIP International Conference, NPC 2012. Gwangju, Korea, September 6-8, 2012. Proceedings*, pp. 217–224, 2012.
- [73] W. Liu *et al.*, “AutoPipe-H: A Heterogeneity-Aware Data-Paralleled Pipeline Approach on Commodity GPU Servers,” *IEEE Transactions on Computers*, pp. 1–14, 2024. DOI: [10.1109/TC.2024.3517748](#).

- [74] S. Lockhart, A. Bienz, W. Gropp, and L. Olson, “Performance Analysis and Optimal Node-aware Communication for Enlarged Conjugate Gradient Methods,” *ACM Transactions on Parallel Computing*, vol. 10, no. 1, pp. 1–25, 2023. DOI: [10.1145/3580003](https://doi.org/10.1145/3580003).
- [75] S. Lockhart, A. Bienz, W. D. Gropp, and L. N. Olson, “Characterizing the performance of node-aware strategies for irregular point-to-point communication on heterogeneous architectures,” *Parallel Computing*, vol. 116, no. September 2022, pp. 1–12, 2023. DOI: [10.1016/j.parco.2023.103021](https://doi.org/10.1016/j.parco.2023.103021).
- [76] V. Lončar, L. E. Young-S., S. Škrbić, P. Muruganandam, S. K. Adhikari, and A. Balaž, “Openmp, openmp/mpi, and cuda/mpi c programs for solving the time-dependent dipolar gross-pitaevskii equation,” *Computer Physics Communications*, vol. 209, pp. 190–196, 2016. DOI: <https://doi.org/10.1016/j.cpc.2016.07.029>.
- [77] R. Matsuo, Y. Degawa, H. Irie, S. Sakai, and R. Shioya, “Flexible Approximate Computing for Mitigating Branch Divergence in GPUs,” *IEEE Micro*, vol. 45, no. 02, pp. 90–100, 2025. DOI: [10.1109/MM.2024.3504261](https://doi.org/10.1109/MM.2024.3504261).
- [78] A. Maurya, M. M. Rafique, F. Cappello, and B. Nicolae, “MLP-Offload: Multi-Level, Multi-Path Offloading for LLM Pre-training to Break the GPU Memory Wall,” *arXiv*, vol. 1, no. 1, pp. 1–12, 2025. DOI: [10.1145/3712285.3759864](https://doi.org/10.1145/3712285.3759864). arXiv: [2509.02480](https://arxiv.org/abs/2509.02480).
- [79] MIT Supercomputing Technologies Group, *OpenCilk: The MIT Cilk Project*, 2025. [Online]. Available: <https://cilk.mit.edu/> (visited on 10/01/2025).
- [80] *MPI Forum*, 2025. [Online]. Available: <https://www.mpi-forum.org/> (visited on 10/01/2025).
- [81] *MPICH*, 2025. [Online]. Available: <https://www.mpich.org/> (visited on 10/01/2025).
- [82] *Multi-GPU Jacobi Solver*, 2025. [Online]. Available: <https://github.com/NVIDIA/multi-gpu-programming-models> (visited on 10/01/2025).

- [83] *Multi-Process Service*, 2025. [Online]. Available: <https://docs.nvidia.com/deploy/mps/> (visited on 10/01/2025).
- [84] G. Murthy, M. Ravishankar, M. Baskaran, and P. Sadayappan, "Optimal loop unrolling for gpgpu programs," in *Proceedings of the IEEE International Symposium on Parallel and Distributed Processing (IPDPS)*, Jan. 2010, pp. 1–11. doi: [10.1109/IPDPS.2010.5470423](https://doi.org/10.1109/IPDPS.2010.5470423).
- [85] *MVAPICH*, 2025. [Online]. Available: <https://mvapich.cse.ohio-state.edu/> (visited on 10/01/2025).
- [86] NASA Advanced Supercomputing (NAS) Division, *NAS Parallel Benchmarks (NPB)*, 2025. [Online]. Available: <https://www.nas.nasa.gov/Software/NPB/> (visited on 10/01/2025).
- [87] A. Nukada, "Performance Optimization of Allreduce Operation for Multi-GPU Systems," in *Proceedings of the IEEE International Conference on Big Data (Big Data)*, IEEE, 2021, pp. 3107–3112.
- [88] A. Nukada, "Performance Optimization of Allreduce Operation for Multi-GPU Systems," in *Proceedings of the International Conference on Big Data (Big Data)*, IEEE, 2022, pp. 1–6. doi: [10.1109/bigdata52589.2021.9672073](https://doi.org/10.1109/bigdata52589.2021.9672073).
- [89] E. Nuriyev and A. Lastovetsky, "Efficient and Accurate Selection of Optimal Collective Communication Algorithms Using Analytical Performance Modeling," *IEEE Access*, pp. 1–19, 2021. doi: [10.1109/ACCESS.2021.3101689](https://doi.org/10.1109/ACCESS.2021.3101689).
- [90] E. Nuriyev, J. A. Rico-Gallego, and A. Lastovetsky, "Model-based selection of optimal MPI broadcast algorithms for multi-core clusters," *Journal of Parallel and Distributed Computing*, pp. 1–16, 2022. doi: [10.1016/j.jpdc.2022.03.012](https://doi.org/10.1016/j.jpdc.2022.03.012).
- [91] NVIDIA, *Advanced nvidia cuda kernel optimization techniques with handwritten ptx*, Accessed: 2025-01-01, 2025. [Online]. Available: <https://developer.nvidia.com/blog/advanced-nvidia-cuda-kernel-optimization-techniques-handwritten-ptx/>.

- [92] NVIDIA, *CUDA*, 2025. [Online]. Available: <https://docs.nvidia.com/cuda/index.html> (visited on 10/01/2025).
- [93] NVIDIA, *CUDA C++ Programming Guide*, 2025. [Online]. Available: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/> (visited on 10/01/2025).
- [94] NVIDIA, *CUDA Graphs*, 2025. [Online]. Available: <https://developer.nvidia.com/blog/cuda-graphs/> (visited on 10/01/2025).
- [95] NVIDIA, *CUDA Graphs in Dynamic Environments*, 2025. [Online]. Available: <https://developer.nvidia.com/blog/employing-cuda-graphs-in-a-dynamic-environment/> (visited on 10/01/2025).
- [96] NVIDIA, *GPU memory copy library*, 2025. [Online]. Available: <https://github.com/NVIDIA/gdrcopy> (visited on 10/01/2025).
- [97] NVIDIA, *GPUDirect Technologies*, 2025. [Online]. Available: <https://developer.nvidia.com/gpudirect> (visited on 10/01/2025).
- [98] NVIDIA, *Multi Instance GPU (MIG)*, 2025. [Online]. Available: <https://www.nvidia.com/en-us/technologies/multi-instance-gpu/> (visited on 10/01/2025).
- [99] NVIDIA, *NVIDIA Collective Communications Library*, 2025. [Online]. Available: <https://github.com/NVIDIA/ncc1> (visited on 10/01/2025).
- [100] NVIDIA, *NVIDIA Corporation*, 2025. [Online]. Available: <https://www.nvidia.com/> (visited on 10/01/2025).
- [101] NVIDIA, *NVLink-Fusion*, 2025. [Online]. Available: <https://www.nvidia.com/en-us/data-center/nvlink-fusion/> (visited on 10/01/2025).
- [102] NVIDIA, *NVSHMEM*, 2025. [Online]. Available: <https://developer.nvidia.com/nvshmem> (visited on 10/01/2025).
- [103] *Open MPI*, 2025. [Online]. Available: <https://www.open-mpi.org/> (visited on 10/01/2025).

- [104] *OpenCL*, 2025. [Online]. Available: <https://www.khronos.org/opencl/> (visited on 10/01/2025).
- [105] *OpenMP*, 2025. [Online]. Available: <https://www.openmp.org/> (visited on 10/01/2025).
- [106] OpenSHMEM.org, *OpenSHMEM: The OpenSHMEM Application Programming Interface*, 2025. [Online]. Available: <http://openshmem.org/> (visited on 10/01/2025).
- [107] J. Pješivac-Grbović, T. Angskun, G. Bosilca, G. E. Fagg, E. Gabriel, and J. J. Dongarra, “Performance analysis of MPI collective operations,” in *Proceedings of the IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, vol. 10, 2004, pp. 127–143. DOI: [10.1007/s10586-007-0012-0](https://doi.org/10.1007/s10586-007-0012-0).
- [108] R. Prat *et al.*, “ExaDEM: a HPC application based on exaNBody targeting scalable DEM simulations with complex particle shapes,” *Journal of Open Source Software*, vol. 10, no. 106, p. 7484, 2025. [Online]. Available: <https://cea.hal.science/cea-05117625>.
- [109] B. Qiao, M. Akif Ozkan, J. Teich, and F. Hannig, “The best of both worlds: Combining CUDA graph with an image processing DSL,” in *Proceedings of the ACM/IEEE Design Automation Conference (DAC)*, 2020, pp. 1–6. DOI: [10.1109/DAC18072.2020.9218531](https://doi.org/10.1109/DAC18072.2020.9218531).
- [110] R. Rabenseifner, “Optimization of collective reduction operations,” in *Proceedings of the IEEE International Conference on Cluster Computing (CLUSTER)*, vol. 3036, 2004, pp. 1–9. DOI: [10.1007/978-3-540-24685-5_1](https://doi.org/10.1007/978-3-540-24685-5_1).
- [111] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, “ZeRO : Memory Optimizations Toward Training Trillion Parameter Models,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2020, pp. 1–16. DOI: [10.5555/3433701.3433727](https://doi.org/10.5555/3433701.3433727).
- [112] S. Rajbhandari, O. Ruwase, J. Rasley, S. Smith, and Y. He, “ZeRO-Infinity : Breaking the GPU Memory Wall for Extreme Scale Deep Learning,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2021, pp. 1–14. DOI: [10.1145/3458817.3476205](https://doi.org/10.1145/3458817.3476205).

- [113] *RDMA over Converged Ethernet (RoCE)*, 2025. [Online]. Available: <https://docs.nvidia.com/networking/display/winofv55053000/rocev2> (visited on 10/01/2025).
- [114] A. Riahi, A. Savadi, and M. Naghibzadeh, “Comparison of analytical and ML-based models for predicting CPU-GPU data transfer time,” *Computing*, vol. 102, no. 9, pp. 2099–2116, 2020. DOI: [10.1007/s00607-019-00780-x](https://doi.org/10.1007/s00607-019-00780-x).
- [115] Rice University, *Coarray Fortran 2.0 at Rice University*, 2025. [Online]. Available: <http://caf.rice.edu/> (visited on 10/01/2025).
- [116] J. A. Rico-Gallego, J. C. Díaz-Martín, R. R. Manumachu, and A. L. Lastovetsky, “A survey of communication performance models for high-performance computing,” *ACM Computing Surveys*, vol. 51, no. 6, pp. 1–36, 2019. DOI: [10.1145/3284358](https://doi.org/10.1145/3284358).
- [117] J. A. Rico-Gallego and J. C. Díaz-Martín, “ τ -Lop: Modeling performance of shared memory MPI,” *Parallel Computing*, vol. 46, pp. 14–31, 2015. DOI: [10.1016/j.parco.2015.02.006](https://doi.org/10.1016/j.parco.2015.02.006).
- [118] J. Ryu, J. Kim, H. Kim, D. Park, and J. Lee, “SPipe : Hybrid GPU and CPU Pipeline for Training LLMs under Memory Pressure,” in *Proceedings of the 20th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2025, pp. 1–19.
- [119] P. Sakdhnagool, A. Sabne, and R. Eigenmann, “Regdem: Increasing GPU performance via shared memory register spilling,” *CoRR*, vol. abs/1907.02894, 2019. arXiv: [1907.02894](https://arxiv.org/abs/1907.02894).
- [120] W. Schonbein, S. Levy, M. G. Dosanjh, W. P. Marts, E. Reid, and R. E. Grant, “Modeling and Benchmarking the Potential Benefit of Early-Bird Transmission in Fine-Grained Communication,” *Proceedings of the International Conference on Parallel Processing (ICPP)*, pp. 306–316, 2023. DOI: [10.1145/3605573.3605618](https://doi.org/10.1145/3605573.3605618).
- [121] T. R. Scogland, W. C. Feng, B. Rountree, and B. R. De Supinski, “CoreTSAR: Adaptive worksharing for heterogeneous systems,” *Lecture Notes in Computer Science (LNCS)*, pp. 172–186, 2014. DOI: [10.1007/978-3-319-07518-1_11](https://doi.org/10.1007/978-3-319-07518-1_11).

- [122] D. D. Sensi *et al.*, “Exploring GPU-to-GPU Communication : Insights into Supercomputer Interconnects,” *arXiv*, pp. 1–13, 2024. arXiv: [arXiv:2408.14090v1](#).
- [123] A. Sewell, K. Fan, L. Dyken, and S. Petruzza, “Bruck Algorithm Performance Analysis for Multi-GPU All-to-All Communication,” in *Proceedings of the International Conference on High Performance Computing in Asia-Pacific Region (HPCAsia)*, 2024, pp. 127–133. DOI: [10.1145/3635035.3635047](#).
- [124] P. Shamis *et al.*, “UCX : An Open Source Framework for HPC Network APIs and Beyond,” in *Proceedings of the IEEE Symposium on High-Performance Interconnects (HOTI)*, IEEE, 2015, pp. 40–43. DOI: [10.1109/HOTI.2015.13](#).
- [125] H. Sharifian, A. Sojoodi, and A. Afsahi, “A Topology- and Load-Aware Design for Neighborhood Allgather,” in *Proceedings of the IEEE International Conference on Cluster Computing (CLUSTER)*, 2024, pp. 1–12. DOI: [10.1109/CLUSTER59578.2024.00019](#).
- [126] S. Shi, Q. Wang, and X. Chu, “Performance modeling and evaluation of distributed deep learning frameworks on GPUs,” *Proceedings of the IEEE International Conference on Dependable, Autonomic and Secure Computing (DASC)*, pp. 943–948, 2018. DOI: [10.1109/DASC/PiCom/DataCom/CyberSciTec.2018.000-4](#).
- [127] V. Shreeyansh, A. S. Gad, B. A. Rao, and N. G. Kini, “Parallelizing exponential search algorithms with mpi and cuda for high-performance computing,” in *2025 International Conference on Next Generation Communication and Information Processing (INCIP)*, 2025, pp. 285–288. DOI: [10.1109/INCIP64058.2025.11019520](#).
- [128] P. Shu, J. Chen, Z. Liu, H. Zhao, X. Li, and T. Liu, “Survey of HPC in US Research Institutions,” *arXiv*, pp. 1–28, 2025. arXiv: [2506.19019](#).
- [129] A. Sojoodi, M. Akbari, H. Sharifian, A. Farzadaghi, R. Grant, and A. Afsahi, “Accelerating intra-node gpu communication: A performance model for multi-path transfers,” in *Proceedings of the 13th Workshop on Extreme Scale MPI (ExaMPI)*, 2025, pp. 1–12. DOI: [10.1145/3731599.3767392](#).

- [130] A. Sojoodi, A. Farazdaghi, H. Sharifian, R. E. Grant, and A. Afsahi, “Collaborative Bandwidth-Efficient Intra-Node Allreduce,” in *Proceedings of the International Workshop on Accelerators and Hybrid Emerging Systems (AsHES)*, 2025, pp. 1–5. DOI: [10.1109/IPDPSW66978.2025.00016](https://doi.org/10.1109/IPDPSW66978.2025.00016).
- [131] A. Sojoodi, I. S. Kotsereas, and A. Afsahi, “Efficient High-Performance Computing Strategies for the Legendre Pairs Search,” *International Journal of Parallel Programmin (JPP) - Submitted*, pp. 1–39, 2025.
- [132] A. Sojoodi, M. Salimi Beni, and F. Khunjush, “Ignite-GPU: a GPU-enabled in-memory computing architecture on clusters,” *Journal of Supercomputing*, pp. 1–28, 2020. DOI: [10.1007/s11227-020-03390-z](https://doi.org/10.1007/s11227-020-03390-z).
- [133] A. Sojoodi, Y. H. Temucin, and A. Afsahi, “Enhancing Intra-Node GPU-to-GPU Performance in MPI + UCX through Multi-Path Communication,” in *Proceedings of the International Workshop on Extreme Heterogeneity Solutions (ExHET)*, 2024, pp. 1–6. DOI: [10.1145/3642961.3643800](https://doi.org/10.1145/3642961.3643800).
- [134] A. Sojoodi, Y. H. Temucin, A. Barati, and A. Afsahi, “Accelerating Intra-Node GPU-to-GPU Communication Through Multi-Path Transfers,” *Journal of Supercomputing - Under Revision*, pp. 1–27, 2025.
- [135] A. Srikanth, C. Trigila, and E. Roncali, “Gpu optimization techniques to accelerate optigan—a particle simulation gan,” *Machine Learning: Science and Technology*, vol. 5, no. 2, p. 027 001, Jun. 2024. DOI: [10.1088/2632-2153/ad51c9](https://doi.org/10.1088/2632-2153/ad51c9).
- [136] Y. Tatsugi and A. Nukada, “Accelerating data transfer between host and device using idle GPU,” in *Proceedings of the Workshop on General Purpose Processing using GPUs (GPGPU)*, 2022, pp. 1–6. DOI: [10.1145/3530390.3532732](https://doi.org/10.1145/3530390.3532732).
- [137] Y. H. Temucin, R. E. Grant, and A. Afsahi, “Micro-Benchmarking MPI Partitioned Point-to-Point Communication,” in *Proceedings of the International Conference on Parallel Processing*

- (ICPP), Association for Computing Machinery, 2022, pp. 1–12. doi: [10.1145/3545008.3545088](https://doi.org/10.1145/3545008.3545088).
- [138] Y. H. Temucin, A. Sojoodi, P. Alizadeh, and A. Afsahi, “Efficient Multi-Path NVLink / PCIe-Aware UCX based Collective Communication for Deep Learning,” in *Proceedings of the IEEE Symposium on High-Performance Interconnects (HOTI)*, 2021, pp. 1–10. doi: [10.1109/HOTI52880.2021.00018](https://doi.org/10.1109/HOTI52880.2021.00018).
 - [139] Y. H. Temucin, A. Sojoodi, P. Alizadeh, B. W. Kitor, and A. Afsahi, “Accelerating Deep Learning using Interconnect-Aware UCX Communication for MPI Collectives,” *IEEE Micro*, pp. 1–9, 2021. doi: [10.1109/MM.2022.3148670](https://doi.org/10.1109/MM.2022.3148670).
 - [140] The Khronos Group, *SYCL: C++ Single-source Heterogeneous Programming for Acceleration Offload*, 2025. [Online]. Available: <https://www.khronos.org/sycl/> (visited on 10/01/2025).
 - [141] The Ohio State University, *The OSU Micro-Benchmark Suite (OMB)*, 2025. [Online]. Available: <https://mvapich.cse.ohio-state.edu/benchmarks/> (visited on 10/01/2025).
 - [142] The Open Group, *The Open Group Base Specifications Issue 7, 2018 edition (IEEE Std 1003.1-2017)*, 2018. [Online]. Available: <https://pubs.opengroup.org/onlinepubs/9699919799/> (visited on 10/01/2025).
 - [143] The OpenACC Organization, *OpenACC: Homepage*, 2025. [Online]. Available: <https://www.openacc.org/> (visited on 10/01/2025).
 - [144] A. Thune, S.-a. Reinemo, T. Skeie, and X. Cai, “Detailed modeling of heterogeneous and communication,” *IEEE Transactions on Parallel and Distributed Systems*, pp. 1–14, 2023. doi: [10.1109/TPDS.2023.3253881](https://doi.org/10.1109/TPDS.2023.3253881).
 - [145] *Top500*, 2025. [Online]. Available: <https://top500.org/> (visited on 10/01/2025).

- [146] T. Tran *et al.*, “Accelerating communication with multi-HCA aware collectives in MPI,” *Concurrency and Computation: Practice and Experience*, vol. 36, no. 1, pp. 1–20, 2024. doi: [10.1002/cpe.7879](https://doi.org/10.1002/cpe.7879).
- [147] J. S. Turner, I. S. Kotsireas, D. A. Bulutoglu, and A. J. Geyer, “A Legendre pair of length 77 using complementary binary matrices with fixed marginals,” *Des. Codes Cryptogr.*, vol. 89, no. 6, pp. 1321–1333, 2021.
- [148] D. Unat *et al.*, “The Landscape of GPU-Centric Communication,” *arXiv*, pp. 1–25, 2024. arXiv: [arXiv:2409.09874v2](https://arxiv.org/abs/2409.09874v2).
- [149] *Unified Collective Communication (UCC)*, 2025. [Online]. Available: <https://github.com/openucx/ucc> (visited on 10/01/2025).
- [150] *Unified Communication X (UCX)*, 2025. [Online]. Available: <https://openucx.org/> (visited on 10/01/2025).
- [151] UXL Foundation, *oneAPI Threading Building Blocks (oneTBB)*, 2025. [Online]. Available: <https://github.com/oneapi-src/oneTBB> (visited on 10/01/2025).
- [152] I. E. Venetis, V. Saltogianni, V. Saltogianni, S. C. Stiros, and E. Gallopoulos, “Multivariable inversion using exhaustive grid search and high-performance gpu processing: A new perspective,” *Geophysical Journal International*, vol. 221, pp. 905–927, 2 2020. doi: [10.1093/GJI/GGAA042](https://doi.org/10.1093/GJI/GGAA042).
- [153] M. G. Venkata *et al.*, “Unified Collective Communication (UCC): An Unified Library for CPU , GPU , and DPU Collectives,” in *Proceedings of the IEEE Symposium on High-Performance Interconnects (HOTI)*, IEEE, 2024, pp. 37–46. doi: [10.1109/HOTI63208.2024.00018](https://doi.org/10.1109/HOTI63208.2024.00018).
- [154] Vetter, Jeffrey S. *et al.*, *vetter/shoc: The Scalable HeterOgeneous Computing (SHOC) Benchmark Suite*, 2025. [Online]. Available: <https://github.com/vetter/shoc> (visited on 10/01/2025).

- [155] G. Wang, S. Venkataraman, A. Phanishayee, J. Thelin, N. Devanur, and I. Stoica, “Blink: Fast and Generic Collectives for Distributed ML,” in *Proceedings of Machine Learning and Systems (MLSys)*, 2020, pp. 1–15.
- [156] G. Wang *et al.*, “ZeRO++: Extremely Efficient Collective Communication for Giant Model Training,” *arXiv*, pp. 1–12, 2023. DOI: [10.48550/arXiv.2306.10209](https://doi.org/10.48550/arXiv.2306.10209). arXiv: [2306.10209](https://arxiv.org/abs/2306.10209).
- [157] J. Wang, T. Zhao, and Y. Wang, “Network states-aware collective communication optimization,” *Cluster Computing*, pp. 1–19, 2024. DOI: [10.1007/s10586-024-04330-9](https://doi.org/10.1007/s10586-024-04330-9).
- [158] Z. Wang, H. Chen, X. Dong, W. Cai, and X. Zhang, “LogSC: Model-based one-sided communication performance estimation,” *Future Generation Computer Systems*, vol. 132, pp. 25–39, 2022. DOI: [10.1016/j.future.2022.02.004](https://doi.org/10.1016/j.future.2022.02.004).
- [159] B. V. Werkhoven, J. Maassen, F. J. Seinstra, and H. E. Bal, “Performance models for CPU-GPU data transfers,” in *Proceedings of the International Symposium on Cluster, Cloud, and Grid Computing (CCGrid)*, IEEE, 2014, pp. 11–20. DOI: [10.1109/CCGrid.2014.16](https://doi.org/10.1109/CCGrid.2014.16).
- [160] B. van Werkhoven, “Kernel tuner: A search-optimizing gpu code auto-tuner,” *Future Generation Computer Systems*, vol. 90, pp. 347–358, 2019. DOI: <https://doi.org/10.1016/j.future.2018.08.004>.
- [161] Y. Wu, X. Liu, S. Jin, C. Xu, and D. C. Apr, “HeterMoE: Efficient Training of Mixture-of-Experts Models on Heterogeneous GPUs,” *arXiv*, pp. 1–14, 2025. arXiv: [arXiv:2504.03871v1](https://arxiv.org/abs/2504.03871v1).
- [162] C.-T. Yang, C.-L. Huang, and C.-F. Lin, “Hybrid cuda, openmp, and mpi parallel programming on multicore gpu clusters,” *Computer Physics Communications*, vol. 182, no. 1, pp. 266–269, 2011, Computer Physics Communications Special Edition for Conference on Computational Physics Kaohsiung, Taiwan, Dec 15-19, 2009. DOI: <https://doi.org/10.1016/j.cpc.2010.06.035>.
- [163] X. Yi, “A Study of Performance Programming of CPU, GPU accelerated Computers and SIMD Architecture,” *arXiv*, pp. 1–19, 2024. arXiv: [2409.10661](https://arxiv.org/abs/2409.10661).

- [164] B. Zhang, M. Zheng, D. Ganguly, X. Zhang, and V. Singh, “Efficient Fine-Grained GPU Performance Modeling for Distributed Deep Learning of LLM,” *arXiv*, pp. 1–11, 2025. doi: [10.48550/arXiv.2509.22832](https://doi.org/10.48550/arXiv.2509.22832). arXiv: [arXiv:2509.22832v1](https://arxiv.org/abs/2509.22832v1).
- [165] C. Zhao, W. Gao, F. Nie, and H. Zhou, “A Survey of GPU Multitasking Methods Supported by Hardware Architecture,” *IEEE Transactions on Parallel and Distributed Systems*, pp. 1–13, 2022. doi: [10.1109/TPDS.2021.3115630](https://doi.org/10.1109/TPDS.2021.3115630).
- [166] L. Zhao *et al.*, “ForestColl : Efficient Collective Communications on Heterogeneous Network Fabrics,” *arXiv*, pp. 1–26, 2024. arXiv: [arXiv:2402.06787v1](https://arxiv.org/abs/2402.06787v1).
- [167] X. Zhao, B. Jia, H. Zhou, Z. Liu, and Y. You, “HeteGen: Efficient Heterogeneous Parallel Inference for Large Language Models on Resource-Constrained Devices,” in *Proceedings of Machine Learning and Systems (MLSys)*, 2024, pp. 1–11. arXiv: [arXiv:2206.09504v1](https://arxiv.org/abs/2206.09504v1).
- [168] Y. Zhao, Q. Sun, Z. He, Y. Bai, and B. Yu, “AutoGraph: Optimizing DNN Computation Graph for Parallel GPU Kernel Execution,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, pp. 1–9, 2023. doi: [10.1609/aaai.v37i9.26343](https://doi.org/10.1609/aaai.v37i9.26343).
- [169] B. Zheng *et al.*, “Grape: Practical and Efficient Graphed Execution for Dynamic Deep Neural Networks on GPUs,” in *Proceedings of the International Symposium on Microarchitecture (MICRO)*, 2023, pp. 1364–1380. doi: [10.1145/3613424.3614248](https://doi.org/10.1145/3613424.3614248).

Appendices

Appendix A

Configuring Multi-Instance GPU (MIG) on NVIDIA GPUs

A.1 MIG Configuration

MIG is a feature of recent NVIDIA **GPUs** that enables partitioning of a single physical **GPU** into multiple isolated instances ¹. Each instance is provisioned with its own dedicated compute cores, memory, and I/O resources, thereby enabling multiple applications or users to share the same **GPU** with predictable performance isolation. This approach is especially advantageous when applications under-utilize the full **GPU** capacity.

Although **MIG** shares some conceptual similarities with NVIDIA **MPS** ², it is fundamentally different in design and implementation. While **MPS** alleviates context switching overhead, **MIG** provides hardware-level partitioning, ensuring that each instance operates independently with dedicated resources. This isolation minimizes interference between workloads, leading to more consistent performance. Therefore, both **MIG** and **MPS** might be used together to improve overall **GPU** utilization. As configuration and management of **MIG** can be complex, this appendix provides a detailed guide on enabling and utilizing **MIG** on supported NVIDIA **GPUs**.

MIG mode can be activated on a per-**GPU** basis. First, the available **GPU** devices should be listed.

¹See: <https://docs.nvidia.com/datacenter/tesla/mig-user-guide/>

²See: <https://docs.nvidia.com/deploy/mps/index.html>

MIG mode can then be enabled for a specific GPU as follows:

```
0 # List available GPUs
1 nvidia-smi -L
2 # Enable MIG mode on GPU 0
3 nvidia-smi -i 0 --mig 1
4 # Query MIG mode status
5 nvidia-smi -i 0 --query-gpu=pci.bus_id,mig.mode.current --format=csv
```

Listing A.1: Enabling MIG on a GPU

Once MIG mode has been enabled, we can create GPU instances by specifying available MIG profiles. The profiles may be queried by:

```
0 $ nvidia-smi mig -lgip
```

Listing A.2: Querying available MIG profiles

MIG profiles are denoted using the format <GPU Slice Count>g.<Memory Size>gb, with an optional suffix +me indicating media extensions. For example, MIG 1g.6gb+me provides 1 GPU slice with 6 GB of memory and additional media features.

Available placement options can be obtained using:

```
0 nvidia-smi mig -lgipp
1 # Example output
2 GPU 0 Profile ID 14 Placements: {0,1,2,3}:1
3 GPU 0 Profile ID 21 Placements: {0,1,2,3}:1
4 GPU 0 Profile ID 5 Placements: {0,2}:2
5 GPU 0 Profile ID 6 Placements: {0,2}:2
6 GPU 0 Profile ID 0 Placement : {0}:4
```

Listing A.3: Placement information of MIG profiles

The syntax of the placement is {<index>}<GPU Slice Count> shows the placement of the instances on the GPU slices. For example, profile ID 5 can be placed on indices 0 and 2, each with 2 GPU slices.

GPU instances can be created in three ways: by profile ID, by short profile name, or by full profile name. These can be combined:

```
0 # Using a combination of profile IDs
```

```

1 nvidia-smi mig -cgi <14,21,5,6,0>
2 # Using short names
3 nvidia-smi mig -cgi <1g.6gb,2g.12gb>
4 # Using full names
5 nvidia-smi mig -cgi <MIG 1g.6gb,MIG 2g.12gb>
6 # Mixed approach
7 nvidia-smi mig -cgi 14,1g.6gb

```

Listing A.4: Creating GPU instances with different specifications

Destroying **GIs** or **CI**s may be required for reconfiguration. All instances can be destroyed as follows:

```

0 # Destroy all Compute and GPU instances
1 nvidia-smi mig -dci # --destroy-compute-instance
2 nvidia-smi mig -dgi # --destroy-gpu-instance
3
4 # Or destroy specific Compute Instances under GPU instance 1
5 nvidia-smi mig -dci -ci 0,1,2 -gi 1
6
7 # Verify the remaining GPU instances
8 nvidia-smi mig -lgi

```

Listing A.5: Destroying MIG instances

A.2 MIG on a Multi-GPU Node

On a system with four NVIDIA A30 **GPUs**, **MIG** mode can be enabled on each **GPU** individually.

As can be seen in the following example, initially, **MIG** mode is disabled:

```

0 $ nvidia-smi -L
1 GPU 0: NVIDIA A30 (UUID: GPU-d8430827-89da-89df-1d70-bde0c9883859)
2 GPU 1: NVIDIA A30 (UUID: GPU-d1ee693e-571b-7664-f22b-1312585630d0)
3 GPU 2: NVIDIA A30 (UUID: GPU-fc7c8f73-47c4-7b21-c9da-858462fa0433)
4 GPU 3: NVIDIA A30 (UUID: GPU-bef3638b-2c97-ea4b-83d9-b1a6b7d2fc29)

```

Listing A.6: Listing GPUs before enabling MIG

After enabling **MIG** mode on **GPU 0**, it can be reconfigured to create multiple **GPU** instances. For example, two instances with profile MIG 2g.12gb can be created:

```

0 $ nvidia-smi -i 0 -mig 1
1 Enabled MIG Mode for GPU 00000000:17:00.0

```

```

2 All done.
3
4 # Create GPU instances
5 $ nvidia-smi mig -cgi 5,5
6 Successfully created GPU instance ID 1 on GPU 0 using profile MIG 2g.12gb (ID
   5)
7 Successfully created GPU instance ID 2 on GPU 0 using profile MIG 2g.12gb (ID
   5)
8
9 # List GPU instances
10 $ nvidia-smi mig -lgi
11 +-----+
12 | GPU instances:                                |
13 | GPU   Name                Profile  Instance  Placement |
14 |                               ID      ID      Start:Size |
15 |=====|
16 |  0  MIG 2g.12gb           5         1         0:2      |
17 +-----+
18 |  0  MIG 2g.12gb           5         2         2:2      |
19 +-----+
20
21 # However there are no Compute Instance (CI) created yet
22 $ nvidia-smi mig -lci
23 No compute instances found: Not Found
24
25 # And the list has not changed
26 $ nvidia-smi -L
27 GPU 0: NVIDIA A30 (UUID: GPU-d8430827-89da-89df-1d70-bde0c9883859)
28 GPU 1: NVIDIA A30 (UUID: GPU-d1ee693e-571b-7664-f22b-1312585630d0)
29 GPU 2: NVIDIA A30 (UUID: GPU-fc7c8f73-47c4-7b21-c9da-858462fa0433)
30 GPU 3: NVIDIA A30 (UUID: GPU-bef3638b-2c97-ea4b-83d9-b1a6b7d2fc29)

```

Listing A.7: Creating and listing MIG instances

A.2.1 Compute Instances (CI)

GPU instances alone are insufficient to execute CUDA workloads, and corresponding CIs must be created first. For example, two GPU instances with profile MIG 2g.12gb can each be subdivided into two CIs with profile MIG 1c.2g.12gb:

```

0 # Create compute instances for GPU instance 1 and 2
1 $ nvidia-smi mig -cci 0,0 -gi 1

```

```

2 Successfully created compute instance ID 0 on GPU 0 GPU instance ID 2 using
  profile MIG 1c.2g.12gb (ID 0)
3 Successfully created compute instance ID 1 on GPU 0 GPU instance ID 2 using
  profile MIG 1c.2g.12gb (ID 0)
4
5 $ nvidia-smi mig -cci 0,0 -gi 2
6 Successfully created compute instance ID 0 on GPU 0 GPU instance ID 2 using
  profile MIG 1c.2g.12gb (ID 0)
7 Successfully created compute instance ID 1 on GPU 0 GPU instance ID 2 using
  profile MIG 1c.2g.12gb (ID 0)
8
9 $ nvidia-smi mig -lci
10 +-----+
11 | Compute instances:                                     |
12 | GPU      GPU      Name          Profile  Instance  Placement |
13 |      Instance          ID          ID        Start:Size |
14 |      ID                                     |
15 |=====|
16 |  0      1      MIG 1c.2g.12gb    0         0         0:1    |
17 +-----+
18 |  0      1      MIG 1c.2g.12gb    0         1         1:1    |
19 +-----+
20 |  0      2      MIG 1c.2g.12gb    0         0         0:1    |
21 +-----+
22 |  0      2      MIG 1c.2g.12gb    0         1         1:1    |
23 +-----+

```

Listing A.8: Creating Compute Instances within MIG

After configuration, the **GPU** reports multiple **MIG** devices, and applications can be dispatched to each CI independently.

A.2.2 Running CUDA Applications on MIG

Applications can be executed on individual **MIG** instances by exporting the appropriate UUID:

```

0 $ nvidia-smi
1 +-----+
2 | MIG devices:                                     |
3 +-----+-----+-----+-----+
4 | GPU  GI  CI  MIG | Memory-Usage | Vol | Shared |
5 |      ID  ID  Dev | BAR1-Usage | SM  Unc | CE  ENC  DEC  OFA  JPG |
6 |                  |            |     ECC |      |      |      |      |
7 |=====|

```

```

 8 | 0 1 0 0 | 25MiB / 11968MiB | 14 0 | 2 0 2 0 0 |
 9 |         | 0MiB / 16383MiB |      |      |
10 +-----+-----+-----+-----+
11 | 0 1 1 1 |         | 14 0 | 2 0 2 0 0 |
12 |         |         |      |      |
13 +-----+-----+-----+-----+
14 | 0 2 0 2 | 25MiB / 11968MiB | 14 0 | 2 0 2 0 0 |
15 |         | 0MiB / 16383MiB |      |      |
16 +-----+-----+-----+-----+
17 | 0 2 1 3 |         | 14 0 | 2 0 2 0 0 |
18 |         |         |      |      |
19 +-----+-----+-----+-----+
20
21 $ nvidia-smi -L
22 GPU 0: NVIDIA A30 (UUID: GPU-d8430827-89da-89df-1d70-bde0c9883859)
23   MIG 1c.2g.12gb Device 0: (UUID: MIG-89b757f7-3c4c-5b1c-9476-a11b14aa9308)
24   MIG 1c.2g.12gb Device 1: (UUID: MIG-bdf043cc-8668-599c-95b0-5b640d416440)
25   MIG 1c.2g.12gb Device 2: (UUID: MIG-6df26cf8-a984-58d3-978e-acb0c808d513)
26   MIG 1c.2g.12gb Device 3: (UUID: MIG-6f79e1f8-f1ed-5121-976a-4ff3f0df8634)
27 GPU 1: NVIDIA A30 (UUID: GPU-d1ee693e-571b-7664-f22b-1312585630d0)
28 GPU 2: NVIDIA A30 (UUID: GPU-fc7c8f73-47c4-7b21-c9da-858462fa0433)
29 GPU 3: NVIDIA A30 (UUID: GPU-bef3638b-2c97-ea4b-83d9-b1a6b7d2fc29)
30
31 $ CUDA_VISIBLE_DEVICES=MIG-89b757f7-... ./my_cuda_code &
32 $ CUDA_VISIBLE_DEVICES=MIG-bdf043cc-... ./my_cuda_code &
33 $ CUDA_VISIBLE_DEVICES=MIG-6df26cf8-... ./my_cuda_code &
34 $ CUDA_VISIBLE_DEVICES=MIG-6f79e1f8-... ./my_cuda_code &

```

Listing A.9: Executing CUDA applications on MIG devices

A.3 Enabling MPS on MIG

MPS can also be enabled on top of **MIG**. The key steps are: (i) configure the **MIG** geometry, (ii) set `CUDA_MPS_PIPE_DIRECTORY` to separate directories for each **MIG** instance, and (iii) launch applications with `CUDA_VISIBLE_DEVICES`. A simple script to automate this procedure is:

```

0 #!/bin/bash
1 set -eux
2 GPU_UUIDs=$(nvidia-smi -L | grep -oE "(MIG)-[0-9a-f-]*")
3
4 for ((index = 0; index < ${#GPU_UUIDs[@]}; index++)); do
5     GPU=${GPU_UUIDs[index]}
6     rm -rf /tmp/mps_${GPU}

```

```

7  rm -rf /tmp/mps_log_${GPU}
8  mkdir /tmp/mps_${GPU}
9  mkdir /tmp/mps_log_${GPU}
10 # Skip setting the GPU compute mode to Exclusive Process (not supported on
    MIG-enabled GPUs)
11 # nvidia-smi -i $index -c EXCLUSIVE_PROCESS
12 export CUDA_VISIBLE_DEVICES=${GPU}
13 export CUDA_MPS_PIPE_DIRECTORY=/tmp/mps_${GPU}
14 export CUDA_MPS_LOG_DIRECTORY=/tmp/mps_log_${GPU}
15 nvidia-cuda-mps-control -d
16 done
17
18 ps -ef | grep mps

```

Listing A.10: Enabling MPS on MIG instances

Now, this is how to run applications on each MIG instance with MPS enabled:

```

0 $ CUDA_MPS_PIPE_DIRECTORY=/tmp/mps_MIG-... \
1   CUDA_MPS_LOG_DIRECTORY=/tmp/mps_log_MIG-... \
2   ./deviceQuery

```

Listing A.11: Running CUDA applications on MIG instances with MPS

Similarly, MPS can be disabled for each MIG instance by terminating the MPS control daemon and cleaning up the directories.

```

0 #!/bin/bash
1 set -eux
2 GPU_UUIDS=$(nvidia-smi -L | grep -oE "(MIG)-[0-9a-f-]*")
3 for ((index = 0; index < ${#GPU_UUIDS[@]}; index++)); do
4   GPU=${GPU_UUIDS[index]}
5   export CUDA_VISIBLE_DEVICES=${GPU}
6   export CUDA_MPS_PIPE_DIRECTORY=/tmp/mps_${GPU}
7   export CUDA_MPS_LOG_DIRECTORY=/tmp/mps_log_${GPU}
8   echo "quit" | nvidia-cuda-mps-control
9   rm -rf /tmp/mps_log_${GPU}
10  rm -rf /tmp/mps_${GPU}
11  # Reset the GPU compute mode to Default (not supported on MIG-enabled GPUs)
12  # nvidia-smi -i $index -c DEFAULT
13 done
14
15 # Verify MPS processes have been terminated
16 ps -ef | grep mps

```

Listing A.12: Disabling MPS on MIG instances